Basics

目录

1 赛场工具 1

1.1 函数内随机对拍 1
1.2 Debug 3
1.3 IO 16
1.4 SafeHash 23
1.5 普通框架 28
1.6 时间优化 28
1.7 Python 算法竞赛回忆手册 29
1.8 PBDS 常见用法 41
1.9 标准库与内置函数速查 47
1.10 常用工具函数 61
1.11 RNG 随机数生成器 63
1.12 Cmake 67

2 基础工具与常见技巧 68

2.0 trick 68
2.1 MInt 117
2.2 Frac 118
2.3 组合数 121
2.4 高精度 124
2.5 Matrix 139
2.6 并查集维护后继（删除跳点 DSU） 141
2.7 二分、三分与黄金分割 143
2.8 并行二分 148
2.9 位运算、二进制集合枚举与 Bitset 149
2.10 选择、区间合并与小并大 159
2.11 进制转换 161
2.12 表达式解析与表达式树 164
2.13 日期与星期计算 169
2.14 时间线段树分治 171
2.15 贪心定式与证明 174

1. 赛场工具

1.1 函数内随机对拍

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;

mt19937_64 rng;
int randint(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}
// ===== 题目相关：按题目修改以下部分 =====
struct Data {};

struct Ans {
    friend bool operator==(const Ans &, const Ans &) = default;
};

Data gen(u64 id) {
    return {};
}

Ans fast(Data a) {
    return {};
}

Ans brute(Data a) {
    return {};
}

// 判断 fast 的答案 x 是否可以接受；y 为暴力 / 标准答案
bool spj(const Data &a, const Ans &x, const Ans &y) {
    return false;
}

// 对拍失败时输出数据、两个答案等调试信息
void print(const Data &a, const Ans &x, const Ans &y) {
}

int main(int argc, char **argv) {
    u64 seed = chrono::steady_clock::now().time_since_epoch().count();
    if (argc >= 2) seed = stoull(argv[1]);
    rng.seed(seed);
    for (u64 id = 1; ; ++ id) {
        Data a = gen(id);
        Ans x = fast(a);
        Ans y = brute(a);
        // ===== 判定方式：二选一 =====
        bool ok = (x == y);          // 严格相等
        // bool ok = spj(a, x, y);   // SPJ

        if (!ok) {
            cout << "\nWA\n";
            cout << "seed = " << seed << "\ncase = " << id << '\n';
            print(a, x, y);
            break;
        }

        if (id % 100000 == 0) {
            cout << "Passed " << id << " cases\r" << flush;
        }
    }
```

```
    return 0;
}
```

类型与函数

- `Data`：单组随机数据，按题目填写成员
- `Ans`：答案类型；默认提供逐成员严格相等判定。若答案只是 `i64`、`vector<int>` 等，可以直接改为对应类型
- `randint(l, r)`：生成闭区间 `[l,r]` 内的均匀随机整数
- `gen(id)`：生成第 `id` 组数据；可以利用 `id` 定期构造特殊数据
- `fast(a)`：待验证的正解
- `brute(a)`：暴力 / 标准算法
- `spj(a, x, y)`：特殊判定，返回 `true` 表示正解答案 `x` 可以接受
- `print(a, x, y)`：对拍失败时输出测试数据以及必要的调试信息

判定方式

默认使用严格相等：

```
bool ok = (x == y);
// bool ok = spj(a, x, y);
```

需要 SPJ 时改为：

```
// bool ok = (x == y);
bool ok = spj(a, x, y);
```

注意事项

- `fast()` 与 `brute()` 的参数故意按值传递，因此二者可以独立修改数据，不会互相影响。若 `Data` 很大且算法完全不修改输入，可以改成 `const Data &a`
- `spj()` 默认返回 `false`，避免切换到 SPJ 后忘记实现判定却一直通过
- `Ans` 使用默认 `operator==`，因此结构体成员都支持 `==` 时，可以直接进行严格比较；如果答案类型特殊，可以自己重载 `operator==`
- 对拍失败必须保留打印出的 `seed`。使用相同种子重新运行，可以复现完全相同的随机序列
- `id` 不只是计数器，也可以用于周期性制造边界数据，避免纯随机数据长期覆盖不到特殊情况
- `print()` 最好输出成题目的正常输入格式，再额外输出 `fast`、`brute` 的答案，方便直接复制失败样例调试

常见用法

复现失败数据

假设失败时输出：

```
seed = 123456789
case = 81724
```

重新运行：

```
./a 123456789
```

会重新生成相同的随机序列，并在相同位置复现问题

周期性生成特殊数据

```
Data gen(u64 id) {
    Data a;
    if (id % 10 == 0) {
        // 极小数据
    } else if (id % 10 == 1) {
        // 极大 / 极端数据
    } else {
        // 普通随机数据
    }
    return a;
}
```

浮点答案 SPJ

```
bool spj(const Data &a, const Ans &x, const Ans &y) {
    constexpr long double eps = 1E-12L;
```

```
    return abs(x.val - y.val) ≤ eps * max({1.0L, abs(x.val), abs(y.val)});
}
```

多解构造题 SPJ

`y` 可以提供暴力求出的最优值，`spj()` 负责检查 `x`：

```
bool spj(const Data &a, const Ans &x, const Ans &y) {
    if (!valid(a, x)) return false;
    return x.value == y.value;
}
```

即不要求两份构造完全相同，只要求正解构造合法且目标值正确

1.2 Debug

调试输出

```
using i128 = __int128_t;
using u128 = __uint128_t;

#ifdef LOCAL
struct Debug {
    // ===== 运行上下文 =====
    static inline int tc{ -1 };
    static inline uint64_t seed{};
    static inline bool has_seed{};
    static void set_seed(uint64_t x) { seed = x; has_seed = true; }
private:
    template<class T, class = void> struct IsRange : false_type {};
    template<class T> struct IsRange<T, void_t<
        decltype(begin(declval<const T &>())),
        decltype(end(declval<const T &>()))>> : true_type {};
    template<class T> struct IsPair : false_type {};
    template<class A, class B> struct IsPair<pair<A, B>> : true_type {};
    template<class T> struct IsTuple : false_type {};
    template<class... T> struct IsTuple<tuple<T...>> : true_type {};
    template<class T> static constexpr bool is_cstr =
        is_same_v<decay_t<T>, const char *> || is_same_v<decay_t<T>, char *>;
    static void print128(u128 x) {
        char s[40];
        int n{};
        do { s[n ++] = char('0' + x % 10); x /= 10; } while (x);
        while (n) cerr << s[-- n];
    }
    template<class... T> static void list(const T &...x) {
        bool fst = true;
        ((cerr << (exchange(fst, false) ? "" : ", "), show(x)), ...);
    }
    static string trim(const string &s) {
        size_t l{}, r = s.size();
        while (l < r && isspace((unsigned char)s[l])) ++ l;
        while (r > l && isspace((unsigned char)s[r - 1])) -- r;
        return s.substr(l, r - l);
    }
    static bool quoted(const string &s) {
        return s.size() ≥ 2 &&
            ((s.front() == '"' && s.back() == '"') ||
             (s.front() == '\'' && s.back() == '\''));
    }
    static vector<string> split(const string &s) {
        vector<string> a;
        string cur;
        int p{}, b{}, k{};
        char q{};
        bool esc{};
        for (char c : s) {
```

```
            if (q) {
                cur += c;
                if (esc) esc = false;
                else if (c == '\\') esc = true;
                else if (c == q) q = 0;
            } else if (c == '\'' || c == '"') q = c, cur += c;
            else if (c == '(') ++ p, cur += c;
            else if (c == ')') -- p, cur += c;
            else if (c == '{') ++ b, cur += c;
            else if (c == '}') -- b, cur += c;
            else if (c == '[') ++ k, cur += c;
            else if (c == ']') -- k, cur += c;
            else if (c == ',' && !p && !b && !k) {
                a.push_back(trim(cur));
                cur.clear();
            } else cur += c;
        }
        if (!cur.empty() || a.empty()) a.push_back(trim(cur));
        return a;
    }
    static void named_impl(const vector<string> &, size_t) { cerr << '\n'; }
    template<class T, class... U>
    static void named_impl(const vector<string> &a, size_t i, const T &x, const U &...rest) {
        if (i >= a.size()) {
            show(x);
            ((cerr << ' ', show(rest)), ...);
            cerr << '\n';
            return;
        }
        if constexpr (is_cstr<T>) {
            if (quoted(a[i])) {
                show(x);
                cerr << ": ";
                named_impl(a, i + 1, rest...);
                return;
            }
        }
        cerr << a[i] << " = ";
        show(x);
        if constexpr (sizeof...(rest) == 0) cerr << '\n';
        else {
            cerr << ' ';
            named_impl(a, i + 1, rest...);
        }
    }
    // ===== 位置与运行上下文：不需要 tc/seed 时删除对应两行 =====
    static void head(const char *tag, const char *file, int line, const char *fn) {
        cerr << '[' << tag << ' ' << file << ':' << line << ' ' << fn << ']';
        if (tc >= 0) cerr << " tc=" << tc;
        if (has_seed) cerr << " seed=" << seed;
        cerr << ' ';
    }
public:
    // ===== 基础输出 =====
    template<class T> static void show(const T &x) {
        using U = decay_t<T>;
        if constexpr (is_same_v<U, bool>) cerr << (x ? "true" : "false");
        else if constexpr (is_same_v<U, char> || is_same_v<U, string> ||
                           is_same_v<U, string_view>) cerr << x;
        else if constexpr (is_cstr<T>) {
            if constexpr (is_array_v<remove_reference_t<T>>) cerr << x;
            else cerr << (x ? x : "null");
        } else if constexpr (is_same_v<U, i128>) {
            if (x < 0) cerr << '-', print128(u128(-(x + 1)) + 1);
            else print128(u128(x));
        } else if constexpr (is_same_v<U, u128>) print128(x);
        else if constexpr (IsPair<U>::value) {
            cerr << '('; show(x.first); cerr << ", "; show(x.second); cerr << ')';
        } else if constexpr (IsTuple<U>::value) {
            cerr << '(';
            apply([](const auto &...v) { list(v...); }, x);
```

```
            cerr << ')';
        } else if constexpr (IsRange<T>::value) {
            cerr << '[';
            bool fst = true;
            for (const auto &v : x) {
                if (!fst) cerr << ", ";
                fst = false;
                show(v);
            }
            cerr << ']';
        } else cerr << x;
    }
    template<class... T>
    static void values(const char *file, int line, const char *fn, const T &...x) {
        head("DBG", file, line, fn);
        bool fst = true;
        ((cerr << (exchange(fst, false) ? "" : " "), show(x)), ...);
        cerr << '\n';
    }
    template<class... T>
    static void named(const char *file, int line, const char *fn, const string &s, const T &...x) {
        head("DBG", file, line, fn);
        named_impl(split(s), 0, x...);
    }
    // ===== 失败检查：不需要时可连同 dcheck / dcheckv 宏删除 =====
    [[noreturn]] static void fail(const char *e, const char *file, int line, const char *fn) {
        head("CHECK", file, line, fn);
        cerr << "failed: " << e << '\n';
        cerr.flush();
        abort();
    }
    template<class... T>
    [[noreturn]] static void fail(const char *e, const char *file, int line,
                                  const char *fn, const string &s, const T &...x) {
        head("CHECK", file, line, fn);
        cerr << "failed: " << e << '\n';
        head("VARS", file, line, fn);
        named_impl(split(s), 0, x...);
        cerr.flush();
        abort();
    }
};
#define dbg(...) (Debug::values(__FILE__, __LINE__, __func__, __VA_ARGS__))
#define dbgv(...) (Debug::named(__FILE__, __LINE__, __func__, #__VA_ARGS__, __VA_ARGS__))
#define dcheck(e) do { if (!(e)) Debug::fail(#e, __FILE__, __LINE__, __func__); } while (false)
#define dcheckv(e, ...) do { if (!(e)) Debug::fail(#e, __FILE__, __LINE__, __func__, #__VA_ARGS__, __VA_ARGS__); } while
(false)
#else
#define dbg(...) ((void)0)
#define dbgv(...) ((void)0)
#define dcheck(e) ((void)0)
#define dcheckv(e, ...) ((void)0)
#endif
```

- `dbg(x...)`：输出若干变量的值，自动附带文件、行号和函数名
- `dbgv(x...)`：输出变量名与值，保留原来的字符串标签用法
- `Debug::tc`：当前测试编号，默认 `-1` 表示不显示
- `Debug::set_seed(x)`：设置随机种子，之后的调试信息自动附带 `seed`
- `dcheck(cond)`：条件不成立时打印位置与表达式，随后调用 `abort()`
- `dcheckv(cond, x...)`：与 `dcheck` 相同，额外输出指定变量的名称和值

自动输出代码位置

```
int n = 10;
vector<int> a{1, 2, 3};

dbg(n, a);
dbgv("Before DFS", n, a);
```

输出示意：

```
[DBG main.cpp:42 solve] 10 [1, 2, 3]
[DBG main.cpp:43 solve] Before DFS: n = 10 a = [1, 2, 3]
```

检查可能导致 RE 的条件

```
dcheck(0 ≤ p && p < int(a.size()));
dcheckv(0 ≤ p && p < int(a.size()), p, a.size(), a);
```

第二个检查失败时，会输出类似：

```
[CHECK main.cpp:51 solve] failed: 0 ≤ p && p < int(a.size())
[VARS main.cpp:51 solve] p = 7 a.size() = 3 a = [1, 2, 3]
```

适合在数组下标、除数、递归入口、图节点编号等高风险位置使用

注意事项

- `LOCAL` 未定义时，`dbg`、`dbgv`、`dcheck`、`dcheckv` 均不会求值参数，因此不要在这些宏里执行必需的修改操作
- `dcheck` 和 `dcheckv` 是调试断言，不应代替正式程序中的合法性判断
- `abort()`、段错误等异常终止通常不会执行 `RuntimeInfo` 和 `ScopeTimer` 的析构函数，因此不保证输出最终时间
- `dbgv` 的变量名拆分支持常见括号、数组及初始化列表，但直接含有模板参数逗号的复杂表达式，必要时用括号包裹
- `Debug::tc` 和 `seed` 只记录主动设置的上下文，无法自动知道任意崩溃时全部局部变量

运行时间

```
#ifdef LOCAL
struct RuntimeInfo {
    using Clock = chrono::steady_clock;
    Clock::time_point st{ Clock::now() };
    i64 cnt{}, id{};
    double sum{}, mx{};
    static double ms(Clock::time_point t) {
        return chrono::duration<double, milli>(Clock::now() - t).count();
    }
    // ===== 多测计时：不需要时可删除 run() 与 cnt/id/sum/mx =====
    template<class F> void run(F &&f) {
        auto t = Clock::now();
        forward<F>(f)();
        double d = ms(t);
        if (!cnt || d > mx) mx = d, id = cnt + 1;
        sum += d;
        ++ cnt;
    }
    ~RuntimeInfo() {
        auto flags = cerr.flags();
        auto prec = cerr.precision();
        cerr << fixed << setprecision(3) << "[Time] total=" << ms(st) << " ms";
        if (cnt) {
            cerr << " solve=" << sum << " ms avg=" << sum / cnt
                 << " ms max=" << mx << " ms (#" << id << ")";
        }
        cerr << '\n';
        cerr.flags(flags);
        cerr.precision(prec);
    }
};
inline RuntimeInfo runtime_info;

// ===== 局部计时 =====
struct ScopeTimer {
    const char *s;
    chrono::steady_clock::time_point st{ chrono::steady_clock::now() };
    explicit ScopeTimer(const char *s_ = "scope") : s(s_) {}
    ~ScopeTimer() {
        auto flags = cerr.flags();
        auto prec = cerr.precision();
        cerr << fixed << setprecision(3) << "[Time] " << s << '='
```

```
            << chrono::duration<double, milli>(chrono::steady_clock::now() - st).count() << " ms\n";
        cerr.flags(flags);
        cerr.precision(prec);
    }
};
#endif
```

如果删除 `run()` 和相关统计成员，还需同步删除析构函数里的 `if (cnt)` 统计输出段；如果完全不需要 `RuntimeInfo`，则可连同 `ScopeTimer` 一起删除

- `runtime_info`：全局计时对象，正常结束时自动输出总耗时
- `runtime_info.run(f)`：执行函数 `f`，记录该次耗时
- `ScopeTimer(s)`：创建局部计时对象，离开作用域时输出耗时
- `cnt`：已计时次数
- `sum`：所有 `run()` 的累计耗时
- `mx`：最慢一次 `run()` 的耗时
- `id`：最慢一次的编号，从 `1` 开始

多测计时与 RE 上下文

```
int t{ 1 };
// cin >> t;
for (int i = 1; i <= t; ++ i) {
#ifdef LOCAL
    runtime_info.run(solve);
#else
    solve();
#endif
}
```

`Start case` 在每组开始时立即输出，即使之后发生段错误，也能通过最后一条记录判断程序执行到了哪组测试
如果已经确认程序正确、不需要调试输出，可以仅保留计时

```
#ifdef LOCAL
runtime_info.run(solve);
#else
solve();
#endif
```

局部计时

```
#ifdef LOCAL
{
    ScopeTimer tm{"build"};
    build();
}
#else
build();
#endif
```

总计时与局部计时相互独立，可以同时保留

RE 定位与现代 C++ 扩展

C++20 `std::source_location`

如果只需要一个能直接打印调用位置的独立函数，可以使用 C++20 标准库，无需宏

```
#if defined(LOCAL) && __cplusplus >= 202002L
#include <source_location>
void here(const std::source_location &loc = std::source_location::current()) {
    cerr << "[HERE] " << loc.file_name() << ':' << loc.line() << ' ' << loc.function_name() << '\n';
}
#endif
```

```
#ifdef LOCAL
here();
```

```
#endif
```

该函数的默认参数会获取调用位置，而非函数定义位置

C++23 `std::stacktrace`

支持该特性的标准库可以在程序仍能正常执行到调试函数时获取调用栈

```
// 放在头文件区域
#if __has_include(<stacktrace>)
#include <stacktrace>

#ifdef __cpp_lib_stacktrace
cerr << std::stacktrace::current() << '\n';
#endif
```

可以放进 `Debug::fail()`，在 `abort()` 前打印调用栈
部分编译器或标准库尚不完整支持 `std::stacktrace`，某些版本还需要额外链接库，因此不建议纳入默认 Debug 核心

Linux 峰值内存统计

如果还希望查看本地测试的内存峰值：

```
#if defined(LOCAL) && defined(__linux__)
#include <sys/resource.h>
void peak_mem() {
    rusage ru{};
    if (getrusage(RUSAGE_SELF, &ru) == 0) {
        cerr << "[Mem] peak=" << ru.ru_maxrss / 1024.0 << " MiB\n";
    }
}
#endif

#if defined(LOCAL) && defined(__linux__)
peak_mem();
#endif
```

可以放到 `RuntimeInfo` 析构函数中，Linux 的 `ru_maxrss` 单位为 KiB，输出换算为 MiB

注意事项

- `run()` 计时包含 `solve()` 内的输入输出，但不包含函数外的全局预处理
- 对真正的 RE，优先使用 Sanitizer；需要进一步查看递归调用栈和函数参数时使用 GDB

Debug 编译、运行与调试命令手册

环境与命令约定

不同环境的区别

环境	编译器	程序运行方式	推荐用途
Linux（Bash）	GCC / Clang	`./main`	完整调试、Sanitizer、性能及内存分析
Windows CMD	MinGW-w64 GCC	`main.exe`	Windows 原生编译、调试
Windows PowerShell	MinGW-w64 GCC	`.\main.exe`	Windows 原生编译、调试
Windows Git Bash	MinGW-w64 GCC	`./main.exe`	使用 Bash 风格命令运行 Windows 程序
Windows WSL	Linux GCC / Clang	`./main`	复现 Linux 评测环境

> [!] 编译参数由编译器决定，运行命令由终端决定，系统功能由操作系统决定
>
> 以下默认使用 GCC。Windows 需要安装 MinGW-w64 / MSYS2 等 GCC 工具链，不能直接把这些参数用于 MSVC 的 `cl.exe`
>
> Git Bash 不等于 Linux，WSL 内使用 Linux GCC 编译产生的是 Linux 程序，而不是 Windows 原生 `.exe`

检查编译环境

Linux / WSL / Git Bash
复制

```
g++ --version
gdb --version
command -v g++
command -v gdb
```

Windows CMD

```
g++ --version
gdb --version
where g++
where gdb
```

Windows PowerShell

```
g++ --version
gdb --version
Get-Command g++
Get-Command gdb
```

若 `g++` 或 `gdb` 无法识别，需安装相应工具并设置 `PATH`

常用命令速查

Linux / WSL

```
# 普通编译
g++ main.cpp -std=gnu++20 -O2 -o main

# 编译警告
g++ main.cpp -std=gnu++20 -O2 -Wall -Wextra -Wshadow -o main

# 调试版本
g++ main.cpp -std=gnu++20 -Og -g3 -DLOCAL -o main

# Sanitizer 检查
g++ main.cpp -std=gnu++20 -O1 -g -DLOCAL -fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all -o
main

# STL 检查
g++ main.cpp -std=gnu++20 -O0 -g -D_GLIBCXX_DEBUG -DLOCAL -o main

# 运行并重定向
./main < in.txt > out.txt 2> err.txt

# 对比输出
diff -u ans.txt out.txt

# 查看时间、峰值内存
/usr/bin/time -v ./main < in.txt > out.txt

# GDB 定位崩溃
gdb -q -batch -ex 'run < in.txt' -ex 'bt full' -ex 'info args' -ex 'info locals' --args ./main
```

Windows CMD

```
REM 普通编译
g++ main.cpp -std=gnu++20 -O2 -o main.exe

REM 编译警告
g++ main.cpp -std=gnu++20 -O2 -Wall -Wextra -Wshadow -o main.exe

REM 调试版本
g++ main.cpp -std=gnu++20 -Og -g3 -DLOCAL -o main.exe

REM STL 检查 (MinGW-w64 GCC + libstdc++)
g++ main.cpp -std=gnu++20 -O0 -g -D_GLIBCXX_DEBUG -DLOCAL -o main.exe
```

```
REM 运行并重定向
main.exe < in.txt > out.txt 2> err.txt

REM 比对输出
fc.exe /N ans.txt out.txt

REM GDB 定位崩溃（需 Windows 版 GDB）
gdb -q -batch -ex "run < in.txt" -ex "bt full" -ex "info args" -ex "info locals" --args main.exe
```

Windows PowerShell

编译命令与 CMD 基本相同，运行时采用 `.\main.exe`

```
# 普通编译
g++ main.cpp -std=gnu++20 -O2 -o main.exe

# 直接运行
.\main.exe

# 使用 CMD 完成输入输出重定向
cmd /c ".\main.exe < in.txt > out.txt 2> err.txt"

# 对比输出
fc.exe /N ans.txt out.txt

# GDB
gdb -q -batch -ex "run < in.txt" -ex "bt full" -ex "info args" -ex "info locals" --args .\main.exe
```

PowerShell 不提供 Bash/CMD 风格的 `< in.txt` 输入重定向，且较旧的 PowerShell 版本对原生程序输出重定向可能进行文本处理，因此竞赛程序的精确 I/O 测试优先通过 `cmd /c` 执行

> ⚠ Windows 原生 GCC 的 Sanitizer 支持取决于工具链及运行库版本，不能保证 `-fsanitize=address,undefined` 在每个 MinGW 环境下都可用
>
> 如果需要稳定使用完整的 ASan + UBSan，推荐 Linux / WSL；Clang 在 Windows 上也有 ASan 支持，但仍需相应的运行库

GCC 编译参数详解

基础参数

通用命令格式：

```
g++ main.cpp -std=gnu++20 -O2 -DLOCAL -g -o main
```

参数	功能	限制与说明
`main.cpp`	待编译的源文件	多文件可填写多个文件名
`-o main`	指定输出文件	Windows 建议写 `main.exe`
`-std=gnu++17`	GNU C++17	支持 `__int128` 等 GNU 扩展
`-std=gnu++20`	GNU C++20	支持 `ranges`、`source_location` 等
`-std=gnu++23`	GNU C++23	新功能还取决于标准库版本
`-std=c++20`	标准 C++20 模式	GNU 扩展可能仍可用，但在严格检查下可能产生诊断
`-DLOCAL`	定义 `LOCAL` 宏	启用你现有的 Debug 代码
`-DONLINE_JUDGE`	定义 `ONLINE_JUDGE` 宏	仅适用于使用该宏判断环境的代码
`-DNDEBUG`	禁用标准 `assert()`	不影响自定义 `dcheck()`
`-g`	生成调试符号	主要影响文件体积，不是运行时检查
`-g3`	生成更多调试信息	包含宏定义等额外信息
`-ggdb3`	针对 GDB 生成详细调试信息	使用 GDB 时可选

`-g3` 与 `-ggdb3` 选择其一即可，后者更偏向 GDB 的调试需求

优化等级

参数	作用	推荐场景
`-O0`	关闭大部分优化	GDB 单步调试
`-Og`	兼顾基本优化与可调试性	日常 Debug
`-O1`	基础优化	Sanitizer
`-O2`	常规优化	正式性能测试
`-O3`	更激进的优化	卡常对比测试
`-Ofast`	允许部分非严格标准优化	确认语义安全后尝试

这些优化等级通常只选择一个，若重复指定，以最后生效的优化等级为准
推荐搭配：

```
# GDB
-O0 -g3

# 日常调试
-Og -g3

# Sanitizer
-O1 -g

# 性能测试
-O2
```

`-Ofast` 可能改变浮点数等运算的语义，不应作为 Debug 选项

编译警告

参数	检查内容	注意事项
`-Wall`	常见潜在错误	并非开启所有警告
`-Wextra`	更多额外警告	可能产生未使用参数等提示
`-Wshadow`	变量名遮蔽	竞赛中的局部重名可能产生较多警告
`-Wshadow=compatible-local`	较有针对性的局部变量遮蔽检查	可替代更严格的 `-Wshadow`
`-Wconversion`	可能改变数值的隐式类型转换	对 `size_t` 到 `int` 等场景较敏感
`-Wsign-conversion`	有符号与无符号类型转换	提示数量可能较多
`-Wfloat-equal`	浮点数直接相等比较	部分精确比较是有意设计的
`-Werror`	将警告视为编译错误	调试时可用，日常不必默认开启
`-fmax-errors=5`	最多显示五个编译错误	不限制普通警告的总数

`-Wshadow` 和 `-Wconversion` 的检查范围可以按 GCC 文档进一步缩小，适合逐项排查警告
推荐组合：

```
-Wall -Wextra -Wshadow
```

需要严格检查转换时：

```
-Wall -Wextra -Wshadow -Wconversion -Wsign-conversion
```

运行时错误检查

Sanitizer 参数

参数	作用	可能的限制
`-fsanitize=address`	检查部分内存越界、释放后访问等	增加内存和运行开销，需要运行库支持
`-fsanitize=undefined`	检查多种未定义行为	不覆盖所有逻辑错误
`-fsanitize=address,undefined`	同时启用 ASan 与 UBSan	Linux 优先使用
`-fno-sanitize-recover=all`	检测到支持的错误后停止	适合快速定位首次错误
`-fno-omit-frame-pointer`	保留栈帧指针	改善部分调用栈信息
`-fno-optimize-sibling-calls`	避免部分尾调用优化	使部分调用栈更完整
`-g`	输出调试符号	便于获得源码位置

GCC 官方建议使用较低优化等级及帧指针辅助获取可读的 Sanitizer 调用栈
Linux / WSL 完整写法：

```
g++ main.cpp -std=gnu++20 -O1 -g -DLOCAL -Wall -Wextra -fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all -o main
./main < in.txt > out.txt 2> err.txt
```

也可以只检查特定错误，例如仅检查未定义行为：

```
g++ main.cpp -std=gnu++20 -O1 -g -fsanitize=undefined -o main
```

Windows 原生工具链应先确认相应 Sanitizer 是否提供运行库，Clang 的 Windows ASan 支持也存在平台和功能上的限制

STL 检查参数

参数	作用	限制
`-D_GLIBCXX_ASSERTIONS`	启用 libstdc++ 的额外前置条件断言	检查范围有限
`-D_GLIBCXX_DEBUG`	开启更完整的容器、迭代器调试	可能改变部分容器布局、增加耗时和内存

轻量检查：

```
g++ main.cpp -std=gnu++20 -O2 -D_GLIBCXX_ASSERTIONS -DLOCAL -o main
```

完整 STL 调试：

```
g++ main.cpp -std=gnu++20 -O0 -g -D_GLIBCXX_DEBUG -DLOCAL -o main
```

以上同样适用于使用 GNU libstdc++ 的 Windows MinGW-w64 GCC
`_GLIBCXX_DEBUG` 会改变部分标准库模板的布局，混合编译多个文件或链接预编译库时需要注意 ABI 兼容性

> **建议分别构建 Sanitizer 版与 STL Debug 版，而不是默认把两者全叠加**
>
> 两者可以在受支持的配置中组合，但性能和内存开销更高，出现问题时也不容易判断是哪一类检查导致的

GDB 调试命令

启动参数

参数	功能
`-q`	不显示 GDB 启动信息
`-batch`	批处理模式，执行完命令自动退出
`-ex "命令"`	在启动时执行一条 GDB 命令，可重复
`-x file`	从文件读取 GDB 命令
`--args`	指定被调试程序及其命令行参数，后续内容交给程序
`-return-child-result`	在适用情况下传回被调试程序的退出码

这些启动参数可以按需要组合

GDB 内部命令

命令	功能
`run` / `r`	启动程序
`run < in.txt`	使用文件作为程序输入
`bt`	查看调用栈
`bt full`	查看调用栈及局部变量
`frame 0`	选择第 0 层栈帧，即当前出错位置
`up` / `down`	在调用栈中移动
`info args`	查看当前函数参数
`info locals`	查看当前函数局部变量
`print x` / `p x`	输出变量或表达式的值

命令	功能
`p a.size()`	求出当前可访问对象的大小
`break solve` / `b solve`	在 `solve()` 设置断点
`break main.cpp:42`	在指定源码行设置断点
`next` / `n`	单步执行，不进入函数
`step` / `s`	单步执行，进入函数
`continue` / `c`	继续运行
`list` / `l`	查看附近源码
`quit` / `q`	退出 GDB

`bt full` 通常比单独执行 `info locals` 更适合定位深层递归中的错误，因为它可以查看多层调用栈

交互式调试

Linux / WSL：

```
g++ main.cpp -std=gnu++20 -O0 -g3 -DLOCAL -o main
gdb ./main
```

进入 GDB 后：

```
break solve
run < in.txt
bt full
info args
info locals
print n
quit
```

Windows CMD：

```
g++ main.cpp -std=gnu++20 -O0 -g3 -DLOCAL -o main.exe
gdb main.exe
```

进入 GDB 后使用相同的 GDB 内部命令
原生 Windows GDB 的运行与输入重定向能力可能受具体发行版及调试目标影响，应先用一份简单输入检查 `run < in.txt` 是否正常工作

运行、重定向与结果检查

重定向符号

符号	功能
`< in.txt`	从文件读取标准输入
`> out.txt`	将标准输出写入文件，覆盖原内容
`>> out.txt`	将标准输出追加到文件末尾
`2> err.txt`	将标准错误输出写入文件
`2>> err.txt`	追加标准错误输出
`2>&1`	将标准错误输出合并到当前标准输出目标
`|`	将左侧程序输出连接到右侧程序输入

注意 `2>&1` 与其他重定向符号的先后顺序可能影响结果
Linux、CMD 和 Git Bash 支持这些常见重定向方式，PowerShell 对输出流的处理规则有所不同，尤其需要注意输入重定向与编码

对比答案

Linux / WSL：

```
./main < in.txt > out.txt
diff -u ans.txt out.txt
```

Windows CMD：

```
main.exe < in.txt > out.txt
fc.exe /N ans.txt out.txt
```

Windows PowerShell:

```
cmd /c ".\main.exe < in.txt > out.txt"
fc.exe /N ans.txt out.txt
```

常用比对参数：

参数	作用	适用环境
`diff -u`	统一格式显示差异	Linux / Bash
`diff -w`	忽略空白数量差异	Linux / Bash
`fc.exe /N`	显示不同文本行及行号	Windows
`fc.exe /B`	逐字节比较文件	Windows
`fc.exe /W`	压缩空白差异	Windows

`fc.exe` 返回 `0` 表示相同、`1` 表示不同、`2` 表示发生比较错误
以上均为普通文本或字节比较，并不能替代浮点误差判定、答案不唯一等题目的 SPJ

获取程序退出码

```
# Linux / Bash：紧跟程序运行之后
echo $?
```

```
REM Windows CMD
echo %ERRORLEVEL%
```

```
# Windows PowerShell
$LASTEXITCODE
```

退出码非零通常表示程序异常结束或自行报告失败，但具体数值会受到程序、操作系统和工具链影响

时间、内存与运行限制

Linux / WSL

```
# 简单计时
time ./main < in.txt > out.txt

# GNU time：详细资源统计
/usr/bin/time -v ./main < in.txt > out.txt

# 把时间与内存报告单独保存
/usr/bin/time -v -o perf.txt ./main < in.txt > out.txt 2> err.txt

# 限制运行时间为 2 秒
timeout 2s ./main < in.txt

# 查看当前栈限制
ulimit -s

# 将当前 Shell 的栈软限制尝试设为 64 MiB
ulimit -s 65536
```

参数或命令	作用	限制
`time`	查看整体执行耗时	不一定提供内存信息
`/usr/bin/time -v`	查看时间、CPU 及峰值内存等	需要 GNU time
`-o perf.txt`	将 GNU time 报告写入文件	不影响程序的标准输出
`timeout 2s`	最多运行约 2 秒	需要 GNU coreutils
`ulimit -s`	查询或设置栈空间限制	仅影响当前 Shell 及后续子进程
`ulimit -c unlimited`	允许尽可能大的 core dump	仍受系统 core dump 配置限制

GNU time 的最大常驻内存指标通常以 KiB 表示，而 `timeout` 超时后默认返回码为 `124`，特殊选项或信号情况除外
`ulimit -s` 不能改变评测机的栈限制，并且修改栈限制可能影响 ASan 的运行环境，不能将其作为通用修复手段
Windows PowerShell

```
# 运行并测量总耗时
(Measure-Command {
    cmd /c ".\main.exe < in.txt > out.txt 2> err.txt"
}).TotalMilliseconds
```

`Measure-Command` 返回一个 `TimeSpan` 对象，可以读取 `TotalMilliseconds`、`TotalSeconds` 等属性
它测量的是整个命令块执行时间，包括启动 `cmd.exe` 和程序的开销，因此只适合比较较长的运行过程，不适合微小函数的精细测速
Windows CMD 的 `timeout /t 2` 只是暂停命令处理器 2 秒，并不是限制目标程序执行 2 秒，与 Linux 的 `timeout 2s ./main` 不同
Windows 原生环境没有与 Linux `/usr/bin/time -v` 完全等价的内置命令。峰值内存可以考虑任务管理器、性能分析工具或专门的 Windows API 实现；想使用本章 Linux 工具时，优先通过 WSL

参数组合规则

推荐组合

使用场景	推荐组合
正式编译	`-std=gnu++20 -O2`
基础警告	`-O2 -Wall -Wextra -Wshadow`
日常 Debug	`-Og -g3 -DLOCAL -Wall -Wextra -Wshadow`
GDB 定位 RE	`-O0 -g3 -DLOCAL -fno-omit-frame-pointer`
Sanitizer	`-O1 -g -DLOCAL -fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all`
STL 轻量检查	`-O2 -D_GLIBCXX_ASSERTIONS`
STL 完整检查	`-O0 -g -D_GLIBCXX_DEBUG`
性能对比	分别使用 `-O2`、`-O3` 编译并测试

不推荐的组合

组合	原因
`-O0 -O3`	同类优化等级重复，按最后生效的设置处理
`-Ofast` + 浮点正确性调试	可能改变浮点语义
`-DNDEBUG` + 依赖 `assert()` 的调试	会禁用断言
`-D_GLIBCXX_DEBUG` + 正式性能测试	改变容器行为与开销
`-fsanitize=address` + `-static`	ASan 静态链接存在支持限制
`-march=native` + 未知评测机 CPU	可能生成目标 CPU 不支持的指令
Sanitizer + 严格时空限制模拟	检查工具会增加运行时间及内存

按需拼接示例

例如想在 Linux 上定位 RE，同时开启警告和变量检查：

```
g++ main.cpp -std=gnu++20 -O1 -g3 -DLOCAL -Wall -Wextra -Wshadow -fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all -o main
./main < in.txt > out.txt 2> err.txt
```

想在 Windows 上检查 STL 越界，并开启你的 `dbg()` / `dbgv()`：

```
g++ main.cpp -std=gnu++20 -O0 -g3 -DLOCAL -D_GLIBCXX_DEBUG -Wall -Wextra -Wshadow -o main.exe
main.exe < in.txt > out.txt 2> err.txt
```

想在 Linux 上统计真实运行耗时和峰值内存：

```
g++ main.cpp -std=gnu++20 -O2 -o main
/usr/bin/time -v -o perf.txt ./main < in.txt > out.txt
```

这时不要使用 Sanitizer 或 STL Debug，因为它们会改变程序的时间与内存表现

注意事项

- `-DLOCAL` 对应你的 `#ifdef LOCAL`，如果忘记添加，调试输出和计时功能就不会启用
- `-g` 本身不会检查越界，`-O0` 本身也不会检查越界
- Sanitizer、STL Debug、GDB 各自解决不同的问题，不能相互完全替代
- Windows 上的 `g++` 不一定是同一个发行版，支持的参数和运行库可能不同
- 尽量保留和评测机相同的 C++ 标准、编译器及标准库版本

- 对于跨平台竞赛项目，最稳妥的是分别保留 Windows 原生调试命令与 Linux / WSL 调试命令，而不是强行合并成一条
  最终，这份手册可以遵循一个简单的拼接顺序：

```
编译器 + 源文件 + C++ 标准 + 优化等级
       + 宏定义 + 警告选项 + 调试信息
       + 检查工具 + 输出文件
```

1.3 IO

FastIO

使用 `fread` / `fwrite` 缓冲输入输出，支持普通整数、`i128`、`u128`、字符、字符串及常见容器

- `read()` 支持连续读取多个参数、数组和已分配空间的容器
- `write()` 自动在不同元素之间添加空格
- `writeln()` 自动换行，`writelines()` 将每个参数分别输出一行
- `get()`、`line()` 分别支持原始字符和整行读取
- 不支持浮点数读写，相关方法放在后面的标准库速查中

```
using i128 = __int128_t;
using u128 = __uint128_t;

namespace FastIO {
struct Input {
private:
    static constexpr size_t S{ 1 << 20 };
    char buf[S];
    size_t p{}, n{};
    static bool digit(int c) { return '0' ⩽ c && c ⩽ '9'; }
    bool skip(int &c) {
        c = get();
        while (c ≠ EOF && c ⩽ ' ') c = get();
        return c ≠ EOF;
    }
    // ===== 整数读取核心 =====
    template<class U>
    bool read_abs(U &v, bool &neg) {
        int c;
        if (!skip(c)) return false;
        neg = false;
        if (c == '-' || c == '+') {
            neg = c == '-';
            c = get();
        }
        if (!digit(c)) return false;
        v = 0;
        bool ok = true;
        constexpr U lim = ~U(0);
        while (digit(c)) {
            U d = U(c - '0');
            if (v > lim / 10 || (v == lim / 10 && d > lim % 10)) ok = false;
            else if (ok) v = v * 10 + d;
            c = get();
        }
        return ok;
    }
public:
    // ===== 原始字符读取 =====
    int get() {
        if (p == n) {
            n = fread(buf, 1, S, stdin);
            p = 0;
            if (!n) return EOF;
        }
        return (unsigned char)buf[p ++];
    }
```

```
    // ===== 标准整数 =====
    template<class T,
        enable_if_t<is_integral_v<T> && !is_same_v<T, bool>
            && !is_same_v<T, i128> && !is_same_v<T, u128>, int> = 0>
    bool read(T &x) {
        using U = make_unsigned_t<T>;
        U v;
        bool neg;
        if (!read_abs(v, neg)) return false;
        if constexpr (is_signed_v<T>) {
            U lim = U(numeric_limits<T>::max()) + U(neg);
            if (v > lim) return false;
            x = neg ? (v ? -T(v - 1) - 1 : T{}) : T(v);
        } else {
            if (neg) return false;
            x = T(v);
        }
        return true;
    }
    // ===== 128 位整数 =====
    bool read(i128 &x) {
        u128 v;
        bool neg;
        if (!read_abs(v, neg)) return false;
        if (v > (u128(1) << 127) - 1 + u128(neg)) return false;
        x = neg ? (v ? -i128(v - 1) - 1 : i128{}) : i128(v);
        return true;
    }
    bool read(u128 &x) {
        u128 v;
        bool neg;
        if (!read_abs(v, neg) || neg) return false;
        x = v;
        return true;
    }
    // ===== 字符与字符串 =====
    bool read(char &c) {
        int x;
        if (!skip(x)) return false;
        c = char(x);
        return true;
    }
    bool read(string &s) {
        int c;
        if (!skip(c)) return false;
        s.clear();
        while (c != EOF && c > ' ') {
            s.push_back(char(c));
            c = get();
        }
        return true;
    }
    template<size_t N>
    bool read(char (&s)[N]) {
        static_assert(N > 0);
        int c;
        if (!skip(c)) return false;
        size_t len{};
        bool ok = true;
        while (c != EOF && c > ' ') {
            if (len + 1 < N) s[len ++] = char(c);
            else ok = false;
            c = get();
        }
        s[len] = '\0';
        return ok;
    }
    // ===== 整行读取 =====
    bool line(string &s) {
        int c = get();
        if (c == EOF) return false;
```

```
        s.clear();
        while (c != EOF && c != '\n') {
            s += char(c);
            c = get();
        }
        if (!s.empty() && s.back() == '\r') s.pop_back();
        return true;
    }
    // ===== 容器读取 =====
    template<class T, size_t N,
        enable_if_t<!is_same_v<T, char>, int> = 0>
    bool read(T (&a)[N]) {
        for (T &x : a) if (!read(x)) return false;
        return true;
    }
    template<class T, size_t N>
    bool read(array<T, N> &a) {
        for (T &x : a) if (!read(x)) return false;
        return true;
    }
    template<class T, class A>
    bool read(vector<T, A> &a) {
        for (T &x : a) if (!read(x)) return false;
        return true;
    }
    // ===== 连续读取 =====
    template<class T, class U, class... V>
    bool read(T &x, U &y, V &...v) { return read(x) && read(y, v...); }
};
struct Output {
private:
    static constexpr size_t S{ 1 << 20 };
    char buf[S];
    size_t p{};
    template<class U>
    void write_unsigned(U x) {
        char s[40];
        int n{};
        do {
            s[n ++] = char('0' + x % 10);
            x /= 10;
        } while (x);
        while (n) put(s[-- n]);
    }
    void sep(bool &first) {
        if (!first) put(' ');
        first = false;
    }
    // ===== 标量输出 =====
    void scalar(char c) { put(c); }
    void scalar(bool x) { put(x ? '1' : '0'); }
    void scalar(string_view s) { for (char c : s) put(c); }
    void scalar(const char *s) { while (*s) put(*s ++); }
    template<class T,
        enable_if_t<is_integral_v<T> && !is_same_v<T, bool>
            && !is_same_v<T, i128> && !is_same_v<T, u128>, int> = 0>
    void scalar(T x) {
        using U = make_unsigned_t<T>;
        U v;
        if constexpr (is_signed_v<T>) {
            if (x < 0) {
                put('-');
                v = U(-(x + 1)) + 1;
            } else v = U(x);
        } else v = U(x);
        write_unsigned(v);
    }
    // ===== 128 位整数输出 =====
    void scalar(i128 x) {
        if (x < 0) {
            put('-');
```

```
            write_unsigned(u128(-(x + 1)) + 1);
        } else write_unsigned(u128(x));
    }
    void scalar(u128 x) { write_unsigned(x); }
    // ===== 容器展开 =====
    template<class T>
    void tokens(bool &first, const T &x) {
        sep(first);
        scalar(x);
    }
    template<class T, size_t N,
        enable_if_t<!is_same_v<T, char>, int> = 0>
    void tokens(bool &first, const T (&a)[N]) {
        for (const T &x : a) tokens(first, x);
    }
    template<class T, size_t N>
    void tokens(bool &first, const array<T, N> &a) {
        for (const T &x : a) tokens(first, x);
    }
    template<class T, class A>
    void tokens(bool &first, const vector<T, A> &a) {
        for (const T &x : a) tokens(first, x);
    }
public:
    ~Output() { flush(); }
    // ===== 基础输出 =====
    void flush() {
        if (p) {
            fwrite(buf, 1, p, stdout);
            p = 0;
        }
    }
    void put(char c) {
        if (p == S) flush();
        buf[p ++] = c;
    }
    // ===== 通用输出 =====
    template<class... T>
    void write(const T &...x) {
        if constexpr (sizeof...(T)) {
            bool first = true;
            (tokens(first, x), ...);
        }
    }
    // ===== 扩展输出 =====
    template<class... T>
    void writeln(const T &...x) { write(x...); put('\n'); }
    template<class... T>
    void writelines(const T &...x) { (writeln(x), ...); }
};
inline Input in;
inline Output out;
}
using namespace FastIO;
```

Input

- `in.get()`：读取一个原始字节，包含空格、换行，EOF 返回 `-1`，均摊 $O(1)$
- `in.read(x)`：读取整数、非空白字符或字符串，成功返回 `true`，失败返回 `false`
- `in.read(a, b, c)`：依次读取多个参数，任一失败则停止
- `in.read(a)`：当 `a` 是 C 数组、`array`、`vector` 时，逐元素读取
- `in.line(s)`：读取一整行，不包含行末换行符，支持空行，遇到 EOF 且没有可读取字符时返回 `false`

Output

- `out.put(c)`：输出一个原始字符
- `out.flush()`：将缓冲区内的数据写入 `stdout`
- `out.write(x...)`：输出多个标量或容器，元素之间自动添加空格，不自动换行
- `out.writeln(x...)`：与 `write()` 相同，末尾添加换行
- `out.writelines(x...)`：每个参数输出一行

- `out.writeln()`：输出一个空行

容器约定

- `vector` 必须事先 `resize()`，不支持直接读取未分配大小的容器
- 支持嵌套的 `vector`、`array` 和普通数组
- 不支持 `vector<bool>` 的代理引用读写
- `char[N]` 作为以 `'\0'` 结尾的字符串处理，而不是逐元素处理

注意事项

- `FastIO` 输入会提前缓存数据，不能与 `cin`、`scanf`、`getchar` 混合读取同一个 `stdin`
- 使用 `FastIO` 输出时，也不应与 `cout`、`printf` 混合输出，以免缓冲顺序与预期不一致
- `read()` 假定整数 token 只含数字及可选的前置正负号，且以空白或 EOF 分隔，不是通用文本语法分析器
- 整数读取溢出、非法符号、EOF 时返回 `false`，单个标量读取失败不会修改目标变量，容器多元素读取失败时，之前的元素可能已经修改
- `read(char[N])` 超长时会截断存储、消费整个 token 并返回 `false`
- `get()` 返回 `int`，不能用 `char` 保存 EOF
- `line()` 在 `\r\n` 行尾时自动去掉末尾 `\r`
- 使用 `read()` 后调用 `line()` 时，整数或 token 读取已消费一个结束字符，注意是否还有同一行的剩余内容
- 正常程序退出时 `Output` 析构函数自动刷新缓冲区，交互题不建议使用该 `fread` 方案
- 本实现的输出不进行特殊的 `boolalpha`、浮点格式化、进制格式化

常见用法

标准 I/O（iostream）

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

- `sync_with_stdio(false)`：关闭 C++ 标准流与 C 标准流之间的同步
- `cin.tie(nullptr)`：取消输入前自动刷新 `cout`
- `'\n'`：只添加换行，通常优先于 `endl`
- `endl`：换行并刷新输出缓冲区

字符、字符串与整行输入

```
char c;
cin >> c;          // 跳过前导空白
cin.get(c);        // 读取一个原始字符，包括空格和换行

string s, line;
cin >> s;          // 读取空白分隔的字符串
getline(cin, line); // 读取整行，不含换行符

char buf[100];
cin >> setw(100) >> buf; // 最多读取 99 个字符
```

读取整数后接着使用 `getline()` 时，需要先处理残留换行符

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string s;
getline(cin, s);
```

也可以写成：

```
getline(cin >> ws, s);
```

但 `ws` 会跳过所有前导空白，也可能跳过空行或行首空格，不能用于需要保留这些信息的场景

未知数量整数输入

读到 EOF 为止

```
vector<int> a;
for (int x; cin >> x;) a.push_back(x);
```

按行读取，一行内的整数个数未知

```
string s;
getline(cin, s);
stringstream ss(s);
vector<int> a;
for (int x; ss >> x;) a.push_back(x);
```

使用 `FastIO` 时：

```
int x;
while (in.read(x)) {
    // ...
}
```

`scanf` / `printf` 格式化输入输出

```
int x;
i64 y;
double a;
long double b;
char c, s[100];

scanf("%d%lld%lf%Lf", &x, &y, &a, &b);
scanf(" %c", &c);       // 跳过前导空白
scanf("%99s", s);       // 限制字符串长度

printf("%d %lld\n", x, y);
printf("%.12f\n", a);
printf("%.12Lf\n", b);
```

常见格式说明符：

类型	`scanf`	`printf`
`int`	`%d`	`%d`
`unsigned int`	`%u`	`%u`
`i64`	`%lld`	`%lld`
`u64`	`%llu`	`%llu`
`float`	`%f`	`%f`
`double`	`%lf`	`%f`
`long double`	`%Lf`	`%Lf`
`char`	`%c`	`%c`
C 字符串	`%s`	`%s`

`scanf` 读取 C 字符串应指定宽度，至少预留一个字符存放 `'\0'`

`stringstream` 常用操作

字符串与整数互转

```
stringstream ss;
ss << 123 << ' ' << -45;
string s = ss.str(); // "123 -45"

stringstream tt(s);
int x, y;
tt >> x >> y;
```

按照指定分隔符拆分

```
stringstream ss("aa,bb,,cc");
string s;
vector<string> a;
while (getline(ss, s, ',')) a.push_back(s);
// {"aa", "bb", "", "cc"}
```

不涉及格式化时，整数与字符串转换优先考虑 `to_string()`、`stoi()`、`stoll()` 或 C++17 的 `from_chars()`、`to_chars()`

浮点数与格式化输出

```
double x = 1.23456789;
cout << fixed << setprecision(3) << x << '\n'; // 1.235
cout << scientific << setprecision(6) << x << '\n';
cout << defaultfloat << x << '\n';

cout << setw(6) << setfill('0') << 42 << '\n'; // 000042
cout << boolalpha << true << '\n';             // true
cout << noboolalpha << true << '\n';           // 1
```

- `fixed`：`setprecision(k)` 表示小数点后保留 `k` 位
- `scientific`：科学计数法，`setprecision(k)` 表示小数点后 `k` 位
- `defaultfloat`：恢复默认浮点格式，此时精度表示有效数字位数
- `setw(k)`：只影响下一次格式化插入
- `setfill(c)`、`setprecision(k)`、`fixed` 等设置会持续生效，必要时需恢复

C++20 `std::format`

需要 `<format>`，适用于格式化生成字符串

```
// C++20，需编译器标准库支持 <format>
string s = std::format("{} {:.3f}", 42, 1.23456);
// s = "42 1.235"
string t = std::format("{:08d}", 42);
// t = "00000042"
```

`std::format()` 返回 `string`，并不直接输出

C++23 `std::print` / `std::println`

需要 `<print>`，可以直接格式化输出，无需构造字符串流

```
// C++23，需标准库支持 <print>
std::print("{} {}", 10, 20);
std::println(" {}", 30); // 10 20 30
std::println("{:.6f}", 1.23456789);
```

`std::print` 不自动换行，`std::println` 自动添加换行，均支持格式说明符

⚠ `std::format` 属于 C++20，`std::print` / `std::println` 属于 C++23，但 `-std=gnu++20` 或 `-std=gnu++23` 并不保证评测机标准库已实现相关接口

FastIO 常用调用

```
int n, m;
i64 x;
string s;
in.read(n, m, x, s);

vector<int> a(n);
in.read(a);

out.writeln(n, m, x);
out.writeln(a);
out.writelines(n, m, x);

// 原始字符与整行
int c = in.get();
string line;
in.line(line);

// 立即刷新
out.flush();
```

输出容器时会递归展开其中的元素

```
vector<vector<int>> a{{1, 2}, {3, 4}};
```

```
out.writeln(a); // 1 2 3 4
```

文件重定向与本地调试

使用条件编译进行文件重定向

```
int main() {
#ifdef LOCAL
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
    return 0;
}
```

GNU C++ 编译：

```
g++ main.cpp -std=gnu++17 -O2 -DLOCAL -o main
./main
```

Shell 重定向：

```
./main < in.txt > out.txt
./main < in.txt > out.txt 2> err.txt
time ./main < in.txt > out.txt
```

Windows CMD：

```
main.exe < in.txt > out.txt
main.exe < in.txt > out.txt 2> err.txt
```

- `stdin`、`stdout`、`stderr` 分别为标准输入、标准输出、标准错误输出
- `2> err.txt` 将标准错误输出重定向到文件
- `freopen()` 应在任何实际读写之前调用
- 本地调试优先使用 Shell 重定向，无需修改源代码

复杂度

操作	时间复杂度	空间复杂度
`in.get()` / `out.put()`	均摊 $O(1)$	$O(1)$
`in.read(x)` 整数	$O(d)$	$O(1)$
`in.read(s)` 字符串	$O(n)$	$O(n)$
`in.line(s)`	$O(n)$	$O(n)$
`in.read(a)` 容器	$O(\sum d_i)$	$O(1)$（不计容器）
`out.write(x...)`	$O(L)$	$O(1)$（不计缓冲区）
`out.flush()`	$O(B)$	$O(1)$
`getline()`	$O(n)$	$O(n)$
`stringstream` 解析	$O(n)$	$O(n)$

其中：

- d 为一个整数的十进制位数
- n 为字符串长度
- L 为输出总字符数
- B 为待刷新缓冲区中的字节数
- FastIO 的固定缓冲区总空间约为 $2\ \mathrm{Mib}$

1.4 SafeHash

整数哈希

这个是最常用的，适合 `int / i64 / u64`，主要用于防卡 `unordered_*`

```
struct IntHash {
    static u64 splitmix64(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(u64 x) const {
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
    }
};
```

直接：

```
unordered_map<i64, int, IntHash> mp;
unordered_set<u64, IntHash> st;
```

`pair<int, int>`

二维坐标、边、状态二元组特别常见。两个 `int` 可以无冲突地压进一个 `u64`，所以没必要做两次字段哈希：

```
struct PairIntHash {
    size_t operator()(const pair<int, int> &p) const {
        u64 x = u64(uint32_t(p.first)) << 32 | uint32_t(p.second);
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        return IntHash::splitmix64(x + seed);
    }
};
```

```
unordered_map<pair<int, int>, int, PairIntHash> mp;
```

这个版本我尤其推荐。`int` 即使是负数也没问题，因为转成 `uint32_t` 后只是保留原来的 32 位比特
如果你的二维点经常写成：

```
array<int, 2>
```

可以准备对应版本：

```
struct Array2Hash {
    size_t operator()(const array<int, 2> &a) const {
        u64 x = u64(uint32_t(a[0])) << 32 | uint32_t(a[1]);
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        return IntHash::splitmix64(x + seed);
    }
};
```

`pair<i64, i64>`

两个 `i64` 已经不能无损塞进一个 `u64`，这时候才需要真正 combine：

```
struct PairI64Hash {
    size_t operator()(const pair<i64, i64> &p) const {
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        u64 x = IntHash::splitmix64(u64(p.first) + seed);
        u64 y = IntHash::splitmix64(u64(p.second) + seed);
        return IntHash::splitmix64(x ^ (y + 0x9e3779b97f4a7c15ULL + (x << 6) + (x >> 2)));
    }
};
```

适合：

```
unordered_set<pair<i64, i64>, PairI64Hash> st;
```

`__int128`

```
struct I128Hash {
    size_t operator()(i128 x) const {
        u128 y = x;
        u64 lo = y;
        u64 hi = y >> 64;
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        return IntHash::splitmix64(lo ^ IntHash::splitmix64(hi + seed));
    }
};
```

```
unordered_map<i128, int, I128Hash> mp;
```

如果你已经定义了 `u128`，这个只有几行，实战价值挺高

任意定长 `array`

如果偶尔会有三维坐标、状态数组，可以准备这个。它比针对 `array<int, 2>` 的版本稍重，但仍远比完整 `SafeHash` 简单：

```
template<class T, size_t N>
struct ArrayHash {
    size_t operator()(const array<T, N> &a) const {
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        u64 h = seed;
        for (const T &x : a) {
            u64 y = IntHash::splitmix64(u64(x) + seed);
            h ^= y + 0x9e3779b97f4a7c15ULL + (h << 6) + (h >> 2);
        }
        return IntHash::splitmix64(h);
    }
};
```

主要面向整数数组：

```
unordered_map<array<int, 3>, int, ArrayHash<int, 3>> mp;
```

自定义哈希

```
struct SafeHash {
    static u64 splitmix64(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    static u64 seed() {
        static const u64 s = [] {
            u64 x = chrono::steady_clock::now().time_since_epoch().count();
            x ^= u64(uintptr_t(&x));
            return splitmix64(x);
        }();
        return s;
    }
    // 基础类型：要求 std::hash<T> 可用
    template<class T>
    static u64 get(const T &x) {
        return splitmix64(u64(hash<T>{}(x)) + seed());
    }
    // 组合多个字段；自定义结构体哈希时直接传入各成员
    template<class T>
    static void combine(u64 &h, const T &x) {
        h ^= splitmix64(get(x) + 0x9e3779b97f4a7c15ULL + (h << 6) + (h >> 2));
    }
    // pair / tuple 可直接作为 unordered_* 的键
    template<class T1, class T2>
    static u64 get(const pair<T1, T2> &p) {
        u64 h{ 0 };
        combine(h, p.first);
        combine(h, p.second);
        return h;
```

```
    }
    template<class... Ts>
    static u64 get(const tuple<Ts...> &t) {
        u64 h{ 0 };
        apply([&](const auto &...x) { (combine(h, x), ...); }, t);
        return h;
    }
    template<class T>
    size_t operator()(const T &x) const { return size_t(get(x)); }
};
```

接口说明

- `SafeHash`：可直接作为 `unordered_map` / `unordered_set` 的哈希器
- `splitmix64(x)`：对一个 `u64` 做高质量位混合
- `seed()`：返回本次程序运行固定、不同运行之间变化的随机种子
- `get(x)`：计算单个对象的哈希；普通类型要求存在 `std::hash<T>`
- `get(pair)`：按顺序组合两个元素的哈希
- `get(tuple)`：按顺序组合 tuple 中所有元素的哈希
- `combine(h, x)`：把字段 `x` 合并到已有哈希值 `h`，主要用于自定义结构体
- `operator()(x)`：标准哈希器接口，使 `SafeHash` 可以直接传给 `unordered_*`

注意事项

- `SafeHash` 主要用于降低 `unordered_map` / `unordered_set` 被构造大量哈希冲突而退化的风险，不是密码学哈希，也不能保证绝对无碰撞
- 普通类型仍依赖 `std::hash<T>` 得到初始值，再经过随机化的 `splitmix64` 混合
- 已直接支持任意可哈希元素组成的 `pair` 和 `tuple`，包括嵌套结构，例如 `pair<pair<int, int>, i64>`
- `combine()` **传原始成员即可**，不要写成 `combine(get(x))`，否则会额外再哈希一次
- `pair` / `tuple` 的组合哈希与元素顺序有关，`{a, b}` 和 `{b, a}` 通常具有不同哈希
- 自定义结构体除哈希器外，还必须提供正确的相等比较，默认即 `operator==`
- 若数据量很大且大致知道元素数量，可以在插入前使用 `reserve()`，减少 `unordered_*` 扩容次数

常见用法

普通整数键

```
unordered_map<i64, int, SafeHash> mp;
unordered_set<i64, SafeHash> s;
mp[x] = 1;
s.insert(x);
```

相比默认的：

```
unordered_map<i64, int> mp;
```

只需要增加第三个模板参数 `SafeHash`

`pair` 作为键

无需再单独写 `PairHash`：

```
unordered_map<pair<int, int>, int, SafeHash> mp;
mp[{x, y}] ++;
```

也可以用于集合：

```
unordered_set<pair<int, int>, SafeHash> s;
s.insert({x, y});
if (s.contains({x, y})) {
}
```

若使用 C++17，将 `contains()` 改为：

```
if (s.find({x, y}) != s.end()) {
}
```

`tuple` 作为键

```
unordered_map<tuple<int, int, i64>, int, SafeHash> mp;
mp[{x, y, z}] ++;
```

不同类型也可以混用：

```
unordered_set<tuple<int, string, i64>, SafeHash> s;
s.emplace(1, "abc", 10);
```

嵌套组合键

`pair` / `tuple` 可以递归组合：

```
using Key = pair<pair<int, int>, tuple<int, i64>>;
unordered_map<Key, int, SafeHash> mp;
mp[{{x, y}, {z, w}}] ++;
```

自定义结构体

例如：

```
struct Point {
    int x, y;
    bool operator==(const Point &p) const {
        return x == p.x && y == p.y;
    }
};
```

为它写一个极短的哈希器：

```
struct PointHash {
    size_t operator()(const Point &p) const {
        u64 h{ 0 };
        SafeHash::combine(h, p.x);
        SafeHash::combine(h, p.y);
        return h;
    }
};
```

然后：

```
unordered_map<Point, int, PointHash> mp;
mp[{1, 2}] ++;
```

多字段结构体同理：

```
struct NodeHash {
    size_t operator()(const Node &p) const {
        u64 h{ 0 };
        SafeHash::combine(h, p.x);
        SafeHash::combine(h, p.y);
        SafeHash::combine(h, p.id);
        return h;
    }
};
```

大量元素时预留容量

```
unordered_map<i64, int, SafeHash> mp;
mp.max_load_factor(0.7);
mp.reserve(n);
```

可以减少 rehash，一般在预计需要插入大量元素时使用

复杂度

设组合键包含 k 个字段

操作	时间复杂度
`splitmix64(x)`	$O(1)$
`get(x)`	$O(1)$ 加 `std::hash<T>` 的复杂度
`get(pair)`	两个成员哈希复杂度之和
`get(tuple)`	$O(k)$ 加各成员自身哈希复杂度
`combine(h, x)`	$O(1)$ 加 `get(x)` 的复杂度

`unordered_map` / `unordered_set` 的查找、插入和删除仍为期望 $O(1)$；`SafeHash` 的作用主要是使恶意构造桶冲突更加困难，而不是改变容器本身的理论复杂度

1.5 普通框架

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
namespace rgs = ranges;
constexpr int mod{ 998244353 };
constexpr int inf{ numeric_limits<int>::max() };

void init() {
}

void solve() {
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t{ 1 };
    // cin >> t;
    init();
    while (t --) {
        solve();
    }
    return 0;
}
```

1.6 时间优化

以下优化选项适用于 GCC / GNU C++，默认不启用，卡常时按需选择。普通题目优先使用评测机提供的 `-O2` 优化

编译优化

```cpp
// ===== 方案一：O3，优先尝试 =====
#pragma GCC optimize("O3")
// ===== 方案二：O3 + 循环展开，实测有效时使用 =====
#pragma GCC optimize("O3,unroll-loops")
// ===== 方案三：Ofast，确认语义安全后使用 =====
#pragma GCC optimize("Ofast,unroll-loops")
```

上述三个方案只保留一个，`O3` 不保证比 `O2` 快，循环展开也可能导致代码体积增大

CPU 指令集优化

```cpp
// ===== AVX2 / POPCNT：确认评测机支持后使用 =====
#pragma GCC target("avx2,popcnt")
// ===== 扩展位运算指令：确认全部支持后使用 =====
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

指令集选项可与前面的编译优化组合，但必须确认目标 CPU 支持，不能因为本机支持就默认评测机也支持

局部函数优化

当只需要优化特定函数时，可以使用 `push_options` / `pop_options`，避免影响其他代码

```
#pragma GCC push_options
#pragma GCC optimize("O3")
#pragma GCC target("avx2,popcnt")

void work() {
    // 需要优化的函数
}

#pragma GCC pop_options
```

以上 pragma 对后续定义的函数生效，`pop_options` 恢复之前的设置

注意事项

- GCC 的 `O3` 包含 `O2` 的优化，但更激进不代表运行速度一定更快
- `Ofast` 会启用 `-ffast-math` 等选项，可能改变浮点运算、NaN、无穷大及有符号零等行为，不能作为通用默认配置
- `target` 指令集优化通常面向特定架构，AVX2、BMI 等属于 x86 指令集，不适用于所有平台
- 未支持的 CPU 执行相应指令可能触发 `Illegal instruction`
- 指令集优化只是允许编译器使用对应指令，不代表程序一定会自动向量化
- 对于单文件程序，编译优化优先使用命令行选项；`#pragma` 适合不能修改评测机编译参数时尝试
- 不建议使用 `#pragma GCC optimize("trapv")` 等与加速无关的选项来卡常

常见用法

本地对比优化等级

```
g++ main.cpp -std=gnu++20 -O2 -o main
g++ main.cpp -std=gnu++20 -O3 -o main
g++ main.cpp -std=gnu++20 -O3 -flto -o main
```

`-flto` 启用链接时优化，单文件竞赛程序的收益不一定明显，测试时仍需保证相同数据、运行环境及正确输出

C++20 分支预测提示

```
if (x == 0) [[unlikely]] {
    // 极少发生的情况
} else {
    // 常见情况
}
```

`[[likely]]` 与 `[[unlikely]]` 是 C++20 属性，只提供分支概率提示，不保证生成更快的代码

代码层面的常用优化

- 减少重复分配：已知元素个数时提前 `reserve(n)`，可减少扩容
- 改善局部性：连续数组通常比大量离散分配的节点更有缓存优势
- 避免不必要的复制：大对象传参优先考虑 `const T &`
- 减少高频操作开销：循环中避免重复执行不变的计算或构造临时对象
- 选择合适的 I/O：大量输入输出优先使用快速 I/O

1.7 Python 算法竞赛回忆手册

用途： 高精度整数、数论、字符串、组合枚举、模拟及标准库能够简化实现的竞赛题
环境： 默认 Python 3.8+，新版本功能单独标注；使用 CPython / PyPy 时以评测机实际版本为准

1. 基础框架与快速 I/O

普通竞赛模板

```
import sys
input = sys.stdin.buffer.readline
write = sys.stdout.write

def solve():
    n = int(input())
```

```
    a = list(map(int, input().split()))
    print(sum(a))

def main():
    t = 1
    # t = int(input())
    for _ in range(t):
        solve()

if __name__ == "__main__":
    main()
```

- `input()` 在本模板中返回 `bytes`，不是通常内置 `input()` 返回的 `str`
- `int(input())`、`map(int, input().split())` 可直接读取整数
- `write(s)` 不自动换行；`print()` 默认自动换行
- 若需要读取 Unicode 文本，使用 `input().decode()` 或改回普通 `input`

快读方法

按行读取：默认选择

```
input = sys.stdin.buffer.readline
n, m = map(int, input().split())
a = list(map(int, input().split()))
s = input().strip().decode()
```

一次性读入：适合输入不太大且以空白分隔的数据

```
a = list(map(int, sys.stdin.buffer.read().split()))
```

一次性读入并迭代：避免立即创建完整的整数列表

```
it = map(int, sys.stdin.buffer.read().split())
n, m = next(it), next(it)
a = [next(it) for _ in range(n)]
```

`read().split()` 仍会建立 `bytes` token 列表并占用内存，特别大的输入优先逐行处理

按字节扫描整数

仅适用于合法的空白分隔整数输入，减少拆分后的 token 对象，但 Python 层逐字节循环不一定比 `map(int, ...)` 快

```
import sys

def ints():
    x = 0
    neg = has = False
    for c in sys.stdin.buffer.read():
        if 48 <= c <= 57:
            x = x * 10 + c - 48
            has = True
        elif has:
            yield -x if neg else x
            x = 0
            neg = has = False
        else:
            neg = c == 45
    if has:
        yield -x if neg else x

it = ints()
# n = next(it)
```

只在输入保证合法整数 token 时使用；大量输入仍全部保存在读取的 `bytes` 对象中

快速输出

```
a = [1, 2, 3]
print(*a)                                  # 1 2 3
print("YES" if ok else "NO")               # 按条件输出
```

```
sys.stdout.write(" ".join(map(str, a)) + "\n")

out = ["YES", "NO"]
sys.stdout.write("\n".join(out) + "\n")

# 很多行输出时，也可定期分批 write，避免积累过多字符串
```

- `print(*a)` 方便，元素特别多时可改为 `join`
- `join` 需要字符串元素，数字序列通常搭配 `map(str, a)`
- 交互题不可延迟刷新

交互题

```
print("?", x, y, flush=True)
res = int(input())
if res == -1:
    sys.exit()
print("!", ans, flush=True)
```

如使用 `sys.stdout.write()`，询问后调用 `sys.stdout.flush()`

2. 高精度与数学

任意精度整数

```
a = 10 ** 100
b = int("9" * 100)

x = a + b
x = a - b
x = a * b
x = a // b
x = a % b
q, r = divmod(a, b)
x = a ** 5
x = pow(a, 5, 998244353)   # 模快速幂
```

Python `int` 可以存储任意精度整数，实际受时间、内存与整数/字符串转换限制约束

超长十进制整数的转换限制

较新的 CPython 默认限制部分十进制整数与字符串互转，常见默认上限为 4300 位；部分旧版本安全更新也引入了限制

```
import sys
if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)
```

- `0` 表示取消限制，建议仅在可信的竞赛数据确实需要超长十进制转换时使用
- 不仅 `int(s)`，`str(x)`、`print(x)` 也可能受影响
- 二、八、十六进制等以 2 的幂为底的转换不受该十进制位数限制

基础运算与 C++ 差异

运算	Python	注意事项
加减乘	`+`、`-`、`*`	整数自动扩展精度
真除法	`/`	产生浮点结果，不适合精确大整数除法
整数整除	`//`	向负无穷方向取整
取余	`%`	与向下整除匹配
商和余数	`divmod(a, b)`	返回 `(q, r)`
乘方	`a ** b`	`^` 是异或，不是乘方
模快速幂	`pow(a, b, mod)`	`mod` $\neq 0$
向上整除	`-(-a // b)`	`b` $\neq 0$，适用于正负除数

```
-7 // 3          # -3, C++ 的 -7 / 3 为 -2
-7 % 3           # 2, C++ 的 -7 % 3 为 -1
```

数学标准库

```
from math import gcd, factorial, isqrt, comb, perm, prod

gcd(12, 18)          # 6
factorial(10)        # 10!
isqrt(10**100)       # 精确整数平方根
comb(10, 3)          # 120
perm(10, 3)          # 720
prod([2, 3, 4])      # 24

# Python 3.9+
from math import lcm
lcm(12, 18)          # 36
```

`comb`、`perm`、`isqrt`、`prod` 需要 Python 3.8+，`lcm` 需要 Python 3.9+

模逆元与组合数

```
mod = 998244353
inv = pow(3, -1, mod)        # Python 3.8+
ans = a * pow(b, -1, mod) % mod
ans = pow(2, 10**18, mod)
ans = comb(100, 30) % mod
```

- `pow(b, -1, mod)` 要求 `gcd(b, mod) == 1`，不要求模数为质数
- 逆元不存在时会抛出 `ValueError`
- 大量组合数模查询仍应考虑阶乘与逆阶乘预处理

精确分数和十进制小数

```
from fractions import Fraction
from decimal import Decimal, getcontext

a = Fraction(1, 3)
b = Fraction(2, 5)
print(a + b)                 # 11/15

getcontext().prec = 50
x = Decimal("0.1")
y = Decimal("0.2")
print(x + y)                 # 0.3
```

- `Fraction` 适用于精确有理数
- `Decimal` 适用于十进制精度控制，应从字符串创建
- 二者运算通常慢于 `int` 或普通 `float`

3. 基础语法速查

循环、条件、赋值

```
for i in range(n):               # [0, n)
    pass
for i in range(l, r):            # [l, r)
    pass
for i in range(n - 1, -1, -1):   # 倒序
    pass
for i, x in enumerate(a):
    pass
for x, y in zip(a, b):
    pass

x, y = y, x                      # 交换
x = a if ok else b               # 条件表达式
if 0 <= x < n:
    pass
if x in st:
    pass
```

`zip(a, b)` 默认在较短序列结束时停止

推导式与生成器

```
a = [0] * n
b = [i * i for i in range(n)]
c = [x for x in a if x > 0]
d = [[0] * m for _ in range(n)]

s = sum(x * x for x in a)
ok = all(x >= 0 for x in a)
ok = any(x == 0 for x in a)

b = [x for row in d for x in row]   # 展平二维列表
mp = {x: i for i, x in enumerate(a)}
st = {x % 10 for x in a}
```

二维列表不要使用 `[[0] * m] * n`，这样会让所有行引用同一个列表

切片、拆包与复制

```
a = [1, 2, 3, 4, 5]
a[-1]             # 最后一个
a[1:4]            # [2, 3, 4]
a[:3]             # 前三个
a[3:]             # 从下标 3 到末尾
a[::-1]           # 逆序副本
a[::2]            # 步长为 2
a[:]              # 浅拷贝

x, y = y, x
x, *mid, y = [1, 2, 3, 4, 5]

def calc(a, b):
    return a + b, a - b
s, d = calc(10, 3)
```

切片一般会创建新对象，避免在大循环里无意复制大量元素

函数、变量作用域、默认参数

```
def f(x, y=0):
    return x + y

g = lambda x: x * x

def outer():
    cnt = 0
    def dfs():
        nonlocal cnt
        cnt += 1
    dfs()
    return cnt
```

- `nonlocal` 修改外层函数绑定，`global` 修改模块级绑定
- `lambda` 只能包含一个表达式
- 避免 `def f(a=[])` 等可变默认参数，通常改为 `None` 再初始化

现代语法

```
# Python 3.8+
if (x := len(a)) > 10:
    print(x)

# Python 3.9+, 字典合并
c = a | b

# Python 3.10+
match op:
    case "+":
```

```
        pass
    case "-":
        pass
    case _:
        pass
```

4. 容器

list：动态数组、栈

```
a = []
a.append(3)
a.extend([4, 5])
x = a.pop()       # 删除最后一个
a.insert(0, 2)    # O(n)
a.remove(3)      # 删除第一个等于 3 的元素
a.count(4)
a.index(4)
a.reverse()
a.clear()
```

`append()`、尾部 `pop()` 均摊 $O(1)$；中间插入、删除通常 $O(n)$

deque：队列与双端队列

```
from collections import deque
q = deque()
q.append(1)
q.appendleft(2)
x = q.popleft()
y = q.pop()
```

两端操作均摊 $O(1)$，BFS 不要用 `list.pop(0)`

set：哈希集合

```
st = set()
st.add(3)
st.discard(3)       # 不存在也不报错
st.remove(3)        # 不存在抛 KeyError

a = {1, 2, 3}
b = {2, 3, 4}
a | b               # 并
a & b               # 交
a - b               # 差
a ^ b               # 对称差
```

`set` 无键排序保证，也不支持按位置下标访问

dict：哈希映射

```
mp = {}
mp["a"] = 3
mp["b"] = 5
x = mp.get("c", 0)
mp.setdefault("d", 0)
mp.pop("a", None)
for k, v in mp.items():
    pass
```

`dict` 保留插入顺序，但不按键排序

Counter / defaultdict

```
from collections import Counter, defaultdict
cnt = Counter([1, 2, 2, 3, 3, 3])
cnt[3]                # 3
cnt[10]               # 0
cnt.most_common(2)    # [(3, 3), (2, 2)]
```

```
mp = defaultdict(int)
mp[5] += 1
g = defaultdict(list)
g[1].append(2)
st = defaultdict(set)
st[1].add(2)
```

5. 排序、二分与堆

排序

```
a.sort()                            # 原地升序
a.sort(reverse=True)                # 原地降序
b = sorted(a)                       # 新列表

p = [(2, 5), (1, 8), (2, 3)]
p.sort()                            # 元组字典序
p.sort(key=lambda x: x[1])          # 按第二项升序
p.sort(key=lambda x: (x[0], -x[1]))  # 第一项升序，第二项降序
```

- 排序是稳定的，一般时间复杂度为 $O(n \log n)$
- `a.sort()` 返回 `None`，不能写 `a = a.sort()`

bisect：有序序列二分

```
from bisect import bisect_left, bisect_right, insort

a = [1, 2, 2, 2, 4, 5]
l = bisect_left(a, 2)     # 1
r = bisect_right(a, 2)    # 4
cnt = r - l               # 等于 2 的个数
insort(a, 3)              # 查找 O(log n)，移动 O(n)
```

- `bisect_left(a, x)`：小于 `x` 的元素个数
- `bisect_right(a, x)`：小于等于 `x` 的元素个数

heapq：小根堆

```
from heapq import heappush, heappop, heapify, heappushpop, heapreplace
h = [5, 2, 8]
heapify(h)              # O(n)
heappush(h, 3)          # O(log n)
x = heappop(h)          # 最小值
x = h[0]                # 查看堆顶
x = heappushpop(h, 4)
x = heapreplace(h, 4)
```

大根堆（通用写法）：

```
h = []
heappush(h, -10)
heappush(h, -30)
x = -heappop(h)      # 30
```

Python 3.14+ 原生大根堆：

```
from heapq import heapify_max, heappush_max, heappop_max
h = [5, 2, 8]
heapify_max(h)
heappush_max(h, 10)
x = heappop_max(h)
```

旧版 Python / PyPy 不能假定已支持 3.14 的接口

6. itertools 与记忆化

前缀和、排列、组合、笛卡尔积

```python
from itertools import accumulate, permutations, combinations, product

a = [1, 2, 3, 4]
pre = list(accumulate(a, initial=0))
# [0, 1, 3, 6, 10], Python 3.8+

for p in permutations([1, 2, 3]):
    pass
for p in combinations([1, 2, 3], 2):
    pass
for p in product(range(2), repeat=3):
    pass
```

合并、分组、相邻元素

```python
from itertools import chain, groupby
b = list(chain([1, 2], [3, 4]))
for x, group in groupby(sorted(a)):
    cnt = sum(1 for _ in group)

# Python 3.10+
from itertools import pairwise
list(pairwise([1, 2, 3]))  # [(1, 2), (2, 3)]

# Python 3.12+
from itertools import batched
list(batched(range(5), 2)) # [(0, 1), (2, 3), (4,)]
```

`groupby()` 只将连续相同值分到同一组，全局计数通常要先排序

记忆化搜索

```python
from functools import lru_cache

@lru_cache(None)
def dfs(x):
    if x <= 1:
        return x
    return dfs(x - 1) + dfs(x - 2)

ans = dfs(30)
dfs.cache_clear()        # 多测时按需清缓存
```

Python 3.9+ 可使用 `from functools import cache` 和 `@cache`

7. 字符串、进制、位运算

字符串方法

```python
s = "  abC,def  "
s.strip()
s.lower()
s.upper()
s.replace("a", "x")
s.find("b")               # 找不到返回 -1
s.count("a")
s.startswith("a")
s.endswith("f")

"1 2 3".split()          # ['1', '2', '3']
"a,b,c".split(",")       # ['a', 'b', 'c']
"-".join(["a", "b"])     # 'a-b'
"abc"[::-1]              # 'cba'
ord("a")                  # 97
chr(97)                   # 'a'
```

`str` 不可原地修改，改动较多时先转为 `list`，完成后再 `"".join(a)`

bytes 与 str

```
s = b"abc"
s[0]                    # 97
s.decode()               # 'abc'
"abc".encode()           # b'abc'
a = list(s)              # [97, 98, 99]
```

`sys.stdin.buffer.readline()` 返回 `bytes`，字符串处理时注意类型差异

进制转换

```
int("101010", 2)        # 42
int("FF", 16)           # 255
int("777", 8)           # 511
bin(42)                 # '0b101010'
oct(42)                 # '0o52'
hex(255)                # '0xff'
format(42, "b")         # '101010'
format(42, "08b")       # '00101010'
format(255, "X")        # 'FF'
f"{42:08b}"             # '00101010'
```

`int(s, base)` 支持 $2 \sim 36$ 进制，超过 36 进制需自行处理字符表

位运算与整数 bitset

```
x & y                   # 与
x | y                   # 或
x ^ y                   # 异或（不是乘方）
~x                      # 按位取反
x << k                  # 左移
x >> k                  # 右移
x & -x                  # lowbit
x & (x - 1)             # 删除最低位的 1
(x >> k) & 1            # 取第 k 位
x | (1 << k)            # 设置第 k 位
x ^ (1 << k)            # 翻转第 k 位
x & ~(1 << k)           # 清除第 k 位
x.bit_length()          # 二进制位数
x.bit_count()           # 1 的数量，Python 3.10+
```

遍历置位：

```
mask = 0b10110
while mask:
    lb = mask & -mask
    i = lb.bit_length() - 1
    mask ^= lb
```

任意精度整数作为位集（0/1 背包可达性）：

```
f = 1
for x in a:              # 假设所有 x ≥ 0
    f |= f << x
ok = bool((f >> target) & 1)
```

移位和按位或的复杂度与位集长度有关，并非严格 $O(1)$

8. 常用算法极简写法

前缀和与差分

```
from itertools import accumulate
pre = list(accumulate(a, initial=0))
ans = pre[r] - pre[l]          # [l, r)

d = [0] * (n + 1)
d[l] += x
```

```
d[r] -= x
a = list(accumulate(d))[:n]
```

去重与离散化

```
b = sorted(set(a))
mp = {x: i for i, x in enumerate(b)}
c = [mp[x] for x in a]
```

BFS

```
from collections import deque

dis = [-1] * n
dis[s] = 0
q = deque([s])
while q:
    u = q.popleft()
    for v in g[u]:
        if dis[v] != -1:
            continue
        dis[v] = dis[u] + 1
        q.append(v)
```

Dijkstra

```
from heapq import heappush, heappop

inf = 10**30            # 必须大于实际可能的有限最短路长度
dis = [inf] * n
dis[s] = 0
q = [(0, s)]
while q:
    d, u = heappop(q)
    if d != dis[u]:
        continue
    for v, w in g[u]:   # (v, w), w ≥ 0
        nd = d + w
        if nd < dis[v]:
            dis[v] = nd
            heappush(q, (nd, v))
```

9. 性能、递归与内存

优化方法速查

方法	适用情况	注意事项
`sys.stdin.buffer.readline`	常规大量按行输入	得到 `bytes`
`sys.stdin.buffer.read().split()`	大量空白分隔输入	整体内存占用较高
`sys.stdout.write()` + `join()`	大量输出	不自动换行
`sum`、`min`、`max`、`sort`	标准库能够直接处理	通常优于手写 Python 循环
`math` 整数函数	高精度数学运算	留意版本
`deque`	队列、单调队列	两端操作快
PyPy	较多 Python 层循环、模拟、DP	启动和内存开销可能更大
迭代代替深递归	深 DFS / 树形问题	减少递归栈风险

修改递归深度

```
import sys
sys.setrecursionlimit(10**6)
```

这只提高解释器递归层数限制，不会扩大操作系统栈，设置过高仍可能导致进程崩溃

减少内存占用

```
vis = bytearray(n)            # 0/1 标记，单元素 1 字节

from array import array
a = array("i", [0]) * n        # 固定类型整数，位宽依平台

vis = 0                       # int 位集
vis |= 1 << i
```

`list[int]` 的对象开销高于 C++ `vector<int>`；`array` 有类型范围限制，不能保存任意大整数

局部绑定与减少重复工作

```
a = []
push = a.append
for x in range(n):
    push(x)
```

是否加速依赖 Python 解释器版本，建议在目标环境实测，不要优先追求这类微优化

程序计时

```
from time import perf_counter
import sys
st = perf_counter()

# 被测代码

print(f"[Time] {(perf_counter() - st) * 1000:.3f} ms", file=sys.stderr)
```

正式测速应关闭大规模调试输出

10. Python 与 C++ 易混淆之处

场景	Python	C++
整数	`int` 任意精度	`int` / `i64` 固定宽度
乘方	`**`	快速幂或 `pow`
异或	`^`	`^`
整数除法	`//` 向下取整	`/` 向零截断
取模	与向下整除匹配	余数符号通常随被除数
浮点除法	`/`	浮点类型的 `/`
逻辑运算	`and`、`or`、`not`	`&&`、`` ` ``
布尔值	`True`、`False`	`true`、`false`
空值	`None`	`nullptr` / `nullopt` 等
自增	`x += 1`	`++x`
条件表达式	`a if ok else b`	`ok ? a : b`
哈希容器	`dict`、`set`	`unordered_map`、`unordered_set`

易忘记的语言细节

```
if not a:                 # 空容器为假
    pass

a = [1, 2]
b = a                     # 引用同一列表，不是复制
b[0] = 10                 # a[0] 也改变
b = a.copy()              # 浅拷贝

from copy import deepcopy
b = deepcopy(a)           # 深拷贝

s = list("abc")
s[0] = "x"
s = "".join(s)           # 'xbc'

# a = a.sort()            # 错误，sort() 返回 None
```

```
x = (5,)                # 单元素元组必须有逗号
True == 1               # True
```

11. 版本、解释器与运行命令

版本功能速查

功能	最低 Python 版本
`math.comb`、`perm`、`isqrt`、`prod`	3.8
`pow(a, -1, mod)`	3.8
`itertools.accumulate(initial=...)`	3.8
海象运算符 `:=`	3.8
`math.lcm`、字典合并 `a | b`	3.9
`functools.cache`	3.9
`int.bit_count`、`itertools.pairwise`	3.10
`match ... case`	3.10
整数十进制转换位数限制	3.11 及部分旧版本安全更新
`itertools.batched`	3.12
`heapq` 大根堆 API	3.14

Linux / WSL / Git Bash（Bash）

```
python3 --version
pypy3 --version
python3 main.py < in.txt > out.txt
pypy3 main.py < in.txt > out.txt
python3 main.py < in.txt > out.txt 2> err.txt
/usr/bin/time -v python3 main.py < in.txt > out.txt
```

`/usr/bin/time -v` 通常仅适用于 Linux / WSL，不是 Windows Git Bash 的通用命令

Windows CMD

```
py -3 --version
py -3 main.py < in.txt > out.txt
py -3 main.py < in.txt > out.txt 2> err.txt
REM 安装了 PyPy 并加入 PATH 后
pypy3 main.py < in.txt > out.txt
```

Windows PowerShell

```
py -3 --version
py -3 .\main.py
cmd /c "py -3 main.py < in.txt > out.txt 2> err.txt"
```

PowerShell 不支持 Bash/CMD 的 `< in.txt` 输入重定向，使用 `cmd /c` 更直接

CPython 与 PyPy 选择

- CPython：标准库兼容性通常更明确，许多内置大整数与字符串操作表现良好
- PyPy：部分 Python 层循环、模拟和 DP 可能更快，内存及启动开销可能更大
- Python 的 `-O` 主要影响 `assert` 和 `__debug__`，不等同于 C++ 的 `-O2` / `-O3`
- 两者速度与内存表现因题而异，优先以题目支持的解释器和实测结果为准

12. 赛场选用速查

需求	优先选择
数百至数千位整数加减乘除	`int`
高精度模快速幂	`pow(a, b, mod)`
模逆元	`pow(a, -1, mod)`
整数平方根	`math.isqrt`
精确组合数	`math.comb`
精确分数	`fractions.Fraction`

需求	优先选择
十进制高精度小数	`decimal.Decimal`
大量输入	`sys.stdin.buffer.readline` / `read().split()`
大量输出	`sys.stdout.write` + `join`
计数与分组	`Counter` / `defaultdict`
BFS / 双端队列	`deque`
堆	`heapq`
排序	`sort(key=...)`
有序数组二分	`bisect_left` / `bisect_right`
组合排列枚举	`itertools`
前缀和	`itertools.accumulate`
状压可达性	`int` 位集
记忆化搜索	`functools.lru_cache` / `cache`

1.8 PBDS 常见用法

公共头文件

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/trie_policy.hpp>
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
```

GNU 环境也可直接使用

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
```

顺序统计红黑树

基于 `rb_tree_tag`，主要用于动态排名、第 k 小等，通常是 PBDS 中最常用的结构

```
template<class T, class C = less<T>>
using RBT = tree<T, null_type, C, rb_tree_tag, tree_order_statistics_node_update>;
```

接口

只记录普通平衡树之外的接口

- `t.order_of_key(x)`：严格小于 `x` 的元素个数
- `t.find_by_order(k)`：第 `k` 小，`0base`，越界返回 `end()`
- `t.split(x, b)`：`less` 下将 `> x` 的部分移入 `b`
- `t.join(b)`：将 `b` 合入 `t`，成功后 `b` 为空

常见用法

第 `k` 小，`k` 为 `0base`

```
auto it = t.find_by_order(k);
if (it != t.end()) {
    auto x = *it;
}
```

严格小于 `x` 的个数

```
int k = t.order_of_key(x);
```

`x` 的 `1base` 排名

```
int rk = t.order_of_key(x) + 1;
```

严格前驱

```
int k = t.order_of_key(x);
if (k) {
    auto pre = *t.find_by_order(k - 1);
}
```

按值切树

```
RBT<int> a, b;
a.split(x, b);
// a: ⩽ x
// b: > x
```

恢复

```
a.join(b);
```

多重集合

不要使用 `less_equal`

```
RBT<int, less_equal<int>> t; // 不要这样写
```

比较器必须满足严格弱序，多重元素统一加入唯一编号

```
using P = pair<i64, int>;
RBT<P> t;
int id{};
t.insert({x, id++});
```

建议保证 `id ⩾ 0`，查询时用 `-1` 和足够大的值作为边界

```
constexpr int INF = 1E9;
```

`< x`

```
int c = t.order_of_key({x, -1});
```

`⩽ x`

```
int c = t.order_of_key({x, INF});
```

`== x`

```
int c = t.order_of_key({x, INF}) - t.order_of_key({x, -1});
```

第 `k` 小，`k` 为 `1base`

```
i64 x = t.find_by_order(k - 1)->first;
```

严格前驱

```
int k = t.order_of_key({x, -1});
if (k) {
    i64 pre = t.find_by_order(k - 1)->first;
}
```

严格后继

```
int k = t.order_of_key({x, INF});
if (k < int(t.size())) {
    i64 suf = t.find_by_order(k)->first;
}
```

删除任意一个 `x`

```
auto it = t.lower_bound({x, -1});
if (it != t.end() && it->first == x) t.erase(it);
```

精确删除时保存编号

```
int p = id++;
t.insert({x, p});
t.erase({x, p});
```

注意事项

- `join` 要求两棵树值域完全分离，即一棵整体小于另一棵
- `split/join` 是 PBDS 的实现接口，不建议在复杂度证明中直接当作手写红黑树的标准 $O(\log n)$ 操作
- 只需要排名和第 k 小一般优先使用 `rb_tree_tag`

复杂度

- 插入、删除、查找：$O(\log n)$
- `order_of_key`：$O(\log n)$
- `find_by_order`：$O(\log n)$

Splay Tree

PBDS 自带按键维护的 Splay，可省掉只需要集合操作、自调整访问和按键切分时的手写 Splay

```
template<class T, class C = less<T>>
using SPT = tree<T, null_type, C, splay_tree_tag, tree_order_statistics_node_update>;
```

如果不需要排名可缩短为

```
template<class T, class C = less<T>>
using SPT = tree<T, null_type, C, splay_tree_tag>;
```

接口

启用 `tree_order_statistics_node_update` 后

- `t.order_of_key(x)`：严格小于 `x` 的元素个数
- `t.find_by_order(k)`：第 `k` 小
- `t.split(x, b)`：按键切成 `≤ x` 与 `> x`
- `t.join(b)`：合并两个值域完全分离的 Splay

常见用法

按值切分

```
SPT<int> a, b;
a.split(x, b);
```

处理后重新合并

```
a.join(b);
```

带排名

```
int rk = t.order_of_key(x);
auto it = t.find_by_order(k);
```

注意事项

- 这是按键排序的 Splay，不是隐式 Splay
- 不能直接完成区间翻转、区间加法、序列搬移等隐式 Splay 操作
- 连续访问相近或重复元素时才较可能发挥 Splay 的局部性优势
- 普通动态排名问题优先 `rb_tree_tag`

复杂度

- 查找、插入、删除：均摊 $O(\log n)$
- 顺序统计：均摊 $O(\log n)$
- 单次操作最坏可退化到 $O(n)$

Ordered Vector Tree

底层为 `ov_tree_tag`，本质更接近有序连续数组，查询多、修改很少时可以使用

```
template<class T, class C = less<T>>
using OVT = tree<T, null_type, C, ov_tree_tag, tree_order_statistics_node_update>;
```

接口

- `t.order_of_key(x)`：严格小于 `x` 的元素个数
- `t.find_by_order(k)`：第 `k` 小
- `t.split(x, b)`：按键切分
- `t.join(b)`：合并值域完全分离的两棵树

常见用法

大量排名查询，数据只进行少量修改

```
OVT<int> t;
for (int x : a) t.insert(x);
int rk = t.order_of_key(x);
int v = *t.find_by_order(k);
```

注意事项

- 插入、删除需要移动大量元素
- 动态修改多时不要使用
- 修改可能导致大量迭代器失效
- 如果数据完全静态，直接排序 `vector` 往往更简单

复杂度

- 查询、顺序统计：$O(\log n)$
- 插入、删除：$O(n)$

Patricia Trie

PBDS 自带压缩字典树，最有价值的是直接提供前缀范围查询，可省掉一整份 Trie

```
using PT = trie<string, null_type, trie_string_access_traits<>,
    pat_trie_tag, trie_prefix_search_node_update>;
```

接口

除普通集合操作外主要使用

- `t.prefix_range(s)`：返回所有以 `s` 为前缀的字符串区间
- `t.split(s, b)`：字典序 `> s` 的部分移入 `b`
- `t.join(b)`：合并两个字典序值域分离的 Trie

`prefix_range` 返回

```
pair<PT::iterator, PT::iterator>
```

常见用法

插入字符串

```
PT t;
t.insert("abc");
```

```
t.insert("abd");
t.insert("acd");
```

枚举所有以 `"ab"` 开头的字符串

```
auto [l, r] = t.prefix_range("ab");
for (auto it = l; it != r; ++ it) {
    cout << *it << '\n';
}
```

判断是否存在某个前缀

```
auto [l, r] = t.prefix_range(s);
if (l != r) {}
```

统计前缀匹配数只能遍历区间

```
auto [l, r] = t.prefix_range(s);
int c{};
for (; l != r; ++ l) ++ c;
```

若题目需要大量前缀计数，应手写带 `cnt` 的 Trie，而不是这样遍历

注意事项

- `prefix_range` 是 Patricia Trie 最值得使用的功能
- 字符串很多且字符集小时，手写数组 Trie 往往更快
- PBDS Trie 常数和内存开销不小
- `trie_order_statistics_node_update` 在部分新 GCC + C++20 环境存在兼容问题，不建议作为赛场板子依赖

复杂度

设字符串长度为 L

- 查找、插入、删除通常与 L 相关
- `prefix_range(p)` 定位与前缀长度相关
- 枚举额外需要 $O(ans)$

GP Hash Table

开放寻址哈希表，主要价值是较小常数，不重复记录标准哈希表已有接口

```
template<class K, class V, class H = hash<K>>
using HT = gp_hash_table<K, V, H>;
```

接口

需要特别注意的接口差异

```
if (mp.find(x) != mp.end()) {}
```

默认不要写

```
mp.count(x);
mp.reserve(n);
mp.resize(n);
```

这些接口不能直接按 `unordered_map` 的习惯使用

常见用法

```
HT<i64, int> mp;
++ mp[x];
```

配合防卡哈希

```
struct H {
    static u64 f(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(u64 x) const {
        static const u64 s = chrono::steady_clock::now().time_since_epoch().count();
        return f(x + s);
    }
};
HT<i64, int, H> mp;
```

注意事项

- 默认 `resize()` 并不能像 `unordered_map::reserve()` 一样直接调用
- 不要默认存在 `count()`
- 需要防构造数据时仍建议自定义哈希

复杂度

- 查询、插入、删除：期望 $O(1)$
- 最坏仍可能退化

Pairing Heap

PBDS 堆最实用的用途是保存节点句柄，从而直接修改或删除堆内任意元素

```
template<class T, class C = less<T>>
using PH = __gnu_pbds::priority_queue<T, C, pairing_heap_tag>;
```

接口

普通堆之外主要使用

- `q.push(x)`：插入并返回节点句柄
- `q.modify(it, x)`：修改指定节点
- `q.erase(it)`：删除指定节点
- `q.join(b)`：合并 `b`，之后 `b` 为空

常见用法

可修改小根堆

```
using E = pair<i64, int>;
PH<E, greater<E>> q;
auto it = q.push({10, u});
q.modify(it, {5, u});
q.erase(it);
```

保存句柄

```
using Q = PH<pair<i64, int>, greater<pair<i64, int>>>;
Q q;
vector<Q::point_iterator> p(n);
p[u] = q.push({d[u], u});
q.modify(p[u], {nd, u});
```

合并两个堆

```
a.join(b);
```

注意事项

- 节点被 `erase()` 或 `pop()` 后，对应句柄失效
- 保存句柄时要额外维护节点当前是否仍在堆中
- Dijkstra 通常用重复入堆更简单，只有确实需要 *decrease-key* 时再使用 `modify`

复杂度

- `push`：$O(1)$
- `modify`、`erase`、`pop`：均摊 $O(\log n)$
- `join`：$O(1)$

结构选择

需求	优先选择
动态排名、第 k 小	`RBT`
按键 Splay、自调整访问	`SPT`
查询很多、修改极少	`OVT`
字符串前缀范围	`PT`
高性能哈希表	`HT`
可修改、可删除、可合并堆	`PH`

1.9 标准库与内置函数速查

IO 与格式化

```
cout << fixed << setprecision(9) << x; // 固定 9 位小数
cout << setprecision(9) << x;          // 9 位有效数字
cout << scientific << x;               // 科学计数法
cout << defaultfloat << x;             // 恢复默认浮点格式
cout << hex << x;                      // 十六进制
cout << oct << x;                      // 八进制
cout << dec << x;                      // 十进制
cout << setw(5) << x;                  // 本次输出宽度至少为 5
cout << setfill('0') << setw(5) << x;  // 不足宽度补 0
cout << boolalpha << b;                 // true / false
```

`setw()` 只影响紧随其后的一次输出，其他格式通常持续生效
整行读入：

```
getline(cin >> ws, s);
```

跳过前导空白后读取整行

查找

传统：

```
auto it = find(a.begin(), a.end(), x);
auto it = find_if(a.begin(), a.end(), pred);
auto it = find_if_not(a.begin(), a.end(), pred);
```

C++20 ranges：

```
auto it = ranges::find(a, x);
auto it = ranges::find_if(a, pred);
auto it = ranges::find_if_not(a, pred);
```

谓词示例：

```
auto it = find_if(a.begin(), a.end(), [](int x) {
    return x < 0;
});
auto it = ranges::find_if(a, [](int x) {
    return x < 0;
});
```

- `find`：第一个等于 `x` 的元素
- `find_if`：第一个满足谓词的元素
- `find_if_not`：第一个不满足谓词的元素

- 找不到时返回 `end()`
  复杂度：$O(n)$

计数

传统：

```
int c1 = count(a.begin(), a.end(), x);
int c2 = count_if(a.begin(), a.end(), pred);
```

ranges：

```
int c1 = ranges::count(a, x);
int c2 = ranges::count_if(a, pred);
```

谓词示例：

```
int c = ranges::count_if(a, [](int x) {
    return x % 2 == 0;
});
```

复杂度：$O(n)$

全部 / 任意 / 均不满足

传统：

```
bool a1 = all_of(a.begin(), a.end(), pred);
bool a2 = any_of(a.begin(), a.end(), pred);
bool a3 = none_of(a.begin(), a.end(), pred);
```

ranges：

```
bool a1 = ranges::all_of(a, pred);
bool a2 = ranges::any_of(a, pred);
bool a3 = ranges::none_of(a, pred);
```

示例：

```
bool ok = ranges::all_of(a, [](int x) {
    return x ≥ 0;
});
```

最坏复杂度：$O(n)$，支持短路

最值位置

传统：

```
auto mn = min_element(a.begin(), a.end());
auto mx = max_element(a.begin(), a.end());
auto [mn, mx] = minmax_element(a.begin(), a.end());
```

ranges：

```
auto mn = ranges::min_element(a);
auto mx = ranges::max_element(a);
auto [mn, mx] = ranges::minmax_element(a);
```

自定义比较：

```
auto cmp = [](int x, int y) {
    return abs(x) < abs(y);
};

auto it1 = min_element(a.begin(), a.end(), cmp);
auto it2 = ranges::min_element(a, cmp);
```

复杂度：$O(n)$

有序范围二分

传统：

```
auto l = lower_bound(a.begin(), a.end(), x); // 第一个 ≥ x
auto r = upper_bound(a.begin(), a.end(), x); // 第一个 > x
bool ok = binary_search(a.begin(), a.end(), x);
auto [l2, r2] = equal_range(a.begin(), a.end(), x);
```

ranges：

```
auto l = ranges::lower_bound(a, x);
auto r = ranges::upper_bound(a, x);
bool ok = ranges::binary_search(a, x);
auto [l2, r2] = ranges::equal_range(a, x);
```

降序数组：

```
ranges::sort(a, greater{});

auto it1 = lower_bound(a.begin(), a.end(), x, greater{});
auto it2 = ranges::lower_bound(a, x, greater{});
```

必须使用与排序相同的比较规则
复杂度：

- 比较次数 $O(\log n)$
- 非随机访问迭代器可能仍需要 $O(n)$ 次迭代器移动

partition_point

对于已经按谓词分成 `true...false` 的范围：
传统：

```
auto it = partition_point(a.begin(), a.end(), pred);
```

ranges：

```
auto it = ranges::partition_point(a, pred);
```

示例：

```
auto it = ranges::partition_point(a, [](int x) {
    return x < 10;
});
```

返回第一个使谓词为 `false` 的位置
复杂度：

- $O(\log n)$ 次谓词判断
- 非随机访问迭代器可能有 $O(n)$ 次移动

排序

传统：

```
sort(a.begin(), a.end());
stable_sort(a.begin(), a.end());
```

ranges：

```
ranges::sort(a);
ranges::stable_sort(a);
```

降序：

```
sort(a.begin(), a.end(), greater{});
ranges::sort(a, greater{});
```

自定义比较：

```
auto cmp = [](const auto &a, const auto &b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
};

sort(a.begin(), a.end(), cmp);
ranges::sort(a, cmp);
```

复杂度：

- `sort`：$O(n \log n)$
- `stable_sort`：通常 $O(n \log n)$，额外内存不足时实现可能退化到 $O(n \log^2 n)$
  `stable_sort` 保持等价元素原来的相对顺序

ranges 投影

`ranges` 可以直接按成员操作，无需 lambda：

```
struct Node {
    int x, y;
};
ranges::sort(a, {}, &Node::x);
auto it = ranges::lower_bound(a, v, {}, &Node::x);
auto it = ranges::min_element(a, {}, &Node::x);
```

降序：

```
ranges::sort(a, greater{}, &Node::x);
```

传统 STL 没有投影参数，需要 lambda：

```
sort(a.begin(), a.end(), [](const Node &a, const Node &b) {
    return a.x < b.x;
});
```

复杂度与对应算法相同

第 k 小

传统：

```
nth_element(a.begin(), a.begin() + k, a.end());
auto x = a[k];
```

ranges：

```
ranges::nth_element(a, a.begin() + k);
auto x = a[k];
```

自定义比较：

```
nth_element(a.begin(), a.begin() + k, a.end(), greater{});
ranges::nth_element(a, a.begin() + k, greater{});
```

此时 `a[k]` 为第 `k` 大
执行后只保证：

- `a[k]` 是完整排序后的第 `k` 个元素
- 左侧元素不大于它
- 右侧元素不小于它
- 两侧内部不保证有序
  平均复杂度：$O(n)$

前 k 小并排序

传统：

```
partial_sort(a.begin(), a.begin() + k, a.end());
```

ranges：

```
ranges::partial_sort(a, a.begin() + k);
```

降序：

```
partial_sort(a.begin(), a.begin() + k, a.end(), greater{});
ranges::partial_sort(a, a.begin() + k, greater{});
```

此时得到前 `k` 大并排序
复杂度：

$$O(n \log k)$$

next_permutation / prev_permutation

传统：

```
bool ok = next_permutation(a.begin(), a.end());
bool ok = prev_permutation(a.begin(), a.end());
```

ranges：

```
bool ok = ranges::next_permutation(a).found;
bool ok = ranges::prev_permutation(a).found;
```

自定义比较：

```
bool ok1 = next_permutation(a.begin(), a.end(), greater{});
bool ok2 = ranges::next_permutation(a, greater{}).found;
```

`next_permutation`：

- 成功时变为下一个字典序排列并返回 `true`
- 已是最大排列时变回最小排列并返回 `false`
  枚举全部不同排列：

```
sort(a.begin(), a.end());
do {
    // a
} while (next_permutation(a.begin(), a.end()));
```

ranges：

```
ranges::sort(a);
do {
    // a
} while (ranges::next_permutation(a).found);
```

即使有重复元素，从最小排列开始也只会枚举不同的值排列
单次复杂度：$O(n)$

is_permutation

传统：

```
bool ok = is_permutation(a.begin(), a.end(), b.begin(), b.end());
```

ranges：

```
bool ok = ranges::is_permutation(a, b);
```

自定义等价关系：

```
auto eq = [](int x, int y) {
    return abs(x) == abs(y);
};
bool ok1 = is_permutation(a.begin(), a.end(), b.begin(), b.end(), eq);
bool ok2 = ranges::is_permutation(a, b, eq);
```

最坏复杂度：$O(n^2)$

remove / remove_if

传统：

```
a.erase(remove(a.begin(), a.end(), x), a.end());
a.erase(remove_if(a.begin(), a.end(), [](int x) {
    return x < 0;
}), a.end());
```

C++20 可以直接：

```
erase(a, x);

erase_if(a, [](int x) {
    return x < 0;
});
```

`remove / remove_if` 本身只把保留元素移动到前面，并不会真正改变容器大小
对于 `vector/string/deque`，整体复杂度通常为 $O(n)$

rotate

传统：

```
rotate(a.begin(), a.begin() + k, a.end());
```

ranges：

```
ranges::rotate(a, a.begin() + k);
```

例如：

```
vector<int> a{1, 2, 3, 4, 5};
ranges::rotate(a, a.begin() + 2);
// 3 4 5 1 2
```

即把 `[0,k)` 搬到末尾
复杂度：$O(n)$

shift_left / shift_right

C++20：

```
shift_left(a.begin(), a.end(), k);
shift_right(a.begin(), a.end(), k);
```

移动后空出的元素值不应继续依赖
复杂度：$O(n)$

shuffle

传统：

```
shuffle(a.begin(), a.end(), rng);
```

ranges：

```
ranges::shuffle(a, rng);
```

例如：

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ranges::shuffle(a, rng);
```

复杂度：$O(n)$

transform

一元版本：
传统：

```
transform(a.begin(), a.end(), b.begin(), [](int x) {
    return x * x;
});
```

ranges：

```
ranges::transform(a, b.begin(), [](int x) {
    return x * x;
});
```

二元版本：
传统：

```
transform(a.begin(), a.end(), b.begin(), c.begin(),
    [](int x, int y) {
        return x + y;
    }
);
```

ranges：

```
ranges::transform(a, b, c.begin(),
    [](int x, int y) {
        return x + y;
    }
);
```

复杂度：$O(n)$

merge

两个输入必须有序
传统：

```
vector<int> c;
merge(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));
```

ranges：

```
vector<int> c;
ranges::merge(a, b, back_inserter(c));
```

自定义比较：

```
ranges::merge(a, b, back_inserter(c), greater{});
```

此时两个输入也必须已经按降序排列
复杂度：

$$O(n+m)$$

inplace_merge

若 `[first,mid)` 与 `[mid,last)` 已分别有序：
传统：

```
inplace_merge(a.begin(), a.begin() + k, a.end());
```

ranges：

```
ranges::inplace_merge(a, a.begin() + k);
```

自定义比较：

```
ranges::inplace_merge(a, a.begin() + k, greater{});
```

复杂度：

- 有足够辅助空间时约 $O(n)$
- 否则可达到 $O(n \log n)$

有序集合运算

两个输入都必须按相同比较方式排序
传统：

```
set_union(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));
set_intersection(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));
set_difference(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));
set_symmetric_difference(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));
```

ranges：

```
ranges::set_union(a, b, back_inserter(c));
ranges::set_intersection(a, b, back_inserter(c));
ranges::set_difference(a, b, back_inserter(c));
ranges::set_symmetric_difference(a, b, back_inserter(c));
```

降序：

```
ranges::set_intersection(a, b, back_inserter(c), greater{});
```

复杂度：$O(n+m)$
这些函数按有序多重集合语义处理重复元素

includes

判断有序多重集合 `a` 是否包含 `b`
传统：

```
bool ok = includes(a.begin(), a.end(), b.begin(), b.end());
```

ranges：

```
bool ok = ranges::includes(a, b);
```

自定义比较：

```
bool ok = ranges::includes(a, b, greater{});
```

两个范围也必须按 `greater{}` 排序
复杂度：$O(n+m)$

partial_sum

```
partial_sum(a.begin(), a.end(), pre.begin());
```

得到：

```
pre[i] = a[0] + ... + a[i]
```

常见 `pre[0] = 0`：

```
vector<i64> pre(a.size() + 1);
partial_sum(a.begin(), a.end(), pre.begin() + 1);
```

自定义运算：

```
partial_sum(a.begin(), a.end(), pre.begin(), multiplies{});
```

复杂度：$O(n)$

inclusive_scan / exclusive_scan

C++17：

```
inclusive_scan(a.begin(), a.end(), pre.begin());
exclusive_scan(a.begin(), a.end(), pre.begin(), 0LL);
```

`inclusive_scan`：

```
a0
a0+a1
a0+a1+a2
```

`exclusive_scan`：

```
init
init+a0
init+a0+a1
```

自定义运算：

```
inclusive_scan(a.begin(), a.end(), pre.begin(), multiplies{});
```

复杂度：$O(n)$

adjacent_difference

```
adjacent_difference(a.begin(), a.end(), d.begin());
```

得到：

```
d[0] = a[0]
d[i] = a[i] - a[i - 1]
```

自定义运算：

```
adjacent_difference(a.begin(), a.end(), d.begin(),
    [](i64 x, i64 y) {
        return x ^ y;
    }
);
```

复杂度：$O(n)$

inner_product

```
i64 ans = inner_product(
    a.begin(), a.end(),
    b.begin(),
    0LL
);
```

默认计算：

$$\sum_i a_i b_i$$

也可以自定义两个运算：

```
auto ans = inner_product(
    a.begin(), a.end(), b.begin(), 0LL,
    plus{}, multiplies{}
);
```

复杂度：$O(n)$

midpoint

C++20：

```
auto m = midpoint(l, r);
```

避免 `(l + r) / 2` 的加法溢出
整数不能整除时向第一个参数方向取整
复杂度：$O(1)$

lerp

C++20：

```
double x = lerp(a, b, t);
```

表示线性插值：

$$a + t(b - a)$$

复杂度：$O(1)$

size / ssize / empty / data

```
size(a);
ssize(a); // C++20，有符号
empty(a);
data(a);
```

也适用于普通数组：

```
int a[10];
int n = size(a);
```

通常均为 $O(1)$

map / set contains

C++20：

```
if (s.contains(x)) {}
if (mp.contains(key)) {}
```

传统：

```
if (s.find(x) != s.end()) {}
if (mp.find(key) != mp.end()) {}
```

如果还需要迭代器或对应值，直接使用 `find`
复杂度：

- `map/set`：$O(\log n)$
- `unordered_map/unordered_set`：均摊 $O(1)$，最坏 $O(n)$

map try_emplace

仅不存在时插入：

```
mp.try_emplace(key, value);
```

不存在则插入，存在则覆盖：

```
mp.insert_or_assign(key, value);
```

`mp[key]`：

```
auto &x = mp[key];
```

若 `key` 不存在会自动插入默认值
复杂度：

- `map`：$O(\log n)$
- `unordered_map`：均摊 $O(1)$

字符判断

```
isdigit(c)
isalpha(c)
isalnum(c)
islower(c)
isupper(c)
isspace(c)
```

转换：

```
tolower(c)
toupper(c)
```

严格写法：

```
isdigit((unsigned char)c);
tolower((unsigned char)c);
```

除 `EOF` 外，不应把负的 `char` 直接传入 `<cctype>` 函数
复杂度：$O(1)$

string 查找

```
s.find(t)
s.rfind(t)
s.find_first_of(t)
s.find_last_of(t)
s.find_first_not_of(t)
s.find_last_not_of(t)
```

找不到返回：

```
string::npos
```

例如：

```
if (s.find(t) != string::npos) {}
```

注意：

```
s.find_first_of("abc")
```

查找的是字符 `'a'/'b'/'c'` 中任意一个，不是子串 `"abc"`
单字符查询通常 $O(n)$，子串查找最坏可达到 $O(nm)$

string 前后缀

C++20：

```
s.starts_with("abc");
s.ends_with("xyz");
```

C++23：

```
s.contains("abc");
```

C++20 判断包含：

```
s.find("abc") != string::npos
```

`starts_with/ends_with` 复杂度 $O(m)$，其中 `m` 为模式长度

字符串与数字转换

```
int x = stoi(s);
i64 x = stoll(s);
u64 x = stoull(s);
double x = stod(s);

string s = to_string(x);
```

指定进制：

```
int x = stoi(s, nullptr, 16);
i64 x = stoll(s, nullptr, 2);
```

获得停止解析的位置：

```
size_t p;
int x = stoi(s, &p);
```

复杂度：$O(|s|)$

stringstream

```
stringstream ss(s);

int x;
while (ss >> x) {
    // x
}
```

混合类型：

```
string name;
int x;
ss >> name >> x;
```

完整解析一遍通常为 $O(|s|)$，常数明显高于直接字符处理

浮点取整

```
floor(x) // 向 -inf
ceil(x)  // 向 +inf
trunc(x) // 向 0
round(x) // 最近整数，正好一半时远离 0
```

例如：

```
x       floor  ceil  trunc  round
 1.7      1      2      1      2
-1.7     -2     -1     -1     -2
```

均视为 $O(1)$

atan2

```
ld ang = atan2(y, x);
```

返回 `(x,y)` 与正 `x` 轴的有向夹角，范围约为：

$$[-\pi, \pi]$$

比 `atan(y / x)` 更适合几何，因为可以自动处理象限和 `x = 0`
视为 $O(1)$

hypot

```
auto d = hypot(x, y);
```

等价目标：

$$\sqrt{x^2 + y^2}$$

C++17：

```
hypot(x, y, z);
```

相比直接写 `sqrt(x*x+y*y)` 更不容易产生中间溢出 / 下溢
视为 $O(1)$

fmod

```
auto r = fmod(x, y);
```

结果等价于：

$$x - \text{trunc}(x/y)y$$

结果符号跟 `x` 一致：

```
fmod(-5.0, 3.0) // -2
```

要求 `[0,y)`：

```
x = fmod(x, y);
if (x < 0) x += y;
```

视为 $O(1)$

isnan / isinf / isfinite / signbit

```
isnan(x)
isinf(x)
isfinite(x)
signbit(x)
```

分别判断 NaN、无穷、有限值和符号位
视为 $O(1)$

copysign

```
auto x = copysign(a, b);
```

绝对值来自 `a`，符号来自 `b`
视为 $O(1)$

fma

```
auto x = fma(a, b, c);
```

计算：

$$ab + c$$

在支持融合乘加时只进行一次最终舍入
视为 $O(1)$

π

C++20：

```
ld pi = numbers::pi_v<ld>;
```

传统：

```
ld pi = acosl(-1);
```

views::iota

C++20：

```
for (int x : views::iota(l, r)) {
    // l ⩽ x < r
}
```

创建视图 $O(1)$，遍历 $O(r-l)$

views::reverse

```
for (auto x : a | views::reverse) {
    // ...
}
```

创建视图 $O(1)$，完整遍历 $O(n)$
传统反向遍历：

```
for (auto it = a.rbegin(); it ≠ a.rend(); ++ it) {
    auto x = *it;
}
```

views::filter

```
for (auto x : a | views::filter([](int x) {
    return x % 2 == 0;
})) {
    // 偶数
}
```

完整遍历最坏 $O(n)$ 次谓词判断
传统：

```
for (auto x : a) {
    if (x % 2 == 0) {
        // ...
    }
}
```

views::transform

```
for (auto x : a | views::transform([](int x) {
    return x * x;
})) {
    // x 为平方后的值
}
```

惰性计算，创建视图 $O(1)$，完整遍历 $O(n)$ 次转换
传统通常直接：

```
for (auto x : a) {
    auto y = x * x;
}
```

pair / tuple 解包

```
auto [x, y] = p;
```

赋值：

```
tie(x, y) = p;
```

忽略部分：

```
tie(x, ignore, z) = t;
```

均为常数级操作，不计成员自身复制 / 移动成本

exchange

```
auto old = exchange(x, y);
```

等价于：

```
auto old = x;
x = y;
```

复杂度取决于对象自身复制 / 移动，标量类型为 $O(1)$

1.10 常用工具函数

```
// ===== 基础运算 =====
template<class T>
T power(T a, i64 b) {
    assert(b ⩾ 0);
    T res{ 1 };
    while (b) {
        if (b & 1) res *= a;
        b >>= 1;
        if (b) a *= a;
    }
    return res;
}
template<class T>
T sqr(T x) { return x * x; }
// ===== 整数平方根与立方根 =====
u64 isqrt(u64 n) {
    u64 x{ u64(sqrt(ld(n))) };
    while (x > 0 && x > n / x) -- x;
    while (x + 1 ⩽ n / (x + 1)) ++ x;
    return x;
}
u64 icbrt(u64 n) {
    u64 x{ u64(cbrt(ld(n))) };
    auto le = [&](u64 v) { return v == 0 || v ⩽ n / v / v; };
    while (!le(x)) -- x;
    while (le(x + 1)) ++ x;
    return x;
}
// ===== 整数根扩展：不需要可整段删除 =====
u64 ceil_sqrt(u64 n) {
    u64 x{ isqrt(n) };
    return x + (x * x ≠ n);
}
u64 ceil_cbrt(u64 n) {
    u64 x{ icbrt(n) };
    return x + (x * x * x ≠ n);
}
bool is_square(u64 n) {
    u64 x{ isqrt(n) };
    return x * x == n;
```

```
}
bool is_cube(u64 n) {
    u64 x{ icbrt(n) };
    return x * x * x == n;
}
// ===== 整数对数 =====
// 要求 x > 0
int floor_log2(u64 x) {
    assert(x > 0);
    return bit_width(x) - 1;
}
int ceil_log2(u64 x) {
    assert(x > 0);
    return bit_width(x - 1);
}
// 要求 base ⩾ 2, x ⩾ 1
int floor_log(u64 x, u64 base) {
    assert(base ⩾ 2 && x ⩾ 1);
    int k{ 0 };
    while (x ⩾ base) {
        x /= base;
        ++ k;
    }
    return k;
}
int ceil_log(u64 x, u64 base) {
    assert(base ⩾ 2 && x ⩾ 1);
    return x == 1 ? 0 : floor_log(x - 1, base) + 1;
}
// ===== 有符号整除与取模 =====
// 要求 b ≠ 0, 且 a / b 可由 T 表示
template<class T>
T floor_div(T a, T b) {
    static_assert(is_integral_v<T> && is_signed_v<T>);
    assert(b ≠ 0 && !(a == numeric_limits<T>::min() && b == T(-1)));
    T q{ a / b }, r{ a % b };
    if (r ≠ 0 && ((r > 0) ≠ (b > 0))) -- q;
    return q;
}
template<class T>
T ceil_div(T a, T b) {
    static_assert(is_integral_v<T> && is_signed_v<T>);
    assert(b ≠ 0 && !(a == numeric_limits<T>::min() && b == T(-1)));
    T q{ a / b }, r{ a % b };
    if (r ≠ 0 && ((r > 0) == (b > 0))) ++ q;
    return q;
}
template<class T>
T floor_mod(T a, T b) {
    static_assert(is_integral_v<T> && is_signed_v<T>);
    assert(b ≠ 0 && !(a == numeric_limits<T>::min() && b == T(-1)));
    T r{ a % b };
    if (r ≠ 0 && ((r > 0) ≠ (b > 0))) r += b;
    return r;
}
// ===== 比较与更新 =====
template<class T, class U>
bool chmin(T &a, const U &b) {
    if (b < a) {
        a = b;
        return true;
    }
    return false;
}
template<class T, class U>
bool chmax(T &a, const U &b) {
    if (a < b) {
        a = b;
        return true;
    }
    return false;
```

```cpp
}
template<class T>
int sign(const T &x) {
    return (T{} < x) - (x < T{});
}
// ===== 二进制辅助：不需要可整段删除 =====
// 返回 ≥ x 的最小 2 的幂；溢出返回 0
u64 ceil_pow2(u64 x) {
    if (x > (1ULL << 63)) return 0;
    return bit_ceil(x);
}
u64 lowbit(u64 x) {
    return x & -x;
}
// ===== 安全绝对差：不需要可直接删除 =====
// 支持完整 i64 范围，结果为 u64
u64 abs_diff(i64 a, i64 b) {
    return a < b ? u64(b) - u64(a) : u64(a) - u64(b);
}
// ===== 整数幂比较与通用 k 次根：不需要可整段删除 =====
// 返回 a^b ≤ lim，避免乘法溢出，要求 b ≥ 0
bool power_le(u64 a, int b, u64 lim) {
    assert(b ≥ 0);
    u64 p{ 1 };
    for (int i = 0; i < b; ++ i) {
        if (a && p > lim / a) return false;
        p *= a;
    }
    return p ≤ lim;
}
// 要求 1 ≤ k ≤ 64，返回 floor(n^(1/k))
u64 kth_root(u64 n, int k) {
    assert(1 ≤ k && k ≤ 64);
    if (k == 1 || n ≤ 1) return n;
    u64 l{ 1 }, r{ 1ULL << ((64 + k - 1) / k) };
    while (r - l > 1) {
        u64 mid{ l + (r - l) / 2 };
        if (power_le(mid, k, n)) l = mid;
        else r = mid;
    }
    return l;
}
```

接口说明

函数	功能
power(a,b)	泛型非负整数快速幂
sqr(x)	求平方
isqrt / icbrt	整数根下取整
ceil_sqrt / ceil_cbrt	整数根上取整
is_square / is_cube	判断完全平方 / 立方
floor_log2 / ceil_log2	底数为 2 的整数对数
floor_log / ceil_log	任意整数底数的整数对数
floor_div / ceil_div	数学意义的向下 / 向上取整除法
floor_mod	与下取整除法一致的余数
chmin / chmax	更新极值，返回是否更新
sign	返回 (-1,0,1)
ceil_pow2	向上取整到 2 的幂
lowbit	最低有效 1 对应的值
abs_diff	有符号 64 位整数安全绝对差
power_le	安全判断 $a^b \le lim$
kth_root	精确整数 k 次根下取整

1.11 RNG 随机数生成器

随机数极简版

仅随机引擎

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
```

已经可以直接：

```
u64 x = rng();
shuffle(a.begin(), a.end(), rng);
int v = uniform_int_distribution<int>(l, r)(rng); // [l, r]
double y = uniform_real_distribution<double>(l, r)(rng); // [l, r)
```

⚠ 异或哈希使用单独的 mt19937 几乎必定能被卡掉，需要添加非线性映射，如开平方或使用 `splitmix64`

常用整数版

如果题目频繁生成区间随机整数，再补一个 `randint()`：

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
template<class T>
T randint(T l, T r) { return uniform_int_distribution<T>(l, r)(rng); }
```

使用：

```
int x = randint(1, n);              // [1, n]
i64 y = randint<i64>(-1E18, 1E18);
```

若主要随机下标，可以额外补：

```
template<class T>
T randrange(T n) { return randint<T>(0, n - 1); }
```

```
int p = randrange(n); // [0, n)
```

Xorshift64* 极短高速版

若连 `<random>` 的分布器都不需要，只想获得高速的随机 `u64`，可以使用：

```
u64 rnd() {
    static u64 x = u64(chrono::steady_clock::now().time_since_epoch().count()) | 1;
    x ^= x >> 12;
    x ^= x << 25;
    x ^= x >> 27;
    return x * 2685821657736338717ULL;
}
```

使用：

```
u64 x = rnd();
int p = rnd() % n;
int v = i64(rnd() % (u64(i64(r) - l) + 1)) + l;
```

以上下标示例要求 `n > 0`；区间示例使用 `int l, r` 且 `l ≤ r`，先提升到宽整数计算区间长度，避免完整 int 区间溢出。整数分布同样要求 `l ≤ r`，`randrange(n)` 要求 `n > 0`；浮点分布要求有限端点且 `l < r`。
状态必须非零，因此初始化时使用 `| 1`
这个版本状态只有一个 `u64`，代码短、速度快，适合：

- Treap 随机优先级
- 随机选取 pivot / 起点
- 随机化搜索
- Pollard--Rho 等随机化算法中的随机扰动

- 不要求严格均匀分布的随机下标
  `rnd() % n` 存在轻微的模偏差。竞赛随机化通常不受影响，但数据生成、概率实验或明确要求均匀随机时，应使用前面的 `uniform_int_distribution`

固定种子

需要复现随机结果时，把时间种子直接换为常量：

```
mt19937_64 rng(114514);
```

或：

```
u64 rnd() {
    static u64 x = 114514;
    x ^= x >> 12;
    x ^= x << 25;
    x ^= x >> 27;
    return x * 2685821657736338717ULL;
}
```

随机数完全体

```
struct RNG {
    using result_type = mt19937_64::result_type;
    mt19937_64 gen;
    static constexpr result_type min() { return mt19937_64::min(); }
    static constexpr result_type max() { return mt19937_64::max(); }
    static u64 splitmix64(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    static u64 seed_now() {
        u64 x{ u64(chrono::steady_clock::now().time_since_epoch().count()) };
        random_device rd;
        x ^= u64(rd()) << 32;
        x ^= u64(rd());
        x ^= u64(reinterpret_cast<uintptr_t>(&x));
        return splitmix64(x);
    }
    RNG() : gen(seed_now()) {}
    explicit RNG(u64 seed) : gen(splitmix64(seed)) {}
    // ===== 基础随机：随机化算法通常只需保留到 shuffle() =====
    result_type operator()() { return gen(); }
    void seed() { gen.seed(seed_now()); }
    void seed(u64 s) { gen.seed(splitmix64(s)); }
    template<class T>
    T randint(T l, T r) {
        static_assert(is_integral_v<T> && !is_same_v<T, bool>);
        assert(l <= r);
        return uniform_int_distribution<T>(l, r)(gen);
    }
    template<class T>
    T randint(T r) { return randint<T>(T{}, r); }
    template<class T>
    T randrange(T l, T r) { assert(l < r); return randint<T>(l, r - 1); }
    template<class T>
    T randrange(T r) { return randrange<T>(T{}, r); }
    template<class T = double>
    T real(T l = T{}, T r = T{1}) {
        static_assert(is_floating_point_v<T>);
        assert(l < r);
        return uniform_real_distribution<T>(l, r)(gen);
    }
    template<class It>
    void shuffle(It first, It last) { std::shuffle(first, last, gen); }
    // ===== 对拍 / 数据生成扩展 =====
    u32 next_u32() { return u32(gen()); }
```

```
    u64 next_u64() { return gen(); }
    bool bit() { return gen() & 1ULL; }
    template<class T = double>
    T normal(T mean = T{}, T stddev = T{1}) {
        static_assert(is_floating_point_v<T>);
        assert(stddev > T{});
        return normal_distribution<T>(mean, stddev)(gen);
    }
    bool bernoulli(double p) { assert(0.0 ⩽ p && p ⩽ 1.0); return bernoulli_distribution(p)(gen); }
    template<class C>
    void shuffle(C &a) { shuffle(begin(a), end(a)); }
    template<class C>
    decltype(auto) choice(C &a) { assert(!a.empty()); return a[randrange<size_t>(a.size())]; }
    template<class C>
    decltype(auto) choice(const C &a) { assert(!a.empty()); return a[randrange<size_t>(a.size())]; }
    template<class C>
    C sample(C a, int k) {
        assert(0 ⩽ k && k ⩽ int(a.size()));
        shuffle(a);
        a.resize(k);
        return a;
    }
    vector<int> perm(int n, int first = 0) {
        assert(n ⩾ 0);
        vector<int> p(n);
        ranges::iota(p, first);
        shuffle(p);
        return p;
    }
    template<class T = int>
    vector<T> vec(int n, T l, T r) {
        assert(n ⩾ 0 && l ⩽ r);
        vector<T> a(n);
        for (T &x : a) x = randint<T>(l, r);
        return a;
    }
};
inline RNG rng;
```

成员变量

- `gen`：底层 `mt19937_64` 随机数引擎
- `rng`：全局随机数生成器，可直接使用

基础接口

- `RNG()`：使用时间、`random_device` 和地址信息生成随机种子
- `RNG(seed)`：指定种子，方便对拍复现
- `seed()`：重新随机播种
- `seed(s)`：使用指定种子重新播种
- `rng()`：返回一个完整的 `mt19937_64::result_type` 随机数
- `randint<T>(l, r)`：均匀生成整数 `[l,r]`
- `randint<T>(r)`：均匀生成整数 `[0,r]`
- `randrange<T>(l, r)`：均匀生成整数 `[l,r)`
- `randrange<T>(r)`：均匀生成整数 `[0,r)`
- `real<T>(l, r)`：均匀生成浮点数 `[l,r)`，默认 `[0,1)`
- `shuffle(first,last)`：随机打乱 `[first,last)`

  `RNG` 补全了 `result_type`、`min()`、`max()` 和 `operator()`，因此本身就是一个 UniformRandomBitGenerator，可以直接：

```
uniform_int_distribution<int> d(1, 100);
int x = d(rng);
```

对拍 / 数据生成接口

- `next_u32()`：返回随机 `u32`
- `next_u64()`：返回随机 `u64`
- `bit()`：随机返回 `0/1`
- `normal(mean,stddev)`：正态分布
- `bernoulli(p)`：以概率 `p` 返回 `true`

- `shuffle(a)`：随机打乱整个容器
- `choice(a)`：随机选择一个元素，返回引用
- `sample(a,k)`：无放回随机选择 `k` 个元素，原容器按值传入，不修改调用者
- `perm(n,first)`：生成 `[first,first+n)` 的随机排列
- `vec<T>(n,l,r)`：生成长度为 `n`、元素位于 `[l,r]` 的随机数组

注意事项

- `randint(l,r)` 是**闭区间** `[l,r]`；`randrange(l,r)` 是**左闭右开** `[l,r)`。平时涉及下标时优先写 `randrange(n)`，不容易多出一个 `n`
- `uniform_int_distribution` 主要用于标准整数类型；不要直接拿它生成 `i128/u128`
- `real()`、`normal()` 只能使用 `float`、`double`、`long double`
- `choice()` 要求容器非空；`sample(a,k)` 要求 $0 \leqslant k \leqslant a.size()$
- `sample()` 要求容器支持随机访问、`shuffle` 和 `resize`，最常用于 `vector` / `string`
- 固定 `RNG rng(123);` 可以稳定复现**底层 `mt19937_64` 序列**
- 如果比赛环境的 `random_device` 有问题，可以删掉 `random_device rd` 以及两行 `rd()`，仅使用时钟和地址信息

常见用法

随机整数与下标

```
int x = rng.randint(-10, 10); // [-10, 10]
int p = rng.randrange(n);     // [0, n)
```

随机浮点数

```
double x = rng.real();          // [0, 1)
double y = rng.real(-1.0, 1.0); // [-1, 1)
```

随机排列

```
vector<int> p = rng.perm(n);
```

若需要 `1..n`：

```
vector<int> p = rng.perm(n, 1);
```

对拍固定种子

```
RNG rng(114514);
auto a = rng.vec<int>(n, -100, 100);
```

出现错误后保留种子即可复现

复杂度

操作	时间复杂度
`rng()` / `randint()` / `randrange()` / `real()` / `bit()` / `normal()` / `bernoulli()` / `choice()`	期望 $O(1)$
`shuffle(a)` / `sample(a,k)` / `perm(n)` / `vec(n,l,r)`	$O(n)$

空间复杂度：`RNG` 本身为 `mt19937_64` 的固定状态空间；`sample`、`perm`、`vec` 额外使用 $O(n)$

1.12 Cmake

```
cmake_minimum_required(VERSION 3.20)
project(Contest LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS ON)

# ===== 默认：O2 优化，关闭 Warning =====
add_compile_options(-O2 -w)

# ===== 调试 RE：使用时建议将 -O2 改为 -O0 =====
# add_compile_options(-g3 -fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all)
```

```
# add_link_options(-fsanitize=address,undefined)

# ===== STL 越界、非法迭代器等检查（GCC/libstdc++）=====
# add_compile_options(-D_GLIBCXX_DEBUG)

file(GLOB SOURCES CONFIGURE_DEPENDS "*.cpp")

foreach(source IN LISTS SOURCES)
    get_filename_component(name "${source}" NAME_WE)
    add_executable(${name} "${source}")
endforeach()
```

编译选项速查

参数	作用
`-O2`	开启常用编译优化
`-w`	关闭所有编译 Warning
`-O0`	关闭优化，方便调试
`-g3`	生成调试信息，帮助定位源码行号
`-fsanitize=address`	检测越界访问、Use After Free 等内存错误
`-fsanitize=undefined`	检测有符号整数溢出、非法移位等未定义行为
`-fno-omit-frame-pointer`	提高错误调用栈的可读性
`-fno-sanitize-recover=all`	检测到运行时错误后立即终止
`-D_GLIBCXX_DEBUG`	检测部分 STL 容器和迭代器的非法操作

推荐使用方式：

- 出现 RE / WA 且怀疑越界或 UB：启用 Sanitizer 两行，将 `-O2` 改为 `-O0`
- 怀疑 STL 使用错误：额外启用 `_GLIBCXX_DEBUG`，但可能显著降低运行速度

注意 Sanitizer 需要编译器和运行时支持，部分 Windows MinGW 工具链可能不支持 AddressSanitizer

2. 基础工具与常见技巧

2.0 trick

2.0.1 数论与代数

LN除以M」模10007计算

$$\lfloor \frac{N}{M} \rfloor \bmod 10007 = \left\lfloor \frac{N \bmod (10007 \times M)}{M} \right\rfloor$$

一个数如果可以不断加k，然后模m，可以到达的数的集合该如何表示呢？

一个数如果可以不断加k，然后模m，可以到达的数的集合该如何表示呢？
可以按照原数取余gcd(k, m)的值，不断加上gcd(k, m)来得到，表达出所有可以达到的数

一个整数如果是11的倍数那么一定满足奇数位偶数位交替相减得到的结果是11的倍数

分析

如果一个整数N表示为

$$N = a_n 10^n + \cdots + a_1 10^1 + a_0 10^0$$

那么在 $\bmod\ 11$意义下

$$N \equiv a_n (-1)^n + \cdots + a_1 (-1)^1 + a_0 (1) \pmod{11}$$

与质数相关的复杂度欺骗问题

适用问题：
给定两个正整数$L, R (1 < L < R < 10^{18})$
寻找满足以下两个条件的整数对 (x, y)：

- $L \le x < y \le R$
- $\gcd(x, y) = 1$
  在所有满足条件的整数对中，求 $y - x$ 可能取得的最大值。

两个数都是质数一定满足条件，又因为要使$y-x$取得最大值，因此直接从端点向内枚举即可。
代码按距离递减枚举，所以找到的第一对互质数是最优解；但“质数间隔很短”不是对所有输入的运行时间证明。最坏枚举上界仍可保守写为 O((R-L)^2 log R)，不能把经验快当作通用复杂度保证。
代码：

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
signed main() {
    int l,r;cin>>l>>r;
    for(int i=r-l;i>=0;i--) {
        for(int j=l;j<=r-i;j++) {
            if(__gcd(j,j+i)==1){cout<<i<<endl;return 0;}
        }
    }
}
```

光速幂用平方根分块预处理

适用场景

当需要大量计算 $a^b \bmod p$，且底数和指数范围较小或可控时，可以用根号分块预处理加速。

核心思路

设块长为 B，将 $b = qB + r$，则 $a^b = (a^B)^q \cdot a^r$。预处理 a^r 和 $(a^B)^q$，单次查询可降到 $O(1)$ 或接近 $O(1)$。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

预处理空间可能很大，只有在查询次数足够多时才划算。

减法变换尝试扩展为取模变换

适用场景

如果操作允许反复执行 $x \leftarrow x - y$，且 y 不变或变化较慢，可以考虑直接扩展为 $x \leftarrow x \bmod y$。

核心思路

连续减去 y 直到小于 y 的结果等价于取模。把多次重复操作压缩为一次取模，常能把复杂度从值域级降低到对数级，类似欧几里得算法。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

只有当中间每次减法都不影响其他状态时才能合并成取模。

同余与整除等价关系

同余 $a \equiv b \pmod{m}$ 的数学等价条件就是 m 能整除 $a - b$。换句话说，m 就是 $|a - b|$ 的因数。

因数倍数 DP 写成迪利克雷卷积

适用场景

当 DP 转移和因数、倍数、gcd、lcm 有关时，可以尝试写成数论函数卷积。

核心思路

把转移式整理成 $f(n) = \sum_{d|n} g(d)h(n/d)$ 或类似形式，即迪利克雷卷积。之后可以利用积性、莫比乌斯反演、筛法或 Dirichlet 前缀和优化。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

先判断函数是否积性。不是所有因数和都能直接按质因子独立拆分。

多项式整数根整除常数项

适用场景

需要寻找整系数多项式的整数根时，可以只枚举常数项的因数。

核心思路

若 $P(x)=a_nx^n+\cdots+a_0$ 且整数 r 是根，则 $r\mid a_0$。因此候选根数量由常数项因数个数控制。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

若常数项为 0，则 0 是根，需要先除掉 x 的因子后再枚举。

完全平方数用质因数奇偶性表示

适用场景

判断若干数乘积是否为完全平方数时，可以看质因数指数的奇偶性。

核心思路

根据唯一分解定理，一个数是完全平方数当且仅当所有质因数指数都是偶数。于是可把每个数映射成一个 01 向量，表示哪些质因数指数为奇数，乘法对应异或。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

大质因数需要正确保留。若值域很大，可以用随机哈希模拟奇偶向量，但要接受概率正确性。

校验补充

直接使用质因数指数奇偶性要求输入为正整数。0 的乘积单独处理；负数还要维护负号出现次数的奇偶性。

小于10^5的数的最大约数个数

在小于 $ 10^5 $ 的数中，一个数最多有128个因子。
在小于 $ 10^6 $ 的数中，一个数最多有240个因子。
在小于 $ 10^7 $ 的数中，一个数最多有448个因子。
在小于 $ 10^8 $ 的数中，一个数最多有768个因子。
在小于 $ 10^9 $ 的数中，一个数最多有1344个因子。
在小于 $ 10^{10} $ 的数中，一个数最多有2304个因子。
在小于 $ 10^{12} $ 的数中，一个数最多有6720个因子。
在小于 $ 10^{15} $ 的数中，一个数最多有26880个因子。
在小于 $ 10^{18} $ 的数中，一个数最多有103680个因子。

斐波那契数列常用二级结论

适用场景

遇到斐波那契数列相关的计数、整除、区间加、递推表示时，可以先考虑斐波那契的常用恒等式，而不是只按定义递推。

常用结论

设 $F_0=0,F_1=1$，则有：

$$\gcd(F_n,F_m)=F_{\gcd(n,m)}.$$

$$F_{n+m}=F_{n+1}F_m+F_nF_{m-1}.$$

$$F_{2n}=F_{n+1}^2-F_{n-1}^2.$$

$$F_{2n+1}=F_{n+1}^2+F_n^2.$$

$$\sum_{i=1}^{n} F_i = F_{n+2} - 1.$$

对于广义斐波那契数列 $G_0 = a, G_1 = b, G_n = G_{n-1} + G_{n-2}$，也可以用普通斐波那契表示：

$$G_n = F_{n-2}G_1 + F_{n-1}G_2, \quad n \geq 2.$$

常见用途

- 枚举 n，把 $F_n a + F_{n+1} b = k$ 转成线性丢番图方程。
- 区间加斐波那契数列时，把 F_{i-l+1} 拆成 F_i 和 F_{i+1} 的线性组合，在线段树上维护两个懒标记。

易错点

- 不同题目对 F_0, F_1 的定义可能不同。
- 使用负下标或平移公式时要先统一定义。
- 取模时除法、逆元、矩阵形式都要按模数处理。

构建三角形

给定正整数直角边 n：当 n<3 时，无法构造正整数边的直角三角形；当 n>=3 时可以。下式中的 x 就是 n。一般三角形不受此限制，例如 1、1、1。

其中当n等于奇数时，另外两条边的值分别为
(x*x-1)/2 和 (x*x+1)/2
当n为偶数时，另外两条边的值分别为
x*x/4-1 和 x*x/4+1

约数小诈骗

一个数的约数个数是奇数等价于这个数是完全平方数。

简单证明

对于任意正整数 n，若 d 为其约数，则 $\frac{n}{d}$ 也是其约数。

而完全平方数有特例 $d = \sqrt{n}$ 时，$d = \frac{n}{d}$。

除此之外，约数成对出现。

线性筛minp加速质因数分解

需要对 $q(q \leq 10^5)$ 个小于等于 $V(V \leq 10^7)$ 的数进行质因数分解，可使用线性筛求出 $minp$，对 x 质因数分解时只需不断将 x 除以 $minp_x$，相较于朴素分解的 $O(q\sqrt{V})$，该做法可将复杂度降至 $O(V + q \log V)$

计算n!的末尾0的个数

公式如下：

$$ans = \lfloor \frac{n}{5} \rfloor + \lfloor \frac{n}{25} \rfloor + \lfloor \frac{n}{125} \rfloor + ...$$

计算 $n!$ 的末尾 0 的个数，实际上就是计算 $ 1 2 3 ... n - 1 * n $ 中因子 10 的个数

$10 = 2 * 5$，且在 $n!$ 中，因子 2 的个数远远多于因子 5 的个数，所以只需要统计 $n!$ 中所有因子 5 的个数即可

已知 1 到 n 中 5 的倍数的个数为 $\lfloor \frac{n}{5} \rfloor$，但如果直接将 $\lfloor \frac{n}{5} \rfloor$ 当作答案会漏掉 5 的个数
因为 $25 = 5 * 5$ 贡献了 2 个 5，$125 = 5 * 5 * 5$ 贡献了 3 个 5 ...

可以通过重复计数来补上这些漏掉的 5

- $\lfloor \frac{n}{5} \rfloor$ 统计所有 5 的倍数贡献的第一个 5
- $\lfloor \frac{n}{25} \rfloor$ 统计所有 25 的倍数贡献的第二个 5
- $\lfloor \frac{n}{125} \rfloor$ 统计所有 125 的倍数贡献的第三个 5
- ……

这样一来就可以完整地统计出 $n!$ 中所有因子 5 的个数了

连续上取整可以合并分母

适用场景

表达式中出现连续的上取整除法时，可以尝试把它们合并。

核心思路

常用恒等式为 $\left\lceil \frac{\lceil a/b \rceil}{c} \right\rceil = \lceil \frac{a}{bc} \rceil$。用整数形式写为 $\lfloor (a-1)/(bc) \rfloor + 1$。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该式适用于正整数除法。存在负数时 C/C++ 的除法取整方向会带来问题。

2.0.2 组合计数、概率与随机化

Ferrers 转化

Ferrers 转化
所有联通子图的 mex 和转化

$$\sum_{S}\operatorname{mex}(S)=\sum_{k=0}^{n-1}\sum_{S}[\operatorname{mex}(S)>k],\ \operatorname{mex}(S)\in[0,n]$$

枚举联通子图，计算其 mex 和再累加 ↔ 枚举其 mex 的范围，计算其大于 mex 的联通子图个数再累加。

LGV 引理处理不交路径计数

适用场景

在 DAG 中统计多条两两点不交路径的方案数时，可以考虑 Lindström–Gessel–Viennot 引理。

核心思路

先计算每个起点到每个终点的路径数 $a_{i,j}$，构成矩阵 A。在满足起终点顺序不交条件时，$\det(A)$ 就给出不交路径组的带符号计数，很多平面图情形符号一致。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

LGV 要求图是 DAG，且需要确认不交路径的符号抵消条件。

O(1)判断组合数奇偶性

$\binom{n}{m}$为奇数$\Leftrightarrow(m\&(n-m))=0$

不少于一半类条件用随机候选

适用场景

当题目保证答案对象在某个集合中出现比例不低，例如不少于一半、不少于三分之一，可以随机抽取候选再检验。

核心思路

随机选取若干个位置作为候选。若真实答案覆盖比例为 p，抽 t 次都没抽到的概率为 $(1-p)^t$。当 $p\geq 1/2$ 时，几十次随机通常就能把失败概率压得很低。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

随机只负责找候选，最后必须有确定性检验候选是否合法。不要把随机结果直接当答案。

再一个矩形统计覆盖特定单元格的子正方形数量

对于一个长度为 N 的线段，包含坐标 i 且长度为 k 的子线段，其合法左端点的方案数为：

$$X_k=\min(k,i,N-i+1,N-k+1)$$

这四个限制分别代表：线段长度本身的限制、左边界距离、右边界距离、以及滑动空间的收缩限制。
同理，列方向上的方案数为：

$$Y_k=\min(k,j,M-j+1,M-k+1)$$

根据乘法原理，边长为 k 且包含 (i,j) 的正方形总数为 $X_k\times Y_k$。因此，包含该单元格的所有正方形总数为：

$$\sum_{k=1}^{\min(N,M)}(X_k\times Y_k)$$

公式推导基于 1-based 索引（即行号 $1 \sim N$，列号 $1 \sim M$）。如果题目是 0-based，需要先将坐标 $+1$ 转化为 1-based 再套用公式。
codeforces有原题目，还在寻找中

判断可重集合是否相等

定义

可重集合（Multiset）：是指允许同一元素出现多次的集合。

问题描述

> 给定数组 a，多次询问：
>
> 判断区间 $[L_1, R_1]$ 与 $[L_2, R_2]$ 所对应的可重集合是否相等。

设每次询问的两个可重集合分别为 S 和 T。

朴素做法

直接的做法包括：

- 将两个区间内的元素排序逐一比较。
- 使用哈希表统计元素出现次数。

设 $|S| \leq |T|$，以上方法，单次询问的时间复杂度为：

$$\mathcal{O}(|S| \cdot \log |S|)$$

随机化做法（随机赋权哈希）

定理

如果两个可重集合相等，那么两个集合的元素和相等。

那么通过比较两个集合的元素和是否能完成问题？

反例

$$S = \{2, 4\}, \quad T = \{3, 3\}$$

改进思路

为每一种元素 x 随机赋予一个权值 $w(x)$，定义集合的哈希值为：

$$H(S) = \sum_{x \in S} w(x)$$

判断两个集合是否相等，只需比较：$H(S)$ 和 $H(T)$。

正确性分析

- 若 $S = T$，则显然有 $H(S) = H(T)$。
- 若 $S \neq T$，设 $cnt_S(x), cnt_T(x)$ 分别表示元素 x 在集合中的出现次数，则：

$$\sum (cnt_S(x) - cnt_T(x)) \cdot w(x) = 0$$

> 这是一个关于随机变量 $w(x)$ 的线性组合。
>
> 在 $w(x)$ 独立随机且值域足够大的情况下，上式成立的概率极低。

其他哈希形式

也可以定义：

$$H(S) = \prod_{x \in S} w(x)$$

同样可以用于判定可重集合相等（本质仍为随机映射）。

区间查询实现

- 不带修，使用前缀和维护，单次查询时间复杂度：$\mathcal{O}(1)$ 。
- 待修，使用数据结构（如树状数组/线段树）维护区间和，单次查询时间复杂度：$\mathcal{O}(\log n)$ 。

✦ 扩展问题

> 给定数组 a，多次询问：
>
> 判断区间 $[L_1, R_1]$ 中所有元素加上 k 后，是否与区间 $[L_2, R_2]$ 的可重集合相等。

设常数 B，定义哈希函数：

$$H(S) = \sum B^{a_i}$$

则有：

$$H([L, R]) = \sum_{i=L}^{R} B^{a_i}$$

区间整体加上 k 后：

$$H'([L, R]) = \sum_{i=L}^{R} B^{a_i+k} = B^k \cdot \sum_{i=L}^{R} B^{a_i} = B^k \cdot H([L, R])$$

所以仅需要判断 $B^k \cdot H([L_1, R_1])$ 与 $H([L_2, R_2])$ 的关系即可

势能法求期望步数

适用场景

随机过程中如果直接列期望方程很复杂，可以尝试构造势能函数。

核心思路

构造函数 $\Phi(state)$，使每一步操作满足期望上 Φ 减少一个常数 c。那么到终态的期望步数就是 $(\Phi(start) - \Phi(end))/c$。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

势能函数是构造出来的，不一定有明显物理意义。关键是验证每一步的期望变化恒定。

校验补充

还需要停止时间与势能满足可选停止/可积性等条件，例如有限状态、终止时间期望有限且增量有界。仅有每步期望势能减少 c，不足以无条件交换无限求和与期望。

在精度允许时舍弃高阶小贡献

适用场景

在期望、概率、浮点 DP 中，如果题目允许一定误差，并且某些高阶状态的贡献会快速衰减，可以只保留前若干个主要状态。

核心思路

设 DP 的某一维理论上很大，但随着下标增大，概率或贡献迅速变小。当这些被舍弃部分对最终答案的影响小于允许误差时，可以设一个阈值 K，只计算 $0 \sim K$ 的状态。

例如某些合并过程里，大数值出现概率极低。若相对误差只要求 10^{-4}，则可以只保留前几十个状态，然后用矩阵快速幂处理很大的 n。

使用条件

- 题目允许误差，而不是要求精确取模。
- 被舍弃项有明确的衰减趋势。
- 阈值 K 需要足够保守，并经过误差分析或经验验证。

易错点

- 取模计数题不能随意舍弃贡献。
- 需要注意绝对误差和相对误差的区别。
- 阈值过小会 WA，过大可能 TLE，应结合数据范围选择。

大 k 的第 k 小查询用随机二分

适用场景

如果有一个 $O(k \operatorname{polylog} n)$ 的求第 k 小算法，但 k 可能很大，可以尝试随机二分值域。

核心思路

随机选择候选值或随机抽样估计答案所在范围，再通过排名查询缩小值域。把直接枚举前 k 个，改成对值域做若干次判定。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

随机只用于缩小范围，最终答案仍应通过精确排名或候选检验确定。

奇怪幂贡献考虑组合意义

适用场景

当答案中出现 x^k、2^x、k^x 这类看起来不好处理的幂贡献时，可以寻找组合解释。

核心思路

x^k 可以理解为 k 个位置各自从 x 个对象中选择；2^x 可以理解为某集合的任意子集数量。把幂解释成选择方案数后，常能交换枚举顺序。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

组合意义要和题目对象一一对应，避免只做形式变形而产生重计。

存在性转为反面全不满足

适用场景

当“存在至少一个满足条件”的方案难以直接计数时，可以统计没有任何对象满足条件的方案。

核心思路

利用 $[\exists x : P(x)] = 1 - [\forall x : \neg P(x)]$。计数时先算总方案，再减去所有对象都不满足的方案；若有多个存在条件，可以进一步容斥。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

补集条件有时比原条件更复杂。使用前要确认补集能被更容易地 DP 或数据结构维护。

将动态可行决策转为随机排列决策

适用场景

当一个随机过程每一步都从“当前仍可行”的决策中随机选择，而决策可行性会随着过程变化时，可以尝试改为：先随机排列所有初始决策，再按顺序执行当前仍可行的决策。

核心思路

很多过程中，某个决策是否最终被执行，只取决于它在相关决策中的相对顺序，而不取决于无关决策。因此可以先生成一个所有决策的随机排列，然后依次扫描：

- 当前决策若仍合法，则执行；
- 当前决策若已无效，则跳过。

这样动态随机过程就转化为排列上的概率问题。

常见用途

利用期望线性性：

$$E[\text{执行次数}] = \sum_{x} P(x \text{ 被执行}).$$

然后分别计算每个决策被执行的概率。

易错点

- 必须证明“先随机排列再跳过无效决策”和原随机过程等价。
- 只适合决策之间主要由相对顺序决定的过程。

- 概率常化成 $\frac{1}{k+1}$ 或组合数形式，要注意边界。

平方和展开恒等式

基于平方和与和平方之间的恒等式

$$(\sum_{i=1}^{n} s_i)^2 = \sum_{i=1}^{n} s_i^2 + \sum_{i \neq j} s_i s_j$$

容易发现：

$$\sum_{i \neq j} s_i s_j = 2 \sum_{1 \leq i < j \leq n} s_i s_j$$

所以有：

$$\sum_{1 \leq i < j \leq n} s_i s_j = \frac{(\sum_{i=1}^{n} s_i)^2 - \sum_{i=1}^{n} s_i^2}{2}$$

应用：数组两两之间的乘积和（$\mathcal{O}(n^2)$ 降为 $\mathcal{O}(n)$）

恰好 k 转钦定 k 后容斥

适用场景

如果“恰好 k 个满足”不易计算，而“钦定若干个满足”或“至少若干个满足”容易计算，可以容斥。

核心思路

把恰好计数改写为对钦定集合大小的交替求和。例如先统计至少包含某些对象的方案数，再通过组合数容斥还原恰好 k。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

钦定对象后剩余对象可以继续满足条件，所以必须用容斥消除多算。

方案数平方和转同类有序对计数

适用场景

当答案形如 $\sum_i cnt_i^2$ 时，可以解释为选出两个同类方案的有序对。

核心思路

cnt_i^2 表示第一种方案和第二种方案都属于第 i 类的有序对数量。把平方和改成对两个方案同时计数，常能交换枚举顺序或做二维 DP。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

平方对应有序对；如果题目要无序对，需要处理除以二和相同方案。

期望用尾和公式改写

适用场景

非负整数随机变量的期望不好直接按取值概率计算时，可以用尾和公式。

核心思路

若 X 是非负整数随机变量，则 $E[X] = \sum_{i \geq 1} P(X \geq i)$。很多最大值、等待时间问题中，$P(X \geq i)$ 比 $P(X = i)$ 更容易算。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

注意求和上界和概率是否为非负整数变量。连续变量有对应积分形式。

用一次项系数统计贡献和

适用场景

当一个算法容易计算“权值乘积的和”，但题目要求“权值和的贡献”时，可以把每个权值 w 替换成一次多项式 $wx+1$，最后取一次项系数。

核心结论

对于若干元素权值 $w_1, w_2, \ldots, w_k$，有：

$$\prod_{i=1}^{k}(1+w_i x) = 1 + \left(\sum_{i=1}^{k} w_i\right) x + O(x^2).$$

因此，在模 x^2 意义下计算乘积，一次项系数就是权值和。

常见用途

如果矩阵树定理能求所有生成树的边权乘积和，那么将每条边的权设为 $1+w_i x$，就能同时统计所有生成树的边权和贡献。

计算时只需要维护形如 $ax+b$ 的一次多项式，并在乘法后丢掉 x^2 项。

易错点

- 只能提取一次项；若需要二次以上贡献，要保留更高阶多项式。
- 做除法或高斯消元时，需要实现一次多项式的逆元。
- 若常数项为 0，多项式逆元不存在，要特殊处理。

逆序对与正序对的总和不变

对于任意一个长度为 n 的排列（允许重复元素），如果我们考虑所有 $n!$ 种排列方式，那么：

所有排列中逆序对的总数 = 所有排列中顺序对的总数

这是因为对于任意两个值不同的位置 i, j，在所有排列中：

- 它们构成正序对和逆序对的概率各占 $1/2$
- 由对称性可知，正序对总数 = 逆序对总数

设数组中有 cnt_x 个值为 x 的元素。

- 任意选取两个位置，共有 $C(n,2)$ 种选法
- 减去值相同的情况：$\sum C(cnt_x, 2)$
- 得到值不同的位置对数量：$C(n,2) - \sum C(cnt_x, 2)$

对于每一对值不同的位置，在所有排列中，正序和逆序各占一半。

因此：

$$\text{逆序对总数} = \frac{n!}{2} \times \left(C(n,2) - \sum C(cnt_x, 2)\right)$$

对于全排列的统计问题，当正序和逆序具有对称性时，可以利用"总对数固定"这一性质简化计算，避免复杂的分情况讨论。

校验补充

上述 n! 统计把相同值的不同出现位置当作带编号对象。若仅计不同的值序列，应将排列总数改为 n!/prod(cnt_x!)；不能仍乘 n!。

镜像法消除 DP 边界限制

适用场景

在路径计数、随机游走、括号序列等 DP 中，如果存在“不能越过某条边界”的限制，可以考虑镜像法。

核心思路

先计算不考虑边界的所有方案，再把越界方案通过反射映射到另一个起点或终点的无约束方案，从而用总方案减去非法方案。典型形式是把第一次越界的位置之后的路径镜像。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

镜像法必须保证非法方案和反射后方案一一对应。边界是严格还是非严格会影响反射线位置。

2.0.3 位运算与二进制

1到n连续按位异或和规律

设

$$S(n) = 1 \oplus 2 \oplus 3 \oplus \cdots \oplus n$$

则有：

$$S(n) = \begin{cases} n, & n \bmod 4 = 0 \\ 1, & n \bmod 4 = 1 \\ n+1, & n \bmod 4 = 2 \\ 0, & n \bmod 4 = 3 \end{cases}$$

Bit-Tricks

异号判定（有符号整数）

```
bool opposite_sign(i64 x, i64 y) { return (x < 0) != (y < 0); }
```

位逆序置换（0 <= n <= 24；内存 O(2^n)）

```
vector<unsigned> bit_reversal(int n) {
    assert(0 <= n && n <= 24);
    unsigned len = 1u << n;
    vector<unsigned> ans(len);
    for (unsigned i=0,j=0;i<len;++i) {
        ans[i]=j;
        if (i+1 == len) break; // 不对最后一个数继续进位，避免 k=0 死循环
        unsigned k=len>>1;
        while (j & k) { j ^= k; k >>= 1; }
        j ^= k;
    }
    return ans;
}
```

枚举子掩码与超掩码（含零子掩码和掩码本身）

```
template<class F> void subsets(unsigned mask, F visit) {
    for (unsigned s=mask;;s=(s-1)&mask) {
        visit(s);
        if (s==0) break;
    }
}
template<class F> void supersets(unsigned t, int n, F visit) {
    assert(0 <= n && n < 32 && t < (1u<<n));
    unsigned limit=1u<<n;
    for (unsigned s=t;s<limit;s=(s+1)|t) visit(s);
}
```

所有 n 位掩码的子掩码总数为 $\sum_{k=0}^{n} \binom{n}{k} 2^k = 3^n$：

```
template<class F> void all_subsets(int n,F visit) {
    assert(0 <= n && n < 32);
    for(unsigned mask=0;mask<(1u<<n);++mask)
        subsets(mask,[&](unsigned sub){visit(mask,sub);});
}
```

下一个相同 popcount 的 u64（Gosper）

没有更大同 popcount 值时返回 false，并保持 x 不变；x=0 也返回 false。

```
bool next_same_popcount(u64& x) {
    if (!x) return false;
    u64 low=x&(~x+1), r=x+low;
    if (!r) return false;
    x=r|(((r^x)>>2)/low);
    return true;
}
```

x和y位运算中1的个数

```
assert(__builtin_popcount(x ^ y) == __builtin_popcount(x) + __builtin_popcount(y) - 2 * __builtin_popcount(x & y));
assert(__builtin_popcount(x | y) == __builtin_popcount(x) + __builtin_popcount(y) - __builtin_popcount(x & y));
assert(__builtin_popcount(x & y) == (__builtin_popcount(x) + __builtin_popcount(y) - __builtin_popcount(x ^ y)) / 2);
```

因为$2 \times \text{popcount}(x \,\&\, y)$ 必然是偶数，所以 $x \oplus y$ 中 1 的个数的奇偶性，完全取决于 x 和 y 中 1 的个数之和。
如果两者同奇偶，那么 $x \oplus y$ 结果中 1 的个数必为偶数；否则为奇数。

三个有关位运算的有用性质

1. 加法等式：`a + b = (a ^ b) + 2 * (a & b)`

   > *`a ^ b` 是不带进位的按位加法， `a & b` 找出均是1的位，乘以2代表进位左移1位*

2. 按位或等式：`a | b = (a ^ b) + (a & b)`

   > *按位或等于“两数不同的位”加上“两数同为1的位”*

3. 由等式1、2可以推得 `a + b = (a & b) + (a | b)`

   > *证明：*
   > *将等式 (1) 中的 `2 (a & b) 拆开：* *a + b = [(a ^ b) + (a & b)] + (a & b) * *把等式（2）代入方括号中，直接得到：* *a + b = (a | b) + (a & b)`**

由于二进制运算独立作用于每一位，我们只需要证明在任意单个比特位上，上述等式成立即可，因此也可以在固定的一位上枚举四种情况证明上述三种性质，在此不再列出

异或的各种性质可以使用按位拆分，使用 mod 2 加法的方式得出

异或的各种性质可以使用按位拆分，使用 mod 2 加法的方式得出。

比如说这个分配律。

$$c_1P_1 \oplus c_2(P_1 \oplus P_2) \oplus c_3(P_2 \oplus P_3) \oplus \cdots \oplus c_k(P_{k-1} \oplus P_k) = 0$$

$$(c_1 \oplus c_2)P_1 \oplus (c_2 \oplus c_3)P_2 \oplus (c_3 \oplus c_4)P_3 \oplus \cdots \oplus (c_{k-1} \oplus c_k)P_{k-1} \oplus c_kP_k = 0$$

对于二进制中的任意一位 x，都有

$$c_1P_{1,x} + c_2(P_{1,x} + P_{2,x}) + \cdots + c_k(P_{k-1,x} + P_{k,x}) \equiv 0 \pmod 2$$

$$(c_1 + c_2)P_{1,x} + (c_2 + c_3)P_{2,x} + \cdots + (c_{k-1} + c_k)P_{k-1,x} + c_kP_{k,x} \equiv 0 \pmod 2$$

那么不难得出上面的结论。

快速求 n个数的 popcount 和

一种以 $O(\log n)$ 求 $\sum_{i=1}^{n} \text{popcount}(i)$ 的方法

逐位统计：令 $N = n + 1$，第 k 位贡献为

$$\lfloor N/2^{k+1} \rfloor 2^k + \max(0, N \bmod 2^{k+1} - 2^k).$$

对 $k = 0, \ldots, 63$ 求和，使用 u128 保存 n+1 和结果。

```
// 返回 sum_{i=0}^n popcount(i)，支持整个 u64 范围。
u128 popcount_sum(u64 n) {
    u128 ans = 0, count = u128(n) + 1;
    for (int k = 0; k < 64; ++k) {
        u128 half = u128(1) << k, period = half * 2;
        ans += (count >> (k + 1)) * half;
        u128 rem = count & (period - 1);
        if (rem > half) ans += rem - half;
    }
    return ans;
}
```

按操作对二进制位的影响划分过程

适用场景

当操作包含加一、异或、或、与等位运算时，整体状态很难维护，但每一位或低若干位的变化规律清楚，可以按二进制位划分过程。

核心观察

在二进制下，$+1$ 会把末尾连续的 1 变成 0，并向更高一位进位。也就是说，低位状态会经历类似：

$$s \to 0 \to t$$

的阶段。

因此，可以把“完整数值变化”拆成“低 i 位如何从初态变为终态”的问题，并维护是否产生进位、是否使用某些异或操作等状态。

常见状态

- 当前处理到第几位；
- 初态来自原数还是 0；
- 终态来自目标数还是 0；
- 已使用的操作集合；
- 是否产生进位。

易错点

- 加法进位会影响更高位，不能逐位完全独立处理。
- 位运算操作如果作用于所有位，需要记录其在低位上的整体效果。
- 位数上界要覆盖所有可能值，例如 2^{40} 就要处理到第 40 位。

整数拍扁成 01 前缀表示差值

适用场景

当需要处理非负整数之间的绝对差，并且值域不大时，可以把整数展开为 01 前缀。

核心思路

把整数 x 表示为长度足够的串 $f_x = \underbrace{11\cdots1}_{x}00\cdots0$。则两个数的差值可以转化为两个 01 串在某些位上的不同数量，从而用 bitset 优化批量计算。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该方法依赖值域长度，值域过大时不可行。

最大化多重集异或和的求和

给定一个序列，两个多重集（初始是空的），把每个数划分到两个多重集中，使得两个多重集的 `xor 和` 再相加，结果最大

根据 $A+B=(A\oplus B)+2(A\ \textbf{and}\ B)=S+2\cdot(A\ \textbf{and}\ B)$
实际上，考虑 $S=\bigoplus_{i=1}^{n}a_i$，这是基础值，是确定的，只需要最大化 $A\ \textbf{and}\ B$

- 选出一个子集 A，最大化 $A\ \textbf{and}\ B$，而 $S=A\oplus B$
- 注意到 S 为 `1` 的位一定形如 `(a(i), b(i)) = (1, 0)`，它的 $\&$ 运算结果一定是 `0`
  而 S 为 `0` 的位呢，形如 `(1, 1) or (0, 0)`，它的 $\&$ 运算结果取决于 a 这一位的值

综上所述，若 $A\oplus B=S$，那么 $A\ \&\ B=A\ \&\ (\sim S)$

这个问题就转化成，给定 S，给定 $a[1\sim n]$，选一个子集构成 A，使得 $A\ \textbf{and}\ (\sim S)$ 最大
实际上就是最大化 $\bigoplus_{i=1}^{m}(a_i\ \textbf{and}\ (\sim S))$

这个让每个数 $a_i=a_i\ \&(\sim S)$，是标准的选取一个子集，使其异或和最大的问题，这就成了线性基模版题了

构造相邻二进制数仅相差一位的数组

想要实现这样的数组,就是在构造格雷码，这里有一个简单的构造方式
构造包含 2^n 个元素的 n 位格雷码数组 a（$0\le i<2^n$）

$$a_i=i\oplus(i>>1)$$

应用

1. 构造相邻异或值之和最小的数组
2. 实现指数级搜索的 $O(1)$ 增量更新 (如 汉诺塔问题，子集枚举优化)
3. 格雷码也可以看作上是 n 维超立方体图的一条哈密顿路径
   …

求连续 111..1 段的个数

给定一个数，求有多少个连续全 1 段，比如 11...1 0...0 111..1，`cnt[ 连续全 1 段 ] = 2`
这个问题，等价于问**对每一位考虑，有多少个相邻的 `01`**

这很重要，因为这可以转化成**状态机**，相邻位就只有 `(00, 01, 10, 11)`，可以用这四种状态来表示

比如给你一个序列，然后若干次操作，每次操作 $\&, \oplus, |$ 上一个随便的数，比如说 x
然后统计每个数**连续全 1 段**的**和**，怎么做呢？
总不能对每个询问挨个去暴力加吧，会超时
考虑**维护整体的和，以及操作之后每一位的状态变化**

- 你只需要维护每一个位的 `(00, 01, 10, 11)` 四种状态
  比如 `cnt(k, "00")` 表示第 `(k+1, k)` 位的状态是 `00` 的数有多少个？
- 比如位运算，$\oplus x$，你只要拿 x 去对齐第 k 位，看看 x 的 `(k+1, k)` 位组成的状态是多少？
  那么是 `mask = x >> k & 3`，是吧？
- 然后就考虑转移，枚举 `4` 种相邻位的状态，`cnt(k, sta)`
  令 `nsta = sta ^ mask`，实际上，所有原来是 `sta` 的状态都变成了 `nsta`，自然对应的 `cnt` 个数也要相等
  于是令 `ncnt[nsta] = cnt[k, sta]`，四个状态都枚举之后
  更新 `cnt[k] = ncnt`，继续考虑更高的位

最高位前补一个 0，才能统计以最高位 1 开头的段。OR/AND 的状态转移可能多对一，必须 ncnt[nsta] += cnt[k,sta]；只有 XOR 的置换转移可以直接赋值。

2.0.4 序列、字符串与状态转化

01 序列 LNDS 转最大前缀和

适用场景

对 01 序列求最长不下降子序列或相关增益时，可以把 0/1 改成权值差。

核心思路

把 0 记为 $+1$，1 记为 -1，某些关于选择前面 0 和后面 1 的问题会变为求最大前缀和或最大子段和。本质是把“多选一个 0 的收益”和“少选一个 1 的损失”统一到权值和中。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该转化依赖 01 序列的特殊结构，不能直接推广到多值序列。

校验补充

准确公式：LNDS = 原串中 1 的总数 + max_{0<=k<=n}(前 k 位 0 数-前 k 位 1 数)。最大值包含空前缀，不能用最大子段和直接代替。

Dilworth 定理：关于最长上升子序列（LIS）

数列的最长**单调增加**子序列的长度，等于可将该数列划分成**单调不增**子序列的最少数量。

数列的最长**单调减少**子序列的长度，等于可将该数列划分成**单调不减**子序列的最少数量。

数列的最长**单调不减**子序列的长度，等于可将该数列划分成**单调减少**子序列的最少数量。

数列的最长**单调不增**子序列的长度，等于可将该数列划分成**单调增加**子序列的最少数量。

> 请注意这里“**子序列**”的定义：从原序列中取出任意多个元素（可以不取，也可以全取），按照其在原序列中的原顺序排列，所得序列即为原序列的子序列

LCS 转 LIS 优化匹配复杂度

适用场景

当一个序列中元素互异，或每种元素出现次数较少时，LCS 可以转为 LIS。

核心思路

把 A 中每个值映射到它在 B 中出现的位置。按 A 的顺序展开这些位置，问题变成在位置序列中选一个严格递增子序列，即 LIS。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

若 B 中一个值出现多次，展开位置时通常要倒序加入，避免同一个 A_i 被使用多次。

NFA 状态集合转 DFA 计数

适用场景

对于“有多少种方案使得存在一条合法路径”的问题，如果合法性可以用非确定自动机描述，可以把可达状态集合当作 DP 状态。

核心思路

设当前 NFA 中能到达的状态集合为 S。读入一个字符或执行一步操作后，S 会转移到新的集合 T。于是可以在状态集合上建 DFA，再做计数或最短路。很多题中理论状态数是 2^n，但实际可达状态很少。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

需要先从初始集合做可达性搜索，不要暴力枚举全部子集。转移时要处理 ϵ 转移或闭包。

Trie 上两串 LCP 等于 LCA 深度

适用场景

在 Trie 中处理两个字符串的最长公共前缀时，可以转为树上 LCA 问题。

核心思路

Trie 上每个节点对应一个前缀。两个字符串对应终止节点 u, v，它们的最长公共前缀就是 $\mathrm{lca}(u, v)$ 对应的字符串，LCP 长度等于该 LCA 的深度。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

如果字符串可能不是完整插入终点，要先明确查询节点对应哪个前缀。

三项等差子序列类比回文处理

适用场景

统计三元组 (i, j, k) 满足 $i < j < k$ 且值或位置成等差时，可以把中间项看作中心。

核心思路

类似回文串围绕中心扩展，固定中间元素 j，左右两侧需要满足对称的差值条件。可以预处理左侧和右侧某类值的出现次数，再乘法统计。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

等差可能发生在下标、值或二者同时，先明确哪一个是中心对称。

二元最大值-最小值计数转化为长度 + 前缀差分

区间二元最大值 / 最小值计数转化为长度 + 前缀差分

交换次数奇偶性

对于一个排列，交换相邻两项必定会使得逆序对个数+1或者-1，那么对于一个排列，它的交换1<=i,j<=n，两个下标使得有序的排序次数的奇偶性取决于逆序对的奇偶性；
同理也有一个性质，对于一个区间循环左移一次，是将第一个元素通过相邻交换r-l将其移动道最后一个位置，那么交换次数就是r-l次，那么循环左移d次就是交换(r-l)*d次相邻元素，对于左右边界外的逆序对个数没有变化，但是对于区间内的进行了(r-l)*d次相邻交换，因为一次相邻交换一定会导致逆序对个数+1或者-1，有因为+1和-1操作在模2意义下相等，所以此时逆序对个数的奇偶性变成(sum+(r-l)*d)%2；

区间翻转排列只影响少量断点

适用场景

维护排列的连续性、段数或相邻关系时，区间翻转不一定需要重算整段。

核心思路

翻转区间内部相邻关系的集合不变，只是方向反过来；真正改变的是区间左右边界附近的连接关系。因此不连续位置数量通常只会在边界处变化，最多影响常数个断点。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该技巧维护的是相邻值关系或断点，不是所有区间统计都只受边界影响。

校验补充

“只影响边界”要求被统计的相邻关系对交换两端不敏感，例如 |a[i+1]-a[i]|=1。若统计 a[i+1]=a[i]+1，翻转会改变整段内部方向，不能只改边界。

合法括号串拼接产生)(断点

适用场景

处理合法括号串的分解、拼接、连通性或段数统计时，可以观察相邻合法块之间的边界。

核心结论

两个非空合法括号串 A 和 B 拼接成 AB 时，拼接处一定会出现形如：

```
)(
```

的相邻字符。

原因是合法括号串一定以 `(` 开头、以 `)` 结尾。于是 A 的最后一个字符是 `)`，B 的第一个字符是 `(`。

作用

如果一个问题中，每次把两个合法括号块拼接会让答案发生一次变化，那么可以直接统计原串中的 `)(` 数量。

例如一个长度为 $2n$ 的合法括号串，如果每个基本括号对初始贡献为 n，每出现一次合法块拼接就减少一个连通块，那么答案可以写成：

$$ans = n - \#\{i \mid s_i = ')', s_{i+1} = '('\}.$$

易错点

- 这里要求参与拼接的两段都是合法括号串。
- 统计的是相邻子串 `)(`，不是所有逆序括号对。
- 空串是否允许要看题意，通常分解中不考虑空块。

大值域互异限制映射到小值域

适用场景

如果题目值域很大，但约束只关心元素是否相同、大小类关系或少量相对关系，可以尝试把值映射到较小集合。

核心思路

只保留会影响答案的等价类。例如只关心互不相同，可以将实际值按出现顺序离散化；只关心是否落在若干阈值区间，可以按阈值划分值域。这样能把大值域状态变成小值域 DP 或图论问题。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

映射必须保持题目需要的全部关系。若题目同时关心大小和差值，普通离散化可能破坏差值信息。

奇偶位取反转化相邻同项操作

适用场景

当题目中有二元字符串，并且操作和相邻两个字符“相同/不同”有关时，可以尝试对所有奇数位或所有偶数位取反。

核心结论

设原串为 s，将某一种奇偶性的下标统一取反得到 t。对于相邻位置 $i, i+1$，由于二者奇偶性不同，恰好只有一个位置被取反，因此：

$$s_i = s_{i+1} \iff t_i \neq t_{i+1},$$

$$s_i \neq s_{i+1} \iff t_i = t_{i+1}.$$

于是，原串中对相邻同项 `00`、`11` 的操作，常常可以转化为新串中对相邻异项 `01`、`10` 的操作。

常见用途

- 将“删除相邻相同字符”转化为“删除相邻不同字符”。
- 将“翻转相邻相同字符”转化为“交换或处理相邻不同字符”。
- 将字符串操作问题转化为计数差、不变量或相邻交换距离问题。

易错点

- 只能统一取反一种奇偶性，不能根据操作动态改变取反规则。
- 初始串和目标串都要做同样的变换后再比较。
- 该技巧通常要求字符集是二元的，或者题目能先压成二元关系。

字符集与长度构造回文串

我们设
字符集内部的字符数量为k （字符集的意思就是这个集合里面都是不同的字符）
例如{A，B，C}为一个字符集，那么字符串只能由这三个构建出来
构建的长度为l

n(种类数)=k*(k-1)*(k-2)^(l-2)

解释：第一个字符可以随便选有k个选择，第二个字符不能与第一个字符相同有（k-1）个选择，第三个字符不能与第二个相同也不能与第一个相同有（k-2）个选择，第四个字符不能与第三个相同也不能与第二个相同也有（k-2）个选择……根据乘法原理就是最后的公式

校验补充

这里计数的是“没有长度 >=2 的回文子串”的字符串，而不是回文串本身。l=0 时为 1，l=1 时为 k；l>=2 且 k>=2 时才可使用 k(k-1)(k-2)^(l-2)，其中 0^0 按 1。k=0、1 的情况单独处理。

将权值减下标消去连续递增限制

适用场景

当题目中出现形如 $a_{i+1}=a_i+1$ 的连续递增条件，或者需要把一段位置变成 $x,x+1,x+2,\dots$ 时，可以考虑令：

$$b_i=a_i-i.$$

核心结论

若一段序列满足：

$$a_{i+1}=a_i+1,$$

则：

$$a_{i+1}-(i+1)=a_i-i.$$

也就是说，连续递增一的段在 $b_i=a_i-i$ 后会变成相等的一段。

常见用途

- 判断区间能否变成连续递增段。
- 将“覆盖为等差数列”转化为“覆盖为常数”。
- 把复杂的线段树等差标记变成普通区间赋值或区间统计。

易错点

- 下标从 0 还是 1 开始会影响 a_i-i 的值，但不影响相等关系。
- 若递增公差不是 1，应改为 $a_i-d\cdot i$。
- 该技巧只消去和位置线性相关的部分。

平均数等于目标转和为零

适用场景

当题目问某段平均值是否为 x，可以把所有数减去 x。

核心思路

令 $b_i=a_i-x$，则区间平均值为 x 等价于区间和为 0。于是可以用前缀和、哈希表、平衡树等方法处理。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

若 x 是分数，建议整体乘分母避免浮点误差。

循环区间滑动时用连续性保证取等

适用场景

当题目要在环形序列或循环区间中选一段，使某个计数恰好等于目标值时，可以考虑滑动窗口的连续变化。

核心思路

在 01 串中，固定窗口长度 m，令 $f(l)$ 表示从 l 开始长度为 m 的循环区间中 1 的个数。

当窗口右移一格时，只会删除一个字符并加入一个字符，因此：

$$|f(l+1)-f(l)|\le 1.$$

如果某些位置的窗口值大于目标 c，另一些位置的窗口值小于目标 c，那么在循环滑动过程中一定会经过 $f(l)=c$。

常见用途

- 构造满足比例的子串。
- 证明答案段数不超过某个小常数。
- 在环上通过滑动找到恰好满足计数的区间。

易错点

- 需要保证滑动过程是连续的，值每次只改变 $-1,0,1$。
- 如果整个序列总量不满足整除条件，目标值可能根本不是整数。
- 环形区间要处理跨越末尾的情况。

按阈值将序列值域转为 01

适用场景

当题目中的操作只关心一个数是否 $\le x$ 或 $>x$，而不是关心具体值时，可以按阈值 x 把序列转成 01 序列。

核心思路

给定一个待判断答案 x，定义：

$$b_i=[a_i>x].$$

这样，原序列中的升序/降序排序、区间统计等操作，可能会变成对 01 序列的简单覆盖操作。

例如区间升序排序时，只需要统计区间内 0 的个数 cnt_0，然后把前 cnt_0 个位置覆盖成 0，剩余位置覆盖成 1。

常见套路

如果最终只询问某个位置的值，可以二分答案 x：

- 将 $a_i\le x$ 视为 0，$a_i>x$ 视为 1；
- 模拟全部操作；
- 根据目标位置最终是 0 还是 1 判断答案方向。

易错点

- 01 转化通常需要配合二分答案。
- 升序和降序排序对应的覆盖方向相反。
- 判断条件中 $\le x$ 和 $>x$ 的边界不能写反。

排列中 mex 与 min 的转化

对于一个排列，前缀 mex 的询问，可以转换为后缀最小值：

$$\operatorname{mex}_{i=1}^{j} a_i=\min_{i=j+1}^{n} a_i$$

约定必须是 0..n-1 的排列；空后缀的最小值定义为 n。若是 1..n 的排列，按通常从 0 开始的 mex 定义，前缀 mex 总为 0，不能直接套此式。

操作可逆时寻找标准中间状态

适用场景

如果操作可逆，直接从起点变到终点很难，可以考虑都变到某个标准状态。

核心思路

由于操作可逆，$A \to B$ 等价于 $A \to C$ 后把 $B \to C$ 的操作逆序反向执行。常见标准状态包括排序后状态、全零状态、规范形。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

需要确保所有合法状态都能到达同一个标准状态，否则要按连通分量分类。

最大子段和的局部最优性质

适用场景

与最大子段和相关的构造、证明或数据结构维护中，可以利用其局部最优性质。

核心思路

一个最优最大子段不存在负的前缀或负的后缀；同时，紧贴它前后的可扩展子段不会具有正贡献。否则可以删去或扩展得到更优答案。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

如果允许空子段，性质中的正负边界要重新处理。

校验补充

若要求非空子段，该论证只适用于删除后仍非空的真前缀/真后缀；全负数组的最优单元素子段可以为负。

构造排列的和，对 n 取模

排列 a 的下标是 0..n-1，令 $f(a) = \sum_{0 \le i < j < n} (a_j - a_i)$。
将每个值改为 $(a_i + d) \bmod n$，各差值模 n 不变，所以 f 模 n 不变。
这叫值域平移，并不等价于循环移动数组位置。
数组左移一次时，f 的变化为 $2na_0 - 2\sum_i a_i$；对 0..n-1 的排列也整除 n，但这是另一个推导。

模糊答案可按二的幂压缩值域

适用场景

如果题目不要求精确答案，只要求输出值 x 满足：

$$ans \le x \le 2ans,$$

或类似的常数倍近似限制，可以考虑按二的幂压缩值域。

核心思路

将每个值 a 映射到某个不小于它、且不超过 $2a$ 的二进制分组代表值。例如把区间：

$$[2^{k-1}, 2^k - 1]$$

内的值统一替换成 $2^k - 1$ 或相邻的二进制边界值。

这样得到的新答案通常不小于原答案，且不超过原答案的常数倍。同时，值域从 $O(n)$ 降为 $O(\log n)$。

作用

压缩后可以枚举值域类别，用原本无法承受的算法处理每一类。

易错点

- 只能用于允许近似答案的题，不能用于精确题。
- 映射后的值必须保证答案方向正确，例如不能小于真实答案。
- 要证明整体答案也保持在允许倍率内，而不仅是单个元素。

用前缀和或差分重解释操作

适用场景

当原操作直接看很复杂时，可以对序列或树上边权做前缀和、前缀异或或差分，观察操作在新变量上的含义。

核心思路

设原数组为 a，差分为：

$$d_i = a_i - a_{i-1}.$$

许多区间操作会在差分数组上变成端点操作。反过来，一些局部操作在前缀变量中可能变成交换、单点修改或多重集比较。

树上也类似。若令点权为根到该点路径上的边权异或和，那么对边的某些操作可能变成相邻点权的交换或整体异或。

常见用途

- 区间修改转端点修改；
- 树边权转根到点的前缀异或；
- 数组操作转差分数组上的相邻交换；
- 判断可达状态时比较变换后的多重集合。

易错点

- 前缀/差分变换通常需要固定根或固定起点。
- 异或和加法差分性质不同，不能混用。
- 变换后要重新翻译初态、终态和边界条件。

逆序对奇偶性由置换环数决定

适用场景

只关心排列逆序对数量的奇偶性时，可以不用真正数逆序对，而看置换分解。

核心思路

一个排列的逆序对奇偶性等于排列符号，而排列符号也等于 $n - c$ 的奇偶性，其中 c 是置换环个数。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该结论只给奇偶性，不能得到逆序对具体数量。

随意交换抽象成置换环

适用场景

当题目允许在一个集合内任意交换元素，或者问把一个排列变成另一个排列的代价时，可以看置换环。

核心思路

把目标位置映射成置换。一个长度为 k 的环通常需要 $k-1$ 次交换才能归位；某些问题的奇偶性、最少操作数也由环数决定。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

如果交换代价和位置或元素权值有关，不能只看环长，还要在环内做最优化。

2.0.5 数据结构与区间维护

BIT 倍增替代二分套树状数组

适用场景

如果需要在树状数组上反复二分某个前缀位置，例如找第一个前缀和达到阈值的位置，直接二分套 BIT 查询是 $O(\log^2 n)$，可以用 BIT 倍增优化到 $O(\log n)$。

核心思路

树状数组节点 x 维护的是区间：

$$[x - \operatorname{lowbit}(x) + 1, x].$$

查找位置时，从最高二进制位到最低二进制位尝试扩展答案。维护当前已经确定的位置 pos 和前缀值 sum。

若加入 $pos+2^k$ 对应的 BIT 块后仍满足条件，就令：

$$pos \leftarrow pos + 2^k,$$

并同步更新 sum。

这样可以避免二分过程中重复查询前缀和。

模板形式

```
int pos = 0;
long long cur = 0;
for (int k = LOG; k ⩾ 0; --k) {
    int nxt = pos + (1 << k);
    if (nxt ⩽ n && cur + bit[nxt] < target) {
        pos = nxt;
        cur += bit[nxt];
    }
}
// pos + 1 是第一个前缀和 ⩾ target 的位置
```

易错点

- 判断条件要根据“找第一个 >= target”还是“找最后一个 < target”调整。
- BIT 下标必须从 1 开始更方便。
- 数组元素必须非负，使前缀和单调；仅有可加性不够。要求 target > 0；超过总和时返回 n+1。LOG 取 floor(log2(n))（n=0 时可取 0），使用 i64 保存前缀和且不允许溢出。

BIT 可线性重构

适用场景

已知原数组，需要初始化树状数组时，不必逐点 add。

核心思路

先令 $tree_i = a_i$，再对每个 i，把 $tree_i$ 加到 $tree_{i+lowbit(i)}$。这样每个位置只向父节点贡献一次，整体 $O(n)$ 建树。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

这是构建静态初始 BIT 的技巧，不改变后续单点修改和前缀查询写法。

校验补充

必须按 i 从小到大传播，且只在 i+lowbit(i)<=n 时更新父节点。

Trie 上交换子树实现值域操作

适用场景

当维护一个数值集合，并支持按位异或、按位或、按位与等值域操作时，可以把数放在 Trie 或动态开点值域线段树上，通过交换或合并子树实现操作。

核心思路

在二进制 Trie 中，每一层对应一个二进制位，每个节点的左右儿子分别表示当前位为 $0/1$。

- 整体异或某一位为 1：交换这一层对应节点的左右儿子。
- 按位或某一位为 1：所有该位为 0 的数会变成 1，相当于把左子树合并到右子树。
- 按位与可以用或和异或转化，或直接对不合法子树进行合并/清空。

常见用途

- 维护集合中不同数个数；
- 对值域区间进行位运算；
- 三进制或其他进制 Trie 中的儿子轮换、交换。

易错点

- Trie 建树方向要和操作匹配，有的进位操作需要从低位向高位建。

- 合并子树时要维护计数和答案信息。
- 区间操作通常需要配合 split / merge 或动态开点线段树。

万能 hash

支持 array、vector、pair、tuple 及其嵌套。使用 `unordered_set<Key, UniversalHash>` 或 `unordered_map<Key, Value, UniversalHash>`。不要为全由标准库类型组成的键向 std 添加 hash 偏特化。哈希可能碰撞，容器仍用相等比较区分键。

```
struct UniversalHash {
    template<class T> static size_t get(const T& x) { return std::hash<T>{}(x); }
    template<class T> static void add(size_t& h, const T& x) {
        h ^= get(x) + size_t(0x9e3779b9) + (h << 6) + (h >> 2);
    }
    template<class A, class B> static size_t get(const pair<A,B>& x) {
        size_t h=0; add(h,x.first); add(h,x.second); return h;
    }
    template<class... T> static size_t get(const tuple<T...>& x) {
        size_t h=0; apply([&](const auto&... y){(add(h,y),...);},x); return h;
    }
    template<class T, size_t N> static size_t get(const array<T,N>& x) {
        size_t h=0; for(const auto& y:x) add(h,y); return h;
    }
    template<class T> static size_t get(const vector<T>& x) {
        size_t h=0; for(const auto& y:x) add(h,y); return h;
    }
    template<class T> size_t operator()(const T& x) const {return get(x);}
};
```

主席树视作静态树套树

适用场景

当一个问题用树套树能做，但没有修改或修改只体现在前缀版本时，可以考虑用主席树降复杂度。

核心思路

主席树的每个版本表示一个前缀或历史状态，相邻版本只差一次插入。区间信息可由两个版本相减得到，相当于把外层位置维静态化，内层权值维用可持久化线段树维护。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

主席树适合静态前缀差分，不适合任意在线修改。值域需要离散化或动态开点。

倍增优化区间并查集合并

适用场景

当题目要求多次把两个等长区间逐位合并，例如要求 $[l_1, r_1]$ 和 $[l_2, r_2]$ 对应位置相等时，直接逐点合并会退化为 $O(nm)$。这时可以用倍增思想优化。

核心思路

类似 ST 表，将长度为 2^k 的区间作为一个整体节点。

令 $fa[k][i]$ 表示区间 $[i, i+2^k-1]$ 所属的并查集代表元。若要合并两个等长区间 $[l_1, r_1]$ 和 $[l_2, r_2]$，令 $k = \lfloor \log_2(r_1 - l_1 + 1) \rfloor$，只需合并：

$$[l_1, l_1 + 2^k - 1] \leftrightarrow [l_2, l_2 + 2^k - 1],$$

以及

$$[r_1 - 2^k + 1, r_1] \leftrightarrow [r_2 - 2^k + 1, r_2].$$

最后再从大长度向小长度下传合并关系，把 2^k 的区间拆成两个 2^{k-1} 的区间。

复杂度

预处理和下传共 $O(n \log n \alpha(n))$，每次区间约束只需要 $O(\alpha(n))$ 次合并。

易错点

- 倍增区间的合并只是记录约束，最终必须从高层向低层拆分传播。
- 区间长度不是 2^k 时，要用左右两个 2^k 区间覆盖。
- 只适合“对应位置相等/合并”这类满足可拆分性的约束。

分块 ST 表降低 RMQ 空间

适用场景

普通 ST 表可以 $O(1)$ 回答静态区间最值，但空间为 $O(n\log n)$。当 n 很大或空间受限时，可以分块后只对块建立 ST 表。

核心思路

设块长为 B。预处理：

1. 每个位置到所在块左端的前缀最值；
2. 每个位置到所在块右端的后缀最值；
3. 每个整块的最值，并对块序列建立 ST 表。

查询 $[l,r]$ 时：

- 若 l,r 在同一块，直接块内处理；
- 否则答案由三部分取最值：
  - l 到其所在块右端的后缀最值；
  - 中间完整块的 ST 表答案；
  - r 所在块左端到 r 的前缀最值。

复杂度

空间约为：

$$O\left(n+\frac{n}{B}\log\frac{n}{B}\right).$$

取合适块长后可明显小于普通 ST 表。

易错点

- 同块查询需要单独处理。
- 块间 ST 表维护的是“整块最值”，不是原数组位置。
- 块长选择要结合时间和空间限制。

删除困难时维护前后缀结构

适用场景

如果某种结构支持加入但不支持删除，例如线性基删除、非质数模除法、某些哈希逆操作，可以考虑前后缀分解。

核心思路

对静态序列预处理前缀结构 pre_i 和后缀结构 suf_i。删除区间或单点时，用删除部分左右两侧的信息合并得到答案。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

结构的合并操作要可行。若删除位置在线修改变化频繁，纯前后缀预处理不够。

区间加组合数用高阶差分

适用场景

当区间更新不是常数，而是关于位置的多项式或组合数形式时，可以用高阶差分。

核心思路

组合数满足 $\binom{x}{k}$ 的差分仍是组合数。把要加的函数展开到组合数基底上，对每一阶差分数组做常数区间加，最后多次前缀和还原。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

组合数基底下边界项容易错，建议用小数据手算验证。

单侧递归线段树处理区间可见性

适用场景

当线段树区间信息无法简单由左右儿子直接合并，但查询时一侧一定完整保留，另一侧需要根据阈值递归判断时，可以使用单侧递归线段树。

核心思路

以“从原点能看到多少楼房”为例。每个位置 i 的可见性由斜率决定。线段树节点维护：

- mx：该区间最大斜率；
- cnt：在没有额外遮挡时，该区间内可见的数量。

查询右子区间时，需要知道左侧已经形成的最大斜率阈值 x。递归函数 $solve(o, x)$ 表示在阈值为 x 的情况下，节点 o 的区间贡献多少。

若左儿子的最大斜率 $\le x$，左儿子完全无贡献，直接递归右儿子；否则先递归左儿子，再把右儿子中超过左侧最大斜率的部分加入。

复杂度

修改一个点后更新路径。由于每层可能触发一次递归，常见复杂度为：

$$O(\log^2 n)$$

每次操作。

易错点

- 合并信息不是普通的 $cnt_L + cnt_R$，右区间贡献依赖左区间最大值。
- 比较斜率时尽量用交叉乘避免精度误差。
- 单侧递归依赖“左侧一定优先影响右侧”这类方向性。

复杂区间修改用矩阵维护

适用场景

当一次修改会同时改变多个统计量，且这些统计量之间是线性关系时，可以用矩阵表示修改。

核心思路

把节点维护的信息写成向量 v，一次操作写成矩阵 M，更新就是 $v \leftarrow Mv$。多个操作复合对应矩阵乘法，因此可作为线段树懒标记。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

矩阵乘法通常不交换，懒标记下传时要保持操作顺序。

将直方图横向分解为笛卡尔树

适用场景

当题目在一个柱状图、直方图或高度数组上进行计数、放置、覆盖等操作时，可以考虑把直方图按高度横向切分，并建立笛卡尔树。

核心思路

对于高度序列 $h_1, h_2, \ldots, h_n$，以最小高度为根建立笛卡尔树。根对应整段中最底部的一层矩形，左右儿子对应被最小值分开的左右子直方图。

这样，原本二维的直方图可以转化为一棵树：

- 每个节点对应一个极大横向矩形；
- 子树对应该矩形上方的若干独立区域；
- 很多放置/计数问题可以在树上做 DP。

常见 DP 形式

设 $f_{u,k}$ 表示节点 u 的子树内放置 k 个对象的方案数。转移时先合并左右儿子，再考虑当前节点对应矩形中新增的行或列贡献。

易错点

- 笛卡尔树通常按最小高度建，表示从下往上切分。
- 当前节点对应的矩形高度应减去父节点高度。
- 如果有相同高度，需要固定稳定的比较规则，避免树结构不唯一导致实现混乱。

并查集维护集合和支持单点改权

适用场景

如果集合只合并不拆分，同时要查询每个连通块权值和，可以在并查集根上维护和。

核心思路

每个根维护当前集合的权值和。合并两个集合时把和加到新根上。单点权值改变时，找到该点当前所在根，将根的和加上权值变化量。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

如果需要移动单个元素到另一个集合，普通并查集不够，需要额外编号或可删除并查集技巧。

摩尔投票维护区间高频候选

适用场景

当需要找区间内出现次数超过一定比例的元素时，可以用摩尔投票合并候选。

核心思路

摩尔投票能在线性扫描中保留可能超过半数的候选。推广到前 k 高频候选时，可以维护有限个候选及抵消次数，并在线段树节点间合并。最后对候选真实出现次数进行验证。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

摩尔投票只保证候选集合包含答案，不保证候选一定合法，必须二次计数。

校验补充

保留 k 个 Misra–Gries 候选保证包含频率严格超过 1/(k+1) 的元素，不保证包含真正频数排名前 k 的所有元素。

数组区间加等差数列

对一个数组 a 进行 $q\left(q \leqslant 10^5\right)$ 次区间 $[l, r]$ 加以 x 为首项，d 为公差的等差数列，相当于对于 $i \in [l, r]$ 加 $x+(i-l)\,d$，即 $id+x-ld$ 。因此可以额外维护俩个差分数组 b 和 c，数组 b 区间加 d，数组 c 区间加 $(x-ld)$ 。求前缀和后新数组的 $a_i^{'}$ 为 $a_i^{'}=a_i+ib_i+c_i$ 。

数颜色转为统计前驱位置

适用场景

静态区间询问不同颜色个数时，可以把颜色是否首次出现转为前驱位置限制。

核心思路

令 pre_i 为位置 i 前一个相同颜色的位置。区间 $[l, r]$ 内颜色数等于满足 $i \in [l, r]$ 且 $pre_i < l$ 的位置数。这就变成二维数点，可用离线树状数组、主席树或莫队处理。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

这里统计的是每种颜色在区间内第一次出现的位置。带修改时需要更复杂的数据结构。

最值比较问题放到笛卡尔树上

适用场景

当区间答案由最大值或最小值支配，并且需要比较大小或分治统计时，可以考虑笛卡尔树。

核心思路

笛卡尔树把区间最值变成树上 LCA 或祖先关系。以区间最值为分治点的递归结构，可以直接对应笛卡尔树的子树结构，从而优化枚举区间。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

最大笛卡尔树和最小笛卡尔树方向不同。同值时要固定稳定规则，否则结构不唯一。

绝对众数可用摩尔投票信息维护

适用场景

当题目涉及绝对众数，即出现次数严格大于区间长度一半的元素时，可以考虑使用摩尔投票信息维护候选值。

核心性质

1. 一个序列至多有一个绝对众数。
2. 摩尔投票信息可以用二元组 (val, cnt) 表示，并且可以合并。
3. 若一个区间存在绝对众数，则摩尔投票得到的候选值一定是它。

合并两个信息 (x, a) 和 (y, b)：

$$(x,a)+(y,b)=\begin{cases}(x,a+b), & x=y,\\ (x,a-b), & x\neq y, a>b,\\ (y,b-a), & x\neq y, b>a,\\ (0,0), & x\neq y, a=b.\end{cases}$$

使用方式

数据结构中维护摩尔投票二元组，查询区间时先得到候选值，再额外统计候选值真实出现次数，判断是否超过区间长度一半。

易错点

- 摩尔投票只能给候选值，不能直接证明它一定是众数。
- 最后必须再查一次出现次数验证。
- 合并顺序不影响结果的候选正确性，适合线段树、平衡树等维护。

静态区间单调栈用倍增处理

适用场景

静态数组中，如果每次查询要模拟单调栈弹栈过程，可以预处理倍增跳跃。

核心思路

先求每个位置在单调栈意义下的下一个支配位置，形成一棵树或一条跳链。区间查询时用倍增从起点向后跳，直到越过右端点。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该技巧依赖数组静态。带修改时 next 关系会变化，不能直接倍增。

2.0.6 离线、分治与复杂度优化

三维偏序判定降为二维偏序

适用场景

当题目要求判断三维点 (a, b, c) 之间是否存在支配关系，或者判断某个三元组是否被另一个三元组完全支配时，可以尝试先固定一维。

核心思路

将其中一维排序，排序后只需要在前缀中维护另外二维的信息。例如按 a 从小到大扫描，问题就变成：之前是否存在某个点满足 b 与 c 的二维限制。这样可以用树状数组、线段树、set 或扫描线维护。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

注意同一第一维下的点不能互相非法影响，通常要分组处理。若要求严格小于，需要区分 $<$ 与 $\leq$。

二维限制排序后一维维护

适用场景

查询或统计形如 $i \in [l, r]$ 且 $v \geq x$ 的二维限制时，常可按其中一维排序，另一维用数据结构维护。

核心思路

把询问和元素按阈值 x 排序扫描。扫描过程中把满足值域条件的元素插入树状数组或线段树，然后在位置维上查询区间。二维问题因此变为一维动态查询。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

离线排序时要注意 $<$、$\leq$ 的边界和同值顺序。

二进制分组应对强制在线

适用场景

如果对象不断在线加入，某种静态结构能批量处理但不支持单次插入，可以用二进制分组。

核心思路

维护若干大小为 2^k 的块。新元素作为大小 1 的块加入，若同大小块已存在就合并，类似二进制进位。每个元素参与 $O(\log n)$ 次重建。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

查询时需要合并所有块的答案，因此单块查询复杂度也要可接受。

先模拟到稳定阶段再换做法

适用场景

有些过程一开始状态复杂，直接套某种算法不成立；但经过有限步模拟后，系统会进入具有单调性、稳定性或可 DP 的阶段。这时可以分阶段处理。

核心思路

将过程拆成两部分：

1. 前若干步直接模拟或暴力 DP；
2. 当状态满足某种稳定性质后，改用更高效的算法。

例如在 DAG 上流动点权，前若干时刻某些点可能反复变成 0 又变为正数。但经过不超过最长路径长度的时间后，影响已经沿 DAG 传播完，此后可以用拓扑 DP 计算。

常见形式

- 先模拟 $O(n)$ 步，之后 DP。
- 小范围用 DP，大范围用贪心。
- 小参数部分精确计算，大参数部分按稳定规律处理。

易错点

- 必须证明进入稳定阶段的步数上界。
- 两个阶段的状态要能无缝衔接。
- 不能只凭感觉截断，截断条件必须由题目性质保证。

出现次数问题优先考虑根号分治

适用场景

当问题围绕某个值出现了多少次、出现位置间距、按频率统计时，常可按出现次数大小分类。

核心思路

设阈值 B。出现次数少的值直接枚举其出现位置，总复杂度受 B 控制；出现次数多的值个数不超过 n/B，可以为它们预处理答案或单独维护。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

阈值一般取 $\sqrt{n}$ 附近，但要根据转移复杂度调整。多组数据时注意清空重频值表。

删除操作用时间线段树分治

适用场景

数据结构不容易支持删除，但容易支持添加且可以回滚时，可以按时间线段树分治。

核心思路

把每个对象存在的时间区间挂到线段树节点上。DFS 时间线段树，进入节点时加入该节点所有对象，离开节点时回滚。叶子处回答对应时刻的询问。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

需要使用可回滚数据结构，例如可撤销并查集。不能使用路径压缩破坏回滚。

取整与按位运算的不同值常为对数级

适用场景

固定一端枚举区间时，若维护的是下取整、gcd、按位与、按位或等会单调变化的值，不同取值通常很少。

核心思路

随着区间扩大，按位与只会从 1 变 0，gcd 只会变成更小因子，下取整也只有 $O(\sqrt{n})$ 或 $O(\log V)$ 段。可以把相同值的区间压缩成若干段维护。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

不同运算的复杂度界不同：gcd/and 常见 $O(\log V)$，整除分块常见 $O(\sqrt{n})$。

启发式分裂保证总复杂度

适用场景

当数据结构需要反复把集合或区间分裂，且普通分裂复杂度可能过高时，可以考虑启发式分裂。

核心思路

每次分裂时只遍历较小的一侧，或把元素从小集合搬到大集合，使每个元素被搬运次数至多 $O(\log n)$。这是启发式合并的对偶形式。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

要保证每次处理的确实是较小部分，否则均摊复杂度不成立。

存在性二维数点只维护另一维最值

适用场景

如果二维查询只问是否存在点，而不问点数，可以把其中一维压成最值。

核心思路

按第一维扫描，在数据结构中对第二维维护能达到的最小或最大另一坐标。查询矩形是否有点时，只需判断区间最值是否越过阈值。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

只能处理存在性或支配性查询。若要计数，仍需真正二维数点。

校验补充

要给出明确的扫描与查询限制；任意四边界矩形不能仅用区间最小/最大值判断存在性。例如 y 集合 {0,10}，查询 [4,6]，min/max 跨过区间仍没有点。

将操作离线倒序处理

适用场景

当正向操作是删除、覆盖、破坏结构，难以维护；而反向操作变成添加、确定答案或合并时，可以把操作离线后倒序处理。

核心思路

很多操作正着做很难，反着做很简单：

- 删除点 → 加入点；
- 删除边 → 加入边；
- 区间覆盖 → 最后一次覆盖先确定答案；
- 断开连通性 → 合并连通块。

倒序处理时，已经确定的部分无需再访问，可以用并查集、链表、set 等维护“下一个未处理位置”。

典型例子

若有多次区间赋值，要求最终数组。倒序枚举操作，某个位置第一次被覆盖时，这个值就是最终答案；之后再遇到它可以跳过。

易错点

- 只有所有询问都能离线时才能倒序。
- 反向操作要和正向操作严格对应。
- 如果正向有查询，倒序时要把答案存下来，最后按原顺序输出。

尾部撤销用线段树分治维护

适用场景

如果操作只允许撤销尾部，或者可以把生命周期分解成若干时间区间，可以用线段树分治加回滚。

核心思路

每个修改影响一个时间区间，把它挂到时间线段树上。递归进入节点时应用修改，退出时撤销。这样能在只有“加”和“撤销”的数据结构上处理复杂时间区间。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

所有修改必须可回滚。使用懒标记或路径压缩的数据结构时要额外记录改动。

枚举区间gcd或按位与的种类数

固定右端点，维护以该点为结尾的所有区间的 gcd 或按位与。
这类运算结果单调不增，一旦变化必严格递减，不同取值仅有 (O(\log V)) 种。
利用这点枚举所有不同值的总时间复杂度为 (O(n\log V))。

校验补充

仅“严格递减”不足以推出对数种类数。对正数 gcd，每次真变小至少减半；AND 每次变化至少清掉一个二进制位。允许 0 时 gcd 可能从 0 变正，另加一个初始 0 状态；负整数不适用此单调论证。

枚举区间最值时只枚举较短一侧

适用场景

当需要枚举每个位置作为区间最大值或最小值时，它能控制一个极长区间 $[L_i, R_i]$。若直接枚举所有包含 i 的子区间会退化为 $O(n^2)$，可以只枚举较短的一侧。

核心结论

对于每个位置 i，设它作为区间最值的最大控制区间为 $[L_i, R_i]$。枚举包含 i 的区间时，只枚举 $[L_i, i]$ 和 $[i, R_i]$ 中较短的一侧，则总枚举量为：

$$O(n \log n).$$

直观证明

全局最值所在位置最多贡献 $n/2$ 个枚举量。它把序列分成左右两段后，每段内的最值又分别最多贡献该段长度的一半。逐层递归，总和为：

$$\frac{n}{2} + 2 \cdot \frac{n}{4} + 4 \cdot \frac{n}{8} + \cdots = O(n \log n).$$

常见用途

- 单调栈求每个元素作为最大值/最小值的控制区间。
- 枚举较短侧，用 Trie、哈希表、前缀信息查询另一侧。

易错点

- 控制区间的边界要处理相等元素，避免重复或漏算。
- 较短侧枚举后，另一侧通常要有数据结构支持查询。
- 复杂度是总枚举量 $O(n \log n)$，不是每个点 $O(\log n)$。

离线后动态维护当前状态

适用场景

多组询问直接在线处理困难时，可以离线排序，让状态按某个维度单调变化。

核心思路

把询问按右端点、阈值、时间或权值排序。扫描过程中只做单调加入或删除，数据结构维护当前状态并回答询问。这样常能把二维或三维问题降成一维动态问题。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

离线会改变回答顺序，最后要按原编号输出。若题目强制在线则不能使用。

2.0.7 动态规划、搜索与贪心

CDQ 外套决策单调性手动分层

适用场景

当 DP 转移不天然按层分开，但贡献函数只适合用指针移动或莫队式维护时，可以用 CDQ 分治制造层次。

核心思路

在 CDQ 的每一层，只处理左半区间对右半区间的贡献，这样转移被分成“上一层到下一层”的形式。若决策单调，还能在每层内用分治找最优决策。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

要确保跨区间贡献不会漏算或重复。递归层数带来的 $\log n$ 因子也要计入复杂度。

DP 取值少时只维护断点

适用场景

如果 DP 数组很长，但取值只有少数段相同或转移后仍保持分段结构，可以只维护断点。

核心思路

把 DP 视为一个分段常数或分段线性函数。每次转移只修改少量断点，或者合并相邻同值段，从而避免逐点更新。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

要证明断点数受控。某些操作会导致断点爆炸，需要配合启发式合并或根号分治。

WQS 二分处理带数量限制最优化

适用场景

要求选恰好 k 个对象并最优化，但直接把数量作为 DP 维度太大时，可以用 WQS 二分。

核心思路

给每个被选对象额外加一个惩罚/奖励 λ，去掉数量限制做普通最优化。随着 λ 变化，最优方案选择数量单调变化，二分 λ 找到数量为 k 的答案。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

需要同时维护最优值和选择数量。若数量不单调或有平局处理问题，要谨慎。

校验补充

数量随惩罚单调还不够：必须证明按数量的最优值具有所需离散凸/凹性。二分可能跳过 k，不能声称一定找到数量正好 k 的解；跳跃和平局处需结合包络及恢复证明。

个数和值域小但答案大时做数位 DP

适用场景

当变量个数很少、每个变量的系数很小，但变量本身可能很大时，可以按二进制位做数位 DP，而不是直接枚举变量值。

核心思路

设需要统计非负整数 $c_1, c_2, \ldots, c_n$ 的方案，并满足若干线性约束。若 n 和系数都很小，但上界 m 很大，可以把每个 c_i 拆成二进制位：

$$c_i = \sum_{k \ge 0} b_{i,k} 2^k.$$

DP 从低位到高位枚举所有 $b_{i,k} \in \{0, 1\}$，同时维护各个线性表达式的进位和是否已经贴着上界。

状态常见组成

- 当前处理到第几位；
- 每个约束表达式的进位；
- 是否仍等于上界前缀；
- 当前方案数。

易错点

- 从低位向高位做时，进位是核心状态。
- 上界限制要维护 tight 状态。
- 进位范围由“变量个数 $\times$ 最大系数”决定，不一定很大。

校验补充

低位到高位不能照搬从高位开始的前缀 tight 限制：当前低位大于上界低位仍可能合法。可维护减法借位，或维护已处理低位的三态比较并由新高位覆盖比较；最后检查整体 <= 上界。

交换值域与状态

适用场景

当原状态维度过大，而值域较小；或者原值域过大但状态数量小，可以尝试交换“位置状态”和“值域状态”。

核心思路

例如原本记录 dp_i 表示到位置 i 的最优值，也可以改为记录 g_x 表示末尾值为 x 的最优状态。交换后常能把转移改成前缀最值、区间查询或小值域枚举。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

交换后要确认状态仍能表示完整历史信息。若值域太大，要配合离散化或动态开点数据结构。

从数组中选取 K 个互不相邻的元素，使其和最大化。

一般做法是用 WQS 二分，或用堆进行贪心
用堆进行贪心可以改为用栈来贪心，好写很多

```
i64 maxMNonAdjacentSum(const vector<i64>& A, int M) {
    assert(M >= 0 && size_t(M) <= (A.size() + 1) / 2);
    using Wide = __int128_t;
    const Wide INF = Wide(1) << 100;
    vector<Wide> S, X;
    auto push = [&](Wide x) {
        S.push_back(x);
        while (S.size() > 2 && S.end()[-3] > S.end()[-2] && S.end()[-2] <= S.end()[-1]) {
            Wide s0 = S.back(); S.pop_back();
            Wide s1 = S.back(); S.pop_back();
            Wide s2 = S.back(); S.pop_back();
            X.push_back(s1);
```

```
            S.push_back(s0 + s2 - s1);
        }
    };
    push(INF);
    for (i64 x : A) push(-Wide(x)); // 最大化原值 = 最小化相反数
    push(INF);
    for (Wide x : S) X.push_back(x);
    nth_element(X.begin(), X.begin() + M, X.end());
    Wide ans = 0;
    for (int i = 0; i < M; ++i) ans -= X[i];
    assert(ans ⩾ LLONG_MIN && ans ⩽ LLONG_MAX);
    return i64(ans);
}
```

约定：恰好选 M 个，0 <= M <= ceil(n/2)，结果必须能放入 i64。支持负权和空数组（M=0）。中间运算使用 i128；只需最小的 M 项之和，无需全排序；nth_element 平均时间 O(n)，空间 O(n)，不宣称最坏线性。

值域限制选数按值从小到大 DP

适用场景

如果选择的数受到值域大小、相邻值差或阈值限制，可以考虑按值而不是按位置 DP。

核心思路

把所有元素按值分组，从小到大处理。状态记录已经选了多少、上一组选取情况或当前值域边界。这样可利用值域单调性，避免在原序列位置上复杂转移。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

按值 DP 会丢失原位置顺序信息。若题目同时依赖位置顺序，需要额外状态或不能这样转。

决策单调性可用整体分治找决策点

适用场景

若 DP 转移的最优决策点随位置单调移动，可以用分治优化。

核心思路

计算区间中点的最优决策，再递归左右区间。左区间决策范围不超过中点最优决策，右区间不小于中点最优决策，从而把一层转移从 $O(n^2)$ 降到 $O(n \log n)$ 或 $O(n)$ 级别。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

必须先证明决策单调性。经验套用容易因为代价函数不满足四边形不等式而 WA。

前 k 大方案用堆维护后继状态

适用场景

如果要输出多个序列、多个状态或多个分组中的前 k 大值，可以为每组维护“下一个候选”。

核心思路

先把每组最大状态放入堆。每次弹出当前最大状态 S，再把它的少量后继状态 $trans(S)$ 放回堆。只要所有状态都能由后继关系到达，就能按顺序生成前 k 大。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

要避免同一状态被重复入堆。后继数量过多时，可把多个后继排成链降低出度。

校验补充

还要求后继的值不大于父状态的值（或另有可靠的剩余状态上界），仅“所有状态可达”不能保证按大小弹出。

区间形态合法性转区间 DP

适用场景

如果一个合法对象天然对应原序列上的一段连续区间，或者一次操作会把两个相邻合法区间合并，可以考虑区间 DP。

核心思路

设 $f_{l,r}$ 表示区间 $[l,r]$ 是否可行、最小代价或方案数。枚举中间断点 k，尝试用 $[l,k]$ 与 $[k+1,r]$ 合并。若合并条件只依赖边界或区间统计量，可先预处理这些信息。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

区间 DP 的关键是证明合并后仍是一段区间。若合法结构会跨越多个不连续部分，直接使用区间 DP 容易漏状态。

反悔贪心处理动态选择

适用场景

当贪心过程中当前选择未来可能被更优选择替换时，可以允许先选，再用数据结构反悔。

核心思路

维护已选集合的最差元素。当遇到更优候选且违反数量或容量限制时，用当前候选替换已选集合中代价最大的元素。优先队列常用于支持这种“后悔”操作。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

必须证明每次替换不会让未来变差，通常依赖 matroid、排序后单调约束或交换论证。

合法性判定同时维护增减方向

适用场景

有些问题不只要判断某个参数是否合法，还能在判定时看出答案应该增大还是减小。

核心思路

对参数 x 做判定时，同时计算“不足量”和“超出量”。如果不足说明 x 太小，超出说明 x 太大，就可以二分出合法区间或唯一答案。该技巧常见于分数规划、构造后修正、平衡约束。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

只有方向具有单调性时才能二分。若可行集合不是连续区间，单纯二分会错。

子集求和使用高维前缀和

适用场景

状压中需要对所有 $T \subseteq S$ 或 $T \supseteq S$ 的 f_T 求和时，使用 SOS DP。

核心思路

对每一维二进制位做一次前缀和转移。求子集和时，若 S 含有第 i 位，则令 $g_S += g_{S\setminus\{i\}}$。总复杂度 $O(n2^n)$。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

子集和与超集和的转移方向相反。不要把两者写混。

差分贡献使 DP 转移单调

适用场景

当 DP 中某些贡献有加有减，导致状态无法单调优化时，可以尝试改成统计贡献的差分量。

核心思路

不直接维护总贡献 F，而维护相邻状态或相邻时刻的增量 ΔF。如果原贡献会反复增减，差分后可能变成只增不减或只在少数位置变化，从而支持单调队列、线段树或扫描线维护。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

差分后的答案要能还原原贡献。多维差分时尤其要注意边界补偿。

延迟贡献处理暂时无法确定的决策

适用场景

在 DP 中，如果某些决策当前还无法判断最终是否贡献，但以后某个状态变化后就能确定，可以先钦定这些对象存在，却暂时不计算贡献，等到后续转移时再统一结算。

核心思路

把对象分成两类：

- 已经确定不再改变贡献的对象；
- 未来可能产生贡献的对象。

DP 状态中只记录第二类对象的数量或必要信息，不立刻乘上它们的贡献。当状态推进到某个阈值时，再把满足条件的对象一次性计入。

典型状态

设 $f_{i,j,k}$ 表示处理到第 i 步，当前已有 j 个失败对象，且有 k 个对象暂时还不能确定贡献。后续当 j 增加时，再从这 k 个对象中选出一部分结算。

易错点

- 延迟的对象必须在状态中保留足够信息，否则后面无法正确结算。
- 结算时要乘组合数和排列数，避免漏算顺序。
- 需要保证每个对象恰好被结算一次。

强制选或禁用询问用前后缀 DP

适用场景

多次询问要求某个物品不能用、必须用，或临时改变一个位置时，可以预处理前缀 DP 和后缀 DP。

核心思路

令 pre_i 表示只考虑前 i 个物品的状态，suf_i 表示只考虑后缀的状态。回答询问时跳过或强制处理中间物品，再把左右两边 DP 合并。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

合并操作需要满足结合性或能在状态数内暴力合并。状态过大时前后缀 DP 仍可能不可行。

折半搜索处理 n 约 40 的状压

适用场景

当 $n \approx 40$，直接 2^n 枚举不可行，但每半 $2^{n/2}$ 可接受时，使用折半搜索。

核心思路

把元素分成两半，分别枚举所有状态并记录信息，再通过排序、哈希、双指针或数据结构合并两边答案。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

折半后的合并方式决定复杂度。不要只枚举两边后再笛卡尔积合并，否则仍是 2^n。

最大化集合最小值时从大到小扫描

适用场景

当目标是最大化某个集合内的最小值，可以枚举或扫描答案阈值。

核心思路

从大到小加入权值不小于当前阈值的对象，维护它们能否形成合法集合。第一次满足条件的阈值就是最大可能的最小值。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

需要合法性随阈值降低单调变强。否则不能直接扫描。

用强制性描述 DP 状态

适用场景

普通 DP 常把状态设成“当前元素选或不选”。但有些题中，真正影响后续转移的不是是否已经选择，而是某个元素是否被强制选、强制不选或强制匹配。

这时可以改变状态含义，用“强制性”来描述状态。

核心思路

普通状态：

$$f_{i,0/1} = \text{第 } i \text{ 个元素不选/选时的最优值。}$$

强制性状态：

$$f_{i,0/1} = \text{第 } i \text{ 个元素是否被强制满足某种条件时的最优值。}$$

这样能把一些重叠、包含、后缀匹配、树形选择的限制变成局部转移。

例子

字符串匹配中，与其记录“当前位置是否匹配成功”，不如记录“当前后缀是否被强制匹配”。树上独立集计数中，也可以记录“当前点是否被强制不选”，从而简化子树之间的限制。

易错点

- 强制状态不是实际选择状态，最后统计答案时要转换回来。
- 初始化要特别小心，强制状态通常会排除一些原本可行的决策。
- 需要证明非强制部分可以由后续转移自然覆盖。

用抽屉原理转化比例上界构造

适用场景

当题目要求构造一个方案，并限制方案大小不超过某个总量的 $\frac{1}{k}$ 时，可以尝试构造 k 个方案，使它们的大小总和恰好不超过该总量。

核心思路

如果能构造 k 个合法方案 $S_1, S_2, \dots, S_k$，并证明：

$$\sum_{i=1}^{k} |S_i| \le T,$$

那么由抽屉原理，至少存在一个方案满足：

$$|S_i| \le \frac{T}{k}.$$

于是原本难以直接构造“小方案”的问题，变成了构造 k 个覆盖方式良好的方案。

常见用途

- 网格染色构造；
- 支配集或覆盖集构造；

- 把奇怪的 $\frac{1}{5}$、$\frac{1}{3}$ 等限制转化为模意义下的若干类方案。

易错点

- 必须保证每个 S_i 都是合法方案。
- 要证明大小总和确实不超过目标总量 T。
- 最后输出最小的那个方案，而不是随便输出一个。

钦定不再操作的元素简化序列过程

适用场景

当序列操作会不断移动、删除或重排元素，直接描述整个序列状态过大时，可以考虑钦定一部分元素从某一时刻起不再被操作。

核心思路

与其记录完整排列，不如记录“已经固定下来、以后不再动”的元素数量或结构。

如果这些元素在合法过程中会形成某种简单形态，例如递增后缀、已确定前缀、已归位集合，那么 DP 状态就可以从排列降维为数量。

例子

对一个排列反复把第一个元素插入到某个位置后，如果钦定最后 j 个元素以后不再操作，并且它们必须形成递增后缀，则状态可以设为：

$$f_{i,j} = \text{操作 } i \text{ 次后，已经钦定 } j \text{ 个后缀元素的方案数。}$$

转移时只需讨论当前第一个元素是否是最后一次被操作。

易错点

- “不再操作”是人为钦定的辅助条件，不一定是题面显式条件。
- 需要证明被钦定的元素确实形成简单结构。
- 统计时要避免同一操作序列被多次钦定导致重复计数。

2.0.8 图论建模与网络流

DAG 的分层图仍可按 DAG 处理

适用场景

当原图缩点后是 DAG，而题目允许使用一次特殊操作、一次反向边、一次状态变化时，可以考虑建立分层图。

核心结论

如果每一层内部都是同一个 DAG，并且跨层边只从低层指向高层，那么整个分层图仍然是 DAG。

因此，原本可以在 DAG 上做的 DP，在分层图上通常仍然可以直接按拓扑序做。

常见建图方式

假设原 DAG 中有边 $u \to v$，若允许最多一次逆向走边，可以建两层：

- 第一层表示“还没有使用特殊操作”；
- 第二层表示“已经使用特殊操作”。

对于原边 $u \to v$：

- 正常走：$u_0 \to v_0$，$u_1 \to v_1$；
- 使用一次反向：$v_0 \to u_1$。

如果跨层后不能回到低层，那么新图仍然无环。

易错点

- 原图不是 DAG 时要先缩点。
- 跨层边必须单向跨层，否则可能重新产生环。
- 层数代表状态，不要把“最多一次”和“恰好一次”混淆。

Dijkstra 出队顺序定向成 DAG

适用场景

最短路问题中，如果每个点只需在最短距离确定后处理一次，可以利用 Dijkstra 的出队顺序构造偏序。

核心思路

Dijkstra 中点被弹出时距离单调不降，因此可以把从较小距离指向较大距离的关系看作 DAG 上的转移。这样原图上的一些计数或 DP 可以改为按最短路距离排序后做。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

只有非负边权下 Dijkstra 出队距离才有该单调性。等距边需要额外处理，可能形成同层边。

以主环为基准求有向环交

适用场景

当题目要求求所有环的公共部分，直接枚举所有环不可行。可以先找一个主环，再判断主环上哪些点一定不在所有环的交中。

核心思路

先在图中找到任意一个环，称为主环。所有环的交如果非空，那么答案一定在主环上。

对于主环上的每个点 u，考虑从 u 出发，不使用主环边，通过非主环边能到达主环上的最远位置 v。那么主环上 u 到 v 中间的点会被某个绕行环避开，因此不可能属于所有环的交。

于是可以：

1. 找出一个主环；
2. 对非主环边做 DFS，计算能到达的最远主环位置；
3. 在主环上做差分，标记不可能成为答案的区间；
4. 剩余点再验证合法性。

易错点

- 主环只是候选范围，最后仍需验证答案。
- 非主环路径可能产生复杂交错情况，不能只贪心取一个点后直接结束。
- 环上区间是循环区间，差分时要处理跨过起点的情况。

校验补充

候选 u 属于所有有向环，当且仅当删除 u 后图无有向环。可用删点+拓扑排序作 O(n(n+m)) 校验基准。非主环部分若自身有环，所有主环候选都被排除；绕行路径也必须保证内部不再经过主环，简单的“最远位置 DFS”不构成完整实现。

传递闭包用 bitset 加速

适用场景

需要求有向图可达性矩阵，且 n 在几千级时，可以用 bitset 优化传递闭包。

核心思路

若在 DAG 上处理，按拓扑序倒序令 $reach_u$ 或上所有后继的 $reach_v$ 并集；一般图可先缩点。bitset 的按位或能把一整行转移压到 $O(n/w)$。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

一般有环图先缩点更稳。Floyd-bitset 写法也要注意循环顺序。

关于二分图最小点覆盖的一种建图方式，以及其操作方式

CF1198E（一个很好的题）

对于给出一个矩阵，然后询问他的黑点取消。这个可以转化等价为行列分别作为二分图的两个集合的点，随后，我们连边就是依靠他已经生成的黑色矩阵连边，我们给点染色，意味着此行或者此列的全部黑色已经被消除，这个染色意味这覆盖，那么问题就可以转化成二分图最小点覆盖问题。

tips:二分图最小点覆盖就是最大匹配的等价，于是可以转化成最大流。

同理，这个题就是如此处理，但是n是1e9该如何处理呢？

我吃饭的时候思考了一下，发现处理这种矩阵，我们通常有一种方法：扫描线思想，那么，我们可以确定，该区间如果在行的位置起点与终点已知（坐标离散化），我们就可以利用该覆盖选满操作（线段树优化建图/离散矩阵建图）变成一个将其变成一个坐标合并信息。将点上信息设定为代价点权，随后跑一个dinic。

tips:离散化的是所有黑色矩形的行上下界和列左右界

图上分治避免重复计算未受影响部分

适用场景

当题目要求枚举删点、删边、保留某些边，且每次都重新计算图上信息会产生大量重复时，可以考虑图上分治。

核心思路

设当前处理的决策范围是 $[l,r]$。递归处理时，可以先钦定一部分元素一定存在或一定被使用，把这些元素对图结构的影响一次性加入，然后递归处理另一半。

这样，未受当前决策影响的部分不用每次重算。

例如枚举删除点并计算全源最短路时，如果当前要处理 $p\in[l,r]$，可以在分治中预先加入不在当前子问题中的中间点，避免每个 p 都重新跑一遍 Floyd。

常见搭配

- Floyd 分治：处理多次删点后的最短路贡献。
- 可撤销并查集：处理边权区间或选边约束的分治。
- 分治 + 回滚：递归进入子问题前加入约束，退出时撤销。

易错点

- 递归时要明确“当前哪些元素已经被固定加入/删除”。
- 使用并查集时一般需要可撤销版本。
- 分治优化不是减少单次复杂度，而是减少重复计算。

图上度数的trick

设有 k 种不同的非负度数，握手定理给出

$$k(k-1)/2\le\sum_v deg(v)=2m.$$

因此 $k\le(1+\sqrt{1+16m})/2=O(1+\sqrt{m})$。
原来的“不超过 sqrt(m) 种”及 sqrt(2m) 常数均不成立。例如三点路径有度数 1、2 两种，而 sqrt(2)<2。
枚举两种度数组合为 O(m+1)。孤立点也要计入度数 0。

图上边权的Trick

在具有 m 条边的图中，最多只有 $O(\sqrt{m})$ 种度数不同的结点。

图直径限制所有点最远距离范围

适用场景

当题目涉及每个点到最远点的距离，而全图直径已知时，可以用直径给出统一上下界。

核心思路

设图直径为 s，点 u 到最远点距离为 f_u，则有 $f_u\le s$，同时由三角不等式可得 $f_u\ge s/2$；仅当距离为整数时才能进一步取上整。这能减少分类或帮助证明某些点必在直径附近。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该结论适用于连通无权图或按最短路度量的连通图。带权图也要保证距离满足三角不等式。

多个无向简单环点集交至多为二

结论

在无向图中，如果存在多于一个本质不同的简单环，那么这些简单环点集的公共交集大小通常可以被限制得很小；特别是在讨论两个不同绕行方向造成的约束时，可以利用“共同点超过两个会迫使环结构重合或分解”的思想。

一个常用判断是：若一个候选集合要同时属于许多简单环，重点只需检查少数关键点，而不是把所有环完整枚举出来。

直观理解

简单环上任取两个点，会把环分成两条简单路径。若另一个简单环也同时经过很多相同点，则这些共同点会把两个环切成多段。只要其中某一段选择不同，就会产生新的环，从而破坏“所有环都必须经过这些点”的强约束。

因此，当题目要求所有无向环的公共点时，可以尝试证明候选点数量很小，再对候选点做验证。

使用方式

1. 先找到若干个环或一个基准环。
2. 通过两个环的交缩小候选集合。
3. 若候选集合超过常数，尝试构造反例环排除多余点。
4. 最后对候选点逐个验证是否在所有环中。

易错点

- 该结论依赖“简单环”的点集交，不是边集交。
- 图中只有一个简单环时，公共交集可能是整个环。
- 使用时应结合具体题目给出严格证明。

校验补充

这里的“所有环”必须是图中全部不同简单环，不能只取任意两个环就声称交点至多二。例如一条长 u-v 路径加两条内部点不同的 u-v 绕路，选出的两个环可以共享整条长路径；第三个环才把公共交限制到 u,v。

完全图特殊边权用 Boruvka 优化 MST

适用场景

完全图有 $O(n^2)$ 条边，无法显式建图；但如果边权由点权通过某种规律计算，可以考虑 Boruvka。

核心思路

Boruvka 每轮只需要为每个连通块找一条连向外部的最小边。若边权如异或、曼哈顿、线性函数等，可以用 Trie、分治、扫描线等结构快速找每个块的最优外连边。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

关键是避免同一连通块内部点被选作目标。每轮合并后需要更新连通块标号。

将二元组约束转化为图上连边

适用场景

当题目给出大量二元组 (a,b)，并且每个二元组表达的是两者之间的约束、影响、依赖或操作关系时，可以考虑把元素看成点，把二元组看成边。

核心思路

二元组常常不是“两个数字”，而是一条关系：

- (l,r) 可以表示区间差分后的两个端点；
- (a,b) 可以表示 b 对 a 的限制；
- 一次操作影响两个状态，可以表示为选择一条边。

建图后，问题可能变成：

- 图上选边改变两个端点状态；
- BFS/DFS 求最长或最短约束链；
- 在生成树上自底向上构造方案；
- 判断连通块内约束是否可满足。

典型例子

对 01 序列做区间取反，可以先做异或差分。区间 $[l,r]$ 取反会变成只修改两个端点，于是可以把一次操作看成连接两个端点的一条边。

易错点

- 建图前要确认二元组是有向还是无向。
- 如果坐标很大，通常要先离散化。
- 差分后端点可能包含 $r+1$，不要漏掉边界。

将无向图问题转到生成树上处理

适用场景

当一个无向图上的操作、构造或判定不好直接处理时，可以先取出一棵生成树，把非树边看作额外自由度或环约束，再在树上做问题。

核心思路

无向连通图可以拆成：

- 一棵生成树；
- 若干条非树边。

树上的结构容易自底向上处理；每条非树边与树上路径会形成一个环。很多图上问题可以转化为：

1. 在生成树上确定唯一或基础方案；

2. 用非树边表示可自由选择的部分、环约束或路径覆盖。

例如求桥时，任意非树边都会覆盖它两端之间的树上路径。没有被任何非树边覆盖的树边就是桥。

常见用途

- 树上差分判桥。
- 图上反色/选边操作转为树上自底向上处理。
- 用非树边贡献自由度，统计方案数。

易错点

- 如果原图不连通，需要对每个连通块取生成森林。
- 非树边形成的环约束不能遗漏。
- 有些问题中不同生成树都可用，但证明要说明答案不依赖具体生成树。

拓扑队列中的零入度点互不可达

适用场景

在 DAG 拓扑排序过程中，同时处于零入度集合的多个点之间不存在可达关系。

核心思路

若零入度点 u 能到达零入度点 v，则路径上进入 v 的最后一条边尚未被删除，v 的入度不可能为零，矛盾。该性质可用于构造、唯一性判断或反链分析。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

这里指当前剩余图中的可达性。原图中已删除点经过的路径不应再参与判断。

无向图边可拆成双向边跑网络流

适用场景

当网络流模型中的边是无向边时，不必强行改模型。通常可以把一条无向边拆成两条方向相反、容量相同的有向边。

核心做法

对于无向边 (u,v)，容量为 c，建边：

$$u \to v, \quad cap = c,$$

$$v \to u, \quad cap = c.$$

这样流量可以选择任意方向通过这条边。

为什么可行

在最大流/最小割模型中，一条无向边连接割的两侧时，无论它原本被看作哪个方向，都会对割贡献 c。用两条反向有向边表示后，从源侧到汇侧的那条有向边会被割掉，贡献仍为 c。

易错点

- 这里说的“双向边”是建模意义上的两条有向边，不是 Dinic 中自动加入的反向残量边。
- 如果题目限制一条无向边总流量不能双向同时使用，要确认双向建边是否仍符合题意。
- 最小割类问题通常可以直接这样建；费用流或带额外限制时需要重新检查。

时间信息转分层图

适用场景

当图上除了点编号还出现时间、步数、次数、阶段等信息时，可以将这些信息作为额外维度建分层图。

核心思路

把状态写成 (u,t)，表示在时间/阶段 t 处于点 u。原图上的一次移动变为层间边。这样带时间限制的可达性、最短路、DP 都能在状态图上统一处理。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

分层数量过大时不能直接建图，需要取模、周期化、矩阵快速幂或最短路剪枝。

最小乘积生成树用凸包分治

适用场景

每棵生成树有两个权值和 (X, Y)，目标最小化 XY 或找二维权值 Pareto 前沿时，可以用凸包思想。

核心思路

对线性组合 $aX + bY$ 求最小生成树，可以得到二维平面凸包上的一个点。递归处理两个已知点之间的支撑线，找新的极值点，从而逼近所有可能最优候选。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

该方法只保证找凸包上的候选。若目标函数最优点可能在非凸包点，需要额外证明。

最小割贡献拆成三类布尔项

适用场景

二元选择问题中，如果代价由点选 0/1 和二元关系决定，可以尝试最小割建模。

核心思路

把每个变量是否在源侧表示成布尔值。单变量代价用源点或汇点连边表示；形如 $[x = 0, y = 1]$ 的二元惩罚用从 x 到 y 的有向边表示；常数项单独加入答案。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

不是所有二元代价都能直接最小割，需要满足次模性。

校验补充

明确编码：若源侧代表 0、汇侧代表 1，边 x->y 的割容量对应 [x=0,y=1]；若源侧代表 1，应反向连边。

次数限制拆点转网络流

适用场景

当某个对象最多被使用若干次、至少使用若干次，或者每种资源有容量限制时，可以考虑拆点建流。

核心思路

把一个点拆成入点和出点，中间连容量为限制次数的边。所有经过该对象的流量都必须通过这条边，于是次数限制转为容量限制。若有限制上下界，可以用有上下界网络流处理。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

拆点后要检查原边连接到入点还是出点。若限制的是边使用次数，不要误拆成点容量。

点权边权互相转化

适用场景

当算法更适合处理边权但题目给点权，或者反过来时，可以考虑点权边权转化。

核心思路

树上常见做法是把边 (fa, u) 的权值放到点 u 上，路径查询时去掉 LCA 点；图上也可通过拆点，把点权变为入点到出点之间的一条边权。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

树上边转点时根节点没有对应父边。路径查询时是否包含 LCA 是最容易错的地方。

用生成树基本环表示无向图任意环

适用场景

在无向图中，如果问题和环的异或和、奇偶性、环覆盖有关，可以先取一棵生成树，用非树边形成的基本环表示所有环。

核心结论

取图的一棵生成树。对于每条非树边 (u,v)，它和树上 $u \to v$ 的路径会形成一个基本环。

无向图中的任意环，都可以由若干个基本环异或得到。

这里的"异或"可以理解为边集按出现次数模 2 合并，出现两次的边抵消。

常见用途

1. 最大异或路径：
   - 任取一条 $s \to t$ 树上路径；
   - 将所有基本环的异或值插入线性基；
   - 用线性基最大化路径异或值。
2. 删除边使图变二分图：
   - 奇环可以由基本奇环组合得到；
   - 统计所有奇环必须经过的边。

易错点

- 非树边形成的是"基本环"，不是所有简单环。
- 路径异或问题中，非树边贡献为 $dis_u \oplus dis_v \oplus w$。
- 若图不连通，需要对每个连通块分别处理。

校验补充

线性基最大化通常得到允许重复点/边的游走 XOR，不保证存在相同 XOR 的简单路径。不能把该结论用于要求简单路径的问题。

用矩阵快速幂加速固定边数 Floyd

适用场景

当题目要求从 s 到 t 经过恰好 k 条边，或一个图上转移需要重复执行很多次时，可以考虑将 Floyd 的转移写成矩阵乘法形式，然后用快速幂优化。

核心思路

普通矩阵乘法是：

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}.$$

最短路中的"乘法"可以换成加法，"加法"可以换成取最小值，也就是 min-plus 乘法：

$$C_{i,j} = \min_k (A_{i,k} + B_{k,j}).$$

若 $A_{i,j}$ 表示从 i 到 j 经过恰好一条边的最短路，那么 A^k 在 min-plus 意义下的结果就表示经过恰好 k 条边的最短路。

使用条件

- 状态转移满足结合律，能被重复合并。
- 常见形式是 $\min$ 与 $+$ 的组合。
- 点数通常较小，允许 $O(n^3 \log k)$。

易错点

- 这是恰好 k 条边，不是至多 k 条边。
- 不存在的边需要设为无穷大。
- 自环是否为 0 要根据题意决定；如果要求恰好走边，一般不能随意加 0 自环。

用耳分解处理强连通或边双连通子图

适用场景

当题目要求在图中选择一个强连通子图或边双连通子图，并在子图上做最优化或计数时，可以考虑耳分解。

核心概念

一条“耳”可以是一条简单路径或简单环。若一张图可以不断删去耳，直到边全部被删除，则称其可耳分解。

常用性质：

- 有向图可耳分解，当且仅当它强连通。
- 无向图可耳分解，当且仅当它边双连通。

因此，构造强连通/边双连通子图时，可以转化为逐步加入耳。

做法轮廓

对于点数较小的图，可以用状压 DP 表示当前已经被加入的点集，以及当前正在构造的一条未闭合的耳的端点状态。

例如设：

$$f_{S,i,j}$$

表示当前已使用点集为 S，正在构造的耳从某个已有点出发，目前末端在 i，最终要连向 j 的最优代价。

易错点

- 耳内部通常不能重复点。
- 开耳的两个端点应接到已有结构中。
- 该技巧常用于小 n 的状压问题，大图上不能直接暴力枚举耳。

校验补充

耳分解不能定义成任意删去简单路径。构造应从一个环出发，每次加入一条内部点全新的耳，两个端点属于已有部分；允许端点重合的闭耳对应边双连通。若无向图仅允许端点不同的开耳，则对应点双连通而非一般边双连通。

用补集转化子集限制

适用场景

当题目限制某个子集难以直接处理时，可以考虑研究它的补集。图论中最常见的是独立集和点覆盖互补。

核心结论

在无向图 $G=(V,E)$ 中，点集 S 是独立集，当且仅当 $V\setminus S$ 是点覆盖。

也就是说：

$$S\text{ 是独立集 }\iff \forall (u,v)\in E,\ u\in V\setminus S\text{ 或 }v\in V\setminus S.$$

因此，求大小为 $n-k$ 的独立集数量，可以转化为求大小为 k 的点覆盖数量。

作用

如果 k 很小，补集大小小，就可以围绕点覆盖进行搜索、剪枝或参数化处理。

例如：若某个点度数大于 k，那么它必须在点覆盖中；否则需要选择它的所有邻点，数量会超过 k。

易错点

- 独立集和点覆盖是互补关系，但只在同一个点集全集 V 下成立。
- 最小点覆盖、最大独立集与“指定大小计数”是不同问题。
- 有重边时覆盖关系不变，但计数实现中要避免重复处理边。

校验补充

度数 >k 的强制选择推理要求简单图，或将“度数”替换成不同邻点的个数。重边不增加必须选择的邻点数。

用质因数虚点优化 gcd 连边图

适用场景

若图中两个点 i,j 是否连边由 $\gcd(a_i,a_j)>1$ 决定，直接两两判断会达到 $O(n^2)$。可以引入质因数虚点优化建图。

核心思路

对于每个原点 i，分解 a_i 的所有不同质因数 p，然后连边：

$$i\leftrightarrow p.$$

这样，两个数 a_i,a_j 有公共质因数，当且仅当它们能通过某个质因数虚点相连：

$$i \to p \to j.$$

原图中一条边权为 1 的边，在新图中会变成长度为 2 的路径；如果要求输出原图路径，BFS 后只保留原点即可。

复杂度

如果预处理最小质因子 $minp$，总建图复杂度约为：

$$O(V + n \log V),$$

其中 $V = \max a_i$。

易错点

- 同一个数的相同质因数只应连一次，避免重复边。
- 新图距离和原图距离可能相差一倍，输出答案时要换回原点路径。
- 若 $a_i = 1$，它没有质因数虚点，通常无法通过 gcd 边连接其他点。

竞赛图

竞赛图：基图为无向完全图的有向简单图。

性质：

- 竞赛图强连通缩点后的DAG呈链状，前面的所有点向后面的所有点连边。
- 任意竞赛图都有哈密顿路径；存在哈密顿回路当且仅当强联通。
- 竞赛图中大小为 n 的强联通子图中存在大小为 $[3, n]$ 的环。
- 兰道定理(Landau's Theoerm): 不降序列$\{s_n\}$是合法的比分序列（即竞赛图的出度序列）当且仅当 (\forall 1 \leqslant k \leqslant n, \sum\limits*{i = 1}^{k}s_i \geqslant \tbinom{k}{2}), 且$\sum\limits*{i = 1}^n s_i = \tbinom{n}{2}$。

比分序列要求为非负整数。哈密顿回路的强连通等价结论约定 n>=3。

给无向边定向优化枚举

适用场景

在无向图中枚举路径、三元环或邻点对时，直接枚举可能达到 $O(\sum d_i^2)$。可以按度数给无向边定向，使每个点的出度受控。

核心做法

对每条无向边 (u, v)，规定：

$$u \to v \iff (deg_u, u) < (deg_v, v).$$

也就是从度数较小的点指向度数较大的点；度数相同时按编号定向。

复杂度依据

这样定向后，每个点的出度可以按根号级别分析：

- 若 $deg_u \le \sqrt{m}$，显然 $out_u \le \sqrt{m}$；
- 若 $deg_u > \sqrt{m}$，这种点最多 $O(\sqrt{m})$ 个，因此出度也为 $O(\sqrt{m})$。

所以很多枚举可以从 $O(\sum d_i^2)$ 降为：

$$O(m\sqrt{m}).$$

典型用途

无向图三元环计数：枚举有向边 $u \to v$，再枚举 v 的出边 $v \to w$，判断 $u \to w$ 是否存在。

易错点

- 定向规则必须形成全序，否则会漏边或重复。
- 统计三元环时要保证每个环只被计一次。
- 复杂度证明依赖边数 m，不是点数 n。

边至少或恰好经过一次考虑欧拉路

适用场景

如果题目要求一组边被至少经过一次或恰好经过一次，并且关心一条行走路径，可以考虑欧拉路模型。

核心思路

图中一笔画路径天然对应每条边经过一次的欧拉路。若要求至少一次，可以补边或重复边，使奇度点数量满足欧拉条件，再计算附加代价。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

欧拉路关注边而不是点。无向图和有向图的度数条件不同，连通性也要检查。

迷宫方向偏移数组模板

当你在做迷宫一类的题目时，肯定会遇到偏移的问题，建议直接把以下部分加入自己的火车头里，在需要的时候直接取消注释即可（也有人喜欢用二维数组的没有关系，因人而异，自己习惯就好）

```
// const int dx[4] = {-1, 1, 0, 0};
// const int dy[4] = {0, 0, -1, 1};
// const int dx[8] = {-1, 0, 1, 1, 1, 0, -1, -1};
// const int dy[8] = {1, 1, 1, 0, -1, -1, -1, 0};
// const int dx[12] = {-1, 0, 1, 1, 1, 0, -1, -1, 0, 2, -2, 0};
// const int dy[12] = {1, 1, 1, 0, -1, -1, -1, 0, 2, 0, 0, -2};
```

2.0.9 树上算法

LCA 路径拆成向上和向下两段

适用场景

树上路径统计中，如果从 u 到 v 的贡献方向不同或不可交换，可以把路径按 LCA 拆开。

核心思路

令 $x=\mathrm{lca}(u,v)$，路径可拆为 $u\to x$ 和 $x\to v$。如果第二段方向不方便，可以预处理从下到上和从上到下的两类信息，查询时分别合并。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

如果运算不满足交换律，合并顺序必须严格保持路径顺序。

动态确定节点带权树的重心

带权重心:
考虑 dfn 序。
存在最小的 x 使 $[1,x]$ 的权和 $\geq S/2$。
*使用 **线段树二分** 确定 x。*
重心在 x 到根的链上，使用 倍增 确定。

校验补充

要求点权非负。总权 S=0 时任意点是重心；S>0 时按 2*prefix>=S 找中位权重点，避免整数 S/2 向下取整。沿其祖先链找到最深的、子树权重满足 2*subtree>=S 的点，即一个带权重心。比较乘 2 时防溢出。

换根后求两点的lca

设 $LCA_{rt}(u,v)$ 表示根为rt时 (u,v) 的最近公共祖先，那么 $LCA_{rt}(u,v)$ 就是 $\{a=lca_1(u,v),\ b=lca_1(u,rt),\ c=lca_1(v,rt)\}$ 中深度最大的点

最远点性质

在边权非负的树中，对于任意一个非空点集 S 和树上任意一点 x（x 可以不在 S 中），S 中距离 x 最远距离可由 S 的一对直径端点中的至少一个取得（并不表示所有并列最远点都是这两个端点）。

树上两条链交用四个 LCA 求

适用场景

需要判断树上两条路径是否相交，或求交集端点时，可以利用 LCA 和深度。

核心思路

路径 (a,b) 与 (c,d) 的交集若非空，交集端点通常来自若干个 LCA 候选。计算 $\mathrm{lca}(a,c),\mathrm{lca}(a,d),\mathrm{lca}(b,c),\mathrm{lca}(b,d)$，按深度筛选即可。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

不同题对“交”是点交还是边交要求不同，边交时需要排除只有端点相交的情况。

校验补充

不能只按深度选点而不验相交。稳妥做法：把四个交叉 LCA 去重，仅保留同时位于两条路径上的候选；用 dist(a,x)+dist(x,b)=dist(a,b) 判断。无候选则空交，否则取候选中相距最远的一对作为交路径端点。

树上多点 LCA

结论

树上多点 LCA，相当于求所有查询点中 dfn 序最小的点和所有查询点中 dfn 序最大的点这两点的 LCA。

做法

通过 ST 表预处理出区间最小 dfn 序和区间最大 dfn 序，然后每次查询两点的 LCA 即可。时间复杂度为 $O(n\log n+q)$。

校验补充

所写 O(n log n+q) 需要 LCA 查询为 O(1)；使用二进制倍增 LCA 时为 O((n+q) log n)。dfn 指固定根 DFS 首次进入序，点集不能为空。

树上操作从叶子向根构造

适用场景

当题目要求在树上选择边、修改状态、调整度数或构造方案时，可以尝试从叶子向根处理。叶子的子树已经没有后续影响，决策通常是被迫的。

核心思路

在有根树中，自底向上处理每个节点。对于一条父边，它是当前子树和外界唯一的连接，因此子树内部若有未满足的奇偶性、颜色、需求等，往往只能通过这条父边传给父亲。

因此可以：

1. 先 DFS 建树；
2. 从叶子开始确定子树内决策；
3. 把未解决的信息上传给父亲；
4. 到根时检查全局条件。

常见用途

- 调整所有点度数为偶数。
- 构造树边选择方案。
- 树上反色、选边覆盖等问题。
- 把一般图先转成生成树，再在树上自底向上做。

易错点

- 根没有父边，最后必须额外检查根的状态。
- 如果原图不是树，通常需要先取生成树，非树边另行处理。
- 自底向上构造依赖子树独立性，不能用于存在跨子树强约束的情况。

树上点集直径合并

树上两个点集的直径可以合并，令S的直径端点为(a,b)，令T的直径端点为(c,d)，那么集合S并上T的直径端点为$\{a,b,c,d\}$中的一对端点；
所以可以用线段树来维护，每个节点存管辖区间的点集的直径端点，每个叶子节点代表一个节点，初始化为$\{i,i\}$，之后树内节点通过左右儿子合并。

校验补充

要求树边权非负，两个点集非空；空集作为合并单位元须另行处理。

树剖边权转点权

适用场景

树链剖分通常在线段树上维护点的 dfn 序；如果题目给的是边权，可以把边权挂到深度更大的端点上。

核心思路

对边 (u,v)，设 v 是较深点，则把该边权存在 v 的位置。查询路径 (x,y) 时，在线段树上查询路径点集，但需要去掉 $\mathrm{lca}(x,y)$ 对应的位置。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。

- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

修改边权时要定位到深度更大的端点。路径查询不能把 LCA 的点值误算进去。

树形 DP 反向保存祖先信息

适用场景

树形 DP 中如果状态需要知道某个子孙信息，但从父亲向儿子转移不自然，可以反过来保存祖先信息。

核心思路

把 $f_{u,son}$ 这类"从 u 看向子树"的状态，改为 $f_{u,fa}$ 或从子节点看祖先的状态。这样一些子孙限制会变成当前路径上的限制，更容易用前缀、倍增或换根维护。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

反向后要重新定义根与边界。不要同时混用两个方向的状态而漏掉转移。

树拓扑序最优化用邻项交换

适用场景

在树形依赖下要求选择一个满足父子先后关系的线性序，并最小化某个代价时，可以考虑邻项交换。

核心思路

先假设两个相邻且互不为祖先后代的块可以交换。比较交换前后的代价差，得到一个排序关键字。若任意逆序交换都不劣，就可以按该关键字合并子树或维护优先队列。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

邻项交换需要保证交换不会破坏拓扑序，所以通常只交换无祖先关系的相邻块。

点到根距离权重和转子树权重和

适用场景

树上维护许多点到根或到某点的带权距离和时，可以转成每条边对子树权重的贡献。

核心思路

固定根后，一个点到根路径经过某条父边，当且仅当该点在这条边下方子树中。因此距离权重和可写成所有边长乘以其子树内点权和的求和。修改点权时更新祖先边贡献即可。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

换根时子树方向会改变，需要重新推导边两侧权重。

点边容斥处理树上集合交

适用场景

树上统计若干路径或连通块的交并关系时，可以在点和边之间做容斥。

核心思路

树上连通结构常满足 $|V|-|E|=1$。对多个结构计数时，点贡献和边贡献的差可以抵消内部重复，只留下连通块或边界数量。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

必须确认统计对象在树上无环。一般图中 $|V|-|E|$ 不再直接等于连通块数。

虚树贡献用 dfn 相邻距离维护

适用场景

动态维护树上一组关键点所形成的虚树大小或最小连通子图边数时，不一定要显式重建虚树。

核心思路

把关键点按 dfn 序排序。关键点集合的最小连通子图边数等于相邻关键点循环距离和的一半。插入或删除一个点时，只会影响它在 dfn 序中的前驱和后继。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

需要把首尾也视为相邻。若维护的是点数而不是边数，还需加入虚点或 LCA 的贡献。

校验补充

若维护原树最小连通子图的点数，非空时直接为边数+1；若维护压缩虚树的节点数，才需要另外维护 LCA 虚点。带权距离公式给出边权和，不是边数。

链加单点与单点加链对偶转化

适用场景

树上路径修改和单点查询、单点修改和路径查询之间常存在差分对偶关系。

核心思路

链加单点查询可以转为若干点加后查询子树和；单点加链查询也可以转成子树加单点查。利用 dfn 序和树上差分，有时能把树剖的 $\log^2 n$ 降到 $\log n$。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

根的选择会影响子树区间。路径加常需对 $u, v, lca, fa(lca)$ 四处打标记。

2.0.10 计算几何与距离

n个点组成的多边形面积

鞋带定理

多边形的面积 S 就是所有相邻两点构成的叉乘之和的一半：

$$S = \frac{1}{2}\left|\sum_{i=1}^{n}(x_i \cdot y_{i+1} - x_{i+1} \cdot y_i)\right|$$

XY 坐标轴旋转 45 度坐标轴

(X,Y)转成(X-Y, X+Y)，可以简化一些问题处理，比如绝对值之类的
有时得结合 y=f(i)-i 的技巧

关于三个数字两两做差的绝对值的最大值转化的小方法

$$\max(|a-b|, |b-c|, |c-a|) = \frac{|a-b| + |b-c| + |c-a|}{2}$$

凸多边形投影期望

一个凸多边形在随机方向上投影，投影的数学期望是 $\frac{C}{\pi}$，其中 C 是该凸多边形的周长。

多维绝对值枚举符号展开

适用场景

求多维点之间绝对值和的最大值、最小值或动态维护时，可以枚举每一维的正负号。

核心思路

利用 $|x| = \max(x, -x)$，有 $\sum_i |a_i - b_i| = \max_{s_i \in \{-1,1\}} \sum_i s_i(a_i - b_i)$。固定符号后就是线性函数，可以维护最大值和最小值。

使用条件

- 转化前后的信息必须等价，不能丢失题目真正需要维护的量。
- 通常适用于该结构能够被预处理、离线排序、分组维护或用数据结构快速合并的场景。

易错点

维度为 d 时需要枚举 2^d 个符号，适合维度较小的场景。

校验补充

枚举符号与取最大值可以交换，因此能求最大曼哈顿距离；求最小距离时一般不能把 min 与 max 交换，需要额外算法。

无精度误差判断线段穿过圆

判断一条穿过点 A 和点 B 的直线是否穿过圆心为 C、半径为 r 的圆，本质上就是计算圆心 C 到直线 AB 的最短距离 d，并判断 d 与 r 的大小关系。

根据题目要求，不能切到菠萝（但边缘相切是可以的，因为交集面积为 0），因此合法的切线必须满足：

$$d \geq r$$

由向量 $\vec{AB}$ 和 $\vec{AC}$ 构成的平行四边形的面积，有两种计算方式：

1. 面积 $S = |\vec{AB} \times \vec{AC}|$ (叉积的绝对值)
2. 面积 $S = \text{底} \times \text{高} = |\vec{AB}| \cdot d$

联立两式，就可以得到点到直线的距离公式：

$$d = \frac{|\vec{AB} \times \vec{AC}|}{|\vec{AB}|}$$

我们要判断 $d \geq r$。代入公式：

$$\frac{|\vec{AB} \times \vec{AC}|}{|\vec{AB}|} \geq r$$

$$|\vec{AB} \times \vec{AC}| \geq r \cdot |\vec{AB}|$$

此时 $|\vec{AB}|$ 中包含平方根（即 $\sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}$）。为了消除根号，我们将不等式两边同时平方：

$$(\vec{AB} \times \vec{AC})^2 \geq r^2 \cdot |\vec{AB}|^2$$

校验补充

上面的公式判断的是无限直线，不是有限线段。线段 AB 与闭圆盘相交需比较圆心到线段的最小距离：A=B 时比较 |AC|²；投影 AC·AB<=0 时比较 |AC|²；投影>=|AB|² 时比较 |BC|²；其余才比较叉积平方与 r²|AB|²。相切是否算相交按题意选 <= 或 <；平方乘积必须使用足够宽的整数。

曼哈顿距离旋转坐标变换

令 $(x, y) \to (x + y, x - y)$。 则$|x_1 - x_2| + |y_1 - y_2| = \max(|x'_1 - x'_2|, |y'_1 - y'_2|)$
处理最大曼哈顿距离时，只需在新坐标系下维护 x' 和 y' 的 $\min / \max$ 值，即可 $O(1)$ 得到结果。（曼哈顿转切比雪夫距离）

用圆上弦端点交错判断圆内直线交点

适用场景

当需要统计距离某点不超过 r 的直线交点数量时，可以把问题转化为圆内弦相交计数。

核心结论

一条直线如果与圆相交，会在圆上截出两个端点，也就是一条弦。两条直线的交点在圆内，当且仅当它们对应的两条弦在圆内相交。

而两条弦在圆内相交，当且仅当它们在圆周上的四个端点交错出现。

若两条弦端点为 (a, b) 和 (c, d)，按圆周顺序编号后，交错条件可写作：

$$a < c < b < d \quad \text{或} \quad c < a < d < b.$$

做法

1. 二分半径 r。
2. 求每条直线和圆的两个交点，按极角排序离散化。
3. 将每条弦表示为端点区间 (l, r)。
4. 按左端点排序，用 BIT 统计右端点落在区间内部的数量。

易错点

- 与圆相切时只有一个交点，需要按题意判断是否计入。

- 极角排序要处理精度误差。
- 端点是循环序，需统一让 $l < r$。

校验补充

严格端点交错对应圆盘内部的交点，并要求四个端点互异。统计距离 <=r 时，圆周上的公共交点也要计入；不能仅检查严格交错。重合直线与多线共点要按题意明确是否按直线对计数。

计算凸多边形的几何中心坐标

设凸多边形的顶点按顺时针或逆时针排列为 $P(x_1,y_1),P(x_2,y_2),.....,P(x_n,y_n)$，且 $P_{n+1}=P_1$ ，那么首先需要计算凸多边形的面积

$$S=\frac{1}{2}\sum_{i=1}^{n}(x_iy_{i+1}-x_{i+1}y_i)$$

那么几何中心 (x_0,y_0) 的坐标为：

$$x_0=\frac{\sum_{i=1}^{n}(x_i+x_{i+1})(x_iy_{i+1}-x_{i+1}y_i)}{6S}$$

$$y_0=\frac{\sum_{i=1}^{n}(y_i+y_{i+1})(x_iy_{i+1}-x_{i+1}y_i)}{6S}$$

校验补充

S 为有向面积，代入重心公式时不要只对分母取绝对值；要求 S!=0。这里是均匀面密度的面积重心，不是顶点坐标算术平均。

计算圆周率

在部分题目中可能会需要圆周率的计算并且需要较高精度，此时
~~直接const double pi 3.14159265358979793不就行了吗~~
就可以优雅的用 `#define pi acos(-1)` 来实现，也就是 $\arccos(-1)=\pi$ 来实现

2.1 MInt

```
constexpr int mod{ 998244353 };

template<uint32_t P>
struct MInt {
    using u32 = uint32_t;
    using u64 = uint64_t;
    static_assert(P > 1);
    u32 x{};
    constexpr MInt() = default;
    constexpr MInt(i64 v) : x(norm(v)) {}
    // ===== 基础接口与四则运算 =====
    static constexpr u32 mod() { return P; }
    static constexpr u32 norm(i64 v) {
        if (v ⩾ P || v ⩽ -i64(P)) v %= P;
        if (v < 0) v += P;
        return u32(v);
    }
    constexpr u32 val() const { return x; }
    explicit constexpr operator u32() const { return x; }
    constexpr MInt &operator+=(MInt b) {
        if (x ⩾ P - b.x) x -= P - b.x;
        else x += b.x;
        return *this;
    }
    constexpr MInt &operator-=(MInt b) {
        if (x < b.x) x += P - b.x;
        else x -= b.x;
        return *this;
    }
    constexpr MInt &operator*=(MInt b) {
        x = u64(x) * b.x % P;
        return *this;
    }
    friend constexpr MInt operator+(MInt a, MInt b) { return a += b; }
    friend constexpr MInt operator-(MInt a, MInt b) { return a -= b; }
    friend constexpr MInt operator*(MInt a, MInt b) { return a *= b; }
    constexpr MInt operator-() const { return x ? P - x : 0; }
```

```
    // ===== 幂、逆元与除法：若只需要加减乘，以下代码可直接删除 =====
    constexpr MInt pow(i64 b) const {
        bool minus{ b < 0 };
        u64 e{ minus ? u64(-(b + 1)) + 1 : u64(b) };
        MInt a = *this, res(1);
        for (; e; e >>= 1, a *= a) if (e & 1) res *= a;
        return minus ? res.inv() : res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return pow(i64(P) - 2);
    }
    constexpr MInt &operator/=(MInt b) { return *this *= b.inv(); }
    friend constexpr MInt operator/(MInt a, MInt b) { return a /= b; }
    // ===== 快捷运算符：若只使用 pow() / inv()，以下代码可直接删除 =====
    constexpr MInt &operator^=(i64 b) { return *this = pow(b); }
    friend constexpr MInt operator^(MInt a, i64 b) { return a.pow(b); }
    friend constexpr MInt operator~(MInt a) { return a.inv(); }
    // ===== 比较与 I/O：不需要时以下代码可直接删除 =====
    friend constexpr bool operator==(MInt a, MInt b) { return a.x == b.x; }
    friend constexpr bool operator!=(MInt a, MInt b) { return a.x != b.x; }
    friend constexpr bool operator<(MInt a, MInt b) { return a.x < b.x; }
    friend istream &operator>>(istream &in, MInt &a) {
        i64 v;
        if (in >> v) a = v;
        return in;
    }
    friend ostream &operator<<(ostream &out, MInt a) { return out << a.val(); }
};
using Z = MInt<mod>;
```

接口说明

- `MInt<P>(v)`：构造模 `P` 意义下的整数，并规范化到 `[0, P)`
- `mod()`：返回模数 `P`
- `val()`：返回 `[0, P)` 内的标准余数
- `+`、`-`、`*`、`/`：模意义下四则运算
- `pow(b)`：计算 a^b；支持负指数
- `inv()`：计算乘法逆元
- `^ b`：等价于 `pow(b)`
- `~a`：等价于 `a.inv()`

注意事项

- 模数 `P` 必须满足 `P > 1`，代码通过 `static_assert` 在编译期检查
- `inv()`、`/`、`~` 和负指数 `pow(-k)` 使用费马小定理，因此要求 `P` 为质数；求逆元素必须非零
- `pow(b)` 支持负指数；`b < 0` 时按 $a^b = (a^{-b})^{-1}$ 计算，代码可以正确处理 `LLONG_MIN`
- 内部始终维护 `0 ≤ x < P`；整数构造和输入会自动规范化负数及超出模数范围的值
- 构造函数与输入直接支持 `i64` 范围内的整数；若输入值可能超过 `i64`，需要使用字符串自行取模
- 模数类型为 `u32`，因此要求 `P ≤ UINT32_MAX`
- 乘法使用 `u64(x) * b.x`，对任意 `u32` 模数均不会在乘法阶段溢出 `u64`
- `operator<` 比较标准余数 `x` 的整数大小，仅用于排序等场景，不表示模意义下存在自然大小关系

2.2 Frac

```
struct Frac {
    using u64 = uint64_t;
    using i128 = __int128_t;
    using ld = long double;
    i64 num{ 0 }, den{ 1 };
    Frac() = default;
    Frac(i64 num_) : num(num_) {}
    Frac(i64 num_, i64 den_) : num(num_), den(den_) { normalize(); }
private:
    // ===== 基础辅助 =====
    static u64 mag(i64 x) { return x < 0 ? u64(-(x + 1)) + 1 : u64(x); }
    static u64 gcd_mag(i64 a, i64 b) {
        u64 x{ mag(a) }, y{ mag(b) };
```

```
        while (y) {
            x %= y;
            swap(x, y);
        }
        return x;
    }
    static i64 gcd_abs(i64 a, i64 b) { return i64(gcd_mag(a, b)); }
    static bool fits_i64(i128 x) {
        return i128(numeric_limits<i64>::min()) ≤ x && x ≤ i128(numeric_limits<i64>::max());
    }
    // 求 gcd(|n|, d), d > 0
    static i64 gcd_num(i128 n, i64 d) {
        if (fits_i64(n)) return gcd_abs(i64(n), d);
        return gcd_abs(i64(n % d), d);
    }
    // 同时约去两个 i64 的 gcd; 额外处理 LLONG_MIN / LLONG_MIN
    static pair<i64, i64> cancel(i64 a, i64 b) {
        u64 g{ gcd_mag(a, b) };
        if (g ≤ u64(numeric_limits<i64>::max())) {
            i64 d{ i64(g) };
            return { a / d, b / d };
        }
        return { -1, -1 };
    }
    void set_reduced(i128 n, i128 d) {
        if (d < 0) {
            n = -n;
            d = -d;
        }
        assert(d > 0 && fits_i64(n) && fits_i64(d));
        num = i64(n);
        den = i64(d);
    }
    // ===== 加减核心: operator+= / operator-= 共用 =====
    Frac &add_assign(const Frac &b, int s) {
        if (b.num == 0) return *this;
        if (num == 0) {
            set_reduced(i128(s) * b.num, b.den);
            return *this;
        }
        if (den == 1) {
            set_reduced(i128(num) * b.den + i128(s) * b.num, b.den);
            return *this;
        }
        if (b.den == 1) {
            set_reduced(i128(num) + i128(s) * b.num * den, den);
            return *this;
        }
        if (den == b.den) {
            i128 n{ i128(num) + i128(s) * b.num };
            i64 g{ gcd_num(n, den) };
            set_reduced(n / g, den / g);
            return *this;
        }
        i64 g{ gcd_abs(den, b.den) };
        i64 a1{ den / g }, b1{ b.den / g };
        i128 n{ i128(num) * b1 + i128(s) * b.num * a1 };
        i64 h{ gcd_num(n, g) };
        set_reduced(n / h, i128(a1) * (b.den / h));
        return *this;
    }
public:
    // ===== 规范化: 直接修改 num / den 后调用; 正常四则运算无需调用 =====
    void normalize() {
        assert(den ≠ 0);
        if (num == 0) {
            den = 1;
            return;
        }
        u64 g{ gcd_mag(num, den) };
        i128 n{ num }, d{ den };
```

```
        if (d < 0) {
            n = -n;
            d = -d;
        }
        set_reduced(n / i128(g), d / i128(g));
    }
    // ===== 四则运算：始终保持最简分数 =====
    Frac &operator+=(const Frac &b) { return add_assign(b, 1); }
    Frac &operator-=(const Frac &b) { return add_assign(b, -1); }
    Frac &operator*=(const Frac &b) {
        if (num == 0 || b.num == 0) {
            num = 0;
            den = 1;
            return *this;
        }
        if (den == 1 && b.den == 1) {
            set_reduced(i128(num) * b.num, 1);
            return *this;
        }
        if (den == 1) {
            i64 g{ gcd_abs(num, b.den) };
            set_reduced(i128(num / g) * b.num, b.den / g);
            return *this;
        }
        if (b.den == 1) {
            i64 g{ gcd_abs(b.num, den) };
            set_reduced(i128(num) * (b.num / g), den / g);
            return *this;
        }
        i64 g1{ gcd_abs(num, b.den) };
        i64 g2{ gcd_abs(b.num, den) };
        set_reduced(
            i128(num / g1) * (b.num / g2),
            i128(den / g2) * (b.den / g1)
        );
        return *this;
    }
    Frac &operator/=(const Frac &b) {
        assert(b.num != 0);
        if (num == 0 || (b.num == 1 && b.den == 1)) return *this;
        auto [a1, b1]{ cancel(num, b.num) };
        if (b.den == 1) {
            set_reduced(a1, i128(den) * b1);
            return *this;
        }
        if (den == 1) {
            set_reduced(i128(a1) * b.den, b1);
            return *this;
        }
        i64 g{ gcd_abs(den, b.den) };
        set_reduced(
            i128(a1) * (b.den / g),
            i128(den / g) * b1
        );
        return *this;
    }
    friend Frac operator+(Frac a, const Frac &b) { return a += b; }
    friend Frac operator-(Frac a, const Frac &b) { return a -= b; }
    friend Frac operator*(Frac a, const Frac &b) { return a *= b; }
    friend Frac operator/(Frac a, const Frac &b) { return a /= b; }
    Frac operator-() const {
        Frac res;
        res.set_reduced(-i128(num), den);
        return res;
    }
    friend bool operator==(const Frac &a, const Frac &b) {
        return a.num == b.num && a.den == b.den;
    }
    friend bool operator!=(const Frac &a, const Frac &b) { return !(a == b); }
    friend bool operator<(const Frac &a, const Frac &b) {
        return i128(a.num) * b.den < i128(b.num) * a.den;
```

```
    }
    friend bool operator>(const Frac &a, const Frac &b) { return b < a; }
    friend bool operator<=(const Frac &a, const Frac &b) { return !(b < a); }
    friend bool operator>=(const Frac &a, const Frac &b) { return !(a < b); }
    // ===== 浮点转换与输出：不需要时可直接删除 =====
    explicit operator ld() const { return ld(num) / ld(den); }
    friend ostream &operator<<(ostream &out, const Frac &x) {
        out << x.num;
        if (x.den != 1) out << '/' << x.den;
        return out;
    }
};
```

必须使用 `i128` 存储时

不能只把成员类型机械改成 `i128`。至少需要恢复安全的非交叉乘比较：

```
using i128 = __int128_t;
using u128 = __uint128_t;
static u128 mag(i128 x) {
    return x < 0 ? u128(-(x + 1)) + 1 : u128(x);
}
// 比较非负分数 a / b 与 c / d，不使用交叉乘
static int cmp_pos(u128 a, u128 b, u128 c, u128 d) {
    bool rev{ false };
    while (true) {
        u128 q1{ a / b }, q2{ c / d };
        if (q1 != q2) {
            int res{ q1 < q2 ? -1 : 1 };
            return rev ? -res : res;
        }
        a %= b;
        c %= d;
        if (!a || !c) {
            int res{ a == c ? 0 : (a ? 1 : -1) };
            return rev ? -res : res;
        }
        swap(a, b);
        swap(c, d);
        rev = !rev;
    }
}
```

数值范围约定

成员必须保持最简且 den > 0。每次运算约分后的分子与分母都必须在 i64 范围内；如 -Frac(LLONG_MIN) 无法表示，断言会失败。直接改成员后必须 normalize()。断言不能在竞赛输入不满足这些约定时替代范围设计。

2.3 组合数

```
template<class T = Z>
struct Comb {
    static constexpr auto mod = T::mod();
    int n{0};
    vector<T> fac{T(1)}, ifac{T(1)}, inv{T(0)};
    explicit Comb(int n_max = 0) {
        inv.push_back(T(1));
        ensure(n_max);
    }
    void ensure(int m) {
        assert(m >= 0 && uint64_t(m) < mod);
        if (m <= n) return;
        int old{ n };
        n = m;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        inv.resize(n + 1);
        for (int i = old + 1; i <= n; ++ i) {
            fac[i] = fac[i - 1] * T(i);
```

```
            if (i == 1) inv[i] = T(1);
            else inv[i] = -T(mod / i) * inv[mod % i];
            ifac[i] = ifac[i - 1] * inv[i];
        }
    }
    T Fac(int m) { ensure(m); return fac[m]; }
    T IFact(int m) { ensure(m); return ifac[m]; }
    T Inv(int m) { assert(m > 0); ensure(m); return inv[m]; }
    T C(int a, int b) {
        if (a < 0 || b < 0 || b > a) return T(0);
        ensure(a);
        return fac[a] * ifac[b] * ifac[a - b];
    }
    // 排列数 P(n,k) = n! / (n-k)!
    T P(int a, int b) {
        if (a < 0 || b < 0 || b > a) return T(0);
        ensure(a);
        return fac[a] * ifac[a - b];
    }
    // 可重复组合（把 K 个相同物品放进 N 类）：C(n+k-1, k)
    T H(int n_types, int k_take) {
        if (n_types < 0 || k_take < 0) return T(0);
        if (n_types == 0) return T(k_take == 0);
        i64 top64{ i64(n_types) + k_take - 1 };
        assert(top64 < mod && top64 <= INT_MAX);
        int top = int(top64);
        return C(top, k_take);
    }
    // ===== Lucas 定理：处理 n >= mod 的组合数 =====
    T Lucas(i64 a, i64 b) {
        if (a < 0 || b < 0 || b > a) return T(0);
        T ans{1};
        while (b) {
            int x = a % mod, y = b % mod;
            if (y > x) return T(0);
            ans *= C(x, y);
            a /= mod;
            b /= mod;
        }
        return ans;
    }
};
```

Lucas

Lucas 定理：

$$\binom{n}{k} \equiv \prod_{i\geq 0} \binom{n_i}{k_i} \pmod{p},$$

其中 `p = mod` 为质数，且

$$n = \sum_i n_i p^i, \quad k = \sum_i k_i p^i, \quad 0 \leq n_i, k_i < p$$

如果某一位满足 $k_i > n_i$，则

$$\binom{n}{k} \equiv 0 \pmod{p}$$

接口说明

- `Lucas(a, b)`：计算 $\binom{a}{b} \bmod mod$，允许 `a >= mod`
- `a < 0`、`b < 0` 或 `b > a` 时返回 `0`
- 内部调用 `C(x, y)` 计算每一位的小组合数
  由于当前 `Comb` 会按需扩容，因此 Lucas 查询只会把预处理范围扩大到实际出现过的最大 `a % mod`，不必预先计算整个 `0..mod-1`

注意事项

- `mod` 必须是质数
- Lucas 解决的是质数模下 `a >= mod` 时普通阶乘组合数失效的问题；合数模不能直接使用
- 若 `mod` 很大，例如 `1E9 + 7`，且某个 `a % mod` 也很大，则不能依靠 `Comb` 预处理到这一位置，否则时间和空间都会不可接受。

- Lucas 本身不会解决"模数很大且每一位也很大"的问题；此时应根据 `b` 的大小选择乘法式求组合数等方法
- 合数模的大组合数应使用扩展 Lucas / 质数幂组合数 + CRT，而不是普通Lucas

Lucas (不预处理)

如果 `mod` 不固定，或者不希望为 Lucas 数位建立阶乘表，可以使用下面的独立版本

```
i64 qpow_mod(i64 a, i64 b, i64 p) {
    assert(p > 1 && b >= 0);
    a %= p;
    if (a < 0) a += p;
    i64 ans{1};
    while (b) {
        if (b & 1) ans = (i128)ans * a % p;
        a = (i128)a * a % p;
        b >>= 1;
    }
    return ans;
}
i64 C_mod_prime(i64 n, i64 k, i64 p) {
    assert(p > 1);
    if (n < 0 || k < 0 || k > n) return 0;
    assert(n < p); // 仅计算 Lucas 的单个数位；n ≥ p 请调用 lucas
    k = min(k, n - k);
    i64 a{1}, b{1};
    for (i64 i = 1; i <= k; ++ i) {
        a = (i128)a * (n - k + i) % p;
        b = (i128)b * i % p;
    }
    return (i128)a * qpow_mod(b, p - 2, p) % p;
}
i64 lucas(i64 n, i64 k, i64 p) {
    assert(p > 1);
    if (n < 0 || k < 0 || k > n) return 0;
    i64 ans{1};
    while (k) {
        i64 a = n % p, b = k % p;
        if (b > a) return 0;
        ans = (i128)ans * C_mod_prime(a, b, p) % p;
        n /= p;
        k /= p;
    }
    return ans;
}
```

使用场景

```
i64 ans = lucas(n, k, p);
```

其中 `p` 可以在运行时给出，但必须为质数
这个版本不需要 $O(p)$ 空间，适合：

- `p` 每组数据不同；
- 查询次数较少；
- 每一位的 $\min(k_i, n_i - k_i)$ 较小
  若同一个小质数 `p` 下有大量查询，优先使用 `Comb + Lucas` 版本

复杂度

`Comb + Lucas`

设一次查询的 Lucas 数位数为

$$d = O(\log_{mod} n)$$

若阶乘表已经覆盖所有需要的数位：

操作	时间复杂度
`Lucas(n, k)`	$O(\log_{mod} n)$
每个数位的 `C()`	$O(1)$

若使用前面的动态 `Comb`，设历史所有 Lucas 查询中实际访问过的最大数位为 `M`，则所有扩容的累计复杂度为：

$$O(M)$$

空间复杂度为：

$$O(M)$$

因此无需强制预处理整个 `0..mod-1`

不预处理版本

设

$$n_i = n \bmod p, \quad k_i = k \bmod p$$

单次查询复杂度为

$$O\left(\sum_i \min(k_i, n_i - k_i) + \log_p n \cdot \log p\right)$$

空间复杂度为 $O(1)$

2.4 高精度

C++ 高精度整数

```
struct BigInt {
    static constexpr int BASE{ 1000000000 };
    static constexpr int DIGITS{ 9 };
    static constexpr int MUL_NAIVE{ 64 };
    static constexpr int DIV_NAIVE{ 64 };
    vector<int> a;
    int sign{ 1 };
private:
    using Vec = vector<int>;
    using u128 = __uint128_t;
    // ===== 1. 基础维护：所有功能依赖 =====
    static void trim_vec(Vec &v) { while (!v.empty() && v.back() == 0) v.pop_back(); }
    void trim() { trim_vec(a); if (a.empty()) sign = 1; }
    static int cmp_vec(const Vec &x, const Vec &y) {
        if (x.size() != y.size()) return x.size() < y.size() ? -1 : 1;
        for (int i = int(x.size()) - 1; i >= 0; -- i) if (x[i] != y[i]) return x[i] < y[i] ? -1 : 1;
        return 0;
    }
    static int abs_cmp(const BigInt &x, const BigInt &y) { return cmp_vec(x.a, y.a); }
    // ===== 2. 无符号向量加减与小整数乘除：快速乘除法内部依赖 =====
    static Vec add_vec(const Vec &x, const Vec &y) {
        int n = max(x.size(), y.size()), carry{ 0 };
        Vec z(n);
        for (int i = 0; i < n; ++ i) {
            i64 cur{ i64(carry) + (i < x.size() ? x[i] : 0) + (i < y.size() ? y[i] : 0) };
            if (cur >= BASE) cur -= BASE, carry = 1;
            else carry = 0;
            z[i] = int(cur);
        }
        if (carry) z.push_back(carry);
        return z;
    }
    // 要求 x >= y
    static Vec sub_vec(const Vec &x, const Vec &y) {
        Vec z{ x };
        int borrow{ 0 };
        for (int i = 0; i < z.size(); ++ i) {
            i64 cur{ i64(z[i]) - borrow - (i < y.size() ? y[i] : 0) };
            if (cur < 0) cur += BASE, borrow = 1;
            else borrow = 0;
            z[i] = int(cur);
        }
        trim_vec(z);
        return z;
    }
```

```
    static Vec mul_small_vec(const Vec &x, int v) {
        if (x.empty() || v == 0) return {};
        if (v == 1) return x;
        Vec z;
        z.reserve(x.size() + 1);
        i64 carry{ 0 };
        for (int d : x) {
            i64 cur{ i64(d) * v + carry };
            z.push_back(int(cur % BASE));
            carry = cur / BASE;
        }
        if (carry) z.push_back(int(carry));
        return z;
    }
    // v 满足 1 ⩽ v < BASE，返回 { 商，余数 }
    static pair<Vec, int> div_small_vec(const Vec &x, int v) {
        assert(1 ⩽ v && v < BASE);
        Vec q(x.size());
        i64 rem{ 0 };
        for (int i = int(x.size()) - 1; i ⩾ 0; -- i) {
            i64 cur{rem * BASE + x[i]};
            q[i] = int(cur / v);
            rem = cur % v;
        }
        trim_vec(q);
        return {move(q), int(rem)};
    }
    // ===== 3. 精确 NTT 乘法：BigInt * BigInt 与 Newton 除法依赖 =====
    // 小规模自动退化为朴素乘法；完全不需要 BigInt 乘除时，本组可整体删除
    template<int MOD, int G>
    struct NTT {
        static int power(int a, int b) {
            i64 r{ 1 };
            for (; b; b >>= 1, a = int(i64(a) * a % MOD)) if (b & 1) r = r * a % MOD;
            return int(r);
        }
        static void transform(vector<int> &a, bool inverse) {
            int n = a.size();
            for (int i = 1, j = 0; i < n; ++ i) {
                int bit{ n >> 1 };
                for (; j & bit; bit >>= 1) j ^= bit;
                j ^= bit;
                if (i < j) swap(a[i], a[j]);
            }
            for (int len = 2; len ⩽ n; len <<= 1) {
                int wlen{ power(G, (MOD - 1) / len) };
                if (inverse) wlen = power(wlen, MOD - 2);
                for (int i = 0; i < n; i += len) {
                    i64 w{ 1 };
                    for (int j = 0; j < len / 2; ++ j) {
                        int u{ a[i + j] }, v{ int(a[i + j + len / 2] * w % MOD) };
                        int x{ u + v };
                        if (x ⩾ MOD) x -= MOD;
                        int y{ u - v };
                        if (y < 0) y += MOD;
                        a[i + j] = x;
                        a[i + j + len / 2] = y;
                        w = w * wlen % MOD;
                    }
                }
            }
            if (inverse) {
                int inv_n{ power(n, MOD - 2) };
                for (int &x : a) x = int(i64(x) * inv_n % MOD);
            }
        }
        static vector<int> convolution(const vector<int> &x, const vector<int> &y, bool square) {
            if (x.empty() || y.empty()) return {};
            int need{ int(x.size() + y.size() - 1) }, n{ 1 };
            while (n < need) n <<= 1;
            assert((MOD - 1) % n == 0);
```

```
            vector<int> a(n);
            copy(x.begin(), x.end(), a.begin());
            transform(a, false);
            if (square) {
                for (int &v : a) v = int(i64(v) * v % MOD);
            } else {
                vector<int> b(n);
                copy(y.begin(), y.end(), b.begin());
                transform(b, false);
                for (int i = 0; i < n; ++ i) a[i] = int(i64(a[i]) * b[i] % MOD);
            }
            transform(a, true);
            a.resize(need);
            return a;
        }
    };
    static Vec convert_base(const Vec &v, int old_digits, int new_digits) {
        if (v.empty()) return {};
        array<i64, 10> p{};
        p[0] = 1;
        for (int i = 1; i <= max(old_digits, new_digits); ++ i) p[i] = p[i - 1] * 10;
        Vec res;
        res.reserve((i64(v.size()) * old_digits + new_digits - 1) / new_digits);
        i64 cur{ 0 };
        int cur_digits{ 0 };
        for (int x : v) {
            cur += i64(x) * p[cur_digits];
            cur_digits += old_digits;
            while (cur_digits >= new_digits) {
                res.push_back(int(cur % p[new_digits]));
                cur /= p[new_digits];
                cur_digits -= new_digits;
            }
        }
        if (cur) res.push_back(int(cur));
        trim_vec(res);
        return res;
    }
    static Vec mul_naive_vec(const Vec &x, const Vec &y) {
        if (x.empty() || y.empty()) return {};
        const Vec *a{ &x }, *b{ &y };
        if (a->size() < b->size()) swap(a, b);
        Vec z(a->size() + b->size());
        for (int i = 0; i < int(b->size()); ++ i) {
            u128 carry{0};
            for (int j = 0; j < int(a->size()) || carry; ++ j) {
                u128 cur{ u128(z[i + j]) + carry };
                if (j < int(a->size())) cur += u128((*b)[i]) * (*a)[j];
                z[i + j] = int(cur % BASE);
                carry = cur / BASE;
            }
        }
        trim_vec(z);
        return z;
    }
    static Vec mul_ntt_vec(const Vec &x, const Vec &y, bool square = false) {
        static constexpr int B{ 10000 }, BD{ 4 };
        static constexpr int M0{ 998244353 }, M1{ 469762049 };
        static constexpr i64 INV_M0_MOD_M1{ 208783132 };
        Vec a4{ convert_base(x, DIGITS, BD) };
        Vec b4{ square ? a4 : convert_base(y, DIGITS, BD) };
        auto c0{ NTT<M0, 3>::convolution(a4, b4, square) };
        auto c1{ NTT<M1, 3>::convolution(a4, b4, square) };
        Vec c4;
        c4.reserve(c0.size() + 8);
        u128 carry{ 0 };
        for (int i = 0; i < c0.size(); ++ i) {
            i64 d{ c1[i] - c0[i] % M1 };
            if (d < 0) d += M1;
            i64 t{ d * INV_M0_MOD_M1 % M1 };
            u128 cur{ u128(c0[i]) + u128(M0) * t + carry };
```

```
            c4.push_back(int(cur % B));
            carry = cur / B;
        }
        while (carry) {
            c4.push_back(int(carry % B));
            carry /= B;
        }
        return convert_base(c4, BD, DIGITS);
    }
    static Vec mul_vec(const Vec &x, const Vec &y) {
        if (x.empty() || y.empty()) return {};
        if (min(x.size(), y.size()) <= MUL_NAIVE) return mul_naive_vec(x, y);
        return mul_ntt_vec(x, y, &x == &y);
    }
    // ===== 4. 无符号长除法：仅供小除数或短商使用；Newton 的初值也依赖 =====
    static pair<Vec, Vec> divmod_naive_vec(const Vec &a, const Vec &b) {
        assert(!b.empty());
        if (a.empty()) return {{}, {}};
        if (b.size() == 1) {
            auto [q, r]{ div_small_vec(a, b[0]) };
            return { move(q), r ? Vec{r} : Vec{} };
        }
        if (cmp_vec(a, b) < 0) return {{}, a};
        int norm{ BASE / (b.back() + 1) };
        Vec x{ mul_small_vec(a, norm) }, y{ mul_small_vec(b, norm) };
        Vec q(x.size() - y.size() + 1);
        Vec r(x.end() - y.size(), x.end());
        trim_vec(r);
        for (int i = int(q.size()) - 1; i >= 0; --i) {
            if (r.size() == y.size()) {
                if (cmp_vec(r, y) >= 0) q[i] = 1, r = sub_vec(r, y);
            } else if (r.size() > y.size()) {
                assert(r.size() == y.size() + 1);
                i64 rb{ i64(r.back()) * BASE + r[r.size() - 2] };
                i64 d{ min<i64>(BASE - 1, rb / y.back()) };
                Vec yd{ mul_small_vec(y, int(d)) };
                while (cmp_vec(r, yd) < 0) --d, yd = sub_vec(yd, y);
                r = sub_vec(r, yd);
                while (cmp_vec(r, y) >= 0) ++d, r = sub_vec(r, y);
                q[i] = int(d);
            }
            if (i) {
                r.insert(r.begin(), x[i - 1]);
                trim_vec(r);
            }
        }
        trim_vec(q);
        auto [rr, rem]{ div_small_vec(r, norm) };
        assert(rem == 0);
        return {move(q), move(rr)};
    }
    // ===== 5. Newton 倒数除法：大整数除法核心；依赖第 3/4 组 =====
    // 计算 floor(BASE^(a.size()+deg) / a)，要求 a.back() >= BASE / 2。
    static Vec calc_inv(const Vec &a, int deg) {
        assert(!a.empty() && a.back() >= BASE / 2);
        int k{ deg }, c = a.size();
        while (k > DIV_NAIVE) k = (k + 1) / 2;
        Vec z(c + k + 1);
        z.back() = 1;
        z = divmod_naive_vec(z, a).first;
        while (k < deg) {
            Vec s{mul_vec(z, z)};
            s.insert(s.begin(), 0);
            int take{ min(c, 2 * k + 1) };
            Vec t(2 * k + 1);
            copy(a.end() - take, a.end(), t.end() - take);
            Vec u{ mul_vec(s, t) };
            if (2 * k + 1 >= u.size()) u.clear();
            else u.erase(u.begin(), u.begin() + 2 * k + 1);
            Vec w(k + 1), w2{ add_vec(z, z) };
            w.insert(w.end(), w2.begin(), w2.end());
```

```
            z = sub_vec(w, u);
            if (!z.empty()) z.erase(z.begin());
            k *= 2;
        }
        int cut{ k - deg };
        if (cut > 0) z.erase(z.begin(), z.begin() + min(cut, int(z.size())));
        trim_vec(z);
        return z;
    }
    static pair<Vec, Vec> divmod_vec(const Vec &a, const Vec &b) {
        assert(!b.empty());
        if (a.empty()) return {{}, {}};
        if (cmp_vec(a, b) < 0) return {{}, a};
        if (b.size() <= DIV_NAIVE || int(a.size()) - int(b.size()) <= DIV_NAIVE) return divmod_naive_vec(a, b);
        int norm{ BASE / (b.back() + 1) };
        Vec x{mul_small_vec(a, norm)}, y{ mul_small_vec(b, norm) };
        int s = x.size(), t = y.size(), deg{ s - t + 2 };
        Vec z{ calc_inv(y, deg) };
        Vec q{ mul_vec(x, z) };
        int cut{ t + deg };
        if (cut >= q.size()) q.clear();
        else q.erase(q.begin(), q.begin() + cut);
        trim_vec(q);
        Vec yq{ mul_vec(y, q) };
        const Vec one{ 1 };
        while (cmp_vec(x, yq) < 0) q = sub_vec(q, one), yq = sub_vec(yq, y);
        Vec r{ sub_vec(x, yq) };
        while (cmp_vec(r, y) >= 0) q = add_vec(q, one), r = sub_vec(r, y);
        auto [rr, rem]{ div_small_vec(r, norm) };
        assert(rem == 0);
        trim_vec(q);
        return {move(q), move(rr)};
    }
    // ===== 6. 绝对值原地加减：公开 +/- 依赖 =====
    void add_abs(const BigInt &v) {
        int n = int(max(a.size(), v.a.size()));
        a.resize(n, 0);
        int carry{ 0 };
        for (int i = 0; i < n; ++ i) {
            i64 cur{ i64(a[i]) + carry + (i < v.a.size() ? v.a[i] : 0) };
            if (cur >= BASE) cur -= BASE, carry = 1;
            else carry = 0;
            a[i] = int(cur);
        }
        if (carry) a.push_back(carry);
    }
    void sub_abs(const BigInt &v) {
        int borrow{0};
        for (int i = 0; i < a.size(); ++ i) {
            i64 cur{ i64(a[i]) - borrow - (i < v.a.size() ? v.a[i] : 0) };
            if (cur < 0) cur += BASE, borrow = 1;
            else borrow = 0;
            a[i] = int(cur);
            if (i >= v.a.size() && !borrow) break;
        }
        trim();
    }
public:
    // ===== 7. 构造、符号与输入输出：基础接口，通常保留 =====
    BigInt() = default;
    BigInt(i64 v) { *this = v; }
    BigInt(const string &s) { read(s); }
    bool is_zero() const { return a.empty(); }
    BigInt abs() const { BigInt res{*this}; res.sign = 1; return res; }
    BigInt &operator=(i64 v) {
        a.clear();
        sign = v < 0 ? -1 : 1;
        u64 x{u64(v)};
        if (v < 0) x = 0 - x;
        while (x) a.push_back(int(x % BASE)), x /= BASE;
        return *this;
```

```
    }
    BigInt operator-() const {
        BigInt res{*this};
        if (!res.is_zero()) res.sign = -res.sign;
        return res;
    }
    void read(const string &s) {
        a.clear();
        sign = 1;
        int pos{ 0 };
        if (!s.empty() && (s[0] == '+' || s[0] == '-')) sign = s[0] == '-' ? -1 : 1, pos = 1;
        a.reserve((s.size() - pos + DIGITS - 1) / DIGITS);
        for (int i = s.size(); i > pos; i -= DIGITS) {
            int x{ 0 };
            for (int j = max(pos, i - DIGITS); j < i; ++ j) x = x * 10 + s[j] - '0';
            a.push_back(x);
        }
        trim();
    }
    string to_string() const {
        if (a.empty()) return "0";
        string s;
        s.reserve(a.size() * DIGITS + (sign < 0));
        if (sign < 0) s.push_back('-');
        s += std::to_string(a.back());
        char buf[DIGITS];
        for (int i = int(a.size()) - 2; i >= 0; -- i) {
            int x{ a[i] };
            for (int j = DIGITS - 1; j >= 0; -- j) buf[j] = char('0' + x % 10), x /= 10;
            s.append(buf, DIGITS);
        }
        return s;
    }
    friend istream &operator>>(istream &in, BigInt &v) { string s; if (in >> s) v.read(s); return in; }
    friend ostream &operator<<(ostream &out, const BigInt &v) { return out << v.to_string(); }
    // ===== 8. 比较：通常保留 =====
    friend bool operator<(const BigInt &x, const BigInt &y) {
        if (x.sign != y.sign) return x.sign < y.sign;
        int c{ abs_cmp(x, y) };
        return x.sign > 0 ? c < 0 : c > 0;
    }
    friend bool operator>(const BigInt &x, const BigInt &y) { return y < x; }
    friend bool operator<=(const BigInt &x, const BigInt &y) { return !(y < x); }
    friend bool operator>=(const BigInt &x, const BigInt &y) { return !(x < y); }
    friend bool operator==(const BigInt &x, const BigInt &y) { return x.sign == y.sign && x.a == y.a; }
    friend bool operator!=(const BigInt &x, const BigInt &y) { return !(x == y); }
    // ===== 9. BigInt 加减：不用 +/- 时可连同第 6 组删除 =====
    BigInt &operator+=(const BigInt &v) {
        if (sign == v.sign) add_abs(v);
        else if (abs_cmp(*this, v) >= 0) sub_abs(v);
        else {
            BigInt t{v};
            t.sub_abs(*this);
            *this = move(t);
        }
        return *this;
    }
    BigInt &operator-=(const BigInt &v) { return *this += -v; }
    friend BigInt operator+(BigInt x, const BigInt &y) { return x += y; }
    friend BigInt operator-(BigInt x, const BigInt &y) { return x -= y; }
    // ===== 10. 与普通整数乘除：不用时可直接删除 =====
    BigInt &operator*=(i64 v) {
        if (v == 0 || is_zero()) return *this = 0;
        if (v < 0) sign = -sign;
        u64 x{u64(v)};
        if (v < 0) x = 0 - x;
        if (x == 1) return *this;
        if (x < BASE) {
            i64 carry{0};
            for (int &d : a) {
                i64 cur{i64(d) * i64(x) + carry};
```

```
                d = int(cur % BASE);
                carry = cur / BASE;
            }
            if (carry) a.push_back(int(carry));
        } else {
            u128 carry{0};
            for (int &d : a) {
                u128 cur{u128(d) * x + carry};
                d = int(cur % BASE);
                carry = cur / BASE;
            }
            while (carry) a.push_back(int(carry % BASE)), carry /= BASE;
        }
        return *this;
    }
    BigInt &operator/=(i64 v) {
        assert(v != 0);
        if (is_zero()) return *this;
        if (v < 0) sign = -sign;
        u64 x{u64(v)};
        if (v < 0) x = 0 - x;
        if (x == 1) return *this;
        if (x < BASE) {
            i64 rem{ 0 };
            for (auto &val : views::reverse(a)) {
//            for (int i = int(a.size()) - 1; i >= 0; --i) {
                i64 cur{ rem * BASE + val };
                val = int(cur / x);
                rem = cur % x;
            }
        } else {
            u128 rem{ 0 };
            for (auto &val : views::reverse(a)) {
//            for (int i = int(a.size()) - 1; i >= 0; --i) {
                u128 cur{ rem * BASE + val };
                val = int(cur / x);
                rem = cur % x;
            }
        }
        trim();
        return *this;
    }
    i64 operator%(i64 v) const {
        assert(v != 0);
        u64 x{u64(v)};
        if (v < 0) x = 0 - x;
        if (x == 1 || is_zero()) return 0;
        u64 rem{ 0 };
        if (x < BASE) {
            for (const auto &val : views::reverse(a)) rem = (rem * BASE + val) % x;
//            for (int i = int(a.size()) - 1; i >= 0; --i) rem = (rem * BASE + a[i]) % x;
        } else {
            u128 r{ 0 };
            for (const auto &val : views::reverse(a)) r = (r * BASE + val) % x;
//            for (int i = int(a.size()) - 1; i >= 0; --i) r = (r * BASE + a[i]) % x;
            rem = u64(r);
        }
        i64 r{i64(rem)};
        return sign < 0 ? -r : r;
    }
    friend BigInt operator*(BigInt x, i64 v) { return x *= v; }
    friend BigInt operator*(i64 v, BigInt x) { return x *= v; }
    friend BigInt operator/(BigInt x, i64 v) { return x /= v; }
    // ===== 11. BigInt 乘法：小规模朴素，大规模双 NTT 精确卷积 =====
    BigInt operator*(const BigInt &v) const {
        if (is_zero() || v.is_zero()) return 0;
        BigInt res;
        res.sign = sign * v.sign;
        res.a = mul_vec(a, v.a);
        res.trim();
        return res;
```

```
    }
    BigInt &operator*=(const BigInt &v) { return *this = *this * v; }
    // ===== 12. BigInt 除法与取模：小规模长除法，大规模 Newton 倒数 =====
    // 同时需要商和余数时务必直接调用 divmod()，避免 / 与 % 各做一遍完整除法。
    friend pair<BigInt, BigInt> divmod(const BigInt &x, const BigInt &y) {
        assert(!y.is_zero());
        auto [qa, ra]{divmod_vec(x.a, y.a)};
        BigInt q, r;
        q.a = move(qa);
        r.a = move(ra);
        if (!q.a.empty()) q.sign = x.sign * y.sign;
        if (!r.a.empty()) r.sign = x.sign;
        return {move(q), move(r)};
    }
    friend BigInt operator/(const BigInt &x, const BigInt &y) { return divmod(x, y).first; }
    friend BigInt operator%(const BigInt &x, const BigInt &y) { return divmod(x, y).second; }
    BigInt &operator/=(const BigInt &v) { return *this = divmod(*this, v).first; }
    BigInt &operator%=(const BigInt &v) { return *this = divmod(*this, v).second; }
    // ===== 13. 数论扩展：不需要 gcd / lcm 时以下代码可直接删除 =====
    friend BigInt gcd(BigInt x, BigInt y) {
        x.sign = y.sign = 1;
        while (!y.is_zero()) {
            auto [q, r]{divmod(x, y)};
            x = move(y);
            y = move(r);
        }
        return x;
    }
    friend BigInt lcm(const BigInt &x, const BigInt &y) {
        if (x.is_zero() || y.is_zero()) return 0;
        return (x / gcd(x, y) * y).abs();
    }
};
```

成员变量

- `a[i]`：绝对值按 10^9 进制低位在前存储，零用空 `vector` 表示
- `sign`：符号，非零时为 `1 / -1`，零固定为 `1`
- `MUL_NAIVE`：乘法朴素 / NTT 的切换阈值，默认 `64`
- `DIV_NAIVE`：长除法 / Newton 除法的切换阈值，默认 `64`

接口

- `BigInt(v)` / `BigInt(s)`：由 `i64` 或十进制字符串构造，$O(n)$
- `read(s)`：读取十进制字符串，$O(n)$
- `to_string()`：转十进制字符串，$O(n)$
- `is_zero()`：判断是否为零，$O(1)$
- `abs()`：返回绝对值副本，$O(n)$
- 比较运算：最坏 $O(n)$
- `+ / -`：高精加减，$O(n)$
- `* / / / %` 与 `i64`：$O(n)$
- `a * b`：小规模朴素乘法，大规模双 NTT 精确卷积
- `divmod(a, b)`：同时返回 `{a / b, a % b}`；小规模 / 短商走长除法，大规模走 Newton 倒数除法
- `a / b`：返回商
- `a % b`：返回余数
- `gcd(a, b)` / `lcm(a, b)`：高精最大公约数 / 最小公倍数

注意事项

- 整数除法语义与 C++ 一致：商向 `0` 截断，余数与被除数同号
- 同时需要商和余数时使用 `divmod(a, b)`，不要分别写 `a / b` 与 `a % b`，否则会完整计算两遍除法
- NTT 使用 `998244353` 与 `469762049` 做 CRT，卷积内部转成 10^4 进制；受第一模数限制，单次 NTT 长度最大为 2^23
- 当前实现覆盖 Library Checker `Big Integer` 的 $2imes10^6$ 十进制位规模；若要处理远超千万位的单个整数，需要更换支持更长 2-adic 长度的 NTT 模数或改成其他卷积实现
- `MUL_NAIVE` / `DIV_NAIVE` 是常数阈值，不同评测机可以在 `32 ~ 128` 左右微调

常见用法

同时求商和余数

```
auto [q, r] = divmod(a, b);
```

只求商或余数

```
BigInt q = a / b;
BigInt r = a % b;
```

与普通整数运算

```
BigInt x = a * 123456789LL;
x /= 998244353;
i64 r = x % 1000000007LL;
```

复杂度

设 `n`, `m` 为 10^9 进制 limb 数，$M(n)$ 为快速乘法复杂度

操作	时间复杂度
比较 / 加减	$O(\max(n,m))$
与 `i64` 乘除模	$O(n)$
小规模 BigInt 乘法	$O(nm)$
大规模 BigInt 乘法	$O(N\log N)$，`N` 为 NTT 长度
小除数 / 短商除法	近似 $O(n\min(m,n-m+1))$
大规模除法	约 $O(M(n)\log n)$
`gcd`	取决于欧几里得迭代次数与每次除法

空间复杂度：加减为 $O(n)$；快速乘除为 $O(N)$

BigInt（赛场短版）

```
struct BigInt {
    static constexpr int BASE{ 1000000000 }, DIGITS{ 9 };
    vector<int> a;
    int sign{ 1 };
private:
    // ===== 1. 核心：所有模块依赖 =====
    void trim() { while (!a.empty() && a.back() == 0) a.pop_back(); if (a.empty()) sign = 1; }
    static int abs_cmp(const BigInt &x, const BigInt &y) {
        if (x.a.size() != y.a.size()) return x.a.size() < y.a.size() ? -1 : 1;
        for (int i = int(x.a.size()) - 1; i >= 0; --i) if (x.a[i] != y.a[i]) return x.a[i] < y.a[i] ? -1 : 1;
        return 0;
    }
public:
    BigInt() = default;
    BigInt(i64 v) { *this = v; }
    BigInt(const string &s) { read(s); }
    bool is_zero() const { return a.empty(); }
    BigInt abs() const { BigInt r{*this}; r.sign = 1; return r; }
    BigInt &operator=(i64 v) {
        a.clear();
        sign = v < 0 ? -1 : 1;
        u64 x{ u64(v) };
        if (v < 0) x = 0 - x;
        while (x) a.push_back(x % BASE), x /= BASE;
        return *this;
    }
    BigInt operator-() const { BigInt r{ *this }; if (!r.is_zero()) r.sign = -r.sign; return r; }
    void read(const string &s) {
        a.clear(); sign = 1;
        int p{ 0 }; if (!s.empty() && (s[0] == '+' || s[0] == '-')) sign = s[0] == '-' ? -1 : 1, p = 1;
        for (int i = int(s.size()); i > p; i -= DIGITS) {
            int x{ 0 };
            for (int j = max(p, i - DIGITS); j < i; ++j) x = x * 10 + s[j] - '0';
            a.push_back(x);
        }
        trim();
```

```
    }
    friend istream &operator>>(istream &in, BigInt &x) { string s; if (in >> s) x.read(s); return in; }
    friend ostream &operator<<(ostream &out, const BigInt &x) {
        if (x.sign < 0) out << '-';
        if (x.a.empty()) return out << 0;
        out << x.a.back();
        for (int i = int(x.a.size()) - 2; i >= 0; --i) out << setw(DIGITS) << setfill('0') << x.a[i];
        return out << setfill(' ');
    }

    // ===== 2. 比较 =====
    friend bool operator<(const BigInt &x, const BigInt &y) {
        if (x.sign != y.sign) return x.sign < y.sign;
        int c{ abs_cmp(x, y) };
        return x.sign > 0 ? c < 0 : c > 0;
    }
    friend bool operator==(const BigInt &x, const BigInt &y) { return x.sign == y.sign && x.a == y.a; }
    friend bool operator!=(const BigInt &x, const BigInt &y) { return !(x == y); }
    friend bool operator>(const BigInt &x, const BigInt &y) { return y < x; }
    friend bool operator<=(const BigInt &x, const BigInt &y) { return !(y < x); }
    friend bool operator>=(const BigInt &x, const BigInt &y) { return !(x < y); }

private:
    // ===== 3. BigInt 加减 =====
    void add_abs(const BigInt &v) {
        a.resize(max(a.size(), v.a.size()));
        int carry{ 0 };
        for (int i = 0; i < a.size(); ++i) {
            i64 cur{ i64(a[i]) + carry + (i < v.a.size() ? v.a[i] : 0) };
            a[i] = cur % BASE; carry = cur / BASE;
        }
        if (carry) a.push_back(carry);
    }
    void sub_abs(const BigInt &v) {
        int borrow{0};
        for (int i = 0; i < a.size(); ++i) {
            i64 cur{ i64(a[i]) - borrow - (i < v.a.size() ? v.a[i] : 0) };
            if (cur < 0) cur += BASE, borrow = 1; else borrow = 0;
            a[i] = cur;
        }
        trim();
    }
public:
    BigInt &operator+=(const BigInt &v) {
        if (sign == v.sign) add_abs(v);
        else if (abs_cmp(*this, v) >= 0) sub_abs(v);
        else { BigInt t{v}; t.sub_abs(*this); *this = move(t); }
        return *this;
    }
    BigInt &operator-=(const BigInt &v) { return *this += -v; }
    friend BigInt operator+(BigInt x, const BigInt &y) { return x += y; }
    friend BigInt operator-(BigInt x, const BigInt &y) { return x -= y; }
    // ===== 4. 与 i64 乘 / 除 / 模：可按函数单独删除 =====
    BigInt &operator*=(i64 v) {
        if (!v || is_zero()) return *this = 0;
        if (v < 0) sign = -sign;
        u64 x{ u64(v) };
        if (v < 0) x = 0 - x;
        __uint128_t carry{ 0 };
        for (int &d : a) {
            __uint128_t cur{__uint128_t(d) * x + carry};
            d = cur % BASE; carry = cur / BASE;
        }
        while (carry) a.push_back(carry % BASE), carry /= BASE;
        return *this;
    }
    BigInt &operator/=(i64 v) {
        assert(v); if (is_zero()) return *this;
        if (v < 0) sign = -sign;
        u64 x{ u64(v) };
        if (v < 0) x = 0 - x;
```

```
        __uint128_t rem{ 0 };
        for (int i = int(a.size()) - 1; i >= 0; --i) {
            __uint128_t cur{ rem * BASE + a[i] };
            a[i] = cur / x; rem = cur % x;
        }
        trim(); return *this;
    }
    i64 operator%(i64 v) const {
        assert(v); u64 x{ u64(v) }; if (v < 0) x = 0 - x;
        __uint128_t r{ 0 };
        for (int i = int(a.size()) - 1; i >= 0; --i) r = (r * BASE + a[i]) % x;
        i64 z{i64(u64(r))};
        return sign < 0 ? -z : z;
    }
    friend BigInt operator*(BigInt x, i64 v) { return x *= v; }
    friend BigInt operator*(i64 v, BigInt x) { return x *= v; }
    friend BigInt operator/(BigInt x, i64 v) { return x /= v; }
    // ===== 5. BigInt 乘法：朴素 O(nm) =====
    friend BigInt operator*(const BigInt &x, const BigInt &y) {
        if (x.is_zero() || y.is_zero()) return 0;
        BigInt z;
        z.sign = x.sign * y.sign;
        z.a.assign(x.a.size() + y.a.size() + 1, 0);
        for (int i = 0; i < x.a.size(); ++i) {
            __uint128_t carry{ 0 };
            for (int j = 0; j < y.a.size() || carry; ++j) {
                __uint128_t cur{ __uint128_t(z.a[i + j]) + carry };
                if (j < y.a.size()) cur += __uint128_t(x.a[i]) * y.a[j];
                z.a[i + j] = cur % BASE; carry = cur / BASE;
            }
        }
        z.trim(); return z;
    }
    BigInt &operator*=(const BigInt &v) { return *this = *this * v; }
private:
    // ===== 6. BigInt 除法：Knuth 长除法；与 +/-、乘法模块无依赖 =====
    static pair<vector<int>, vector<int>> divmod_abs(const vector<int> &aa, const vector<int> &bb) {
        if (bb.size() == 1) {
            vector<int> q(aa.size()); i64 r{ 0 };
            for (int i = int(aa.size()) - 1; i >= 0; --i) {
                i64 cur{r * BASE + aa[i]}; q[i] = cur / bb[0]; r = cur % bb[0];
            }
            while (!q.empty() && q.back() == 0) q.pop_back();
            return { move(q), r ? vector<int>{ int(r) } : vector<int>{} };
        }
        if (aa.size() < bb.size() || (aa.size() == bb.size() && lexicographical_compare(aa.rbegin(), aa.rend(), bb.rbegin(),
bb.rend()))) return {{}, aa};
        int norm{ BASE / (bb.back() + 1) };
        auto mul = [&](const vector<int> &x) {
            vector<i64> z; z.reserve(x.size() + 1); i64 carry{0};
            for (int d : x) { i64 cur{i64(d) * norm + carry}; z.push_back(cur % BASE); carry = cur / BASE; }
            if (carry) z.push_back(carry);
            return z;
        };
        vector<i64> u{ mul(aa) }, v{ mul(bb) };
        u.push_back(0);
        int n = v.size(), m{ int(u.size()) - n - 1 };
        vector<int> q(m + 1);
        for (int j = m; j >= 0; --j) {
            i64 num{ u[j + n] * BASE + u[j + n - 1] };
            i64 qt{ num / v[n - 1] }, rt{ num % v[n - 1] };
            if (qt == BASE) --qt, rt += v[n - 1];
            while (n > 1 && rt < BASE && qt * v[n - 2] > BASE * rt + u[j + n - 2]) --qt, rt += v[n - 1];
            i64 carry{ 0 };
            for (int i = 0; i < n; ++i) {
                __int128 p{ __int128(qt) * v[i] + carry };
                i64 cur{ u[j + i] - i64(p % BASE) };
                carry = p / BASE;
                if (cur < 0) cur += BASE, ++carry;
                u[j + i] = cur;
            }
```

```
            u[j + n] -= carry;
            if (u[j + n] < 0) {
                -- qt; i64 c{ 0 };
                for (int i = 0; i < n; ++ i) {
                    i64 cur{ u[j + i] + v[i] + c };
                    if (cur >= BASE) cur -= BASE, c = 1; else c = 0;
                    u[j + i] = cur;
                }
                u[j + n] += c;
            }
            q[j] = int(qt);
        }
        while (!q.empty() && q.back() == 0) q.pop_back();
        vector<int> r(n);
        for (int i = 0; i < n; ++ i) r[i] = u[i];
        i64 rem{ 0 };
        for (int i = n - 1; i >= 0; -- i) {
            i64 cur{ rem * BASE + r[i] }; r[i] = cur / norm; rem = cur % norm;
        }
        while (!r.empty() && r.back() == 0) r.pop_back();
        return {move(q), move(r)};
    }
public:
    // 同时需要商和余数时调用 divmod()，不要分别写 / 和 %
    friend pair<BigInt, BigInt> divmod(const BigInt &x, const BigInt &y) {
        assert(!y.is_zero());
        auto [qa, ra]{ divmod_abs(x.a, y.a) };
        BigInt q, r; q.a = move(qa); r.a = move(ra);
        if (!q.a.empty()) q.sign = x.sign * y.sign;
        if (!r.a.empty()) r.sign = x.sign;
        return {move(q), move(r)};
    }
    friend BigInt operator/(const BigInt &x, const BigInt &y) { return divmod(x, y).first; }
    friend BigInt operator%(const BigInt &x, const BigInt &y) { return divmod(x, y).second; }
    BigInt &operator/=(const BigInt &v) { return *this = divmod(*this, v).first; }
    BigInt &operator%=(const BigInt &v) { return *this = divmod(*this, v).second; }
};
```

5B. BigInt 专用 FFT

模块 `5A`（上面结构中的朴素乘法）和本模块 `5B` 二选一
这一版不依赖外部 FFT / NTT 板子，而是在 `BigInt` 内部放一套专用短 FFT：

- 固定使用 `complex<double>`
- `BASE = 1e9` 的一个 limb 恰好拆成三个 `FFT_BASE = 1e3` 系数
- 把两个实数组分别放进复数的实部和虚部，只做 1 次正 FFT + 1 次逆 FFT
- 不提供通用多项式卷积接口，只服务于高精乘法

```
private:
    // ===== 5B. BigInt 专用 FFT 乘法：与模块 5A 二选一 =====
    using C = complex<double>;
    static constexpr double PI{ 3.14159265358979323846264338327795 };
    static constexpr int FFT_BASE{ 1000 };
    static void fft(vector<C> &a, bool inv) {
        int n = a.size();
        for (int i = 1, j = 0; i < n; ++ i) {
            int bit{ n >> 1 };
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int len = 2; len <= n; len <<= 1) {
            double ang{ 2 * PI / len * (inv ? -1 : 1) };
            C wl{ cos(ang), sin(ang) };
            for (int i = 0; i < n; i += len) {
                C w{ 1 };
                for (int j = 0; j < len / 2; ++ j) {
                    C u{ a[i + j]}, v{a[i + j + len / 2] * w };
                    a[i + j] = u + v;
                    a[i + j + len / 2] = u - v;
                    w *= wl;
```

```
                }
            }
        }
        if (inv) for (C &x : a) x /= n;
    }
    static vector<int> fft_digits(const vector<int> &a) {
        vector<int> d;
        d.reserve(a.size() * 3);
        for (int x : a) {
            d.push_back(x % FFT_BASE); x /= FFT_BASE;
            d.push_back(x % FFT_BASE); x /= FFT_BASE;
            d.push_back(x);
        }
        while (!d.empty() && d.back() == 0) d.pop_back();
        return d;
    }

public:
    friend BigInt operator*(const BigInt &x, const BigInt &y) {
        if (x.is_zero() || y.is_zero()) return 0;
        vector<int> a{ fft_digits(x.a) }, b{fft_digits(y.a) };
        int need{ int(a.size() + b.size() - 1) }, n{ 1 };
        while (n < need) n <<= 1;
        // 实部放 a，虚部放 b；利用共轭对称性一次 FFT 同时得到两者频谱。
        vector<C> f(n);
        for (int i = 0; i < a.size(); ++ i) f[i].real(a[i]);
        for (int i = 0; i < b.size(); ++ i) f[i].imag(b[i]);
        fft(f, false);
        for (int i = 0; i <= n / 2; ++ i) {
            int j{ (n - i) & (n - 1) };
            C A{ (f[i] + conj(f[j])) * 0.5 };
            C B{ (f[i] - conj(f[j])) * C(0, -0.5) };
            C z{ A * B };
            f[i] = z;
            if (i != j) f[j] = conj(z);
        }
        fft(f, true);
        vector<int> d;
        d.reserve(need + 8);
        i64 carry{0};
        for (int i = 0; i < need; ++ i) {
            i64 cur{ llround(f[i].real()) + carry };
            d.push_back(cur % FFT_BASE);
            carry = cur / FFT_BASE;
        }
        while (carry) d.push_back(carry % FFT_BASE), carry /= FFT_BASE;
        while (!d.empty() && d.back() == 0) d.pop_back();
        BigInt z;
        z.sign = x.sign * y.sign;
        z.a.reserve((d.size() + 2) / 3);
        for (int i = 0; i < d.size(); i += 3) {
            int v{ d[i] };
            if (i + 1 < d.size()) v += d[i + 1] * FFT_BASE;
            if (i + 2 < d.size()) v += d[i + 2] * FFT_BASE * FFT_BASE;
            z.a.push_back(v);
        }
        z.trim();
        return z;
    }
    BigInt &operator*=(const BigInt &v) { return *this = *this * v; }
```

裁剪索引

需求	保留模块
只存储 / 输入输出	1
比较	1 + 2
`+ -`	1 + 3
`* / % i64`	1 + 4 中对应函数

需求	保留模块
`BigInt * BigInt`，小规模	1 + 5A
`BigInt * BigInt`，普通 (O(n\log n))	1 + 5B FFT（替换 5A）
`BigInt / BigInt`、`%`	1 + 6
普通完整高精	1 ~ 6

模块 `5` 与模块 `6` **互不依赖**：只需要除法时完全不用抄高精乘法，只需要乘法时也不用抄除法

复杂度

设 `n, m` 为 10^9 进制 limb 数

操作	时间复杂度
输入输出 / 比较 / 加减	$O(n+m)$
与 `i64` 乘除模	$O(n)$
`BigInt * BigInt`（5A）	(O(nm))
`BigInt * BigInt`（5B）	$O((n+m)\log(n+m))$
`BigInt divmod BigInt`	(O(nm))

极短应急方案

若赛场允许 Boost，且数据规模不是专门卡超大整数算法：

```cpp
#include <boost/multiprecision/cpp_int.hpp>
using BigInt = boost::multiprecision::cpp_int;
```

普通高精题优先考虑这一方案；只有需要自定义接口、性能或特殊算法时再抄上面的短版

Python 原生高精度整数

Python 的 `int` 原生支持任意精度整数，不需要额外高精度类

基础模板

```python
import sys
input = sys.stdin.readline

# 超长十进制整数输入 / 输出时关闭字符串转换位数限制
# 老版本 Python 没有该接口，因此先判断
if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)
```

之后直接使用普通整数：

```python
a = int(input())
b = int(input())

c = a + b
d = a - b
e = a * b
q = a // b
r = a % b

x = pow(a, k)       # a^k
y = pow(a, k, mod)  # a^k mod mod，直接使用快速模幂
```

常用整数函数：

```python
from math import gcd, lcm, isqrt, factorial, comb

g = gcd(a, b)
l = lcm(a, b)
r = isqrt(a)        # floor(sqrt(a))，精确整数运算
f = factorial(n)
c = comb(n, k)
```

Python 负数除法与 C++ 不同

Python：

```
-7 // 3 == -3
-7 % 3 == 2
```

因为 `//` 向负无穷方向取整
C++ 的整数除法向 `0` 截断：

```
-7 / 3 == -2
-7 % 3 == -1
```

若 Python 必须模拟 C++ 的语义：

```
q = abs(a) // abs(b)
if (a < 0) ^ (b < 0):
    q = -q
r = a - q * b
```

Python 高精度十进制浮点数

需要超过普通 `float` 精度时使用 `Decimal`：

```
from decimal import Decimal, getcontext

getcontext().prec = 100

a = Decimal(input())
b = Decimal(input())

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

`prec` 表示运算结果保留的有效数字位数
输入十进制小数时优先：

```
x = Decimal("0.1")
```

或：

```
x = Decimal(input())
```

不要先经过 `float`：

```
x = Decimal(0.1)    # 得到的是二进制 float 0.1 的精确值，不是十进制 0.1
```

Python 精确分数

若题目要求有理数始终保持精确，可以直接使用：

```
from fractions import Fraction

a = Fraction(1, 3)
b = Fraction(2, 5)

print(a + b)
print(a * b)
print(a.numerator, a.denominator)
```

没有有理数需求时整段忽略

已明确的数值与输入边界

BigInt 的 read/流输入要求有效十进制整数字符串（允许正负号与前导零）；不是带错误恢复的通用解析器。完整版精确 NTT 要求转换后的卷积长度不超过 2^23（由 998244353 限制）。短版 5B 使用 double FFT，存在浮点舍入风险，不能承诺任意长度精确；要求严格整数正确性时使用完整版精确 NTT 或短版 5A。

2.5 Matrix

```cpp
template<typename T = i64>
struct Matrix {
    int n, m;
    vector<vector<T>> mat;
    Matrix(int n, int m) : n(n), m(m), mat(n, vector<T>(m, T{})) {}
    static Matrix E(int n) {
        Matrix res(n, n);
        for (int i = 0; i < n; ++ i) res[i][i] = T{1};
        return res;
    }
    vector<T>& operator[](int i) { return mat[i]; }
    const vector<T>& operator[](int i) const { return mat[i]; }
    Matrix operator+(const Matrix& o) const {
        assert(n == o.n && m == o.m);
        Matrix res(n, m);
        for (int i = 0; i < n; ++ i) {
            for (int j = 0; j < m; ++ j) res[i][j] = mat[i][j] + o[i][j];
        }
        return res;
    }
    Matrix operator*(const Matrix& o) const {
        assert(m == o.n);
        Matrix res(n, o.m);
        for (int i = 0; i < n; ++ i) {
            for (int k = 0; k < m; ++ k) {
                if (mat[i][k] == T{0}) continue;
                for (int j = 0; j < o.m; ++ j) {
                    res[i][j] = res[i][j] + mat[i][k] * o[k][j];
                }
            }
        }
        return res;
    }
};
template<class T>
Matrix<T> matrix_power(Matrix<T> a, u64 e) {
    assert(a.n == a.m);
    Matrix<T> res = Matrix<T>::E(a.n);
    for (; e; ) {
        if (e & 1) res = res * a;
        e >>= 1;
        if (e) a = a * a;
    }
    return res;
}
```

定长 Matrix

```cpp
template<int N, int M = N, typename T = i64>
struct Matrix {
    array<array<T, M>, N> mat;
    Matrix() { for (auto& row : mat) row.fill(T{0}); }
    static Matrix E() {
        Matrix res;
        for (int i = 0; i < min(N, M); ++ i) res[i][i] = T{1};
        return res;
    }
    array<T, M>& operator[](int i) { return mat[i]; }
```

```
    const array<T, M>& operator[](int i) const { return mat[i]; }
    Matrix operator+(const Matrix& o) const {
        Matrix res;
        for (int i = 0; i < N; ++ i) {
            for (int j = 0; j < M; ++ j) res[i][j] = mat[i][j] + o[i][j];
        }
        return res;
    }
    template<int K>
    Matrix<N, K, T> operator*(const Matrix<M, K, T>& o) const {
        Matrix<N, K, T> res;
        for (int i = 0; i < N; ++ i) {
            for (int k = 0; k < M; ++ k) {
                if (mat[i][k] == T{0}) continue;
                for (int j = 0; j < K; ++ j) {
                    res[i][j] = res[i][j] + mat[i][k] * o[k][j];
                }
            }
        }
        return res;
    }
};
using Mat2 = Matrix<2>;
template<int N, class T>
Matrix<N, N, T> power(Matrix<N, N, T> a, u64 e) {
    auto res = Matrix<N, N, T>::E();
    for (; e; ) {
        if (e & 1) res = res * a;
        e >>= 1;
        if (e) a = a * a;
    }
    return res;
}
```

热带 / 常用半环矩阵速查

普通矩阵	Min-Plus	Max-Plus	Min-Max	Max-Min
`T ans = 0`	`T ans = INF`	`T ans = -INF`	`T ans = INF`	`T ans = -INF`
`ans += x * y`	`ans = min(ans, x + y)`	`ans = max(ans, x + y)`	`ans = min(ans, max(x, y))`	`ans = max(ans, min(x, y))`
零矩阵全 `0`	全 `INF`	全 `-INF`	全 `INF`	全 `-INF`
单位阵对角 `1`	对角 `0`	对角 `0`	对角 `-INF`	对角 `INF`
单位阵非对角 `0`	非对角 `INF`	非对角 `-INF`	非对角 `INF`	非对角 `-INF`

Min-Plus

图论含义

若

```
A[u][v] = edge_weight(u, v);
```

那么

$$(A^k)_{uv}$$

表示：

> 从 u 到 v，**恰好经过 k 条边**的最短路长度

因此适合：

- 固定边数最短路
- 最小转移代价
- DP + 矩阵快速幂

Max-Plus

常见意义

$$(A^k)_{uv}$$

表示：

> 从 (u) 到 (v)，恰好进行 (k) 次转移能够得到的最大收益

常用于：

- 固定步数最大权路径
- 最大收益 DP
- 调度
- Max-Plus 线性系统

Min-Max

图论意义

路径的代价不是边权之和，而是

$$\max_{e\in P} w_e$$

然后在所有路径中取最小：

$$\min_{P} \max_{e\in P} w_e$$

也就是典型的 minimax path / 最小瓶颈路

Max-Min

图论意义

一条路径的容量为：

$$\min_{e\in P} w_e$$

然后选容量最大的路径：

$$\max_{P} \min_{e\in P} w_e$$

即：

- widest path
- 最大瓶颈路径
- 最大运输容量

Boolean 半环

若 `A[u][v] = true` 表示有边，则

$$(A^k)_{uv}$$

表示：

> 是否存在从 u 到 v 恰好走 k 条边的路径
> 普通整数矩阵要求每次乘加及必要的中间矩阵均不溢出 T；指数为 1 时不会额外平方。动态版本要求加法同形、乘法维度匹配、幂为方阵。

2.6 并查集维护后继（删除跳点 DSU）

```cpp
struct NextDSU {
    int n;
    vector<int> pa;
    explicit NextDSU(int n_) : n(n_), pa(n + 1) { ranges::iota(pa, 0); }
    // 返回第一个 ≥ x 且尚未删除的位置；返回 n 表示不存在
    int next(int x) {
        while (x != pa[x]) x = pa[x] = pa[pa[x]];
        return x;
    }
    // 删除位置 x，并返回删除后第一个 > x 且尚未删除的位置
    int erase(int x) { return pa[x] = next(x + 1); }
};
```

成员变量

- `n`：有效位置为 `[0,n)`
- `pa[n]`：额外的哨兵，表示“后面已经没有可用位置”

接口

- `next(x)`：返回第一个不小于 `x` 的未删除位置，仅路径压缩、未按秩合并时可用均摊 $O(\log n)$ 上界
- `erase(x)`：删除位置 `x`，并返回删除后的下一个未删除位置，仅路径压缩、未按秩合并时可用均摊 $O(\log n)$ 上界
  这里“删除”的实际操作就是：

```cpp
pa[x] = next(x + 1);
```

也就是把 `x` 永久接到自己的后继上。之后所有经过 `x` 的查询都会直接跳过去

注意事项

- 有效位置是 `[0,n)`，`n` 是**哨兵**，不能作为普通位置使用
- `next(x) == n` 表示 `[x,n)` 已经没有未删除位置
- 只适用于**单向删除、不会恢复**的场景；普通并查集的路径压缩无法撤销
- 一个位置一旦删除，之后查询会永久跳过它
- `erase(x)` 可以重复调用；对于已经删除的位置，只会重新指向其当前后继，不会把后继一起删除
- 如果既要删除又要恢复，通常改用 `set`、Fenwick、线段树，或者考虑离线倒序处理

常见用法

查询下一个未删除位置

```cpp
NextDSU dsu(n);
dsu.erase(3);
dsu.erase(4);
int x = dsu.next(3); // 5
```

这就是最直接的 successor with delete

跳过已经处理的位置

```cpp
for (int x = dsu.next(l); x < r; x = dsu.erase(x)) {
    // 处理 x
}
```

例如“把 `[l,r)` 中所有**第一次出现的、尚未处理的位置**赋值”：

```cpp
for (int x = dsu.next(l); x < r; x = dsu.erase(x)) {
    a[x] = value;
}
```

即使有很多区间操作，每个位置也只真正进入循环一次

区间只处理一次

例如有 q 个区间，每个位置一旦被覆盖后以后都不用处理：

```cpp
NextDSU dsu(n);
for (auto [l, r] : queries) {
    for (int x = dsu.next(l); x < r; x = dsu.erase(x)) {
        // x 第一次被覆盖
```

```
        vis[x] = true;
    }
}
```

所有循环加起来最多真正处理 n 个位置，而不是每个询问重新扫描整个区间

找下一个空闲位置

还有一类很常见的写法是：

```
int p = dsu.next(x);
if (p == n) {
    // 不存在位置
} else {
    // 使用 p
    dsu.erase(p);
}
```

可以表示：

- 第一个未使用位置
- 第一个未分配槽位
- 第一个还没处理的元素
- 从 `x` 开始第一个可用位置

复杂度

操作	时间复杂度
构造	$O(n)$
`next(x)` / `erase(x)`	仅路径压缩、未按秩合并时可用均摊 $O(\log n)$ 上界

空间复杂度：$O(n)$
如果每个位置最多删除一次，那么像

```
for (int x = dsu.next(l); x < r; x = dsu.erase(x)) {
    ...
}
```

这样的所有区间扫描中，每个位置最多被真正处理一次，因此总复杂度通常可以看成

$$O((n+q)\log n)$$

2.7 二分、三分与黄金分割

整数二分

```
i64 mid_floor(i64 l, i64 r) { return i64(i128(l) + (i128(r) - l) / 2); }
i64 mid_ceil(i64 l, i64 r) { return i64(i128(l) + (i128(r) - l + 1) / 2); }

template<class F>
i64 fst_true(i64 l, i64 r, F f) {
    while (l < r) {
        i64 m = mid_floor(l, r);
        if (f(m)) r = m;
        else l = m + 1;
    }
    return l;
}

template<class F>
i64 lst_true(i64 l, i64 r, F f) {
    while (l < r) {
        i64 m = mid_ceil(l, r);
        if (f(m)) l = m;
        else r = m - 1;
```

```
    }
    return l;
}
```

接口说明

- `mid_floor(l,r)`：$\lfloor (l+r)/2 \rfloor$，要求 `l ⩽ r`
- `mid_ceil(l,r)`：$\lceil (l+r)/2 \rceil$，要求 `l ⩽ r`
- `fst_true(l,r,f)`：在 `false...true` 中找第一个真，要求 `f(r) = true`
- `lst_true(l,r,f)`：在 `true...false` 中找最后一个真，要求 `f(l) = true`

注意事项

- 搜索区间为闭区间 `[l,r]`
- 使用 `i128` 计算区间长度，可处理负数区间和完整 `i64` 范围
- 答案可能不存在时需要额外哨兵或返回后验证
- 不要改成 `(l + r) / 2`，既可能溢出，也不能正确表达负数情况下的向下取整

常见用法

```
i64 ans = fst_true(l, r, [&](i64 x) {
    return ok(x);
});
```

```
i64 ans = lst_true(l, r, [&](i64 x) {
    return ok(x);
});
```

复杂度

$O(\log(r-l+1))$ 次判定，额外空间 $O(1)$

标准库二分

已有有序范围时优先使用标准库

```
auto p1 = ranges::lower_bound(a, x); // 第一个 ⩾ x
auto p2 = ranges::upper_bound(a, x); // 第一个 > x
```

一般的单调谓词可以使用 `partition_point`

```
auto p = ranges::partition_point(a, [&](const auto &x) {
    return check(x);
});
```

整数候选区间也可以配合 `views::iota`

```
auto rg = views::iota(l, r);
auto p = ranges::partition_point(rg, [&](i64 x) {
    return !ok(x);
});
if (p ≠ rg.end()) {
    i64 ans = *p;
}
```

接口说明

- `ranges::lower_bound(a,x)`：第一个 `⩾ x`
- `ranges::upper_bound(a,x)`：第一个 `> x`
- `ranges::partition_point(a,f)`：`true...false` 中第一个假
- `views::iota(l,r)`：生成整数范围 `[l,r)`

注意事项

- `partition_point` 要求范围已经按谓词分为 `true...false`
- 若目标谓词为 `false...true`，通常传入 `!ok(x)`
- `views::iota(l,r)` 是半开区间，与手写整数二分的闭区间不同

- `p == end()` 表示不存在对应位置，不能直接解引用
- `ranges` / `views` 写法需要 C++20

复杂度

随机访问范围上为 $O(\log n)$ 次比较或谓词判断

实数二分

固定迭代次数通常优先使用

```
template<class F>
ld bin_real(ld l, ld r, F f, int it = 100) {
    while (it --) {
        ld m = l + (r - l) / 2;
        if (f(m)) r = m;
        else l = m;
    }
    return (l + r) / 2;
}
```

接口说明

- `bin_real(l,r,f,it)`：在单调实数谓词上逼近 `false...true` 的分界点
- 返回分界点的近似值

注意事项

- 固定次数不会因为浮点精度导致死循环
- `long double` 下 `100` 次通常已经足够
- 若谓词方向为 `true...false`，反转更新方向或谓词

复杂度

`it` 次判定，额外空间 $O(1)$

整数单峰：斜率二分

对于先不增后不减的整数函数，优先使用相邻值比较
找最左极小点：

```
template<class F>
i64 argmin(i64 l, i64 r, F f) {
    while (l < r) {
        i64 m = mid_floor(l, r);
        if (f(m) ⩽ f(m + 1)) r = m;
        else l = m + 1;
    }
    return l;
}
```

若存在一段极小值平台，找最右极小点：

```
template<class F>
i64 argmin_r(i64 l, i64 r, F f) {
    while (l < r) {
        i64 m = mid_floor(l, r);
        if (f(m) < f(m + 1)) r = m;
        else l = m + 1;
    }
    return l;
}
```

接口说明

- `argmin(l,r,f)`：返回 `[l,r]` 中最左极小点
- `argmin_r(l,r,f)`：返回 `[l,r]` 中最右极小点

注意事项

- 要求 `f` 在整数域上先不增后不减
- 本质是在二分 `f(x+1)-f(x)` 的符号
- `m < r`，因此 `f(m + 1)` 始终合法
- 求最大值时反转比较符号
- 若 `f(x)` 很贵，可自行缓存相邻位置的函数值

常见用法

```
i64 p = argmin(l, r, [&](i64 x) {
    return calc(x);
});
auto ans = calc(p);
```

复杂度

$O(\log(r-l+1))$ 轮，每轮至多两次函数计算，额外空间 $O(1)$

整数三分

无法方便使用相邻斜率时可使用三分

```
template<class F>
auto ter_int(i64 l, i64 r, F f) {
    while (i128(r) - l > 6) {
        i128 d = (i128(r) - l) / 3;
        i64 m1 = i64(i128(l) + d);
        i64 m2 = i64(i128(r) - d);
        if (f(m1) ≤ f(m2)) r = m2 - 1;
        else l = m1 + 1;
    }
    i64 p = l;
    auto v = f(l);
    for (i64 x = l; x ≠ r; ) {
        ++ x;
        auto y = f(x);
        if (y < v) v = y, p = x;
    }
    return pair{p, v};
}
```

接口说明

- `ter_int(l,r,f)`：求 `[l,r]` 上单峰函数的最小值
- 返回 `{最小值位置, 最小值}`

注意事项

- 对离散凸函数通常优先使用 `argmin`
- 必须保证函数确实单峰
- 最后直接枚举剩余常数个位置，避免离散三分的边界问题
- 求最大值时反转比较符号

复杂度

$O(\log(r-l+1))$ 次函数计算，额外空间 $O(1)$

实数三分

```
template<class F>
ld ter_real(ld l, ld r, F f, int it = 150) {
    while (it --) {
        ld d = (r - l) / 3;
        ld m1 = l + d, m2 = r - d;
        if (f(m1) ≤ f(m2)) r = m2;
        else l = m1;
    }
```

```
    return (l + r) / 2;
}
```

接口说明

- `ter_real(l,r,f,it)`：求实数区间上单峰函数的最小值位置

注意事项

- 要求 `f` 先下降后上升，允许极小值平台
- 若导数容易计算且单调，通常直接二分导数更合适
- 求最大值时反转比较符号

复杂度

`2it` 次函数计算，额外空间 $O(1)$

二分导数

连续凸函数且导数可计算时，可直接对导数零点二分

```
ld x = bin_real(l, r, [&](ld x) {
    return df(x) ⩾ 0;
});
```

注意事项

- 要求导数单调不减
- 不要求真的存在 `df(x) == 0`，搜索的是导数由负变非负的分界位置
- 通常比实数三分更直接

复杂度

与实数二分相同

黄金分割

单次函数计算较贵时，比普通实数三分少一次重复计算

```
template<class F>
ld golden(ld l, ld r, F f, int it = 150) {
    constexpr ld k = 0.6180339887498948482L;
    ld x1 = r - (r - l) * k;
    ld x2 = l + (r - l) * k;
    auto y1 = f(x1), y2 = f(x2);
    while (it --) {
        if (y1 ⩽ y2) {
            r = x2;
            x2 = x1;
            y2 = y1;
            x1 = r - (r - l) * k;
            y1 = f(x1);
        } else {
            l = x1;
            x1 = x2;
            y1 = y2;
            x2 = l + (r - l) * k;
            y2 = f(x2);
        }
    }
    return (l + r) / 2;
}
```

接口说明

- `golden(l,r,f,it)`：黄金分割搜索实数单峰函数最小值位置

注意事项

- 与实数三分前提相同
- 初始化计算两次 `f`，之后每轮只新增一次计算
- `f(x)` 较便宜时普通三分通常已经足够

复杂度

共 `it + 2` 次函数计算，额外空间 $O(1)$

2.8 并行二分

有 `m` 次按顺序发生的更新和 `q` 个询问，每个询问都要求最早在哪次更新后满足单调条件

```
template<class R, class A, class C>
vector<int> pbs(int q, int m, R rst, A add, C ok) {
    vector<int> l(q, -1), r(q, m + 1);
    vector<vector<int>> buc(m + 1);
    while (true) {
        bool any = false;
        for (auto &v : buc) v.clear();
        for (int id = 0; id < q; ++ id) if (l[id] + 1 < r[id]) {
            any = true;
            int mid = l[id] + (r[id] - l[id]) / 2;
            buc[mid].push_back(id);
        }
        if (!any) break;
        rst();
        for (int t = 0; t <= m; ++ t) {
            if (t) add(t);
            for (int id : buc[t]) {
                if (ok(id)) r[id] = t;
                else l[id] = t;
            }
        }
    }
    return r; // 0: 初始满足，m + 1: 始终不满足
}
```

接口说明

- `pbs(q,m,rst,add,ok)`：求每个询问第一次满足条件的更新时间
- 询问编号为 `0..q-1`
- 更新编号为 `1..m`
- `rst()`：恢复到所有更新发生前的状态
- `add(t)`：加入第 `t` 次更新
- `ok(id)`：判断询问 `id` 在当前状态下是否满足
- 返回 `ans[id] = 0` 表示不执行更新已经满足
- 返回 `ans[id] = m + 1` 表示执行全部更新仍不满足
- 否则第 `ans[id]` 次更新后第一次满足

注意事项

- 对每个询问，`ok(id)` 必须随 `t` 单调地从假变真
- 每轮都会 `rst()`，随后严格按 `1..m` 的顺序重新加入更新
- `ok(id)` 只能依赖当前已经加入的更新，不能残留上一轮状态
- 更新若会使询问从真重新变假，则不能直接使用并行二分
- `rst()` 较贵时可使用时间戳、记录修改位置后撤销等方式降低清空代价
- 所有询问必须共享同一条更新时间轴，否则通常需要重新建模
- 若只是单个询问，普通二分通常更简单

常见用法

下面只展示接口语义，实际题目通常把 `cur` 换成 BIT、线段树、并查集等结构

```
i64 cur = 0;
auto ans = pbs(q, m,
    [&] {
        cur = 0;
    },
    [&](int t) {
```

```
        cur += a[t];
    },
    [&](int id) {
        return cur >= need[id];
    }
);
```

判断返回值：

```
for (int id = 0; id < q; ++ id) {
    if (ans[id] == 0) {
        // 初始状态已经满足
    } else if (ans[id] == m + 1) {
        // 全部更新后仍不满足
    } else {
        // 第 ans[id] 次更新后第一次满足
    }
}
```

典型 BIT 型接入形式：

```
auto ans = pbs(q, m,
    [&] {
        // 清空 BIT
    },
    [&](int t) {
        // 加入第 t 次修改
    },
    [&](int id) {
        // 查询当前 BIT，判断询问 id 是否满足
        return true;
    }
);
```

复杂度

设一轮中：

- `rst()` 总代价为 R
- 执行全部 `add()` 的总成本为 A
- 执行全部 `ok()` 的总成本为 C
  总时间复杂度为

$$O((m+q+R+A+C)\log(m+2))$$

其中 `m+q` 来自分桶和扫描
若 `add`、`ok` 均为 $O(\log n)$，且清空代价可忽略，则为

$$O((m+q)\log n\log m)$$

空间复杂度为 $O(m+q)$

2.9 位运算、二进制集合枚举与 Bitset

标准位函数与 GNU 内建函数

以下默认 `x` 为 `u64`

功能	C++20 / C++23	GNU / C++17
1 的数量	`popcount(x)`	`__builtin_popcountll(x)`
1 的数量奇偶性	`popcount(x) & 1`	`__builtin_parityll(x)`
末尾 0 数量	`countr_zero(x)`	`x ? __builtin_ctzll(x) : 64`
前导 0 数量	`countl_zero(x)`	`x ? __builtin_clzll(x) : 64`
末尾 1 数量	`countr_one(x)`	`~x ? __builtin_ctzll(~x) : 64`
前导 1 数量	`countl_one(x)`	`~x ? __builtin_clzll(~x) : 64`

功能	C++20 / C++23	GNU / C++17
是否为二的幂	`has_single_bit(x)`	`x && !(x & (x - 1))`
二进制位宽	`bit_width(x)`	`x ? __lg(x) + 1 : 0`
不大于 `x` 的最大二次幂	`bit_floor(x)`	`x ? 1ULL << __lg(x) : 0`
不小于 `x` 的最小二次幂	`bit_ceil(x)`	`x <= 1 ? 1 : 1ULL << (__lg(x - 1) + 1)`
$\lfloor \log_2 x \rfloor$	`bit_width(x) - 1`	`__lg(x)`
$\lceil \log_2 x \rceil$	`bit_width(x - 1)`	`x <= 1 ? 0 : __lg(x - 1) + 1`
最低 `1` 的 `1base` 位置	`x ? countr_zero(x) + 1 : 0`	`__builtin_ffsll(x)`
字节翻转	`byteswap(x)` C++23	`__builtin_bswap64(x)`

其中 `__lg(x)` 是 GNU libstdc++ 扩展，不属于 ISO C++，要求 `x > 0`
也可以完全不用 `__lg`：

```
int lg = 63 - __builtin_clzll(x); // x > 0
int bw = x ? 64 - __builtin_clzll(x) : 0;
```

注意事项

- `<bit>` 中上述计数、二次幂和循环移位接口主要接受无符号整数，位运算默认优先使用 `u64`
- `countr_zero(0ULL) == 64`、`countl_zero(0ULL) == 64`
- `__builtin_ctzll(0)`、`__builtin_clzll(0)` 未定义，必须提前判断
- `__lg(0)` 未定义，不要拿它代替带零处理的 `bit_width`
- `bit_floor(0) == 0`，`bit_ceil(0) == 1`
- `bit_ceil(x)` 要求结果能由参数类型表示；对 `u64` 即 `x <= 1ULL << 63`，否则行为未定义
- `byteswap` 是 C++23，其余表中的标准位操作均为 C++20
- GNU 内建函数按类型有无后缀、`l`、`ll` 三组，`u64` 通常使用 `ll`

循环移位

C++20 直接使用：

```
u64 a = rotl(x, k);
u64 b = rotr(x, k);
```

C++17 没有对应的标准函数，可以写：

```
u64 rotl64(u64 x, int k) {
    k %= 64;
    if (k < 0) k += 64;
    return k ? (x << k) | (x >> (64 - k)) : x;
}
u64 rotr64(u64 x, int k) {
    k %= 64;
    if (k < 0) k += 64;
    return k ? (x >> k) | (x << (64 - k)) : x;
}
```

接口说明

- `rotl(x,k)` / `rotl64(x,k)`：循环左移 `k` 位
- `rotr(x,k)` / `rotr64(x,k)`：循环右移 `k` 位
- `k` 可以为负数，负左移等价于正右移

复杂度

$O(1)$

GNU 特有函数

以下没有常用的标准库一一对应接口：

函数	含义
`__builtin_ffsll(x)`	最低 `1` 的位置，`1base`，`x = 0` 返回 `0`
`__builtin_parityll(x)`	`popcount(x) & 1`
`__builtin_clrsbll(x)`	除符号位外的冗余符号位数量
`__builtin_bswap64(x)`	64 位整数按字节翻转

`parity`、`ffs` 在现代 C++ 中通常直接由 `popcount`、`countr_zero` 表达，不需要额外封装函数

基础位操作

第 `k` 位从最低位开始 `0base`

操作	写法
读取第 `k` 位	`(x >> k) & 1ULL`
判断第 `k` 位为 `1`	`x & (1ULL << k)`
第 `k` 位设为 `1`	`x |= 1ULL << k`
第 `k` 位设为 `0`	`x &= ~(1ULL << k)`
翻转第 `k` 位	`x ^= 1ULL << k`
第 `k` 位赋值为 `v`	`x = (x & ~(1ULL << k)) | (u64(v) << k)`
删除最低位 `1`	`x &= x - 1`
保留最低位 `1`	`x & -x`
将最低位 `0` 设为 `1`	`x | (x + 1)`
清除末尾连续 `1`	`x & (x + 1)`

低位掩码：

```
u64 mask(int k) { return k == 64 ? ~0ULL : (1ULL << k) - 1; }
u64 mask(int l, int r) { return mask(r) ^ mask(l); }
u64 bits(u64 x, int l, int r) { return (x >> l) & mask(r - l); }
```

接口说明

- `mask(k)`：低 `k` 位为 `1`，要求 $0 \le k \le 64$
- `mask(l,r)`：第 `[l,r)` 位为 `1`
- `bits(x,l,r)`：取出 `x` 的 `[l,r)` 位并移动到最低位

注意事项

- `x << k`、`x >> k` 要求 $0 \le k < 64$
- `1ULL << 64` 未定义，因此低 `64` 位掩码必须单独处理
- 常量使用 `1ULL`，不要写 `1 << k`
- `~x` 会翻转全部 `64` 位，只翻低 `n` 位使用 `x ^ mask(n)`
- 无符号整数中的 `-x` 按模 2^{64} 运算，因此 `x & -x` 定义良好

最低位与最高位

最低位：

```
u64 lb = x & -x;
x &= x - 1; // 删除最低位 1
```

对于 `x > 0`：

$$\text{lowbit}(x) = 2^{\text{countr_zero}(x)}$$

最高位：

```
u64 hb = bit_floor(x);                    // C++20
u64 hb2 = x ? 1ULL << __lg(x) : 0;        // GNU
```

常用恒等式：

表达式	效果
`x & -x`	保留最低位 `1`
`x & (x - 1)`	删除最低位 `1`
`x | (x - 1)`	最低位 `1` 以下全部置 `1`，要求 `x > 0`
`x ^ (x - 1)`	最低位 `1` 及以下全部置 `1`，要求 `x > 0`
`x | (x + 1)`	最低位 `0` 置 `1`
`x & (x + 1)`	清除末尾连续 `1`
`~x & (x + 1)`	保留最低位 `0`
`x ^ (x + 1)`	最低位 `0` 及以下全部置 `1`

对于 `x > 0`：

$$x \oplus (x-1) = 2\,\mathrm{lowbit}(x) - 1$$

二进制位宽与对数

对于 `x > 0`：

```
int fl = bit_width(x) - 1; // floor(log2(x))
int ce = bit_width(x - 1); // ceil(log2(x))
```

GNU 写法：

```
int fl = __lg(x);
int ce = x <= 1 ? 0 : __lg(x - 1) + 1;
```

并且：

$$\mathrm{bit_width}(x) = \begin{cases} 0, & x = 0 \\ \lfloor \log_2 x \rfloor + 1, & x > 0 \end{cases}$$

二的幂

现代 C++：

```
bool ok = has_single_bit(x);
u64 lo = bit_floor(x);
u64 hi = bit_ceil(x);
```

老版本：

```
bool ok = x && !(x & (x - 1));
u64 lo = x ? 1ULL << __lg(x) : 0;
u64 hi = x <= 1 ? 1 : 1ULL << (__lg(x - 1) + 1);
```

对于 `x > 0`，以下条件等价：

```
has_single_bit(x)
(x & (x - 1)) == 0
popcount(x) == 1
(x & -x) == x
bit_floor(x) == x
bit_ceil(x) == x
```

注意事项

判断二的幂时，裸写

```
(x & (x - 1)) == 0
```

会错误地把 `0` 判为二的幂

二的幂取模与对齐

设 `p = 2^k`
对于非负整数：

```
x % p == (x & (p - 1))
```

向下对齐：

```
u64 y = x & ~(p - 1);
```

向上对齐：

```
u64 y = (x + p - 1) & ~(p - 1);
```

注意事项

- 要求 `p` 为二的幂
- 向上对齐必须保证 `x + p - 1` 不溢出

加法与位运算

对于非负整数：

$$a + b = (a \oplus b) + 2(a \,\&\, b)$$

$$a + b = (a \mid b) + (a \,\&\, b)$$

$$a \mid b = (a \oplus b) + (a \,\&\, b)$$

因此 `(a & b) == 0` 时：

$$a + b = a \oplus b = a \mid b$$

这也是判断区间和等于区间异或和时常用的无进位条件
无符号整数可以无溢出地计算平均数：

```
u64 fl = (a & b) + ((a ^ b) >> 1);
u64 ce = (a | b) - ((a ^ b) >> 1);
```

二进制集合

低 `n` 位分别表示 `n` 个元素

集合操作	位运算
$A \cap B$	`a & b`
$A \cup B$	`a | b`
$A \triangle B$	`a ^ b`
$A \setminus B$	`a & ~b`
全集	`mask(n)`
补集	`mask(n) ^ a`
加入 `i`	`a |= 1ULL << i`
删除 `i`	`a &= ~(1ULL << i)`
翻转 `i`	`a ^= 1ULL << i`

集合操作	位运算
大小	`popcount(a)` / `__builtin_popcountll(a)`

子集关系：

```
(a & b) == a   // a ⊆ b
(a | b) == b   // a ⊆ b
(a & ~b) == 0  // a ⊆ b
(a & b) == 0   // a ∩ b = ∅
```

数量结论

一个 `n` 元集合：

- 子集数为 2^n
- 大小恰好为 `k` 的子集数为 $\binom{n}{k}$
- 每个元素出现在 2^{n-1} 个子集中
- 所有子集大小之和为 $n2^{n-1}$
  状态 `s`：
- 子集数为 $2^{\text{popcount}(s)}$
- `n` 位内超集数为 $2^{n-\text{popcount}(s)}$
  所有 `(s,t)` 满足 `t ⊆ s` 的有序对共有

$$3^n$$

枚举所有状态

要求 `0 ≤ n < 64`

```
for (u64 s = 0, lim = 1ULL << n; s < lim; ++ s) {
    // s
}
```

复杂度

$O(2^n)$

枚举状态中的所有元素

```
for (u64 t = s; t; t &= t - 1) {
    int k = countr_zero(t);// __builtin_ctzll(t)
}
```

若只需要对应的二次幂：

```
for (u64 t = s; t; t &= t - 1) {
    u64 lb = t & -t;
}
```

复杂度

$O(\text{popcount}(s))$

枚举子集

非空子集：

```
for (u64 t = s; t; t = (t - 1) & s) {
    // t
}
```

包括空集：

```
for (u64 t = s;; t = (t - 1) & s) {
    // t
    if (!t) break;
}
```

非空真子集，要求 $s \neq 0$：

```
for (u64 t = (s - 1) & s; t; t = (t - 1) & s) {
    // t
}
```

注意事项

不能对无符号数写：

```
for (u64 t = s; t ⩾ 0; ...)
```

因为 $t \geqslant 0$ 永远成立

复杂度

$O(2^{\text{popcount}(s)})$
全部 n 位状态再分别枚举子集的总复杂度为 $O(3^n)$

枚举超集

在低 n 位内枚举 s 的全部超集：

```
u64 all = mask(n);
for (u64 t = s;; t = (t + 1) | s) {
    // s ⊆ t
    if (t == all) break;
}
```

复杂度

$O(2^{n-\text{popcount}(s)})$

集合二分

枚举所有有序划分 $a \cup b = s, a \cap b = \varnothing$：

```
for (u64 a = s;; a = (a - 1) & s) {
    u64 b = s ^ a;
    // a, b
    if (!a) break;
}
```

若 (a,b) 与 (b,a) 视为相同，可以固定最低位属于 a：

```
if (s) {
    u64 lb = s & -s;
    for (u64 a = s; a; a = (a - 1) & s) if (a & lb) {
        u64 b = s ^ a;
        // 每个无序划分一次
    }
}
```

Gosper Hack

按数值递增枚举 n 位中恰有 k 个 1 的状态
要求 $0 < k \leqslant n < 64$

```
u64 s = (1ULL << k) - 1;
u64 lim = 1ULL << n;
while (s < lim) {
    // s
    u64 lb = s & -s;
    u64 x = s + lb;
    s = x | (((x ^ s) >> 2) / lb);
}
```

`k == 0` 时唯一状态为 `0`

复杂度

共枚举 $\binom{n}{k}$ 个状态，每次转移 $O(1)$

Gray Code

编码：

```
u64 gray(u64 x) { return x ^ (x >> 1); }
```

解码：

```
u64 igray(u64 g) {
    u64 x = 0;
    for (; g; g >>= 1) x ^= g;
    return x;
}
```

枚举：

```
for (u64 i = 0; i < (1ULL << n); ++ i) {
    u64 s = gray(i);
}
```

相邻 Gray Code 只改变一位，第 `i - 1 → i` 次改变的位置为：

```
int k = countr_zero(i);          // C++20
int k = __builtin_ctzll(i);      // GNU
```

且：

```
gray(i) ^ gray(i - 1) == (i & -i)
```

复杂度

编码 $O(1)$，解码 $O(\log g)$

区间异或

```
u64 xor_pre(u64 n) {
    if ((n & 3) == 0) return n;
    if ((n & 3) == 1) return 1;
    if ((n & 3) == 2) return n + 1;
    return 0;
}
u64 xor_sum(u64 l, u64 r) {
    return xor_pre(r) ^ (l ? xor_pre(l - 1) : 0);
}
```

接口说明

- `xor_pre(n)`：$0 \oplus 1 \oplus \cdots \oplus n$
- `xor_sum(l,r)`：闭区间 `[l,r]` 的异或和

复杂度

$O(1)$

两个只出现奇数次的元素

若恰有两个不同元素出现奇数次，其余元素出现偶数次：

```
u64 s = 0;
for (u64 v : a) s ^= v;

u64 lb = s & -s;
u64 x = 0, y = 0;
for (u64 v : a) {
    if (v & lb) x ^= v;
    else y ^= v;
}
```

利用 `x ^ y` 任意一个为 `1` 的位把两个答案分组

复杂度

$O(n)$

成对编号

若对象严格按

```
0 ⟷ 1
2 ⟷ 3
4 ⟷ 5
```

连续成对插入，则另一对象为：

```
int rev = id ^ 1;
```

常用于网络流反向边

注意事项

只有每一对连续编号且第一对从偶数开始时才能使用

逐位统计数对贡献

设第 `k` 位为 `1` 的元素数为 `c`，总数为 `n`

运算	所有无序对在第 `k` 位为 `1` 的对数
XOR	$c(n-c)$
AND	$\binom{c}{2}$
OR	$\binom{n}{2}-\binom{n-c}{2}$

例如所有无序数对 XOR 之和：

```
i64 ans = 0;
for (int k = 0; k < 60; ++ k) {
    i64 c = 0;
    for (u64 x : a) c += x >> k & 1ULL;
    ans += c * (a.size() - c) * (1LL << k);
}
```

所有有序对：

运算	第 `k` 位为 `1` 的对数
XOR	$2c(n-c)$
AND	c^2
OR	$n^2-(n-c)^2$

注意事项

答案可能远大于元素本身范围，应先估算乘法与答案是否需要 `i128` 或取模

AND / OR 重分配

把两个数

```
x, y
```

替换为

```
x & y, x | y
```

每一位上 `1` 的总数不变，因此：

$$x+y=(x\ \&\ y)+(x\mid y)$$

这种操作可以重新分配各二进制位，但不会改变每一位出现 `1` 的总次数

拆成若干二次幂

正整数 `n` 能拆成恰好 `k` 个正的二次幂之和，当且仅当：

$$\text{popcount}(n)\le k\le n$$

二进制分解给出最少项数，每个 2^i 又可以继续拆成两个 2^{i-1}

Bitset

当位数在编译期确定且可能超过 `64` 时使用：

```
bitset<N> b;
```

常用接口：

```
b[i]
b.test(i)

b.set(i)
b.reset(i)
b.flip(i)

b.set()
b.reset()
b.flip()

b.count()
b.any()
b.none()
b.all()

b.to_string()
b.to_ullong()
```

整体位运算：

```
a & b
a | b
a ^ b
~a
a << k
a >> k
```

从字符串构造：

```
bitset<8> b("00101101");
```

字符串最右侧字符对应第 `0` 位

注意事项

- `bitset<N>` 的 `N` 必须为编译期常量
- `test(i)` 会检查越界，`operator[]` 通常用于竞赛中的直接访问
- `to_ullong()` 要求结果能够表示为 `unsigned long long`

GNU Bitset 枚举置位位置

---

libstdc++ 提供非标准扩展：

```
for (size_t i = b._Find_first(); i < N; i = b._Find_next(i)) {
    // 第 i 位为 1
}
```

注意事项

- `_Find_first()` / `_Find_next()` 不是 ISO C++ 接口，只能依赖 GNU libstdc++
- 找不到时返回 `N`

Bitset 子集和

所有数非负且总和上界可控时，0/1 子集和可写成：

```
constexpr int S = 200000;
bitset<S + 1> dp;
dp[0] = 1;
for (int x : a) dp |= dp << x;
```

处理完成后 `dp[s]` 表示是否存在子集和为 `s`

注意事项

- 每个元素最多使用一次
- 负数不能直接套用
- 左移后超过 `S` 的部分自动丢弃

复杂度

设机器字宽为 `w`，时间复杂度约为 $O(nS/w)$，空间复杂度为 $O(S/w)$

总体注意事项

- 位运算默认使用 `u64`
- 所有 `1ULL << k` 都要求 `k < 64`
- 枚举 `2^n` 个状态时通常要求 `n < 64`，且实际复杂度往往远早于位宽成为瓶颈
- `ctz/clz` 的 GNU builtin 不能传 `0`
- `mask - 1` 的无符号下溢本身有定义，但循环终止条件必须正确

2.10 选择、区间合并与小并大

第 k 小选择

只需要第 `k` 小而不需要完整排序时使用 `nth_element`
C++20：

```
ranges::nth_element(a, a.begin() + k);
auto x = a[k]; // 0base 第 k 小
```

传统写法：

```
nth_element(a.begin(), a.begin() + k, a.end());
auto x = a[k];
```

接口说明

- `nth_element(first,nth,last)`：重排 `[first,last)`，使 `*nth` 等于完整排序后该位置的元素
- `k` 为 `0base`，要求 `0 ≤ k < a.size()`

注意事项

执行后只保证：

- `a[k]` 为第 `k` 小
- `[0,k)` 中没有元素大于 `a[k]`
- `(k,n)` 中没有元素小于 `a[k]`
- 左右两部分内部均不保证有序
  若需要前 `k` 小且内部有序，可以先 `nth_element` 再只排序前半部分：

```
ranges::nth_element(a, a.begin() + k);
ranges::sort(a.begin(), a.begin() + k);
```

复杂度

平均时间复杂度 $O(n)$，额外空间由实现决定

区间合并

默认处理左闭右开区间 `[l,r)`，忽略空区间，相交或首尾相接的区间都会合并

```
template<class T>
vector<pair<T, T>> merge_seg(vector<pair<T, T>> a) {
    ranges::sort(a);
    vector<pair<T, T>> res;
    for (auto [l, r] : a) if (l < r) {
        if (res.empty() || res.back().second < l) res.push_back({l, r});
        else res.back().second = max(res.back().second, r);
    }
    return res;
}
```

接口说明

- `merge_seg(a)`：合并所有相交或首尾相接的区间
- 返回结果按左端点递增，且两两严格分离

注意事项

当前判断：

```
res.back().second < l
```

因此 `[1,3)` 与 `[3,5)` 会合并

若首尾相接也要视为分离，改为：

```
res.back().second ≤ l
```

若题目使用闭区间 `[l,r]`，需按题意重新判断“相邻”是否也应合并，不建议把不同区间语义混进同一个接口

复杂度

排序 $O(n\log n)$，扫描 $O(n)$，额外空间 $O(n)$

小并大

核心规则：始终把较小容器中的元素搬到较大容器
`map` / `unordered_map` 统计：

```
if (a.size() < b.size()) swap(a, b);
for (auto [x, v] : b) a[x] += v;
b.clear();
```

集合：

```
if (a.size() < b.size()) swap(a, b);
for (auto x : b) a.insert(x);
b.clear();
```

原理

若一个元素每次都从较小集合搬入较大集合，则搬运后它所在集合的大小至少翻倍
因此一个元素最多被搬运

$$O(\log n)$$

次

注意事项

- 小并大的关键不是容器类型，而是每次必须小集合并入大集合
- 合并后通常不再需要原来的小集合，因此可以直接 `clear()`
- 如果后续仍需分别保留两个集合，就不能直接套用这种复杂度分析
- 实际题目中的合并逻辑通常不同，因此不建议封装成统一的 `small_to_large()` 回调函数
- 若使用 `unordered_map`，复杂度依赖哈希性能；担心被卡时使用安全哈希

常见用法

树上 DFS 中每个节点维护一个颜色计数表：

```
void dfs(int u, int fa) {
    for (int v : g[u]) if (v != fa) {
        dfs(v, u);
        if (cnt[u].size() < cnt[v].size()) swap(cnt[u], cnt[v]);
        for (auto [c, x] : cnt[v]) cnt[u][c] += x;
        cnt[v].clear();
    }
    ++ cnt[u][col[u]];
}
```

这里 `cnt[u]` 始终保存当前子树合并后的结果

复杂度

若共有 n 个初始元素：

- 单次插入均摊 $O(1)$ 的哈希容器：通常 $O(n\log n)$
- `map` / `set` 单次插入 $O(\log n)$：通常 $O(n\log^2 n)$
- 还需额外计入每道题自身的统计、删除或答案维护代价

2.11 进制转换

支持 $2\sim62$ 进制整数转换，包含普通整数转进制字符串、任意长度整数的跨进制转换

- $2\sim36$ 进制：`0-9A-Z`，输入字母不区分大小写
- $37\sim62$ 进制：`0-9A-Za-z`，输入字母区分大小写
- 不依赖高精度整数类，支持负数、前导零和任意长度整数

```cpp
// ===== 字符映射 =====
int digit_value(char c, int base) {
    int x = -1;
    if ('0' ≤ c && c ≤ '9') x = c - '0';
    else if ('A' ≤ c && c ≤ 'Z') x = c - 'A' + 10;
    else if ('a' ≤ c && c ≤ 'z') x = c - 'a' + (base ≤ 36 ? 10 : 36);
    return x < base ? x : -1;
}
char digit_char(int x) {
    if (x < 10) return char('0' + x);
    if (x < 36) return char('A' + x - 10);
    return char('a' + x - 36);
}
// ===== 普通整数转进制 =====
string to_base(i64 x, int base) {
    bool neg = x < 0;
    u64 y = neg ? u64(-(x + 1)) + 1 : u64(x);
    string s;
    do {
        s.push_back(digit_char(y % base));
        y /= base;
    } while (y);
    if (neg) s.push_back('-');
    reverse(s.begin(), s.end());
    return s;
}
// ===== 任意长度进制转换 =====
string convert_base(const string &s, int from_base, int to_base) {
    bool neg = s[0] == '-';
    int p = (s[0] == '-' || s[0] == '+');
    while (p < int(s.size()) && s[p] == '0') ++ p;
    if (p == int(s.size())) return "0";
    vector<int> a;
    a.reserve(s.size() - p);
    for (int i = p; i < int(s.size()); ++ i) {
        a.push_back(digit_value(s[i], from_base));
    }
    if (from_base == to_base) {
        string t;
        if (neg) t += '-';
        for (int x : a) t += digit_char(x);
        return t;
    }
    vector<int> q;
    q.reserve(a.size());
    string out;
    while (!a.empty()) {
        q.clear();
        int rem = 0;
        for (int x : a) {
            int cur = rem * from_base + x;
            int d = cur / to_base;
            rem = cur % to_base;
            if (d || !q.empty()) q.push_back(d);
        }
        out += digit_char(rem);
        a.swap(q);
    }
    if (neg) out += '-';
    reverse(out.begin(), out.end());
    return out;
}
```

`to_base()` 可以单独使用，但需要 `digit_char()`；`convert_base()` 需要两个字符映射函数，不依赖 `to_base()`

接口

- `digit_value(c, base)`：返回字符 `c` 在指定进制下的数值，不合法时返回 `-1`，$O(1)$
- `digit_char(x)`：返回数字 `x` 对应的字符，要求 $0 \le x$ 二进制字符串

```
string s = bitset<8>(42).to_string(); // "00101010"
// 二进制字符串 → 整数 u64 x = bitset<64>("101010").to_ullong(); // 42
// 直接输出固定位宽二进制 cout << bitset<32>(42) << '\n';
```

`bitset<N>` 的宽度 `N` 必须是编译期常量。输入位数不足 `N` 时高位补零；整数转换超过指定宽度时保留低 `N` 位。`to_ullong()` 的结果若无法放入 `unsigned long long`，会抛出 `overflow_error`
若需要去除前导零，直接使用板子中的 `to_base(x, 2)` 更方便

使用 `stoi` / `stoll` 转换字符串

适用于字符串表示的 $2 \sim 36$ 进制整数转换为十进制整数

```
int a = stoi("101010", nullptr, 2); // 42
int b = stoi("FF", nullptr, 16);     // 255
i64 c = stoll("777", nullptr, 8);    // 511
u64 d = stoull("FFFFFFFFFFFFFFFF", nullptr, 16);
// 自动识别十进制、0 开头的八进制和 0x 开头的十六进制
int x = stoi("0xFF", nullptr, 0); // 255
```

`stoi` 返回 `int`，`stoll` 返回 `i64`，`stoull` 返回 `u64`。遇到非法输入或数值超出范围时会抛出异常；如果只转换了合法前缀，默认不会报错，可使用第二个参数取得已解析的字符数

使用 `stringstream` 转换八、十、十六进制

```
// 整数 → 十六进制字符串
ostringstream os;
os << uppercase << hex << 255;
string s = os.str(); // "FF"

// 十六进制字符串 → 整数
istringstream is("FF");
int x = 0;
is >> hex >> x; // 255

// 直接按八进制、十六进制输出
cout << oct << 255 << '\n'; // 377
cout << hex << 255 << '\n'; // ff
cout << dec << 255 << '\n'; // 255
```

- `dec` / `oct` / `hex`：切换十、八、十六进制
- `uppercase`：十六进制字母使用大写
- `showbase`：输出进制前缀，例如十六进制的 `0x`
- 操纵符会影响流的后续输入输出，使用后需要时应切回 `dec`

使用 `from_chars` / `to_chars`

适用于整数与 $2 \sim 36$ 进制字符串之间的转换。无需构造字符串流，也不通过异常报告解析失败
整数转字符串

```
char buf[70];
auto [p, ec] = to_chars(buf, buf + 70, 255, 16);
string s(buf, p); // "ff", 要求 ec == errc{}
```

字符串转整数

```
string s = "FF";
i64 x = 0;
auto [p, ec] = from_chars(s.data(), s.data() + s.size(), x, 16);
bool ok = (ec == errc{} && p == s.data() + s.size());
// ok 为 true, x 为 255
```

⚠ `from_chars()` 不跳过前导空白、不接受前置 `+`，指定 16 进制时也不接受 `0x` 前缀。必须同时检查 `ec` 和 `p`，才能确认整个字符串合法

`to_chars()` 不附加字符串终止符 `'\0'`，默认输出小写字母，缓冲区不足时通过错误码报告

任意长度整数的进制转换

```
to_base(255, 16);               // "FF"
to_base(-255, 2);                // "-11111111"
convert_base("FF", 16, 2);      // "11111111"
convert_base("-255", 10, 16);   // "-FF"
convert_base("000FF", 16, 16);  // "FF"
convert_base("zz", 62, 10);     // "3843"
```

输入数字超过 `i64` 范围时，仍然可以使用 `convert_base()`，无需先转换为整数

二进制与八、十六进制快速互转

对于 2^k 进制，可以直接按每 k 个二进制位分组。八进制每组 3 位，十六进制每组 4 位，无需使用任意长度进制转换算法
十六进制字符串转二进制字符串

```
string s = "00AF", bin;
for (char c : s) {
    bin += bitset<4>(digit_value(c, 16)).to_string();
}
// bin = "0000000010101111"
```

二进制字符串转十六进制字符串

```
string bin = "10101111", hex;
bin.insert(0, (4 - bin.size() % 4) % 4, '0');
for (int i = 0; i < int(bin.size()); i += 4) {
    hex += digit_char(bitset<4>(bin.substr(i, 4)).to_ulong());
}
// hex = "AF"
```

上述写法会按分组保留前导零；需要规范化整数表示时，再删除前导零即可
将每组 4 位改为 3 位、`bitset<4>` 改为 `bitset<3>`，即可处理二进制与八进制的转换

复杂度

设 L 为输入数字位数，M 为输出数字位数，B 为目标进制，N 为 `bitset` 宽度

操作	时间复杂度	额外空间复杂度
`digit_value` / `digit_char`	$O(1)$	$O(1)$
`to_base`	$O(\log_B(\|x\|+1)+1)$	$O(M)$
`convert_base`	$O(LM)$，即 $O(L^2)$	$O(L+M)$
`convert_base`（进制相同）	$O(L)$	$O(L)$
`stoi` / `stoll` / `from_chars`	$O(L)$	$O(1)$（解析本身）
`to_chars`	$O(M)$	$O(1)$（不计调用方缓冲区）
`bitset<N>::to_string()`	$O(N)$	$O(N)$
二进制分组转换	$O(L)$	$O(L)$

2.12 表达式解析与表达式树

支持十进制非负整数字面量、变量、括号，以及 `+ - * / % ^` 和一元正负号

- 中缀转后缀：支持语法检查、运算符优先级和结合性
- 后缀求值：支持变量赋值，检查除零、负指数和 `i64` 溢出
- 表达式树：通过后缀表达式建树，支持一元与二元运算符，以及恢复完全括号化的中缀表达式

```
struct Expr {
    // ===== 基础辅助函数：被 postfix / eval / ExprTree 依赖 =====
    static bool op(const string &s) { return s == "+" || s == "-" || s == "*" || s == "/" || s == "%" || s == "^" || s == "u+"
|| s == "u-"; }
    static bool unary(const string &s) { return s == "u+" || s == "u-"; }
    static int pri(const string &s) {
        if (s == "^") return 4;
        if (unary(s)) return 3;
        if (s == "*" || s == "/" || s == "%") return 2;
        return 1;
    }
    static bool right(const string &s) { return s == "^" || unary(s); }

    // ===== 中缀转后缀：若只处理现成的后缀表达式，可删除本函数及 pri / right =====
    static vector<string> postfix(const string &s) {
        vector<string> out, st;
        bool need = true;
        for (int i = 0, n = s.size(); i < n; ) {
            unsigned char c = s[i];
            if (isspace(c)) { ++ i; continue; }
            if (isdigit(c) || isalpha(c) || c == '_') {
                if (!need) return {};
                int j = i;
                if (isdigit(c)) {
                    while (j < n && isdigit((unsigned char)s[j])) ++ j;
                } else {
                    while (j < n && (isalnum((unsigned char)s[j]) || s[j] == '_')) ++ j;
                }
                out.push_back(s.substr(i, j - i));
                i = j;
                need = false;
            } else if (c == '(') {
                if (!need) return {};
                st.push_back("(");
                ++ i;
            } else if (c == ')') {
                if (need) return {};
                while (!st.empty() && st.back() != "(") {
                    out.push_back(st.back());
                    st.pop_back();
                }
                if (st.empty()) return {};
                st.pop_back();
                ++ i;
                need = false;
            } else {
                string t(1, c);
                if (need && (c == '+' || c == '-')) {
                    st.push_back("u" + t); // 一元运算符直接入栈，允许 2^-3
                    ++ i;
                    continue;
                }
                if (need || !op(t)) return {};
                while (!st.empty() && st.back() != "(") {
                    int a = pri(t), b = pri(st.back());
                    if (!(b > a || (b == a && !right(t)))) break;
                    out.push_back(st.back());
                    st.pop_back();
                }
                st.push_back(t);
                ++ i;
                need = true;
            }
        }
        if (need) return {};
        while (!st.empty()) {
            if (st.back() == "(") return {};
            out.push_back(st.back());
            st.pop_back();
        }
```

```
        return out;
    }
    // ===== 后缀求值 =====
    static optional<i64> eval(const vector<string> &p, const unordered_map<string, i64> &val = {}) {
        vector<i64> st;
        auto fit = [](i128 x) { return x >= numeric_limits<i64>::min() && x <= numeric_limits<i64>::max(); };
        for (const string &s : p) {
            if (op(s)) {
                if (st.size() < (unary(s) ? 1U : 2U)) return nullopt;
                i128 x = st.back(); st.pop_back();
                if (unary(s)) {
                    if (s == "u-") x = -x;
                } else {
                    i128 b = x, a = st.back(); st.pop_back();
                    if (s == "+") x = a + b;
                    else if (s == "-") x = a - b;
                    else if (s == "*") x = a * b;
                    else if (s == "/" || s == "%") {
                        if (!b) return nullopt;
                        x = s == "/" ? a / b : a % b;
                    } else {
                        if (b < 0) return nullopt;
                        x = 1;
                        for (i64 e = (i64)b; e; e >>= 1) {
                            if (e & 1) {
                                x *= a;
                                if (!fit(x)) return nullopt;
                            }
                            if (e > 1) {
                                a *= a;
                                if (!fit(a)) return nullopt;
                            }
                        }
                    }
                }
                if (!fit(x)) return nullopt;
                st.push_back((i64)x);
            } else {
                i64 x;
                if (!s.empty() && isdigit((unsigned char)s[0])) {
                    auto [p, ec] = from_chars(s.data(), s.data() + s.size(), x);
                    if (ec != errc{} || p != s.data() + s.size()) return nullopt;
                } else {
                    auto it = val.find(s);
                    if (it == val.end()) return nullopt;
                    x = it->second;
                }
                st.push_back(x);
            }
        }
        if (st.size() != 1) return nullopt;
        return st.back();
    }
};
// ===== 表达式树 =====
struct ExprTree {
    struct Node {
        string s;
        int l{ -1 }, r{ -1 };
    };
    vector<Node> tr;
    int root{ -1 };
    bool build(const vector<string> &p) {
        tr.clear();
        root = -1;
        vector<int> st;
        for (const string &s : p) {
            int l = -1, r = -1;
            if (Expr::op(s)) {
                if (st.size() < (Expr::unary(s) ? 1U : 2U)) return false;
                r = st.back(); st.pop_back();
```

```
                if (Expr::unary(s)) swap(l, r);
                else { l = st.back(); st.pop_back(); }
            }
            st.push_back(tr.size());
            tr.push_back({s, l, r});
        }
        if (st.size() != 1) { tr.clear(); return false; }
        root = st.back();
        return true;
    }
    // ===== 恢复中缀表达式：以下函数可直接删除 =====
    void dump(int u, string &out) const {
        const auto &[s, l, r] = tr[u];
        if (l == -1) { out += s; return; }
        out += '(';
        if (r == -1) out += s[1];
        dump(l, out);
        if (r != -1) { out += s; dump(r, out); }
        out += ')';
    }
    string str() const {
        string out;
        if (root != -1) dump(root, out);
        return out;
    }
};
```

Expr 表达式解析与求值

- `Expr::op(s)`：判断 `s` 是否为支持的运算符，$O(1)$
- `Expr::unary(s)`：判断是否为一元正负号，$O(1)$
- `Expr::pri(s)`：返回合法运算符的优先级，$O(1)$
- `Expr::right(s)`：判断运算符是否右结合，$O(1)$
- `Expr::postfix(s)`：中缀表达式转后缀表达式，返回 `vector<string>`，语法不合法时返回空数组，$O(n)$
- `Expr::eval(p, val)`：求后缀表达式的整数值，返回 `optional<i64>`，失败时返回 `nullopt`，变量表 `val` 可省略

ExprTree 表达式树

- `ExprTree::build(p)`：根据后缀表达式建树，成功返回 `true`，失败返回 `false`，$O(n)$
- `ExprTree::root`：根节点编号，未成功建树时为 `-1`
- `ExprTree::tr[u].s`：节点保存的数字、变量或运算符
- `ExprTree::tr[u].l / r`：左右儿子，`-1` 表示不存在
- `ExprTree::str()`：恢复完全括号化的中缀表达式，$O(n)$
- `ExprTree::dump(u, out)`：将节点 `u` 对应子树的中缀表达式追加到 `out`，$O(k)$，其中 k 为输出长度

  节点编号为 `0base`，一元运算符只使用左儿子

注意事项

运算符优先级

优先级	运算符	结合性
4（最高）	`^`	右结合
3	`u+`、`u-`	右结合
2	`*`、`/`、`%`	左结合
1（最低）	`+`、`-`	左结合

- `u+`、`u-` 是内部表示，输入中仍写普通的 `+`、`-`
- `-2^2` 解析为 `-(2^2)`，结果为 `-4`
- `(-2)^2` 解析为 `(-2)^2`，结果为 `4`
- `2^3^2` 解析为 `2^(3^2)`，结果为 `512`
- `2^-3` 可以正确解析，但整数求值不支持负指数
- `0^0` 按整数快速幂约定返回 `1`

语法和数值范围

- 数字仅支持非负十进制整数字面量，变量名支持 `[A-Za-z_][A-Za-z0-9_]*`
- 不支持小数、隐式乘法、函数调用、赋值以及 `&&`、`||` 等其他运算符
- `postfix()` 只负责语法检查，不检查数值范围、变量是否存在以及除零

- `eval()` 对所有运算检查 `i64` 范围，不允许任何中间结果越界
- 单个数字字面量不能超过 `LLONG_MAX`，因此不能直接写 `-9223372036854775808`，但可以通过 `(-2)^63` 得到 `LLONG_MIN`
- `/` 和 `%` 遵循 C++ 有符号整数的截断除法规则
- `ExprTree::build()` 需要传入有效的后缀表达式，失败后不可访问节点
- `str()` 使用递归，极深的表达式树可能导致栈溢出

> ⊗ 如果题目使用浮点数、模运算或布尔表达式，可以保留 `postfix()` 和 `ExprTree`，只修改词法分析和求值规则，不必重写建树算法

常见用法

中缀表达式直接求值

```
auto p = Expr::postfix("-(12 + 3) * 4");
auto ans = Expr::eval(p);
if (ans) cout << *ans << '\n'; // -60
```

输出后缀表达式

```
auto p = Expr::postfix("a + b * c");
for (const string &s : p) cout << s << ' ';
// a b c * +
```

带变量的表达式求值

```
unordered_map<string, i64> val{
    {"a", 2}, {"b", 10}
};
auto ans = Expr::eval(Expr::postfix("a * (b - 3)"), val);
// ans = 14
```

构建表达式树

```
auto p = Expr::postfix("(a+b)*(c-d)");
ExprTree t;
if (t.build(p)) {
    cout << t.root << '\n';
    cout << t.str() << '\n'; // ((a+b)*(c-d))
}
```

输出前缀表达式

建树后先序遍历即可得到前缀表达式，不需要额外实现转换算法

```
vector<string> pre;
vector<int> st{t.root};
while (!st.empty()) {
    int u = st.back();
    st.pop_back();
    pre.push_back(t.tr[u].s);
    if (t.tr[u].r != -1) st.push_back(t.tr[u].r);
    if (t.tr[u].l != -1) st.push_back(t.tr[u].l);
}
// (a+b)*(c-d) -> * + a b - c d
```

复杂度

设 n 为表达式字符总数，k 为后缀表达式的 Token 数，h 为表达式树高度

操作	时间复杂度	空间复杂度
`postfix()`	$O(n)$	$O(n)$
`eval()`	$O(n+\sum \log(e+1))$	$O(k)$
`build()`	$O(n)$	$O(n)$
`str()`	$O(n)$	$O(n+h)$

其中 e 为各次乘方运算的非负指数，`eval()` 假设变量哈希查询均摊常数时间，且计算使用固定宽度整数，因此整体也可视为 $O(n)$

2.13 日期与星期计算

采用公历（Gregorian calendar），支持公元 $1 \sim 9999$ 年

- 日期与整数天数互转，`0001-01-01` 对应第 `0` 天
- 判断闰年、月份天数、日期合法性
- 日期差、日期加减天数、年内第几天
- 星期一为 `1`，星期日为 `7`

```
struct Date {
    int y{}, m{}, d{};
    // ===== 基础日期函数 =====
    static bool leap(int y) { return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0); }
    static int mdays(int y, int m) {
        static constexpr int a[13]{0,31,28,31,30,31,30,31,31,30,31,30,31};
        return a[m] + (m == 2 && leap(y));
    }
    bool valid() const { return 1 <= y && y <= 9999 && 1 <= m && m <= 12 && 1 <= d && d <= mdays(y, m); }
    // ===== 日期与天数互转 =====
    i64 days() const {
        int yy = y - (m <= 2);
        int era = yy / 400, yo = yy % 400;
        int mm = m > 2 ? m - 3 : m + 9;
        return 1LL * era * 146097 + 365LL * yo + yo / 4 - yo / 100 + (153 * mm + 2) / 5 + d - 307;
    }
    static Date from_days(i64 z) {
        z += 306;
        i64 era = z / 146097;
        int doe = z % 146097;
        int yo = (doe - doe / 1460 + doe / 36524 - doe / 146096) / 365;
        int y = int(era * 400 + yo);
        int doy = doe - (365 * yo + yo / 4 - yo / 100);
        int mp = (5 * doy + 2) / 153;
        int d = doy - (153 * mp + 2) / 5 + 1;
        int m = mp < 10 ? mp + 3 : mp - 9;
        y += m <= 2;
        return {y, m, d};
    }
    // ===== 扩展功能 =====
    int wday() const { return int(days() % 7) + 1; }
    int yday() const { return int(days() - Date{y, 1, 1}.days()) + 1; }
    optional<Date> add(i64 k) const {
        i64 z = days(), mx = Date{9999, 12, 31}.days();
        if (k < -z || k > mx - z) return nullopt;
        return from_days(z + k);
    }
};
```

独立星期公式（Kim Larsen）

只需计算星期时，可以直接使用以下函数，不依赖 `Date`

```
int wday(int y, int m, int d) {
    if (m <= 2) m += 12, --y;
    return (d + 2 * m + 3 * (m + 1) / 5 + y + y / 4 - y / 100 + y / 400) % 7 + 1;
}
```

成员变量

- `y`：年份，范围 $[1, 9999]$
- `m`：月份，范围 $[1, 12]$
- `d`：当月日期，从 `1` 开始

接口

- `Date{y,m,d}`：创建日期对象，$O(1)$
- `Date::leap(y)`：判断是否为闰年，$O(1)$
- `Date::mdays(y,m)`：返回当月天数，要求月份合法，$O(1)$

- `valid()`：判断是否为合法日期，$O(1)$
- `days()`：日期转整数天数，$O(1)$
- `Date::from_days(z)`：整数天数转日期，$O(1)$
- `wday()`：返回星期编号，星期一为 `1`，星期日为 `7`，$O(1)$
- `yday()`：返回年内第几天，范围 $[1,365]$ 或 $[1,366]$，$O(1)$
- `add(k)`：日期增加 `k` 天，`k` 可为负数，超出支持范围返回 `nullopt`，$O(1)$

注意事项

- 使用前推公历，即将现代公历的闰年规则延伸至公元 1582 年之前，不处理历史上的儒略历与历法切换
- 所有日期计算均不涉及时区、夏令时和闰秒
- `days()`、`wday()`、`yday()`、`add()` 均要求当前日期合法，`from_days(z)` 要求 $0 \le z \le 3652058$
- `mdays(y,m)` 要求月份在 $[1,12]$ 内，不能直接传入非法月份
- `0001-01-01` 为星期一，`1970-01-01` 为星期四
- `days()` 返回的天数从 `0001-01-01` 开始计算，与 Unix 时间戳的纪元不同
- 日期相减按实际经过天数计算，不包含起点当天；若题目要求首尾日期均计入，需要额外加 `1`
- `from_days()` 不负责越界检查，带偏移的日期计算优先调用 `add()`

常见用法

闰年、日期合法性和月份天数

```
Date a{2024, 2, 29};
bool ok = a.valid();       // true
bool lp = Date::leap(2024); // true
int cnt = Date::mdays(2024, 2); // 29
```

计算两个日期相差的天数

```
Date a{2024, 2, 28}, b{2024, 3, 1};
i64 diff = b.days() - a.days(); // 2

// 计算绝对天数差
i64 dis = abs(diff);
```

日期增加或减少指定天数

```
Date a{2024, 2, 29};
auto b = a.add(10); // 2024-03-10
auto c = a.add(-30); // 2024-01-30

if (b) cout << b->y << ' ' << b->m << ' ' << b->d << '\n';
```

计算星期、年内第几天

```
Date a{2024, 2, 29};
int w = a.wday(); // 4，星期四
int d = a.yday(); // 60

// 只需要星期时直接使用独立公式
int x = wday(2024, 2, 29); // 4
```

日期与整数天数转换

```
Date a{1970, 1, 1};
i64 z = a.days(); // 719162

Date b = Date::from_days(z);
// b = {1970, 1, 1}
```

日期字符串的输入输出

```
string s = "2024-02-29";
Date a;
sscanf(s.c_str(), "%d-%d-%d", &a.y, &a.m, &a.d);

// 格式化为 YYYY-MM-DD
```

```
ostringstream os;
os << setfill('0') << setw(4) << a.y << '-'
   << setw(2) << a.m << '-' << setw(2) << a.d;
string t = os.str(); // "2024-02-29"
```

输入格式需要自行保证合法，读取完成后可以使用 `valid()` 检查

C++20 `chrono` 标准库写法

若比赛使用 C++20，可以直接通过标准库完成日期合法性、日期差和日期加减

```
using namespace chrono;

// 创建日期、检查合法性
year_month_day a{year{2024}, month{2}, day{29}};
bool ok = a.ok(); // true

// 闰年、月份最后一天
bool lp = year{2024}.is_leap(); // true
auto last_day = year{2024}/February/last;
int cnt = unsigned(last_day.day()); // 29

// 日期加减、日期差
sys_days z{a};
year_month_day b{z + days{10}}; // 2024-03-10
i64 diff = (sys_days{b} - z).count(); // 10

// 星期：1~7 对应星期一至星期日
int w = weekday{z}.iso_encoding(); // 4

// 年内第几天
int yd = (z - sys_days{year{2024}/January/day{1}}).count() + 1; // 60

// 2024 年 10 月的第二个星期二
auto t = year{2024}/October/Tuesday[2];
year_month_day d{sys_days{t}}; // 2024-10-08
```

`sys_days` 表示精度为天的时间点，可直接相减得到天数差
需要注意：`year_month_day + months` 是日历月份运算，例如 1 月 31 日加一个月可能得到非法的 2 月 31 日，应当检查 `.ok()`；按实际天数加减则使用 `sys_days + days`

复杂度

操作	时间复杂度	空间复杂度
闰年判断、月份天数、合法性	$O(1)$	$O(1)$
`days()` / `from_days()`	$O(1)$	$O(1)$
`wday()` / `yday()`	$O(1)$	$O(1)$
`add()`	$O(1)$	$O(1)$
独立星期公式	$O(1)$	$O(1)$

2.14 时间线段树分治

将对象的有效时间区间挂到时间线段树上，DFS 时加入对象、离开节点时回滚，适用于离线动态维护。

```
template<class Item>
struct TimeSegmentTree {
    int q, n{1};
    vector<vector<Item>> seg;
    explicit TimeSegmentTree(int q_) : q(q_) {
        while (n < q) n <<= 1;
        seg.resize(2 * n);
    }
    // ===== 有效区间挂载：O(log q) =====
    void add(int l, int r, const Item &item) {
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) seg[l ++].push_back(item);
```

```
            if (r & 1) seg[-- r].push_back(item);
        }
    }
    // ===== DFS 与回滚：被 run() 依赖 =====
    template<class State, class Answer>
    void dfs(int p, int l, int r, State &state, Answer &answer) const {
        if (l >= q) return;
        auto snap = state.snap();
        for (const Item &item : seg[p]) state.apply(item);
        if (r - l == 1) {
            answer(l, as_const(state));
        } else {
            int m = (l + r) / 2;
            dfs(2 * p, l, m, state, answer);
            dfs(2 * p + 1, m, r, state, answer);
        }
        state.rollback(snap);
    }
    template<class State, class Answer>
    void run(State &state, Answer &&answer) const {
        if (q) dfs(1, 0, n, state, answer);
    }
};
// ===== 可撤销异或线性基 =====
template<int B = 64>
struct RollbackBasis {
    static_assert(1 <= B && B <= 64);
    array<u64, B> p{};
    array<int, B> his{};
    int sz{};
    int snap() const { return sz; }
    void apply(u64 x) {
        if constexpr (B < 64) assert((x >> B) == 0);
        for (int b = B - 1; b >= 0; -- b) {
            if (!(x >> b & 1ULL)) continue;
            if (p[b]) x ^= p[b];
            else {
                p[b] = x;
                his[sz ++] = b;
                return;
            }
        }
    }
    void rollback(int t) {
        while (sz > t) p[his[-- sz]] = 0;
    }
    u64 maximum(u64 x = 0) const {
        for (int b = B - 1; b >= 0; -- b) x = max(x, x ^ p[b]);
        return x;
    }
    // ===== 扩展查询：不需要秩/可表示性时，可删除以下函数 =====
    int rank() const { return sz; }
    bool contains(u64 x) const {
        if constexpr (B < 64) if (x >> B) return false;
        for (int b = B - 1; b >= 0; -- b) {
            if ((x >> b & 1ULL) && p[b]) x ^= p[b];
        }
        return x == 0;
    }
};
```

接口说明

`TimeSegmentTree<Item>`

- `TimeSegmentTree(q)`：建立 `q` 个时间点 `0, 1, ..., q-1`；内部叶子数 `n` 为不小于 `max(1, q)` 的最小二次幂。初始化时间、空间均为 $O(q+1)$。
- `add(l, r, item)`：令对象 `item` 在 `[l, r)` 内生效，允许空区间；时间 $O(log(q+1))$，新增存储 $O(log(q+1))$。
- `run(state, answer)`：按时间顺序访问叶子，调用 `answer(t, const State&)`；若 `snap/apply/rollback` 符合约定，结束后 `state` 恢复到运行前。`q = 0` 时不调用回调。
- `dfs(...)`：`run()` 使用的内部递归函数，无需直接调用。
  `State` 的必需接口：

```
auto snap() const;                // 记录当前状态
void apply(const Item &item);     // 加入一个有效对象
void rollback(Snapshot snapshot); // 恢复到指定快照
```

`Snapshot` 可以是任意能正确保存和传递的快照类型，不一定是整数。`answer` 应为**只读查询**。

`RollbackBasis<B>`

- `B`：整数位数，`1 ≤ B ≤ 64`；`B < 64` 时插入值必须小于 2^B。
- `snap()`：返回当前历史记录长度，$O(1)$。
- `apply(x)`：向异或线性基插入 `x`，$O(B)$；线性相关时不产生状态变化。
- `rollback(t)`：撤销到历史记录长度 `t`，每撤销一个主元 $O(1)$。
- `maximum(x = 0)`：计算 `x` 异或当前有效值的某个子集后可得到的最大无符号整数，$O(B)$。
- `rank()`：当前向量空间的维数，而非有效对象个数，$O(1)$。
- `contains(x)`：判断 `x` 能否由当前有效值异或得到，$O(B)$。

注意事项

- **离线算法**：所有生效区间应在 `run()` 前确定。原操作时间点按 `0base` 编号，区间统一为 `[l, r)`，要求 `0 ≤ l ≤ r ≤ q`。
- **操作顺序**：对象会按线段树节点的 DFS 路径顺序加入，**不保证按原始时间或输入顺序加入**。要求结果与有效对象的加入顺序无关；并非任意可撤销操作都能直接套用。
- **回滚完整性**：`apply()` 改写的全部状态都必须能由 `rollback()` 恢复；不能使用未记录修改的路径压缩、缓存、懒标记更新等。回滚 DSU 通常不使用路径压缩。
- **查询只读**：叶子回调接收 `const State&`，不允许在查询过程中破坏当前状态。对存在内部修改的查询，需要另外设计可回滚的实现。
- **时间端点**：若操作 `t` 先修改、再观察状态，则第 `t` 次插入通常从 `t` 开始有效，第 `t` 次删除通常从 `t` 开始失效。实际端点须根据题目查询发生的时刻确定。
- **重复对象**：同一个值可以出现多次，应按对象 ID 或每次插入分别配对删除，不可随意用数值唯一标识生命周期。
- **线性基限制**：`RollbackBasis` 只记录新出现的主元；依赖插入不修改 `p`，故无需入栈。`rank()` 不是当前元素数量；`maximum()` 的选择允许空子集。
- **不适用情形**：普通 LCT 的 `access/splay/link/cut` 不自带完整回滚能力，不能直接放入 `State`；普通可持久化结构/CDQ 也不是本框架的直接替代实现。
- **多测**：每组数据重新构造时间树与状态；通常一次 `run()` 即可完成全部查询。若 `Item` 很大，可存对象 ID 并在 `State::apply(id)` 中读取原对象，减少区间挂载的复制开销。

常见用法

1. 有效时间区间内的最大子集异或

```
TimeSegmentTree<u64> time(q);
time.add(l, r, value);

RollbackBasis<> basis;
vector<u64> ans(q);
time.run(basis, [&](int t, const auto &cur) {
    ans[t] = cur.maximum();
});
```

2. 将插入、删除操作变成有效区间

以下假设 `op[t]` 为 `'+'`、`'-'` 或 `'?'`，每个对象有唯一 ID `id[t]`，并且 `value[id]` 固定；每个 ID 不会重复插入而不先删除。

```
TimeSegmentTree<u64> time(q);
vector<int> start(m, -1);
for (int t = 0; t < q; ++ t) {
    if (op[t] == '+') {
        start[id[t]] = t;
    } else if (op[t] == '-') {
        int x = id[t];
        time.add(start[x], t, value[x]);
        start[x] = -1;
    }
}
for (int x = 0; x < m; ++ x) {
    if (start[x] != -1) time.add(start[x], q, value[x]);
}

RollbackBasis<> basis;
time.run(basis, [&](int t, const auto &cur) {
    if (op[t] == '?') cout << cur.maximum() << '\n';
});
```

3. 自定义 `Item` 类型

```
struct ActiveVector { u64 value; int id; };
struct ActiveBasis : RollbackBasis<> {
    void apply(const ActiveVector &x) { RollbackBasis<>::apply(x.value); }
};

TimeSegmentTree<ActiveVector> time(q);
time.add(l, r, {value, id});
ActiveBasis state;
time.run(state, [&](int t, const auto &cur) {
    ans[t] = cur.maximum();
});
```

复杂度

令 `m` 为有效区间个数，`q` 为时间点个数，`A` 为一次 `apply()` 的最坏时间，`R` 为撤销一次 `apply()` 所引起状态变化的摊还时间，`Q` 为一次叶子查询时间，`S` 为 `State` 的空间开销。

操作	时间复杂度	额外空间
构造时间树	$O(q+1)$	$O(q+1)$
挂载一个区间	$O(log(q+1))$	$O(log(q+1))$
全部挂载	$O(mlog(q+1))$	$O(mlog(q+1))$
DFS 回滚与查询	$O(q+mlog(q+1)\cdot(A+R)+qQ)$	$O(S+log(q+1))$（不计已存储的区间对象与答案）

总空间 $O(q+mlog(q+1)+S)$，不计答案和对象自身额外持有的存储

`RollbackBasis<B>` 的插入与最大异或查询均为 $O(B)$，一次主元撤销 $O(1)$，状态空间 $O(B)$。若每个时间点都计算最大异或，总时间为 $O((mlog(q+1)+q)B+q)$，总空间为 $O(q+mlog(q+1)+B)$。

2.15 贪心定式与证明

> 所有结论均须同时满足其模型约束；以下默认整数运算不会溢出。复杂度中的 n 表示输入元素个数。

2.15.1 Huffman 与最优合并

二叉 / k 叉最优合并

问题。 给定 n 个非负权值 $w_1,\dots,w_n$。每次将至多 k 个当前对象（$k\ge 2$）合并成一个新对象，新权值等于这些对象的权值之和，本次操作的代价也等于这个和。要求最终合并成一个对象，**最小化总代价**。普通合并果子对应 $k=2$；最优前缀码对应构造最小加权路径长度的树。

贪心方法。 每次取当前权值最小的 k 个对象进行合并，把新权值重新加入小根堆。对于 $n>1$ 的 k 叉版本，为统一每步取恰好 k 个，先补充权值为 0 的虚叶，使 $(n-1)\bmod(k-1)=0$。当 $k>n$ 时，可将 k 限制为 n。

该模型等价于最小化叶子的加权深度和：

$$\text{cost}=\sum_{u\ \text{为原叶}} w_u\ \text{depth}(u).$$

简证。 在某棵最优满 k 叉树中，最小的 k 个权值可以安排在最深层的一组同父兄弟上：若小权叶比大权叶浅，交换二者不会增大代价。将这 k 个兄弟收缩成权值为其和的新叶，其他叶深度不变，问题缩小，归纳成立。虚叶权值为零，补满树不会改变原叶的代价。

注意事项。 $n\le 1$ 时总代价为 0；所有权值都要求非负。若题目规定每步**必须恰好合并 k 个原有对象且不允许补零**，应先检查可行性，不能把虚叶视为真实对象。若要求只能合并原序列中**相邻**对象，则不属于普通 Huffman 问题。

仅计算最优合并代价

```
// 传入非负权值；k ≥ 2。无需构造树时只复制本函数。
i64 huffman_cost(vector<i64> a, int k = 2) {
    int n = a.size();
    if (n <= 1) return 0;
    assert(k >= 2);
    k = min(k, n);
    int pad = (k - 1 - (n - 1) % (k - 1)) % (k - 1);
    priority_queue<i64, vector<i64>, greater<>> pq;
    for (i64 x : a) pq.push(x);
    while (pad --) pq.push(0);

    i64 ans = 0;
    while (pq.size() > 1) {
        i64 sum = 0;
```

```
        for (int j = 0; j < k; ++ j) {
            sum += pq.top();
            pq.pop();
        }
        ans += sum;
        pq.push(sum);
    }
    return ans;
}
```

需要构造 Huffman 树

```
struct Huffman {
    vector<i64> w;
    vector<vector<int>> ch;
    vector<int> fa;
    int root = -1, leaves = 0;
    i64 cost = 0;

    explicit Huffman(const vector<i64> &a, int k = 2) : leaves(a.size()) {
        assert(k >= 2);
        int n = a.size();
        if (!n) return;
        k = min(k, n);
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> pq;
        auto add = [&](i64 x) {
            int id = w.size();
            w.push_back(x);
            ch.emplace_back();
            fa.push_back(-1);
            pq.push({x, id});
        };
        for (i64 x : a) add(x);
        if (n > 1) {
            int pad = (k - 1 - (n - 1) % (k - 1)) % (k - 1);
            while (pad --) add(0);
        }
        while (pq.size() > 1) {
            vector<int> son;
            i64 sum = 0;
            for (int j = 0; j < k; ++ j) {
                auto [x, id] = pq.top();
                pq.pop();
                sum += x;
                son.push_back(id);
            }
            int u = w.size();
            w.push_back(sum);
            ch.push_back(move(son));
            fa.push_back(-1);
            for (int v : ch[u]) fa[v] = u;
            cost += sum;
            pq.push({sum, u});
        }
        root = pq.top().second;
    }
};
```

接口说明。 仅求代价时调用 `huffman_cost(a, k)`；需要编码或输出合并方案时用 `Huffman(a, k)`，其中 `cost` 为总代价，`root` 为根，`ch[u]`、`fa[u]` 分别为孩子和父亲，原始叶子编号为 `[0, leaves)`，新添虚叶与内部节点紧随其后。

复杂度。 两种实现均为 $O(n \log n)$ 时间、$O(n)$ 空间（这里 k 已限制为不超过 n）。

2.15.2 区间贪心

最多选择多少个互不相交区间

问题。 给定 n 个非空半开区间 $[l_i, r_i)$，选择尽可能多的区间，使选出的区间两两不相交。端点相接视为不相交。

贪心方法。 按右端点升序排列，每次选择第一个满足 $l \geq \text{last_r}$ 的区间，并更新最后一个所选区间的右端点。

简证（交换）。 取任意最优方案的第一个区间，将其替换成所有区间中结束最早的区间，后续区间仍然能放下，最优数量不减；归纳处理剩余区间。

变式。 最少删除区间使剩下两两不交 = n 减去最多能选择的区间数。带价值的最大权独立区间集合通常需要 DP，不能仅按右端点贪心。

最少选多少点刺穿全部区间

问题。 给定 n 个非空闭区间 $[l_i, r_i]$，在数轴上选择尽可能少的点，使每个区间都包含至少一个所选点。

贪心方法。 按区间右端点 r 升序排列，遇到尚未覆盖的区间时，直接选择它的右端点 r。

简证（交换）。 对最先未覆盖区间，任何可行解都要在其中选一个点；将所选点右移到 r 不会失去对之后尚未处理区间的覆盖（这些区间的右端点都不早于 r）。

边界。 区间必须是闭区间或整数点意义下的闭区间。若是实数半开 $[l, r)$，r 本身不属于区间，不能直接在 r 选点。

用最少区间覆盖目标 $[L, R)$

问题。 给定目标半开区间 $[L, R)$ 和若干可用区间 $[l_i, r_i)$，选出最少个数的可用区间，使它们的并集覆盖整个目标区间；若无法覆盖则报告无解。

贪心方法。 按左端点升序排列。当前已经覆盖到 `cur` 时，在全部满足 $l \le \text{cur}$ 的候选区间中取右端点最大的一个，将 `cur` 向右推进；若无法严格推进则无解。

简证（交换）。 任意合法覆盖下一步都必须选一个左端点不超过 `cur` 且向右延伸的区间，改选能延伸最远者只会扩大后续可覆盖范围，不增区间数。

最少资源容纳所有区间

问题。 给定 n 个任务的占用时段 $[l_i, r_i)$，每个任务必须完整执行，同一份资源不能同时执行重叠任务。求同时完成所有任务至少需要多少份资源。

贪心方法。 按左端点排序；用小根堆维护当前正在占用资源的任务右端点，处理新区间前弹出所有 $r \le l$ 的已结束区间，再加入当前右端点。答案是堆大小的历史最大值。

简证（下界达到）。 任意时刻重叠 d 个区间至少需要 d 份资源；该算法恰好只在所有已有资源都被占用时新开资源，使用资源数始终不超过最大重叠数。

边界。 与 18.2.1 相同，半开区间允许一个区间结束时另一个立即开始；$[l, r]$ 闭区间则不能这样复用端点。

区间三件套

```cpp
// [l, r): 最多不相交区间个数，要求 l < r
i64 max_disjoint(vector<pair<i64, i64>> a) {
    sort(a.begin(), a.end(), [](auto x, auto y) {
        return tie(x.second, x.first) < tie(y.second, y.first);
    });
    i64 ans = 0, last = numeric_limits<i64>::min();
    for (auto [l, r] : a) {
        if (l >= last) {
            ++ ans;
            last = r;
        }
    }
    return ans;
}

// [l, r]: 最少打点个数及具体选点，要求 l <= r
vector<i64> stab_intervals(vector<pair<i64, i64>> a) {
    sort(a.begin(), a.end(), [](auto x, auto y) {
        return tie(x.second, x.first) < tie(y.second, y.first);
    });
    vector<i64> ans;
    for (auto [l, r] : a) {
        if (ans.empty() || ans.back() < l) ans.push_back(r);
    }
    return ans;
}

// [l, r): 覆盖 [L, R) 最少区间数；无解 -1；L >= R 时返回 0
int min_interval_cover(vector<pair<i64, i64>> a, i64 L, i64 R) {
    if (L >= R) return 0;
    sort(a.begin(), a.end());
    int n = a.size(), i = 0, ans = 0;
    i64 cur = L;
    while (cur < R) {
        i64 far = cur;
        while (i < n && a[i].first <= cur) {
            far = max(far, a[i].second);
            ++ i;
        }
        if (far == cur) return -1;
```

```
        cur = far;
        ++ ans;
    }
    return ans;
}
```

复杂度。 排序均为 $O(n\log n)$，扫描 $O(n)$；空间 $O(n)$（排序缓冲及答案）。

2.15.3 单机任务调度

以下模型均假设只有一台机器、全部任务在时刻 0 可用、任务没有前置依赖且不可抢占。p_i 表示处理时间，w_i 表示任务权值，d_i 表示截止时间，C_i 表示完成时刻。**目标不同，贪心方法也不同。**

全部任务必须执行：最优排序

问题。 给定 n 个任务，必须安排全部任务执行，要求分别最小化所有完成时刻之和、带权完成时刻之和，或最大延误。每种目标的最优顺序如下。

目标	贪心方法	简证
最小化 $\sum_i C_i$	按 p_i 升序（SPT）	交换相邻两项，短任务在前不增大总完成时间
最小化 $\sum_i w_i C_i$	按 p_i/w_i 升序（Smith 法则）	i 在 j 前更优当且仅当 $p_i w_j \le p_j w_i$
最小化 $\max_i(C_i-d_i)$	按 d_i 升序（EDD）	交换截止时间逆序的相邻两项，不增大最大延误

边界。 上述 p_i, w_i 非负；Smith 法则对 $w_i>0$ 的任务用交叉乘积 `i128(p[i]) * w[j] < i128(p[j]) * w[i]` 比较，避免浮点误差。零权任务安排在末尾；$p_i=0, w_i>0$ 的任务在最前。最大延误 $\max_i(C_i-d_i)$ 可以为负；EDD 对最大拖期 $\max_i \max(0, C_i-d_i)$ 也最优，但两者目标值不同。

任意耗时：最多按时完成多少任务

问题。 给定处理时间 $p_i \ge 0$、截止时间 $d_i \ge 0$ 的任务，可以放弃部分任务，要求**最大化能够在各自截止时间前完成的任务数量**（不是总收益）。

贪心方法。 按截止时间升序枚举任务，暂时加入当前任务；如果已选任务总耗时超过当前截止时间，则从已选任务中**删除耗时最长的一个**，用大根堆支持反悔。

简证。 截止时间升序处理时，当前所选任务可以按此顺序执行。若新加入的任务导致超时，就必须在当前候选任务中舍弃至少一个；删除耗时最长者能在保留相同任务数的情况下腾出最多的时间。结合归纳交换论证，保留最有利于后续扩展的可行解，最终完成任务数最大。

注意事项。 本策略允许舍弃任务，但不允许拆分、并行或存在额外前置约束；存在释放时间、不同收益或其他资源限制时不能直接套用。

Moore–Hodgson 反悔调度

```
// 任务 (duration, deadline)，均非负；返回选中的原始下标，按 deadline 排好
vector<int> max_ontime_jobs(const vector<pair<i64, i64>> &job) {
    int n = job.size();
    vector<int> ord(n);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int i, int j) {
        return tie(job[i].second, i) < tie(job[j].second, j);
    });
    priority_queue<pair<i64, int>> pq;
    vector<char> use(n);
    i64 sum = 0;
    for (int id : ord) {
        auto [p, d] = job[id];
        pq.push({p, id});
        sum += p;
        use[id] = true;
        if (sum > d) {
            auto [len, j] = pq.top();
            pq.pop();
            sum -= len;
            use[j] = false;
        }
    }
    vector<int> res;
    for (int id : ord) if (use[id]) res.push_back(id);
    return res;
}
```

单位耗时：截止时间内收益最大

问题。 每个任务恰好占用一个整数时间槽，任务 i 必须安排在不晚于 d_i 的时槽，完成后获得收益 w_i；允许不执行某些任务，要求**最大化总收益**。最早时间槽为 1。

贪心方法。 按收益从大到小处理正收益任务，把每个任务安排到不晚于截止时间的**最晚空闲时槽**。用并查集寻找不超过指定时槽的最大空位。

简证。 可安排任务集合满足交换性质：若高收益任务能够加入某可行集合，则可用它替换某个收益不更高的已选任务而不减总收益。放入最晚空位不会减少此前时槽的可用性，故可继续选择后续任务。

注意事项。 必须是单位耗时且时槽离散；如果每个任务耗时不同，不能用这份收益贪心。截止时间大于 n 时可截断到 n，因为总共最多执行 n 个任务。

并查集维护最晚空闲时槽

```
// job[i] = {deadline, profit}; 一个任务耗时 1，最早时槽为 1
// 返回 {最大收益，各时槽执行的原下标}; slot[0] 为 -1
pair<i64, vector<int>> schedule_unit_profit(vector<pair<int, i64>> job) {
    int n = job.size();
    vector<int> ord(n), fa(n + 1), slot(n + 1, -1);
    iota(ord.begin(), ord.end(), 0);
    iota(fa.begin(), fa.end(), 0);
    auto find = [&](int x) {
        while (fa[x] != x) {
            fa[x] = fa[fa[x]];
            x = fa[x];
        }
        return x;
    };
    sort(ord.begin(), ord.end(), [&](int i, int j) {
        return tie(job[i].second, i) > tie(job[j].second, j);
    });
    i64 ans = 0;
    for (int id : ord) {
        auto [d, w] = job[id];
        if (d <= 0 || w <= 0) continue;
        int t = find(min(d, n));
        if (!t) continue;
        ans += w;
        slot[t] = id;
        fa[t] = find(t - 1);
    }
    return {ans, slot};
}
```

复杂度。 上述两份任务选择实现均为 $O(n\log n)$ 时间、$O(n)$ 空间；三种排序型调度排序为 $O(n\log n)$。

2.15.4 堆上动态候选与最少次数

最少加油次数（油箱无限容量）

问题。 给定起点、终点距离 D、初始油量，以及沿途若干加油站的位置和可提供油量。每单位距离耗油 1，油箱容量无限，每站最多使用一次；求到达终点的**最少加油次数**，无法到达则返回无解。

贪心方法。 按位置从小到大考虑加油站。行驶到下一站或终点所需的油量不足时，从此前经过的站点中选择**可提供油量最多**的站补加，使用大根堆维护候选。

证明（交换）。 在必须补一次油的位置，换用此前可选站中油量最大的补给，不增加次数并使剩余可达距离不短于任何其他选择。将选择视为“之前经过站点时已经加油”，不会影响可达性；归纳成立。

```
// stations[i] = {从起点的距离，可取得的油量}; target ≥ 0
// 初始燃料 fuel，可获得的额外油量和耗油量均非负，单位距离耗油 1
int min_refuel_stops(vector<pair<i64, i64>> stations, i64 target, i64 fuel) {
    vector<pair<i64, i64>> a;
    for (auto [x, v] : stations) {
        if (0 <= x && x <= target) a.push_back({x, v});
    }
    a.push_back({target, 0});
    sort(a.begin(), a.end());
    priority_queue<i64> pq;
    i64 prev = 0;
    int ans = 0;
    for (auto [x, v] : a) {
        fuel -= x - prev;
        while (fuel < 0) {
            if (pq.empty()) return -1;
            fuel += pq.top();
            pq.pop();
            ++ ans;
```

```
        }
        pq.push(v);
        prev = x;
    }
    return ans;
}
```

边界。 油箱容量没有上限；站点油量不可拆分限制不影响上式（加整份即可），但若有油箱容量、油价、每站限额或必须现场决定加多少，结论可能失效。`fuel` 与最终可达距离可能超出 `i64` 时需改用更大类型。

最少跳跃次数（从下标 0 跳到 $n-1$）

问题。 给定非负数组 a，从下标 0 出发，在位置 i 可以前进 1 至 a_i 步，求到达 $n-1$ 的最少跳跃次数；不可达返回 -1。

贪心方法。 扫描当前跳跃次数可到达的整层下标 $[L,R]$，计算这层所有位置能扩展出的最大右端点 $\max(i+a_i)$，将其作为下一层边界；无法向右扩展则无解。

简证（BFS 分层）。 使用 t 次跳跃可到达的下标恰形成从起点起的连续前缀；其右端点由上一步可达点的最大延伸确定。这是隐式图的 BFS 分层，不是“每次只选择跳得最远的下一格”。

```
// 非负步长；若不可达返回 -1。空数组返回 0。
int min_jumps(const vector<int> &a) {
    int n = a.size();
    if (n <= 1) return 0;
    int ans = 0, r = 0, far = 0;
    for (int i = 0; i < n - 1; ++ i) {
        if (i > r) return -1;
        far = max(far, int(min<i64>(n - 1, i64(i) + a[i])));
        if (i == r) {
            if (far == r) return -1;
            r = far;
            ++ ans;
            if (r == n - 1) break;
        }
    }
    return ans;
}
```

气站环路：最低前缀和确定起点

问题。 给定一个环上每站的供油量 gas_i 和从该站到下一站的耗油量 cost_i，油箱无上限、初始燃料为 0；求是否存在出发站，使车辆按固定方向恰好绕环一圈。

贪心方法。 令净增量 $b_i=\mathrm{gas}_i-\mathrm{cost}_i$。若 $\sum_i b_i<0$ 则无解；否则令 $s_0=0$、$s_{i+1}=s_i+b_i$，取 $s_0,\dots,s_{n-1}$ 的一个最小值位置 k，从站 k 出发一定可行。

简证。 从最小前缀之后开始，未绕回的相对前缀和都非负；绕回后的相对前缀和等于 总和加上某个更早的前缀和，再减去最小前缀和，也非负。总和为负则不可能完成全环。

2.15.5 排序、配对与数值结论

重排不等式

问题。 给定两个长度相同的数组，只能重排其中一个数组，要求求出元素逐项相乘之和的最大值、最小值。

贪心方法。 最大值同向排序后配对，最小值反向排序后配对。对非降数组 $a_0,\dots,a_{n-1}$ 和 $b_0,\dots,b_{n-1}$：

$$\sum_i a_i b_{n-1-i} \le \sum_i a_i b_{\pi(i)} \le \sum_i a_i b_i.$$

简证（去逆序）。 当 $a_i\le a_j$、$b_x\le b_y$ 时，顺序配对相对于交叉配对多出 $(a_j-a_i)(b_y-b_x)\ge 0$；逐个消除配对逆序即可。元素可以为负；仅需两数组都按同一方向排序。

最小化两列的绝对差之和

问题。 给定两个长度相同的数组 a,b，建立一一匹配，最小化所有匹配对绝对差之和。

贪心方法。 将两数组分别升序排序，按相同下标匹配：

$$\min_{\pi}\sum_i |a_i-b_{\pi(i)}| = \sum_i |a_i^{\uparrow}-b_i^{\uparrow}|.$$

简证（消交叉）。 对 $a\le b$、$c\le d$，满足 $|a-c|+|b-d|\le |a-d|+|b-c|$；因此交换交叉匹配边不会变坏。

最小化最大配对和

问题。 给定长度相同的两个数组 a, b，建立一一匹配，要求最小化所有匹配对之和的最大值 $\max_i(a_i + b_{\pi(i)})$。同一数组元素两两配对也是常见变式（此时数组长度为偶数）。

贪心方法。 将一个数组升序、另一个降序排序，小配大；若是同一个数组，则反复将最小元素与最大元素配对。

简证。 若 $a \le b$, $c \le d$ 却顺序配成 $(a,c),(b,d)$，换成 $(a,d),(b,c)$ 后，两组新和都不超过原来的较大者 $b+d$，反复交换即可。

中位数最优、平均数最优、极差中心

问题。 给定实数序列 a_i，选择同一个值 x，使所有元素到 x 的某种距离总量（绝对差、平方差、最大绝对差）最小。

贪心方法与最优结论。 根据题目具体的目标函数分别选择中位数、均值或极值中点：

- $\min_x \sum_i |a_i - x|$：任意中位数最优；偶数长度时两中位数之间均最优。证明：函数在相邻样本点间的斜率为“左侧数目 - 右侧数目”，穿过中位数时由负转正。
- $\min_x \sum_i w_i|a_i - x|$ $(w_i \ge 0)$：取加权中位数，即左右侧权值都不超过总权值的一半的 x。证明同上，把数量换成权值。
- $\min_x \sum_i (a_i - x)^2$：取算术平均数；若 x 必须为整数，检查均值的上下取整。证明：展开平方或令导数为零。
- $\min_x \max_i |a_i - x|$：取 $\frac{\min_i a_i + \max_i a_i}{2}$，最优值 $\frac{\max_i a_i - \min_i a_i}{2}$；若 x 整数，取中点附近的整数。证明：最远距离至少为极差的一半，取中点达到下界。

已排序数轴上两两距离之和

问题。 给定数轴上的 n 个点，快速求所有无序点对的距离之和 $\sum_{i<j} |a_i - a_j|$。

方法。 将点按位置升序排序，按每个点作为较大数或较小数的次数统计贡献。设 $a_0 \le \cdots \le a_{n-1}$：

$$\sum_{0 \le i < j < n} |a_j - a_i| = \sum_{i=0}^{n-1} (2i - n + 1) a_i.$$

简证（贡献计数）。 每个 a_i 以正号出现在 i 对中，以负号出现在 $n-1-i$ 对中。

定和正整数乘积最大

问题。 把整数 $S \ge n$ 拆成恰好 n 个正整数之和，要求最大化这 n 个数的乘积。

贪心方法。 尽量均分总和，得到的数两两差至多为 1；设 $q = \lfloor S/n \rfloor$, $r = S \bmod n$，最优乘积为 $q^{n-r}(q+1)^r$（可能需高精度）。

简证。 若 $a + 2 \le b$，将 (a, b) 改成 $(a+1, b-1)$，乘积增加 $b - a - 1 > 0$；反复均衡直至差不超过 1。允许零、负数或对每个元素另有限制时，要重新判断。

分数背包

问题。 给定背包容量 $W \ge 0$，每件物品重量 $m_i > 0$、价值 $v_i \ge 0$，允许只取某件物品的一部分且价值按取用比例计算，求最大总价值。

贪心方法。 按单位重量价值 $\frac{v_i}{m_i}$ 从大到小取，最后一件可以只取部分。

简证。 若先取了低密度物品，却仍有更高密度未取满，则交换等重量的两份可以提高收益。

边界。 0/1 背包中整件不可分割，按单位价值排序不保证最优；需要 DP 或其他算法。

整除链硬币体系

问题。 给定面额严格递增的硬币 $c_0 = 1, c_1, \ldots$，满足 $c_i \mid c_{i+1}$，每种硬币数量无限；求凑出非负整数 X 的最少硬币数。

贪心方法。 从面额最大者开始尽量取，余数继续由较小面额凑出。

简证。 c_{i+1}/c_i 个 c_i 可以替换为一个 c_{i+1} 而严格减少硬币数，故较小面额数量在最优方案中可规范化。

边界。 对一般面额不成立，例如面额 $\{1, 3, 4\}$ 凑 6，取 $4+1+1$ 是 3 枚，最优 $3+3$ 是 2 枚。

2.15.6 字符串与字典序构造

固定长度的字典序最小子序列

问题。 给定长度为 n 的字符串 s，必须保留恰好 k 个字符并保持原有相对顺序，要求得到字典序最小的长度为 k 的子序列。

贪心方法。 用单调栈维护当前答案，最多删除 $n-k$ 个字符。遇到较小字符时，只要还有删除额度，就不断弹出栈顶更大的字符；遍历结束后截取前 k 个字符。

简证。 在仍能补齐剩余长度的前提下，尽量让最早的尚未确定位置更小，必优于保留一个更大的字符。操作对每个字符至多进出栈一次。

```
string min_subsequence(const string &s, int k) {
    int n = s.size();
    assert(0 <= k && k <= n);
    int rem = n - k;
    string st;
    for (char c : s) {
        while (rem && !st.empty() && st.back() > c) {
            st.pop_back();
```

```
            -- rem;
        }
        st.push_back(c);
    }
    st.resize(k);
    return st;
}
```

变式。 删除 k 个十进制数字使数值最小：保留 $n-k$ 个数字按同样方法求最小序列，最后去掉前导零；若结果全为零或空，则输出 0。若题目要求结果固定长度、禁止前导零或要求恰好若干非零数字，必须调整可行性判断。

字典序最小拓扑序

问题。 给定一张有向无环图（DAG），在所有满足依赖先后顺序的拓扑排序中，找出按顶点编号比较时字典序最小的那个。

贪心方法。 每次从当前入度为 0 的顶点中取编号最小者（小根堆），将其加入答案并删除其出边。

简证。 任何合法拓扑序的下一点都必须是入度为 0 的点，选最小可行首点使当前位最优；余下仍是一个 DAG，归纳。

边界。 图存在环时不存在完整拓扑序。要求字典序最大的拓扑序改用大根堆。普通队列只能得到某种拓扑序，不保证字典序最小。

2.15.7 拟阵贪心（可行集合的交换性质）

问题。 给定有限元素集 E，每个元素具有权值，并规定哪些子集可行。要求从中选择一个大小达到最大可能值的可行子集（称为基），并使所选元素权值之和最大。何时可以直接按权值从大到小选择？

贪心方法。 若可行集合族构成拟阵，即同时满足以下两个条件，则将元素按权值降序考虑，只要加入后仍可行就选入，最终得到最大权基。

- **遗传性：** 任一可行集的所有子集仍然可行。
- **交换性：** 若 A, B 可行且 $|A| < |B|$，则存在 $e \in B \setminus A$，使 $A \cup \{e\}$ 仍然可行。

简证。 由交换公理，每当贪心选择一项较大权值的可行元素时，可以将某个最优基中的相应元素换成它，仍保持可行且不减小权和。逐步交换，最终使一个最优基与贪心所选基完全一致。

例子。 “至多选择 k 个元素”对应均匀拟阵；“每种类别最多选择给定数量”对应划分拟阵；“单位耗时任务必须在各自截止时间前完成”也具有相应的交换性质。

注意事项。 上面保证的是最大权基（固定最大可行元素数）。如果允许选任意数量且只最大化权值总和，应跳过非正权元素；若选出的集合不需要达到最大基数，则不能把负权元素强行选进去。一般的 $0/1$ 背包约束不构成拟阵，不能只按价值或密度排序贪心。

2.15.8 反例速查（容易误判成贪心）

想当然的策略	反例 / 应用边界
带权区间选结束最早的	$[0,2)$ 权 100；$[0,1)$、$[1,2)$ 权各 1。结束早贪心仅得 2，应区间 DP
0/1 背包按价值密度排序	容量 50；(重量,价值) 为 $(10,60),(20,100),(30,120)$，密度法取前两件仅 160，最优 220
任意硬币体系先拿最大面额	$\{1,3,4\}$ 凑 6，贪心需 3 枚而最优 2 枚
Huffman 用于必须相邻合并	例如 $(1,100,1)$，任意合并最优总代价 104；只能相邻合并最优总代价 203
单机任务全部按最短耗时排列，总能按时做最多任务	任务 (p,d)：$(2,2),(1,100),(1,100)$；最短优先可能先做后两件，导致第一个超时，而按截止时间可做 3 件
资源分配的区间按结束最早即可	若目标是同时安排所有任务且资源数最少，下界是最大重叠数，应该按起点排序配堆
一般集合覆盖每次选新增覆盖元素最多者	这只保证经典对数级近似（一般集合覆盖问题），不是精确最优

2.15.9 判断贪心正确性的速查方法

1. **交换论证。** 交换最优解中的一部分，使其先选当前贪心选择且不变差（Huffman、区间选取、排序配对、Smith 法则）。
2. **始终不劣的前缀。** 保证贪心在处理前 i 项后，留下的余量、可达端点或累计用时不比任何同等规模方案差（覆盖、最少跳跃、反悔调度）。
3. **下界构造。** 先证明任何方案至少需要这么多资源或代价，再给出贪心达到下界的构造（区间分组、中位数、极差中心）。
4. **可扩展交换系统。** 可行集满足拟阵交换公理时，按权重排序就能做最大权基；不满足时需找反例或另用 DP、流、匹配。
5. **先审条件。** 区间是否闭合、任务能否拆分、是否有释放时间、是否只能相邻合并、物品能否分割、权值能否为负，都会直接改变结论。

2.15.10 复杂度总览

模型	时间	额外空间
Huffman 代价 / 建树（n 叶，k 叉，代码已限制 $k \le n$）	$O(n \log n)$	$O(n)$
区间最大互斥 / 最小打点 / 最少覆盖	$O(n \log n)$	$O(n)$
区间最少资源（堆）	$O(n \log n)$	$O(n)$
SPT / Smith / EDD	$O(n \log n)$	$O(n)$（包括排序及存储）
Moore–Hodgson 最多按时完成	$O(n \log n)$	$O(n)$
单位任务最大收益（并查集）	$O(n \log n)$	$O(n)$
最少加油次数（堆）	$O(n \log n)$	$O(n)$

模型	时间	额外空间
最少跳跃 / 气站环路	$O(n)$	$O(1)$
字典序最小定长子序列	$O(n)$	$O(n)$
重排 / 中位数 / 排序匹配	$O(n \log n)$（排序主导）	视排序实现